CLOUD DEVELOPERS

Cloud Developers — Exercise Book

Hands-on Labs

Lars Appel

v2026.04.29-98722a8

Contents

Preface 11

Portal Interface 15

1. Provisioning a VM via Azure Portal 15

2. Provisioning a VM with SSH Keys and Exploring Linux 24

3. Automating Nginx Installation with Custom Data Scripts 33

Command Line Interface ... 39

4. Creating a Resource Group Using AZ CLI 39

5. Provisioning a VM Using AZ CLI, Configuring Nginx, and Allowing HTTP Traffic 44

6. Automating VM Creation, Nginx Installation, and Port Configuration Using a Bash Script 53

ARM and Bicep 61

7. Exporting a Template from Azure Portal 61

8. Minimal ARM Template: Ubuntu VM + Network 64

9. From ARM to Bicep: Ubuntu VM + Network 72

10. Nginx via Cloud-init (bash) and Custom Script Extension 78

Portal Interface 15
1. Creating a Virtual Network 87
2. Creating a Virtual Network with Enhanced Security 108

Command Line Interface 39
3. Virtual Network with Enhanced Security 131

CI/CD to Azure Container Apps 155
First Pipeline: Build and Push to Docker Hub 155
First Pipeline: Build and Push to Docker Hub 155
Private Registry and Automated Deployment 179
Private Registry and Automated Deployment 179
Passwordless Deployment with OIDC Federation 207
Passwordless Deployment with OIDC Federation 207

Logging and Monitoring 237
Structured Logging with ILogger 237
Structured Logging with ILogger<T> 237
Container Logs to Log Analytics 272
Container Logs to Log Analytics 272
Application Insights and Distributed Tracing 300
Application Insights and Distributed Tracing 300

REST API and DTOs 339
REST Controllers, DTOs, and Swagger 339
REST Controllers, DTOs, and Swagger 339
Securing the API with an API Key Middleware 380
Securing the API with an API Key Middleware 380
JWT Bearer Authentication and Cleanup 410
JWT Bearer Authentication and Cleanup 410

Portal Interface 15
1. Provisioning CosmosDB via Portal 461
Provisioning CosmosDB via Portal 461

Command Line Interface 39
2. Provisioning CosmosDB via AZ CLI 477
Provisioning CosmosDB via AZ CLI 477

ARM and Bicep 61
3. Provisioning CosmosDB with Bicep 497
Provisioning CosmosDB with Bicep 497

Connecting from Code 515
4. CosmosDB PoC with .NET Script 515
CosmosDB PoC with .NET Script 515

Azure Blob Storage 535
5. Provisioning Azure Blob Storage 535
Provisioning Azure Blob Storage 535

File Uploads and Deep Health Probes 551
MVC Uploads, PDF Validation, and First Deploy. 551
MVC Uploads, PDF Validation, and First Deploy. 551
CosmosDB and Azure Blob via Managed Identity. 603
CosmosDB and Azure Blob via Managed Identity. 603
Deep Health Probes and Cleanup 641
Deep Health Probes and Cleanup 641

Presentation Layer 677

1. Hello World 677

Hello World MVC Application 677

2. Create A Form With Basic HTML 682

Create A Form With Basic HTML 682

You have a form! 686

3. Helper Tags and Model Binding 686

Helper Tags and Model Binding 686

4. Data Annotations and Client-Side Validation 691

Data Annotations and Client-Side Validation 691

5. Server-Side Validation and Improved Feedback. 695

Server-Side Validation and Improved Feedback 695

6. Displaying a List of Subscribers 701

Displaying a List of Subscribers 701

7. Implement Unsubscribe Functionality 705

Implement Unsubscribe Functionality 705

Exercise Done! 710

8. Enhancing UI with Bootstrap 711

Exercise Done! 719

9. Landing Page with Hero Section 719

Exercise Done! 727

Service Layer 729
10. Implement Service Layer 729
Implement Service Layer 729
Exercise Done! 740
11. Implement Unit Tests and Organize Solution 740
Exercise Done! 747

Data Layer 749
12. Implement the Repository Pattern 749
Implement the Repository Pattern 749
Exercise Done! 759
13. Implementing MongoDB Repository 760
Implementing MongoDB Repository (Development) 760
Exercise Done! 778
14. Migrating to CosmosDB 778
Migrating to CosmosDB 778
15. Azure Blob Storage for Images 794
Azure Blob Storage for Images 794
Exercise Done! 812

Authentication and Authorization 813

1. Cookie Authentication and the Who Am I Page 813

Cookie Authentication and the Who Am I Page 813

2. Roles and the [Authorize] Attribute 831

Roles and the `[Authorize]` Attribute 831

3. Claims, Policies, and CSRF Protection 840

Claims, Policies, and CSRF Protection 840

Identity and User Stores 851

1. Introducing ASP.NET Core Identity 851

Introducing ASP.NET Core Identity 851

2. User Store: InMemory vs SQLite with a Feature Flag 864

User Store: InMemory vs SQLite with a Feature Flag 864

3. Seeding the First Admin 874

Seeding the First Admin 874

4. Registration and Role Promotion 883

Registration and Role Promotion 883

Preface

Hands-on exercises to build practical skills in cloud development and infrastructure.

PART I

Server Foundation

Learn the fundamentals of Azure infrastructure through progressive exercises, from visual portal interface to command-line automation.

0.1 Learning Path

This track guides you through server provisioning and configuration using two complementary approaches:

1. ***Portal Interface** - Graphical approach for understanding Azure resource management visually*
2. ***Command Line Interface** - Scriptable approach for infrastructure automation and repeatability*

EXERCISE 1

Portal Interface

1.1 1. Provisioning a VM via Azure Portal

1.1.1 Goal

Create your first Virtual Machine in Azure using the portal interface to understand how cloud resources are provisioned and managed.

WHAT YOU'LL LEARN:

- How to navigate the Azure Portal interface
- When to use resource groups for organizing cloud resources
- Best practices for VM configuration and security settings

1.1.2 Prerequisites

> **BEFORE STARTING, ENSURE YOU HAVE:**
>
> - ✓ An active Azure account with an active subscription
> - ✓ Access to a web browser
> - ✓ Basic understanding of what a virtual machine is

1.1.3 Exercise Steps

Overview

1. **Log in to Azure Portal**
2. **Create a Resource Group**
3. **Create a Virtual Machine**
4. **Connect to the VM Using Azure Cloud Shell**
5. **Install Nginx (Optional)**
6. **Clean Up Resources (Optional)**

Step 1: *Log in to Azure Portal*

The Azure Portal is your central management interface for all Azure resources. This web-based console provides a graphical way to create, configure, and monitor your cloud infrastructure without needing command-line expertise.

1. **Open** a web browser and navigate to the Azure Portal
2. **Sign in** using your Azure account credentials

> **QUICK CHECK:**
>
> You should see the Azure Portal dashboard after successful login

Step 2: *Create a Resource Group*

Before creating any resources, you need a container to hold them. Resource groups provide logical organization for

related resources and enable you to manage permissions, costs, and lifecycle as a single unit.

1. **Search for** "Resource groups" in the Azure Portal search bar
2. **Click** "Create" and fill in the following:
   - **Subscription**: Choose your subscription
   - **Resource Group Name**: `LabResourceGroup`
   - **Region**: Choose a region (e.g., `North Europe`)
3. **Click** "Review + Create" and then "Create"

> **CONCEPT DEEP DIVE**
>
> A **Resource Group** is a container that holds related resources for an Azure solution. It allows for easy management and deletion of all resources together. Think of it as a folder that groups everything related to a specific project or environment.
>
> ✓ **Quick check:** The resource group appears in the Resource Groups list

Step 3: *Create a Virtual Machine*

Creating the VM is the core of this exercise. You'll configure compute resources, choose an operating system, set up authentication, and define network access rules. Each setting affects security, cost, and functionality of your server.

1. **Search for** "Virtual Machines" in the Azure Portal and click "Create" > "Azure Virtual machine"
2. **Configure** the VM with these settings:
   - **Subscription**: Select your subscription
   - **Resource Group**: Choose `LabResourceGroup`
   - **Virtual Machine Name**: `LabVM`
   - **Region**: Same as the resource group

- **Zone options**: Select *Azure-selected zone*
- **Image**: Select `Ubuntu Server 24.04 LTS`
- **Size**: Choose a size like `Standard_B1s`
- **Authentication Type**: Select **Password**
- **Username**: `azureuser`
- **Password**: Create a secure password and confirm it

3. **Configure** inbound ports:
   - Check **Allow selected ports** and select **HTTP (80)** and **SSH (22)**
4. **Click** "Review + Create", and then "Create"

> **CONCEPT DEEP DIVE**
>
> - **VM Size:** Determines the CPU, memory, and storage capacity. For this exercise, `Standard_B1s` is cost-effective for learning purposes.
> - **Inbound Ports:** Ensure HTTP (for web traffic) and SSH (for secure access) are open for connectivity.
>
> ⚠ **Common Mistakes**
>
> - Forgetting to open port 22 will prevent SSH access to your VM
> - Choosing the wrong region can increase latency and costs
> - Using a weak password compromises security

1. **Wait** for Azure to provision the VM. Once completed, navigate to the **Virtual Machines** section and locate `LabVM`

> ☁️ ***Cloud Concept: On-Demand***
> *Notice how you just created a server in minutes? This is* ***on-demand provisioning*** *- a fundamental characteristic of cloud computing. In traditional IT, ordering a physical server could take weeks: procurement, shipping, rack installation, cabling, OS installation. With cloud, compute resources are available instantly, billed by the hour or minute, and can be deleted when no longer needed. This changes how organizations think about infrastructure - from capital expenditure to operational expenditure, from planning months ahead to responding in real-time.*
> √ ***Quick check:*** *Ensure the VM status is "Running" in the Virtual Machines overview*

Step 4: *Connect to the VM Using Azure Cloud Shell*

Now that your VM is running, you need to access it remotely. Azure Cloud Shell provides a browser-based terminal with pre-installed tools, eliminating the need for local terminal setup.

1. **Click** the **Cloud Shell** icon in the Azure Portal (top-right corner)
2. **Select** "Bash" as your shell environment
3. **Connect** to your VM via SSH:

```
ssh azureuser@<VM_Public_IP>
```

   - Replace `<VM_Public_IP>` with the public IP address of your VM, found in the *Overview* tab of the VM

- Answer *yes* on the question: *Are you sure you want to continue connecting (yes/no/[fingerprint])?*

> **CONCEPT DEEP DIVE**
>
> SSH (Secure Shell) is a protocol used for secure remote login to your server. It encrypts all traffic between your terminal and the server, protecting your credentials and commands from interception.
>
> Note that there is no feedback in the terminal when writing a password. It will look like you don't write anything - this is a security feature.
>
> √ **Quick check:** You should see a command prompt for `azureuser@LabVM`

Step 5: *Install Nginx (Optional)*

With access to your VM, you can now install software. Nginx is one of the most popular web servers, known for its performance and low resource usage. This step demonstrates the basic package management workflow on Ubuntu.

1. **Install** Nginx:

```
sudo apt update
sudo apt install nginx -y
```

CONCEPT DEEP DIVE

Nginx is a very popular Web Server used by millions of websites worldwide.

The `apt update` command refreshes the list of available packages. The `-y` flag confirms the operation without additional prompts (user interaction). `sudo` is used in order to run commands as administrator.

⚠ **Common Mistakes**

- Forgetting `sudo` will result in permission denied errors
- Skipping `apt update` may install outdated package versions

Verify the installation using any of these three methods:

1. **Check service status:**

```
sudo systemctl status nginx
```

Ensure the output shows `active (running)` for the Nginx service. Get back to the prompt by pressing `Ctrl-C`.

2. **Test locally with curl:**

```
curl localhost
```

Ensure you get some HTML code back. This confirms Nginx runs correctly regardless of firewall settings.

3. **Test from browser:**

Open a browser and go to `http://<VM_Public_IP>`. You should see the "Welcome to Nginx!" default page.

> **QUICK CHECK:**
> At least one of the three verification methods confirms Nginx is running

Step 6: *Clean Up Resources (Optional)*

Cleaning up resources when you're done prevents unnecessary charges and keeps your Azure environment organized. Deleting the resource group removes all contained resources in one operation.

1. **Navigate to** the Resource Groups section by searching for "Resource Groups" in the search bar
2. **Locate** the resource group `LabResourceGroup`. You can use the search bar in the Resource Groups section if you have multiple groups
3. **Click** on `LabResourceGroup` to open the resource group details
4. **Click** the "Delete resource group" button at the top of the page
5. **Confirm** deletion:
   - Check the *Apply force delete ...* checkbox
   - Enter the exact name `LabResourceGroup` in the confirmation box
   - Click "Delete" and then "Delete" again

CONCEPT DEEP DIVE

Deleting the resource group will permanently remove all resources contained within it, including the virtual machine, network interface, and any associated disks or public IPs. This action helps avoid unnecessary costs and keeps your environment clean.

√ **Quick check:** After deletion, go back to the Resource Groups page and confirm that `LabResourceGroup` no longer appears in the list. Refresh the page if necessary. It can take a minute or two before it has finalized the operation.

1.1.4 Common Issues

> ***If you encounter problems:***
> ***Cannot connect via SSH:*** *Verify port 22 is open in the VM's network security group and you're using the correct public IP address*
> ***Nginx page not loading:*** *Ensure port 80 is open and Nginx service is running*
> ***Password not working:*** *Remember there's no visual feedback when typing passwords in SSH*
> ***Still stuck?*** *Check the VM status in Azure Portal - it should show "Running"*

1.1.5 Summary

You've successfully provisioned a Virtual Machine in Azure using the portal which:

- √ Demonstrates the complete VM creation workflow
- √ Establishes secure remote access via SSH
- √ Provides a foundation for web server deployment

> **KEY TAKEAWAY:**
> The Azure Portal provides a visual interface for resource management that's ideal for learning and one-off deployments. You'll use this workflow whenever you need to quickly create and configure cloud resources.

1.1.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Try changing the Nginx default page content*
> - *Explore other VM sizes and their cost implications*
> - *Research the difference between SSH key and password authentication*
> - *Investigate Network Security Groups and their rules*

1.1.7 Done 🎉

You have successfully provisioned a Virtual Machine in Azure using the portal and connected to it using Azure Cloud Shell. You also verified its operation by installing and testing the web server Nginx.

1.2 2. Provisioning a VM with SSH Keys and Exploring Linux

1.2.1 Goal

Provision an Ubuntu virtual machine with secure SSH key authentication and explore fundamental Linux filesystem navigation and bash scripting.

WHAT YOU'LL LEARN:

- How to create and use SSH key pairs for secure VM authentication
- How to connect to remote servers using SSH from your local terminal
- How to navigate the Linux filesystem and understand its hierarchical structure
- How to create and execute bash scripts to automate server configuration

1.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ An active Azure account with an available subscription
- ✓ A terminal application (Git Bash on Windows, Terminal on Mac/Linux)
- ✓ Completed Exercise 1 or familiarity with Azure Portal navigation

1.2.3 Exercise Steps

Overview

1. **Log in to Azure Portal**
2. **Create a Virtual Machine with SSH Key Pair**
3. **Secure the SSH Private Key**
4. **Connect to the VM**
5. **Explore the Linux Filesystem**
6. **Create and Execute a Bash Script to Install Nginx**
7. **Clean Up Resources**

Step 1: *Log in to Azure Portal*

Before creating any resources, you need to access the Azure management interface. The Azure Portal provides a web-based dashboard for managing all your cloud resources.

1. **Open** a web browser and navigate to the Azure Portal.
2. **Sign in** using your Azure account credentials.

Step 2: *Create a Virtual Machine with SSH Key Pair*

This step creates your Ubuntu server with secure SSH key authentication. Unlike password authentication, SSH keys provide cryptographic security that is virtually impossible to brute-force attack.

1. **Navigate to Virtual Machines** and click **Create** > **Azure Virtual machine**.
2. **Configure** the VM:
   - **Subscription**: Select your subscription.
   - **Create a new Resource Group**: `LabSSHResourceGroup`. (You can create a new resource group here if you don´t have one already)
   - **Virtual Machine Name**: `LabSSHVM`.
   - **Region**: Same as the resource group.
   - **Zone options**: Select *Azure-selected zone*
   - **Image**: Select `Ubuntu Server 24.04 LTS`.
   - **Size**: Choose `Standard_B1s`.
   - **Authentication Type**: Select **SSH Public Key**.
   - **Username**: `azureuser`.
3. **Select Generate new key pair** under **SSH Public Key Source**.
4. **Configure** inbound ports:
   - Check **Allow selected ports** and select **HTTP (80)** and **SSH (22)**.

5. **Note** the name for your key pair (e.g., `LabSSHKey`) and click **Download private key and create resource**.

> **CONCEPT DEEP DIVE**
>
> The **Generate new key pair** option allows you to securely create an SSH key pair. The private key will be downloaded to your computer and should be stored securely. Azure automatically applies the corresponding public key to the VM.

Step 3: *Secure the SSH Private Key (Only Mac and Linux Users)*

SSH requires that private key files have restricted permissions. This security measure prevents other users on your system from reading your private key, which would compromise your server's security.

1. **Locate** the downloaded private key file (e.g., `LabSSHKey.pem`) on your local machine.
2. **Open** the Terminal and change file permissions:

```
chmod 400 ~/Downloads/LabSSHKey.pem
```

> **CONCEPT DEEP DIVE**
>
> The `chmod 400` command restricts permissions on the private key file, ensuring it is readable only by the file's owner. This is required for secure SSH connections. Failure to set these permissions will result in SSH errors.
>
> Make sure the key is in the Downloads folder. Otherwise you need to adjust the command above accordingly.

Step 4: *Connect to the VM*

Now you will establish a secure shell connection to your virtual machine. SSH encrypts all traffic between your local

computer and the remote server, protecting your commands and data from interception.

1. **Navigate to** your VM's **Overview** tab in the Azure Portal and copy the public IP address of `LabSSHVM`.
2. **Open** your terminal:
   - **Windows**: Use **Git Bash**.
   - **Mac/Linux**: Use your default terminal.
3. **Connect** to the VM using SSH:

```
ssh -i ~/Downloads/LabSSHKey.pem
        azureuser@<VM_Public_IP>
```

   - Replace `<VM_Public_IP>` with the public IP address you copied.
   - On Windows you might need to use the absolute path for the key. You can usually do that by drag-and-drop the key `LabSSHKey.pem` from the file explorer into the Terminal

> **QUICK CHECK:**
> Ensure you are logged into the VM and see the `azureuser@LabSSHVM` prompt.

Step 5: *Explore the Linux Filesystem*

Understanding the Linux filesystem structure is essential for server administration. The filesystem follows a hierarchical tree structure starting from the root directory, with each directory serving a specific purpose.

1. **List** the contents of the root directory:

```
ls /
```

- Key directories:
  - `/home`: User home directories.
  - `/etc`: Configuration files.
  - `/var`: Logs and variable data.

2. **Check** your current working directory:

```
pwd
```

> **CONCEPT DEEP DIVE**
>
> The Linux filesystem is hierarchical, starting from `/` (root).

Step 6: *Create and Execute a Bash Script to Install Nginx*

Bash scripts automate repetitive tasks and ensure consistent server configuration. By writing installation steps as a script, you can reproduce the same setup across multiple servers and document exactly what was done.

1. **Create** the Bash Script:
   - While logged into the VM, create a new script file:

```
nano install_nginx.sh
```

   - **Add** the following content to the file:

```
#!/bin/bash
apt update
apt install nginx -y
```

   - **Save** and exit by pressing `CTRL+X`, `y`, `Enter`.

2. **Make** the Script Executable:

```
chmod +x install_nginx.sh
```

3. **Run** the Script with Elevated Privileges:

```
sudo ./install_nginx.sh
```

4. **Verify** the Installation:
   - Check if Nginx is running:

     ```
     systemctl status nginx
     ```

     Ensure the status shows `active (running)`.
   - Test Nginx using `curl`:

     ```
     curl localhost
     ```

     The output should include the default Nginx HTML content, such as `Welcome to nginx!`.

> **CONCEPT DEEP DIVE**
>
> **Systemctl and the Init System**
>
> The `systemctl` command is used to interact with the system's **init system** (often `systemd` on modern Linux distributions). It is responsible for managing services, such as starting, stopping, enabling, or checking the status of services like Nginx.
>
> - `start` : Starts the service immediately.
> - `enable` : Configures the service to start automatically at boot.
> - `status` : Displays the current status of the service.

Step 7: *Clean Up Resources (Optional)*

To avoid unnecessary charges, you should delete resources when they are no longer needed. Deleting the entire resource group removes all associated resources in one operation.

1. **Navigate to Resource Groups** in the Azure Portal and locate `LabSSHResourceGroup`.
2. **Click Delete resource group**.

1.2.4 Common Issues

> ***If you encounter problems:***
> ***Permission denied (publickey):*** *Ensure you ran* `chmod 400` *on your private key file and that you're using the correct path to the key.*
> ***Connection timed out:*** *Verify that SSH (port 22) is allowed in your VM's network security group inbound rules.*
> ***Key file path not found:*** *On Windows, use the absolute path by dragging the key file into Git Bash, or navigate to the Downloads folder first.*

1.2.5 Summary

You've successfully provisioned an Ubuntu VM with SSH key authentication and explored Linux fundamentals which:

- ✓ Created a secure SSH key pair using Azure's built-in key generation
- ✓ Established encrypted SSH connections from your local terminal
- ✓ Navigated the Linux filesystem structure
- ✓ Automated Nginx installation using a bash script

> **KEY TAKEAWAY:**
> SSH keys provide cryptographic authentication that is far more secure than passwords, and bash scripts enable repeatable, documented server configuration.

1.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Move your private key to* `~/.ssh/` *and create an SSH config file for easier connections*
> - *Explore other Linux directories like* `/var/log` *for system logs*
> - *Modify the bash script to also enable Nginx to start on boot with* `systemctl enable nginx`
> - *Try connecting from a different device using the same key pair*

1.2.7 Done 🎉

You have successfully provisioned a VM using Azure's Generate Key Pair feature, connected securely using SSH, and explored the Linux filesystem and basic commands.

1.3 3. Automating Nginx Installation with Custom Data Scripts

1.3.1 Goal

Automate the installation and configuration of Nginx during VM provisioning using Azure custom data scripts, then customize the default web page.

> **WHAT YOU'LL LEARN:**
>
> - How to use custom data scripts to automate software installation during VM provisioning
> - The relationship between cloud-init and Azure's custom data feature
> - How to modify Nginx's default web page content

1.3.2 Prerequisites

> **BEFORE STARTING, ENSURE YOU HAVE:**
>
> - An active Azure subscription
> - An SSH key pair available (or ability to generate one)
> - Basic familiarity with SSH connections

1.3.3 Exercise Steps

Overview

1. **Log in to Azure Portal**
2. **Create a Virtual Machine with Custom Data**
3. **Edit the Nginx Default Page**
4. **Clean Up Resources (Optional)**

Step 1: *Log in to Azure Portal*

Accessing the Azure Portal is the starting point for all Azure resource management. You'll use the portal's graphical interface to create and configure your virtual machine with automation scripts.

1. **Open** a web browser and navigate to the Azure Portal
2. **Sign in** using your Azure account credentials

Step 2: *Create a Virtual Machine with Custom Data*

Custom data scripts allow you to automate configuration tasks that would otherwise require manual SSH access after VM creation. This approach demonstrates infrastructure-as-code principles where your server configuration is defined before deployment.

1. **Navigate** to **Virtual Machines** in the Azure Portal and click **Create** > **Virtual machine**
2. **Configure** the VM with these settings:
   - **Subscription**: Select your subscription
   - **Create a new Resource Group**: `LabCustomDataRG`
   - **Virtual Machine Name**: `LabCustomDataVM`
   - **Region**: Same as the resource group
   - **Zone options**: Select *Azure-selected zone*
   - **Image**: Select `Ubuntu Server 24.04 LTS`
   - **Size**: Choose `Standard_B1s`

- **Authentication Type**: Select **SSH Public Key**
- **Username**: `azureuser`
- **SSH Public Key Source**: Use an existing SSH key or generate a new one
- **Allow selected ports** and select **HTTP (80)** and **SSH (22)**

3. **Navigate** to the **Advanced** tab
4. **Paste** the following script in the **Custom Data** field:

```
#!/bin/bash
apt update
apt install nginx -y
```

5. **Click Review + Create**, and then **Create**

> **CONCEPT DEEP DIVE**
>
> The **Custom Data** field allows you to execute a script during the initial boot of the VM, automating configurations and software installations.
>
> This script:
>
> - Updates the system's package list (`apt update`)
> - Installs the Nginx web server (`apt install nginx -y`)
>
> It is important that you 1) don't use `sudo` 2) make sure it is *non-interactive* 3) use a shebang
>
> **Common Mistakes**
>
> - Adding `sudo` to commands (custom data runs as root)
> - Forgetting the shebang (`#!/bin/bash`)
> - Using interactive prompts that hang the script
>
> **Quick check:** Open a browser and go to `http://<VM_Public_IP>`. You should see the **"Welcome to Nginx!"** default page. This might take a minute, so be patient and refresh the page.

Step 3: *Edit the Nginx Default Page*

Now that Nginx is running, you'll connect to the server and modify its default page. This demonstrates that while automation handles initial setup, you can still make manual changes when needed.

1. **Connect** to the VM using SSH:

```
ssh -i <private-key> azureuser@<VM_Public_IP>
```

   - Replace `<private-key>` with your private key file
   - Replace `<VM_Public_IP>` with the VM's public IP address
   - Don't forget to `chmod 400` the key if you are on Mac or Linux

2. **Navigate** to the Nginx *root* directory:

```
cd /var/www/html/
```

3. **Open** `index.nginx-debian.html` in a text editor:

```
sudo nano index.nginx-debian.html
```

4. **Modify** the content (e.g., add "Hello World" to the `<h1>` tag)
5. **Save** and exit by pressing `Ctrl+x`, `y`, and `Enter`

> ***Quick check:*** *Refresh the page in your browser or rerun the* `curl localhost` *command to see the updated content.*

Step 4: *Clean Up Resources (Optional)*

Deleting resources when you're finished prevents unnecessary charges and keeps your Azure subscription organized.

1. **Navigate** to **Resource Groups** in the Azure Portal
2. **Locate** `LabCustomDataRG`
3. **Click Delete resource group**

1.3.4 Common Issues

> ***If you encounter problems:***
> ***Nginx not responding after VM creation:*** *Wait 2-3 minutes for the custom data script to complete. The script runs during first boot and needs time to update packages and install Nginx.*
> ***Permission denied on SSH:*** *Ensure your private key has correct permissions (*`chmod 400 <private-key>`*).*
> ***Cannot edit index file:*** *Remember to use* `sudo` *when editing files in* `/var/www/html/`*.*

1.3.5 Summary

You've successfully automated Nginx installation using Azure custom data scripts which:

- Eliminates manual configuration steps after VM creation
- Demonstrates infrastructure-as-code principles
- Shows how cloud-init processes startup scripts
- Provides a foundation for more complex automation

> **KEY TAKEAWAY:**
>
> Custom data scripts transform VM provisioning from a multi-step manual process into a single automated deployment, making your infrastructure reproducible and consistent.

1.3.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Create a custom data script that also configures a custom HTML page automatically*
> - *Research cloud-init directives for more advanced configuration options*
> - *Experiment with multiple custom data scripts for different services*

1.3.7 Done 🎉

You have successfully automated the installation of Nginx using a custom data script and edited the default web page to display custom content.

EXERCISE 2

Command Line Interface

2.1 4. Creating a Resource Group Using AZ CLI

2.1.1 Goal

Create and manage an Azure resource group using command-line tools instead of the portal interface.

WHAT YOU'LL LEARN:

- How to access and use Azure Cloud Shell
- How to create resource groups using Azure CLI commands
- How to verify and list resources from the command line

2.1.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ An active Azure subscription
- ✓ Access to the Azure Portal

2.1.3 Exercise Steps

Overview

1. **Log in to Azure Cloud Shell**
2. **Create a Resource Group**
3. **Clean Up Resources (Optional)**

Step 1: *Log in to Azure Cloud Shell*

The Azure Cloud Shell provides a browser-based terminal that comes pre-configured with the Azure CLI. Using Cloud Shell eliminates the need to install tools locally while giving you full access to Azure management capabilities.

1. **Open** the Azure Portal and **click** the **Cloud Shell** icon (top-right corner).
2. **Select Bash** as your shell environment if prompted.

> **CONCEPT DEEP DIVE**
>
> The Azure Cloud Shell is a Terminal in the Azure Portal that comes pre-configured with the Azure CLI. It enables you to run CLI commands without additional setup.

Step 2: *Create a Resource Group*

Resource groups serve as containers for all resources within Azure. Creating one via CLI demonstrates how automation and scripting can replace manual portal operations, making deployments repeatable and faster.

1. **Run** the following command to create a resource group:

```
az group create --name MyCLIDemoGroup --location
        northeurope
```

   - `--name`: Specifies the name of the resource group (e.g., `MyCLIDemoGroup`).
   - `--location`: Sets the region where the resources in the group will be located (e.g., `northeurope`).

CONCEPT DEEP DIVE

A **Resource Group** acts as a container for all resources within Azure, making it easier to organize, manage, and delete resources as a single unit.

√ **Quick check:** Review the output to confirm successful creation. You should see a JSON response similar to:

```
{
  "id": "/subscriptions/<subscription-id>/resource-
        Groups/MyCLIDemoGroup",
  "location": "eastus",
  "name": "MyCLIDemoGroup",
  "properties": {
    "provisioningState": "Succeeded"
  }
}
```

In the Azure Portal under **Resource groups** you should also see it listed. You can also list all resource groups in your subscription by running the command:

```
az group list --output table
```

Step 3: *Clean Up Resources (Optional)*

Deleting unused resource groups prevents unnecessary charges and keeps your subscription organized. The CLI makes cleanup quick and can be incorporated into automated workflows.

1. **Delete** the resource group to avoid unnecessary costs:

```
az group delete --name MyCLIDemoGroup --no-wait --yes
```

- `--no-wait`: Ensures the command returns immediately without waiting for the operation to complete.
- `--yes`: Confirms the deletion without additional prompts.

2.1.4 Common Issues

> ***If you encounter problems:***
> ***Cloud Shell not loading:*** *Refresh the browser or try a different browser. Ensure pop-ups are allowed for portal.azure.com.*
> ***"Subscription not found" error:*** *Run* `az account list` *to verify your subscription is active and selected.*

2.1.5 Summary

You've successfully created and managed a resource group using Azure CLI which:

- √ Demonstrated command-line resource management
- √ Introduced you to Azure CLI syntax and commands
- √ Showed how to verify resources from the terminal

> **KEY TAKEAWAY:**
> The Azure CLI enables scriptable, repeatable infrastructure management that forms the foundation for automation and Infrastructure as Code practices.

2.1.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Create multiple resource groups in different regions using a single script*
> - *Explore* `az group list` *with different output formats (json, yaml, tsv)*
> - *Use* `az configure` *to set default values for location and resource group*

2.1.7 Done 🎉

You have successfully created and managed a resource group using Azure CLI, gaining hands-on experience with command-line operations in Azure.

2.2 5. Provisioning a VM Using AZ CLI, Configuring Nginx, and Allowing HTTP Traffic

2.2.1 Goal

Provision a Virtual Machine using Azure CLI, install Nginx web server, and configure network security to allow HTTP traffic.

> **WHAT YOU'LL LEARN:**
>
> - How to create and manage Azure resources using the Azure CLI
> - How to generate and use SSH keys for secure VM authentication
> - How to configure Azure Network Security Groups to control traffic

2.2.2 Prerequisites

> **BEFORE STARTING, ENSURE YOU HAVE:**
>
> - ✓ Azure CLI installed and configured
> - ✓ VSCode with integrated terminal (Git Bash for Windows users)
> - ✓ An active Azure subscription

2.2.3 Exercise Steps

Overview

1. **Set up the environment**
2. **Create a resource group**
3. **Create a VM**
4. **Log in to the VM**
5. **Install Nginx**
6. **Open port 80 for HTTP traffic**
7. **Clean up resources (optional)**

Step 1: *Set Up the Environment*

Before creating Azure resources, you need to ensure your local environment is properly configured. This includes verifying your Azure CLI installation and authenticating with your Azure account to establish the connection between your terminal and Azure's management plane.

1. **Open** VSCode and launch the **Integrated Terminal**:
    - On Windows, use **Git Bash** inside VSCode.
    - On Mac or Linux, use the default terminal.
2. **Verify** that Azure CLI is installed by running:

```
az --version
```

- If Azure CLI is not installed, follow the official installation guide.

3. **Log in** to your Azure account:

```
az login
```

- Follow the on-screen instructions to complete the authentication.

Step 2: *Create a Resource Group*

Resource groups serve as logical containers for all related Azure resources. By creating a dedicated resource group for this exercise, you can easily manage, monitor, and delete all associated resources together when you're finished.

1. **Set** variables in the terminal session for the resource group and VM name:

```
resource_group="MyCLIVMGroup"
vm_name="MyCLIVM"
```

2. **Create** the resource group in the `northeurope` location:

```
az group create --name $resource_group --location
        northeurope
```

> **QUICK CHECK:**
> Review the output to confirm successful creation. You should see a JSON response similar to:

```
{
  "id": "/subscriptions/<subscription-id>/resourceGroups/
        MyCLIDemoGroup",
  "location": "eastus",
  "name": "MyCLIVMGroup",
  "properties": {
    "provisioningState": "Succeeded"
  }
}
```

Step 3: *Create a VM*

Creating a VM through Azure CLI gives you precise control over the machine's configuration. The command below provisions a cost-effective B1s instance with Ubuntu 24.04 and automatically configures SSH key-based authentication, which is more secure than password-based access.

1. **Create** a VM using the default SSH keys:

```
az vm create \
   --resource-group $resource_group \
   --location northeurope \
   --name $vm_name \
   --image Ubuntu2404 \
   --size Standard_B1s \
   --admin-username azureuser \
   --generate-ssh-keys
```

CONCEPT DEEP DIVE

This command:

- Creates a VM named `MyCLIVM`.
- Uses the latest Ubuntu 22.04 image.
- Generates SSH keys if not already available in `~/.ssh/id_rsa`.

If you don't have default SSH keys (`~/.ssh/id_rsa`), Azure CLI will generate them automatically. You can also create them manually using:

```
ssh-keygen -t rsa -b 2048
```

> *This generates:*
>
> - `~/.ssh/id_rsa` *(private key)*
> - `~/.ssh/id_rsa.pub` *(public key)*
>
> √ ***Quick check:*** *Review the output to confirm successful creation. You should see a JSON response similar to:*

```
{
  "fqdns": "",
  "id": "/subscriptions/
        ca0a7799-8e2e-4237-8616-8cc0e947ecd5/resource-
        Groups/MyCLIVMGroup/providers/Microsoft.Compute/
        virtualMachines/MyCLIVM",
  "location": "northeurope",
  "macAddress": "60-45-BD-DD-A5-5F",
  "powerState": "VM running",
  "privateIpAddress": "10.0.0.4",
  "publicIpAddress": "13.74.100.155",
  "resourceGroup": "MyCLIVMGroup",
  "zones": ""
}
```

Step 4: *Log in to the VM*

SSH (Secure Shell) provides encrypted communication between your local machine and the VM. By retrieving the public IP address and connecting via SSH, you establish a secure remote session that allows you to configure the server directly.

1. **Retrieve** the public IP address of the VM:

```
vm_ip=$(az vm show --resource-group $resource_group --
        name $vm_name --show-details --query publicIps
        -o tsv)
echo $vm_ip
```

2. **Log in** to the VM using your default SSH key:

```
ssh azureuser@$vm_ip
```

Step 5: *Install Nginx*

Nginx is a high-performance web server commonly used to serve static content and act as a reverse proxy. Installing it on your VM transforms the machine from a basic compute instance into a functional web server capable of responding to HTTP requests.

1. **Update** the system and install Nginx:

```
sudo apt update
sudo apt install nginx -y
```

> **QUICK CHECK:**

1. Run the command `curl localhost`. Ensure you get some HTML code back. This means that it runs correctly regardless of firewall settings.

2. Open a browser and go to the `<VM_Public_IP>`. You should **NOT** see the Nginx default page. This is because we haven't opened the firewall on port 80 yet.

> **CONCEPT DEEP DIVE**
>
> Azure VMs block HTTP traffic on port 80 by default for security reasons. This defense-in-depth approach means you need to explicitly allow HTTP traffic through the Network Security Group before external users can access your web server.

Step 6: *Open Port 80 for HTTP Traffic*

Network Security Groups (NSGs) act as virtual firewalls for your Azure resources. By default, Azure blocks incoming traffic on common service ports. Opening port 80 creates an inbound security rule that permits HTTP traffic to reach your Nginx server.

1. **Exit** back to your laptop in the Terminal:

```
exit
```

2. **Allow** traffic on port 80:

```
az vm open-port --resource-group $resource_group --name
        $vm_name --port 80
```

> **QUICK CHECK:**
>
> Verify the rule is applied:

```
az network nsg rule list --resource-group $resource_group
        --nsg-name ${vm_name}NSG --output table
```

- Ensure a rule exists for port 80 allowing inbound traffic.

> *Return to the browser and refresh the page. Confirm the default Nginx page loads in the browser, indicating port 80 is open and traffic is allowed.*

Step 7: *Clean Up Resources (Optional)*

Deleting unused resources prevents unexpected charges from accumulating on your Azure subscription. Since we placed all resources in a single resource group, you can remove everything with a single command.

1. **Delete** the resource group to avoid unnecessary costs:

```
az group delete --name $resource_group --no-wait --yes
```

COMMON MISTAKES

- Forgetting to delete test resources leads to ongoing charges
- The `--no-wait` flag returns immediately while deletion continues in background

2.2.4 Common Issues

> ***If you encounter problems:***
> ***SSH connection refused:*** *Verify the VM is running and you're using the correct public IP address*
> ***Nginx page not loading:*** *Ensure port 80 is open in the NSG and Nginx service is running*
> ***Azure CLI not authenticated:*** *Run* `az login` *again if your session has expired*

2.2.5 Summary

You've successfully provisioned a VM using Azure CLI which:

- √ Created a resource group to organize Azure resources
- √ Deployed an Ubuntu VM with SSH key authentication
- √ Installed and configured Nginx web server
- √ Modified Network Security Group rules to allow HTTP traffic

> **KEY TAKEAWAY:**
> Azure CLI enables scriptable, repeatable infrastructure provisioning that forms the foundation for infrastructure automation and Infrastructure as Code practices.

2.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Create a bash script that automates the entire VM provisioning process*
> - *Configure HTTPS with a self-signed certificate on port 443*
> - *Explore Azure CLI commands for scaling and monitoring your VM*

2.2.7 Done 🎉

You have successfully provisioned a VM using Azure CLI, logged in with SSH, installed and tested Nginx, and configured port 80 for HTTP traffic. This exercise demonstrates the fundamental workflow for command-line infrastructure management in Azure.

2.3 6. Automating VM Creation, Nginx Installation, and Port Configuration Using a Bash Script

2.3.1 Goal

Create a Bash script that provides a “one-click” solution for deploying an Azure VM with Nginx automatically installed and configured.

WHAT YOU’LL LEARN:

- How to create reusable automation scripts for Azure resource provisioning
- How to use custom data files to configure VMs during creation
- How to combine multiple Azure CLI commands into an automated workflow

2.3.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- Completed the previous exercises in the server foundation track
- Azure CLI installed and configured with your subscription
- A code editor (VS Code recommended) for creating script files

2.3.3 Exercise Steps

Overview

1. **Prepare the custom data file** - Create the cloud-init script for Nginx installation
2. **Create the automation script** - Write the Bash script that orchestrates all Azure commands
3. **Execute the automation script** - Run your one-click deployment
4. **Clean up resources** - Remove Azure resources when finished

Step 1: *Prepare the Custom Data File*

Custom data files allow you to run scripts automatically when a VM boots for the first time. This eliminates manual SSH access and configuration, making your deployments truly automated and repeatable.

1. **Create** a file named `custom_data_nginx.sh` (you can use VS Code)
2. **Add** the following content to the file:

```
#!/bin/bash
apt update
apt install nginx -y
```

CONCEPT DEEP DIVE

The custom data script runs during VM provisioning with root privileges. It:

- Updates package lists to ensure latest versions
- Installs Nginx with the `-y` flag to auto-confirm installation

This approach is known as "cloud-init" and is the standard way to bootstrap cloud VMs.

Step 2: *Create the Automation Script*

The automation script combines all the individual Azure CLI commands you've learned into a single executable file. This transforms a multi-step manual process into a repeatable, error-free deployment that anyone on your team can run.

1. **Create** a Bash script named `provision_vm.sh`
2. **Add** the following content to the script:

```bash
#!/bin/bash

# Variables
resource_group="MyOneClickGroup"
vm_name="MyOneClickVM"
location="northeurope"
custom_data_file="custom_data_nginx.sh"

# Create a resource group
echo "Creating resource group: $resource_group..."
az group create --name $resource_group --location $loca-
        tion

# Create a virtual machine with custom data
echo "Creating virtual machine: $vm_name..."
az vm create \
   --resource-group $resource_group \
   --location $location \
   --name $vm_name \
   --image Ubuntu2404 \
   --size Standard_B1s \
   --admin-username azureuser \
   --generate-ssh-keys \
   --custom-data @$custom_data_file

# Open port 80 for HTTP traffic
echo "Opening port 80 for HTTP traffic..."
az vm open-port --resource-group $resource_group --name
        $vm_name --port 80

# Retrieve public IP address of the VM
vm_ip=$(az vm show --resource-group $resource_group --
        name $vm_name --show-details --query publicIps
        -o tsv)

echo "Deployment complete! Access your server at http://
        $vm_ip"
```

3. **Make** the script executable:

```
chmod +x provision_vm.sh
```

CONCEPT DEEP DIVE

The script demonstrates several important automation patterns:

- **Variables at the top** make the script easy to customize
- **Echo statements** provide progress feedback during execution
- **The `@` prefix** tells Azure CLI to read the custom data from a file
- **Command substitution** (`$()`) captures the output of one command for use in another

Common Mistakes

- Forgetting to make the script executable with `chmod +x`
- Not having the custom data file in the same directory as the script
- Using spaces around the `=` in variable assignments (Bash doesn't allow this)

Step 3: *Execute the Automation Script*

Running the script will execute all Azure CLI commands in sequence, creating your complete infrastructure with a single command. This is the "one-click" deployment in action.

1. **Run** the script:

```
./provision_vm.sh
```

CONCEPT DEEP DIVE

The script will:

- Create a resource group
- Provision a VM with the custom data script to install and configure Nginx
- Open port 80 for HTTP traffic
- Output the public IP address of the VM

Quick check: Open a browser and navigate to `http://<VM_Public_IP>`. You should see the default Nginx page.

Step 4: *Clean Up Resources (Optional)*

Cleaning up resources prevents unnecessary charges and ensures a tidy Azure environment. This step demonstrates how to tear down everything created by your automation script.

1. **Delete** the resource group if you no longer need the resources:

```
az group delete --name MyOneClickGroup --no-wait --yes
```

CONCEPT DEEP DIVE

The `--no-wait` flag returns control immediately while deletion continues in the background. The `--yes` flag skips the confirmation prompt, making this suitable for automated cleanup scripts.

It might take a minute or two before the resource group is completely deleted.

2.3.4 Common Issues

> ***If you encounter problems:***
> ***"Permission denied" when running the script:*** *Ensure you ran* `chmod +x provision_vm.sh` *to make the script executable.*
> ***"File not found" for custom data:*** *Make sure* `custom_data_nginx.sh` *is in the same directory where you run* `provision_vm.sh`*.*
> ***Nginx page doesn't load:*** *Wait 2-3 minutes for cloud-init to complete. You can SSH into the VM and check* `/var/log/cloud-init-output.log` *for progress.*

2.3.5 Summary

You've successfully created a one-click automation solution which:

- Provisions all Azure resources with a single command
- Automatically installs and configures Nginx using cloud-init
- Demonstrates the power of combining Azure CLI commands into reusable scripts

> **KEY TAKEAWAY:**
> Bash scripts transform manual, error-prone processes into repeatable, shareable automation. This is the foundation of Infrastructure as Code (IaC) and is essential for professional DevOps workflows.

2.3.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add error handling to your script using* `set -e` *to exit on any command failure*
> - *Parameterize the script to accept resource group name and location as command-line arguments*
> - *Add a custom HTML page to Nginx by extending the custom data script*
> - *Create a corresponding cleanup script that deletes resources when you're done*

2.3.7 Done 🎉

You have successfully created a "one-click" solution to deploy an Azure VM, configure Nginx, and enable HTTP traffic using Bash scripting and custom data. This automation approach will serve as the foundation for more advanced Infrastructure as Code techniques in upcoming exercises.

EXERCISE 3

ARM and Bicep

3.1 7. Exporting a Template from Azure Portal

Goal: See your week-one Ubuntu VM and its network as **Infrastructure as Code** by exporting the deployment to an **ARM template**. This will give you a concrete reference before we author templates ourselves.

Estimated time: 15–25 minutes

3.1.1 Learning outcomes

- Understand how Azure resources are represented in **ARM**.
- Identify **parameters**, **variables**, **resources**, and **dependsOn**.
- Map resource types to our diagram: VNet/Subnet, NSG, Public IP, NIC, Ubuntu VM.

3.1.2 Prerequisites

- You already created the VM in the **Azure Portal** (from Exercise 1/2/3/5/6).
- Access to the **Resource Group** that holds the VM and its networking.

3.1.3 What you'll produce

- A downloaded ZIP containing `template.json` and `parameters.json` (generated by the portal).
- A short note with the resource types you found and how they relate.

3.1.4 Steps

1. **Open your Resource Group**
   - Azure Portal → *Resource groups* → select the RG that contains your VM, VNet, NSG, Public IP, NIC.
2. **Export the template**
   - In the RG, look for **Export template** (sometimes under *Settings* or *Automation*).
   - Click **Export template** → **Download** (ZIP).
3. **Unpack & open in VS Code**
   - Unzip locally. Open the folder in **VS Code**.
   - Files you should see: `template.json`, `parameters.json`, and a README snippet.
   - (Optional) Install *Azure Resource Manager (ARM) Tools* extension for JSON schema help.
4. **Read the template (guided tour)**
   - **parameters** → values the template expects (e.g., location, admin username).

- **variables** → name building or convenience values.
- **resources** → look for these types and **write them down**:
  - `Microsoft.Network/virtualNetworks`
  - `Microsoft.Network/networkSecurityGroups`
  - `Microsoft.Network/publicIPAddresses`
  - `Microsoft.Network/networkInterfaces`
  - `Microsoft.Compute/virtualMachines`
- **dependsOn** → note how NIC depends on Public IP/NSG/VNet, and how VM depends on NIC.
- **outputs** → check if any are present (often none in exports).

5. **Map to the diagram (quick check)**
   - Internet → **Public IP** → **NIC/NSG** → **VM (Nginx via cloud-init)**.
   - Confirm each hop exists in `resources` and note the names.
6. **Notes on exports (know the limits)**
   - Exports are **best-effort** and may not reflect all features or your naming conventions.
   - Parameterization is basic; we'll improve this with **Bicep** in the next exercise.

3.1.5 Verification

Create a short note (5–8 bullets) answering: - Which **resource types** did you find? - What does the **VM** depend on? - Which **NSG rules** exist for SSH (22) and HTTP (80)? - Where is the **location** set (parameter or literal)? - Anything **surprising** or unclear in the export?

3.1.6 Stretch (optional)

- If you have Bicep installed, try:

```
az bicep decompile --file template.json
```

  Open the generated `template.bicep` and compare readability.

3.1.7 Cleanup

None. Keep the exported template for reference; we'll author a minimal template next.

3.2 8. Minimal ARM Template: Ubuntu VM + Network

Goal: Author the *smallest viable* **ARM template** that deploys an **Ubuntu VM** with a minimal network (VNet/Subnet, NSG, Public IP, NIC). Preview with `what-if`, deploy, and capture the public IP as an output.

Estimated time: 40–60 minutes

3.2.1 Learning outcomes

- Structure an ARM template: **parameters**, **variables**, **resources**, **outputs**.
- Deploy to a **resource group** with `az deployment group` and use **what-if**.
- Wire up **NSG → NIC → VM** and output the **Public IP**.

3.2.2 Prerequisites

- Azure CLI, VS Code.
- A resource group (or create one).

- Your **SSH public key** (`~/.ssh/id_rsa.pub` or `id_ed25519.pub`).

3.2.3 What you'll produce

- `main.json` — minimal template.
- `parameters.dev.json` — parameter file for your environment.
- A successful deployment and the **public IP** printed as an **output**.

3.2.4 Steps

1. **Scaffold files** Create a new folder and two files: `main.json` and `parameters.dev.json`.

main.json (paste & save):

```
{
  "$schema": "https://schema.management.azure.com/schemas/
        2019-04-01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "location": {"type": "string", "defaultValue":
        "[resourceGroup().location]"},
    "namePrefix": {"type": "string", "defaultValue":
        "demo"},
    "adminUsername": {"type": "string", "defaultValue":
        "azureuser"},
    "adminPublicKey": {"type": "string"},
    "addressSpace": {"type": "string", "defaultValue":
        "10.10.0.0/16"},
    "subnetPrefix": {"type": "string", "defaultValue":
        "10.10.1.0/24"}
  },
  "variables": {
    "vnetName": "[format('{0}-vnet', paramet-
        ers('namePrefix'))]",
    "subnetName": "default",
    "nsgName": "[format('{0}-nsg', paramet-
        ers('namePrefix'))]",
    "pipName": "[format('{0}-ip', paramet-
        ers('namePrefix'))]",
    "nicName": "[format('{0}-nic', paramet-
        ers('namePrefix'))]",
    "vmName": "[format('{0}-vm', paramet-
        ers('namePrefix'))]"
  },
  "resources": [

    {
      "type": "Microsoft.Network/virtualNetworks",
      "apiVersion": "2024-07-01",
      "name": "[variables('vnetName')]",
      "location": "[parameters('location')]",
      "properties": {
        "addressSpace": {"addressPrefixes": ["[paramet-
        ers('addressSpace')]"]},
        "subnets": [
          {
            "name": "[variables('subnetName')]",
            "properties": {"addressPrefix": "[paramet-
        ers('subnetPrefix')]" }
          }
```

```
        ]
      }
    },

    {
      "type": "Microsoft.Network/networkSecurityGroups",
      "apiVersion": "2024-07-01",
      "name": "[variables('nsgName')]",
      "location": "[parameters('location')]",
      "properties": {
        "securityRules": [
          {
            "name": "SSH",
            "properties": {
              "protocol": "TCP",
              "sourcePortRange": "*",
              "destinationPortRange": "22",
              "sourceAddressPrefix": "*",
              "destinationAddressPrefix": "*",
              "access": "Allow",
              "priority": 300,
              "direction": "Inbound"
            }
          },
          {
            "name": "HTTP",
            "properties": {
              "protocol": "TCP",
              "sourcePortRange": "*",
              "destinationPortRange": "80",
              "sourceAddressPrefix": "*",
              "destinationAddressPrefix": "*",
              "access": "Allow",
              "priority": 320,
              "direction": "Inbound"
            }
          }
        ]
      }
    },

    {
```

```
      "type": "Microsoft.Network/publicIPAddresses",
      "apiVersion": "2024-07-01",
      "name": "[variables('pipName')]",
      "location": "[parameters('location')]",
      "sku": {"name": "Standard", "tier": "Regional"},
      "properties": {
        "publicIPAddressVersion": "IPv4",
        "publicIPAllocationMethod": "Static"
      }
    },

    {
      "type": "Microsoft.Network/networkInterfaces",
      "apiVersion": "2024-07-01",
      "name": "[variables('nicName')]",
      "location": "[parameters('location')]",
      "dependsOn": [

        "[resourceId('Microsoft.Network/publicIPAd-
        dresses', variables('pipName'))]",
        "[resourceId('Microsoft.Network/virtualNetworks',
        variables('vnetName'))]",
        "[resourceId('Microsoft.Network/networkSecurity-
        Groups', variables('nsgName'))]"
      ],
      "properties": {
        "ipConfigurations": [
          {
            "name": "ipconfig1",
            "properties": {
              "subnet": {"id":
        "[resourceId('Microsoft.Network/virtualNetworks/
        subnets', variables('vnetName'), vari-
        ables('subnetName'))]"},
              "publicIPAddress": {"id":
        "[resourceId('Microsoft.Network/publicIPAd-
        dresses', variables('pipName'))]"},
              "privateIPAllocationMethod": "Dynamic",
              "primary": true
            }
          }
        ],
        "networkSecurityGroup": {"id":
        "[resourceId('Microsoft.Network/networkSecurity-
        Groups', variables('nsgName'))]"}
```

```
      }
    },

    {
      "type": "Microsoft.Compute/virtualMachines",
      "apiVersion": "2024-11-01",
      "name": "[variables('vmName')]",
      "location": "[parameters('location')]",
      "dependsOn": [

        "[resourceId('Microsoft.Network/networkInter-
        faces', variables('nicName'))]"
      ],
      "properties": {
        "hardwareProfile": {"vmSize": "Standard_B1s"},
        "storageProfile": {
          "imageReference": {
            "publisher": "canonical",
            "offer": "ubuntu-24_04-lts",
            "sku": "server",
            "version": "latest"
          },
          "osDisk": {
            "createOption": "FromImage",
            "managedDisk": {"storageAccountType":
        "Premium_LRS"},
            "diskSizeGB": 30
          }
        },
        "osProfile": {
          "computerName": "[variables('vmName')]",
          "adminUsername": "[parameters('adminUsername')]",
          "linuxConfiguration": {
            "disablePasswordAuthentication": true,
            "ssh": {
              "publicKeys": [
                {
                  "path": "[format('/home/{0}/.ssh/author-
        ized_keys', parameters('adminUsername'))]",
                  "keyData": "[paramet-
        ers('adminPublicKey')]"
                }
              ]
            }
```

```
          }
        },
        "networkProfile": {
          "networkInterfaces": [{ "id":
        "[resourceId('Microsoft.Network/networkInter-
        faces', variables('nicName'))]" }]
        }
      }
    }
  ],
  "outputs": {
    "publicIpAddress": {
      "type": "string",
      "value": "[reference(resourceId('Microsoft.Network/
        publicIPAddresses', variables('pipName')),
        '2024-07-01').ipAddress]"
    }
  }
}
```

parameters.dev.json (paste & adjust):

```
{
  "$schema": "https://schema.management.azure.com/schemas/
        2015-01-01/deploymentParameters.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "namePrefix": {"value": "demo"},
    "adminUsername": {"value": "azureuser"},
    "adminPublicKey": {"value": "<paste your SSH public
        key>"}
  }
}
```

1. **Deploy**

```
# (optional) create an RG
az group create -n rg-arm-demo -l northeurope

# preview
az deployment group what-if -g rg-arm-demo -f main.json -p
        @parameters.dev.json

# deploy
az deployment group create -g rg-arm-demo -f main.json -p
        @parameters.dev.json
```

1. **Get the public IP (output)**

```
az deployment group show -g rg-arm-demo -n main --query
        properties.outputs.publicIpAddress.value -o tsv
```

1. **Verify**

```
ssh azureuser@<public-ip>   # should prompt to trust the
        host (no password)
```

3.2.5 Verification

- Paste your **public IP** output.
- Show your **what-if** summary (brief snippet or screenshot).
- Short note: which resource **dependsOn** which?

3.2.6 Stretch (optional)

- **Add cloud-init** (Nginx home page): add to `osProfile` → `customData` with base64 of a small script file that installs Nginx.
- Add your default public SSH key programatically as a AZ CLI parameter.

3.2.7 Cleanup

```
az group delete -n rg-arm-demo --yes --no-wait
```

3.3 9. From ARM to Bicep: Ubuntu VM + Network

Goal: Rebuild **Exercise 8** using **Bicep** instead of ARM JSON. Produce a clean `main.bicep` and a `.bicepparam` file, preview with `what-if`, then deploy.

Estimated time: 40–60 minutes

3.3.1 Learning outcomes

- Author resources in **Bicep** and understand how it compiles to ARM.
- Use **.bicepparam** files for environment-specific values.
- Output the **Public IP** and verify SSH/Nginx like in Exercise 8.

3.3.2 Prerequisites

- Exercise 8 completed (or read through).
- **Azure CLI** + **Bicep** installed (`az bicep install` or standalone).
- SSH public key available.

3.3.3 What you'll produce

- `main.bicep` — minimal VM + network in Bicep.
- `dev.bicepparam` — parameters for your environment.
- A successful deployment and the **public IP** printed as an output.

3.3.4 Steps

1. **Create a bicep template**

   Create `main.bicep` and paste:

```
param location string = resourceGroup().location
param namePrefix string = 'demo'
param adminUsername string = 'azureuser'
param adminPublicKey string
param addressSpace string = '10.10.0.0/16'
param subnetPrefix string = '10.10.1.0/24'

var vnetName = '${namePrefix}-vnet'
var nsgName  = '${namePrefix}-nsg'
var pipName  = '${namePrefix}-ip'
var nicName  = '${namePrefix}-nic'
var vmName   = '${namePrefix}-vm'
var subnetName = 'default'

resource vnet 'Microsoft.Network/virtualNet-
works@2024-07-01' = {
  name: vnetName
  location: location
  properties: {
    addressSpace: { addressPrefixes: [ addressSpace ] }
    subnets: [{ name: subnetName, properties: { addressPre-
fix: subnetPrefix } }]
  }
}

resource nsg 'Microsoft.Network/
networkSecurityGroups@2024-07-01' = {
  name: nsgName
  location: location
  properties: {
    securityRules: [
      {
        name: 'SSH'
        properties: {
          protocol: 'TCP'
          sourcePortRange: '*'
          destinationPortRange: '22'
          sourceAddressPrefix: '*'
          destinationAddressPrefix: '*'
          access: 'Allow'
          priority: 300
```

```
          direction: 'Inbound'
        }
      }
      {
        name: 'HTTP'
        properties: {
          protocol: 'TCP'
          sourcePortRange: '*'
          destinationPortRange: '80'
          sourceAddressPrefix: '*'
          destinationAddressPrefix: '*'
          access: 'Allow'
          priority: 320
          direction: 'Inbound'
        }
      }
    ]
  }
}

resource pip 'Microsoft.Network/publicIPAd-
dresses@2024-07-01' = {
  name: pipName
  location: location
  sku: { name: 'Standard', tier: 'Regional' }
  properties: {
    publicIPAddressVersion: 'IPv4'
    publicIPAllocationMethod: 'Static'
  }
}

resource nic 'Microsoft.Network/networkInter-
faces@2024-07-01' = {
  name: nicName
  location: location
  properties: {
    ipConfigurations: [{
      name: 'ipconfig1'
      properties: {
        subnet: { id: resourceId('Microsoft.Network/virtu-
alNetworks/subnets', vnet.name, subnetName) }
        publicIPAddress: { id: pip.id }
```

```
        privateIPAllocationMethod: 'Dynamic'
        primary: true
      }
    }]
    networkSecurityGroup: { id: nsg.id }
  }
}

resource vm 'Microsoft.Compute/virtualMachines@2024-11-01'
= {
  name: vmName
  location: location
  properties: {
    hardwareProfile: { vmSize: 'Standard_B1s' }
    storageProfile: {
      imageReference: {
        publisher: 'canonical'
        offer: 'ubuntu-24_04-lts'
        sku: 'server'
        version: 'latest'
      }
      osDisk: {
        createOption: 'FromImage'
        managedDisk: { storageAccountType: 'Premium_LRS' }
        diskSizeGB: 30
      }
    }
    osProfile: {
      computerName: vmName
      adminUsername: adminUsername
      linuxConfiguration: {
        disablePasswordAuthentication: true
        ssh: {
          publicKeys: [{ path: '/home/${adminUser-
name}/.ssh/authorized_keys', keyData: adminPublicKey }]
        }
      }
    }
    networkProfile: {
      networkInterfaces: [{ id: nic.id }]
    }
  }
```

```
}

output publicIpAddress string = pip.properties.ipAddress
```

1. **Create `dev.bicepparam`**

```
using './main.bicep'

param namePrefix = 'demo'
param adminUsername = 'azureuser'
param adminPublicKey = '<paste your SSH public key>'
```

1. **Preview & deploy**

```
# (optional) create an RG
az group create -n rg-bicep-demo -l northeurope

# preview
az deployment group what-if -g rg-bicep-demo -f main.bicep
        -p dev.bicepparam

# deploy
az deployment group create -g rg-bicep-demo -f main.bicep
        -p dev.bicepparam
```

1. **Get the public IP (output)**

```
az deployment group show -g rg-bicep-demo -n main --query
        properties.outputs.publicIpAddress.value -o tsv
```

1. **Verify**

```
ssh azureuser@<public-ip>
```

3.3.5 Verification

- Paste your **public IP** output.

- Show your **what-if** summary.
- Short note: what changed moving from **ARM → Bicep** (readability, params, outputs).

3.3.6 Cleanup

```
az group delete -n rg-bicep-demo --yes --no-wait
```

3.4 10. Nginx via Cloud-init (bash) and Custom Script Extension

Goal: Install **Nginx** on an Ubuntu VM using two approaches and compare them:

1. **Cloud-init (`customData`)** — runs **at VM creation**, script as a separate **bash** file
2. **Custom Script extension** — can run **on an existing VM**, script as a separate **bash** file

Estimated time: 45–60 minutes

3.4.1 Learning outcomes

- Pass a **bash** script to cloud-init via `customData` and `loadTextContent()`.
- Use the **Custom Script** extension to execute a **bash** script on an existing VM.
- Know when to choose **cloud-init** vs **extension**.

3.4.2 Prerequisites

- Exercise 9 (Bicep) or similar `main.bicep` + `dev.bicep-param` in place.
- Azure CLI and Bicep installed.

- An Ubuntu VM template (from earlier exercises).

3.4.3 What you'll produce

- `cloud-init.sh` — bash script for cloud-init.
- `install-nginx.sh` — bash script for the Custom Script extension.
- Updated `main.bicep` and `dev.bicepparam` that wire both methods.

3.4.4 Part A — Cloud-init (runs on new VM creation)

1. **Create `cloud-init_nginx.sh`** (keep it minimal; no HTML changes):

```
#!/bin/bash
apt-get update
apt-get install -y nginx
```

1. **Update `main.bicep`** — add parameter and pass it to `osProfile.customData`:

```
@description('User-data passed to cloud-init; start the
script with #!/bin/bash')
param cloudInitContent string = ''

// in the VM resource:
osProfile: {
  computerName: vmName
  adminUsername: adminUsername
  customData: base64(cloudInitContent)
  linuxConfiguration: {
    disablePasswordAuthentication: true
    ssh: {
      publicKeys: [{ path: '/home/${adminUsername}/.ssh/
authorized_keys', keyData: adminPublicKey }]
    }
  }
}
```

1. **Update `dev.bicepparam`** — load the bash script from file:

```
param cloudInitContent = loadTextContent('./cloud-
init_nginx.sh')
```

1. **Deploy (requires a** new** VM) **Cloud-init is only read during** creation**:

```
az deployment group create -g rg-bicep-demo2 -f main.bicep
        -p dev.bicepparam adminPublicKey=@~/.ssh/
        id_rsa.pub

az deployment group show -g rg-bicep-demo2 -n main --query
        properties.outputs.publicIpAddress.value -o tsv
```

3.4.5 Verification

- Browse to your **public IP** and verify Nginx welcome page

3.4.6 Part B — Custom Script Extension (works on existing VM)

1. **Create `cse_deploy_webpage.sh`** (sh; minimal):

```
#!/bin/sh
set -eu # Exit on error, treat unset variables as an error

# Create the web root even if nginx hasn't created it yet
install -d -m 0755 /var/www/html

# Write the web page
cat >/var/www/html/index.html <<'HTML'
<!doctype html>
<html lang="en">
<head><meta charset="utf-8"><title>Azure VM — Nginx</
        title></head>
<body><h1>It works 🎉</h1><p>Deployed via Azure
        <strong>Custom Script Extension</strong>.</p></
        body>
</html>
HTML
```

1. **Add parameter in `dev.bicepparam`** for script content:

```
param customScriptContent = loadTextContent('./cse_de-
ploy_webpage.sh')
```

1. **Add extension resource to `main.bicep`** (after VM resource):

```
@description('Custom Script content to execute on the VM
(bash)')
param customScriptContent string = ''

resource vmExt 'Microsoft.Compute/virtualMachines/exten-
sions@2024-07-01' = {
  name: 'deploy-webpage'
  parent: vm
  location: location
  properties: {
    publisher: 'Microsoft.Azure.Extensions'
    type: 'CustomScript'
    typeHandlerVersion: '2.1'
    autoUpgradeMinorVersion: true
    settings: {
      // Inline script executed by /bin/sh (extension
decodes Base64 and runs it as script.sh)
      script: base64(customScriptContent)
    }
  }
}
```

1. **Deploy (can be applied to an existing VM)** This will add/update only the extension:

```
az deployment group what-if -g rg-bicep-demo2 -f
        main.bicep -p dev.bicepparam adminPub-
        licKey=@~/.ssh/id_rsa.pub
az deployment group create -g rg-bicep-demo2 -f main.bicep
        -p dev.bicepparam adminPublicKey=@~/.ssh/
        id_rsa.pub
```

3.4.7 Verify

```
curl <public-ip>
```

You can decode the string from what-if like this

```
# print to screen
printf '%s' 'IyEvYmluL3NoCns...Cg==' | base64 --decode

# write to a file (recommended)
printf '%s' 'IyEvYmluL3NoCns...Cg==' | base64 --decode >
        script.sh
```

3.4.8 When to use which?

- **Cloud-init (`customData`)** → base bootstrap on first boot; immutable after creation.
- **Custom Script extension** → make changes post-creation; great for iterative tweaks.

3.4.9 Cleanup

Delete test resources if created for this lab:

```
az group delete -n rg-bicep-demo2 --yes --no-wait
```

PART II

Network Foundation

Learn the fundamentals of Azure networking through progressive exercises, from visual portal interface to command-line automation.

3.5 Learning Path

This track guides you through virtual network provisioning and security configuration using two complementary approaches:

1. ***Portal Interface** - Graphical approach for understanding Azure network resources and security groups visually*

2. ***Command Line Interface*** *- Scriptable approach for network automation and repeatability*

EXERCISE 4

Portal Interface

4.1 1. Creating a Virtual Network

4.1.1 Goal

Build a secure cloud network infrastructure with three specialized servers to demonstrate network segmentation, security groups, and the principle of least privilege in Azure.

WHAT YOU'LL LEARN:

- How to create and configure Azure Virtual Networks and subnets
- When to use Network Security Groups for firewall rules
- Best practices for server role separation and secure access patterns
- How reverse proxies and bastion hosts protect internal infrastructure

4.1.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Active Azure subscription with resource creation permissions
- ✓ Basic understanding of IP addresses and CIDR notation
- ✓ SSH client installed on your local machine
- ✓ Familiarity with Linux command line basics

4.1.3 Exercise Steps

Overview

1. **Create the Virtual Network Foundation**
2. **Deploy and Configure Web Server**
3. **Deploy and Configure Reverse Proxy**
4. **Deploy Bastion Host**
5. **Secure with Network Security Groups**
6. **Test Your Implementation**

Step 1: *Create the Virtual Network Foundation*

Establish a Virtual Network (VNet) as the foundation for your cloud infrastructure. A VNet provides isolated network space in Azure where resources can communicate securely, similar to a traditional network in a data center but with cloud scalability and flexibility.

1. **Navigate to** the Azure Portal at https://portal.azure.com
2. **Sign in** with your Azure account credentials
3. **Search for** "Virtual Networks" using the search bar at the top
4. **Select** Virtual Networks from the search results
5. **Click** the **+ Create** button to start the creation wizard
6. **Configure** the Basics tab with the following settings:
   - **Subscription**: Select your subscription
   - **Resource Group**: Create new named `DemoRG`
   - **Name**: Enter `DemoVNet`
   - **Region**: Choose `North Europe` or a region close to you
7. **Click** Review + Create, then **click** Create to deploy the Virtual Network

CONCEPT DEEP DIVE

A Virtual Network (VNet) is the foundation of cloud networking. The default address space `10.0.0.0/16` provides over 65,000 IP addresses, allowing extensive growth. Azure automatically creates a default subnet `10.0.0.0/24` with 256 addresses, which is sufficient for small to medium deployments. Subnets enable logical segmentation within the VNet for security and resource management.

⚠ **Common Mistakes**

- Choosing overlapping address spaces with on-premises networks causes VPN connectivity issues
- Selecting different regions for VNet and VMs increases latency and may incur bandwidth charges
- Forgetting to create a resource group first interrupts the workflow

√ **Quick check:** Navigate to Virtual Networks in the portal and verify `DemoVNet` shows Address Space `10.0.0.0/16` and default subnet `10.0.0.0/24`

[image: Network Overview — not found]

Step 2: *Deploy and Configure Web Server*

Create an Ubuntu VM that will host your web application on a non-standard port. This simulates a typical application server that runs behind a reverse proxy and isn't directly exposed to the internet.

1. **Navigate to** Virtual Machines in the Azure Portal
2. **Click** + Create to start the VM creation wizard
3. **Configure** the Basics tab:
    - **Subscription**: Select the subscription used in Step 1
    - **Resource Group**: Select `DemoRG`

- **Virtual Machine Name**: Enter `WebServer`
- **Region**: Use the same region as the VNet
- **Image**: Choose **Ubuntu Server 24.04 LTS**
- **Size**: Select `Standard_B1s` (cost-effective for learning)
- **Authentication Type**: Select **SSH Public Key**
- **Username**: Enter `azureuser`
- **SSH Public Key**: Choose **Generate new key pair**

4. **Navigate to** the Networking tab
5. **Configure** networking settings:
   - **Virtual Network**: Select `DemoVNet`
   - **Subnet**: Select `default (10.0.0.0/24)`
   - **Public IP**: Ensure a public IP is assigned
   - **NIC Network Security Group**: Select **Basic**
   - **Public Inbound Ports**: Select **Allow SSH (22)**
6. **Navigate to** the Advanced tab
7. **Add** the following custom data to install nginx:

```
#!/bin/bash

apt update
apt install nginx -y
```

8. **Click** Review + create, then **click** Create
9. **Download** the SSH private key when prompted and save it to your Downloads folder
10. **Wait** for the deployment to complete (approximately 2-3 minutes)
11. **Connect** to the WebServer using SSH:

```
chmod 400 ~/Downloads/WebServer_key.pem
ssh -i ~/Downloads/WebServer_key.pem azureuser@<Web-
        Server_PublicIP>
```

12. **Configure** nginx to listen on port 8080 and customize the welcome page:

```
sudo sed -i 's/listen 80 default_server;/listen 8080
        default_server;/' /etc/nginx/sites-available/
        default
sudo sed -i '/listen \[::\]:80 default_server;/d' /etc/
        nginx/sites-available/default
sudo nginx -s reload
sudo sed -i 's/Welcome to nginx/Hello World/g' /var/www/
        html/index.nginx-debian.html
```

13. **Verify** nginx is responding:

```
curl localhost:8080
```

CONCEPT DEEP DIVE

Using custom data (cloud-init) automates software installation during VM provisioning. This Infrastructure-as-Code approach ensures consistent configuration and reduces manual errors. Changing nginx to port 8080 demonstrates that application servers can run on non-standard ports - a common practice when multiple services share a machine or when a reverse proxy handles standard ports.

The `sed` commands perform in-place editing of nginx configuration files. The first two commands change the listening port, while the last command customizes the default page content.

⚠ **Common Mistakes**

- Forgetting to reload nginx after configuration changes means changes won't take effect
- Incorrect SSH key permissions (must be 400 or 600) cause "permissions are too open" errors
- Using the wrong public IP address results in connection timeouts
- Not removing the IPv6 listener can cause port conflicts on systems with dual-stack networking

√ **Quick check:** The `curl localhost:8080` command returns HTML containing "Hello World"

Step 3: *Deploy and Configure Reverse Proxy*

Create a second VM that acts as a reverse proxy, forwarding external HTTP requests to your internal web server. This architecture pattern provides a security boundary and

enables advanced features like load balancing, SSL termination, and caching.

1. **Repeat** the VM creation process from Step 2 with these changes:
   - **Virtual Machine Name**: Enter `ReverseProxy`
   - **All other settings**: Keep identical to WebServer
   - **Custom data**: Use the same nginx installation script
2. **Download** the SSH private key for ReverseProxy
3. **Connect** to the ReverseProxy:

```
chmod 400 ~/Downloads/ReverseProxy_key.pem
ssh -i ~/Downloads/ReverseProxy_key.pem
        azureuser@<ReverseProxy_PublicIP>
```

4. **Test** internal connectivity to the WebServer:

```
curl <WebServer_PrivateIP>:8080
```

5. **Verify** you receive the “Hello World” response
6. **Configure** nginx as a reverse proxy by editing the default site configuration:

```
sudo nano /etc/nginx/sites-available/default
```

7. **Replace** the entire file contents with the following configuration (update the IP and port):

```
server {
  listen        80 default_server;
  location / {
    proxy_pass         http://<Web-
Server_PrivateIP>:8080/;
    proxy_http_version 1.1;
    proxy_set_header   Upgrade $http_upgrade;
    proxy_set_header   Connection keep-alive;
    proxy_set_header   Host $host;
    proxy_cache_bypass $http_upgrade;
    proxy_set_header   X-Forwarded-For $proxy_add_x_for-
warded_for;
    proxy_set_header   X-Forwarded-Proto $scheme;
  }
}
```

8. **Test** the nginx configuration syntax:

```
sudo nginx -t
```

9. **Reload** nginx to apply changes:

```
sudo nginx -s reload
```

10. **Verify** the reverse proxy is working:

```
curl localhost
```

CONCEPT DEEP DIVE

A reverse proxy sits between clients and backend servers, forwarding requests and responses. This architecture provides several benefits: security (backend servers aren't directly exposed), performance (caching, compression), and flexibility (load balancing, SSL termination).

The nginx configuration uses `proxy_pass` to forward requests to the backend. The `proxy_set_header` directives preserve important request information like the original client IP (`X-Forwarded-For`) and protocol (`X-Forwarded-Proto`). The `Connection keep-alive` header enables persistent connections for better performance.

Resources within the same VNet can communicate using private IP addresses without traversing the internet. No firewall exists between VMs in the same VNet by default, enabling seamless internal communication.

⚠ **Common Mistakes**

- Forgetting to update `<WebServer_PrivateIP>` and port causes proxy errors
- Missing semicolons in nginx configuration cause syntax errors
- Not testing configuration before reload can break the service
- Using the public IP instead of private IP routes traffic through the internet unnecessarily

✓ **Quick check:** The `curl localhost` command on ReverseProxy returns the "Hello World" page from WebServer

Step 4: *Deploy Bastion Host*

Create a jump server (bastion host) that provides secure SSH access to internal servers. This implements the principle

of least privilege by reducing the attack surface - only the bastion host is exposed to SSH from the internet.

1. **Create** a third VM with these settings:
   - **Virtual Machine Name**: Enter `BastionHost`
   - **All other settings**: Keep identical to previous VMs
   - **Custom data**: Leave empty (nginx not needed)
2. **Download** the SSH private key for BastionHost
3. **Test** SSH access to the BastionHost:

```
chmod 400 ~/Downloads/BastionHost_key.pem
ssh -i ~/Downloads/BastionHost_key.pem azureuser@<Bas-
        tionHost_PublicIP>
```

4. **Exit** the SSH session to return to your local machine:

```
exit
```

5. **Start** the SSH agent on your local machine:

```
eval $(ssh-agent)
```

6. **Add** all three private keys to the agent:

```
ssh-add ~/Downloads/WebServer_key.pem
ssh-add ~/Downloads/ReverseProxy_key.pem
ssh-add ~/Downloads/BastionHost_key.pem
```

7. **Verify** keys are loaded:

```
ssh-add -l
```

8. **Connect** to BastionHost with agent forwarding enabled:

```
ssh -A azureuser@<BastionHost_PublicIP>
```

9. **From BastionHost**, SSH into WebServer using its private IP:

```
ssh -A azureuser@<WebServer_PrivateIP>
```

CONCEPT DEEP DIVE

A bastion host (jump server) is a security best practice for accessing internal infrastructure. Instead of exposing SSH on every server, only the bastion is publicly accessible. All other servers accept SSH only from internal network addresses.

SSH agent forwarding (`-A` flag) allows you to use your local SSH keys on remote servers without copying private keys to those servers. The agent holds keys in memory and authenticates on your behalf. This is crucial for security - never store private keys on bastion hosts or other shared systems.

Each VM has its own SSH key pair for security. If one key is compromised, it doesn't affect other servers. The agent provides convenience without sacrificing this security boundary.

⚠ **Common Mistakes**

- Forgetting the `-A` flag means agent forwarding won't work
- Adding keys after connecting to bastion won't help - keys must be added locally first
- Using `ssh-copy-id` to copy keys to the bastion violates security best practices
- Not verifying keys are loaded (`ssh-add -l`) leads to confusing authentication failures

✓ **Quick check:** You can successfully SSH from your laptop → BastionHost → WebServer using private IPs

Step 5: *Secure with Network Security Groups*

Apply Network Security Groups (NSGs) to implement firewall rules that enforce the principle of least privilege. Each server will accept only the minimum required network traffic for its role.

1. **Navigate to** the WebServer VM in the Azure Portal

2. **Select** Networking → Network settings from the left menu
3. **Click** the link for `WebServer-nsg` (the Network Security Group)
4. **Navigate to** Settings → Inbound security rules
5. **Delete** the SSH rule by clicking the ellipsis (...) and selecting Delete
6. **Verify** deletion by refreshing the page
7. **Test** that SSH is now blocked from the internet:

```
ssh -i ~/Downloads/WebServer_key.pem azureuser@<Web-
        Server_PublicIP>
```

8. **Verify** SSH still works from BastionHost:

```
ssh -A azureuser@<BastionHost_PublicIP>
ssh azureuser@<WebServer_PrivateIP>
```

9. **Navigate to** the ReverseProxy VM in the Azure Portal
10. **Follow** the link to `ReverseProxy-nsg`
11. **Delete** the SSH rule for ReverseProxy
12. **Add** a new inbound security rule:
    - **Click** + Add
    - **Service**: Select **HTTP**
    - **Click** Add
13. **Test** HTTP access from the internet by opening a browser:

```
http://<ReverseProxy_PublicIP>
```

14. **Verify** you see the "Hello World" page served by WebServer

CONCEPT DEEP DIVE

Network Security Groups (NSGs) act as virtual firewalls controlling inbound and outbound traffic. Each NSG contains security rules that allow or deny traffic based on source, destination, port, and protocol. Rules are evaluated by priority number (lower numbers first).

This exercise implements a layered security model:

- **WebServer**: No public access (no SSH, no HTTP) - completely internal
- **ReverseProxy**: HTTP only (no SSH) - serves web traffic
- **BastionHost**: SSH only (no HTTP) - administrative access point

This architecture follows the principle of least privilege - each component has exactly the permissions it needs and no more. The WebServer is completely isolated from the internet, reducing its attack surface. All administrative access flows through the hardened bastion host.

⚠ **Common Mistakes**

- Deleting all rules locks you out of VMs completely
- Forgetting to add the HTTP rule to ReverseProxy breaks web access
- Testing with wrong IPs (using WebServer public IP instead of ReverseProxy) gives false results
- Not verifying rules are applied immediately after changes can cause confusion

√ **Quick check:** You cannot SSH to WebServer or ReverseProxy from internet, but you can SSH from BastionHost, and HTTP works on ReverseProxy from internet

Step 6: *Test Your Implementation*

Verify that your secure network architecture functions correctly and enforces the intended security boundaries. This

comprehensive testing ensures all components work together as designed.

1. **Test WebServer isolation:**
   - Try to SSH to WebServer from your laptop (should fail)
   - Try to access `http://<WebServer_PublicIP>:8080` in browser (should timeout)
   - SSH through BastionHost to WebServer (should succeed)
   - Run `curl localhost:8080` on WebServer (should return "Hello World")
2. **Test ReverseProxy security:**
   - Try to SSH to ReverseProxy from your laptop (should fail)
   - SSH through BastionHost to ReverseProxy (should succeed)
   - Access `http://<ReverseProxy_PublicIP>` in browser (should return "Hello World")
   - Verify the content is served from WebServer, not ReverseProxy itself
3. **Test BastionHost access:**
   - SSH directly to BastionHost from your laptop (should succeed)
   - From BastionHost, SSH to both WebServer and ReverseProxy using private IPs (should succeed)
   - Try to access any HTTP service on BastionHost (should fail - no web server installed)
4. **Test internal connectivity:**
   - From ReverseProxy, run `curl <WebServer_PrivateIP>:8080` (should succeed)

- From WebServer, run `ping <ReverseProxy_PrivateIP>` (should succeed)
- Verify all servers can communicate internally

5. **Test security boundaries:**
   - Confirm WebServer has no direct internet exposure
   - Confirm only ReverseProxy accepts HTTP from internet
   - Confirm only BastionHost accepts SSH from internet

√ ***Success indicators:***

- *WebServer is completely inaccessible from the internet (SSH and HTTP both blocked)*
- *ReverseProxy serves web content but blocks SSH from internet*
- *BastionHost allows SSH but has no HTTP service*
- *All servers are accessible from BastionHost using private IPs*
- *Web traffic flows: Internet → ReverseProxy → WebServer*
- *Administrative traffic flows: Internet → BastionHost → Internal Servers*

√ ***Final verification checklist:***

☐ *Virtual Network created with address space 10.0.0.0/16*

☐ *All three VMs deployed and running*

☐ *WebServer responds on port 8080 with "Hello World"*

☐ *ReverseProxy forwards HTTP traffic to WebServer*

☐ *SSH agent forwarding works through BastionHost*

☐ *NSG rules enforce correct security boundaries*

☐

- [ ] *No direct SSH or HTTP access to WebServer from internet*
- [ ] *HTTP works on ReverseProxy, SSH blocked*
- [ ] *SSH works on BastionHost*

4.1.4 Common Issues

> ***If you encounter problems:***
> ***SSH connection timeout:*** *Verify you're using the correct public IP and that your local firewall allows outbound SSH*
> ***Permission denied (publickey):*** *Check SSH key file permissions with* `ls -l`*, should be 400 or 600. Use* `chmod 400` *to fix*
> ***nginx: configuration file test failed:*** *Review nginx config file for missing semicolons or curly braces*
> ***Reverse proxy returns 502 Bad Gateway:*** *Verify WebServer_PrivateIP and port 8080 are correct in ReverseProxy nginx config*
> ***Agent forwarding not working:*** *Ensure you started ssh-agent and added keys before connecting, use* `ssh-add -l` *to verify*
> ***Cannot access WebServer from BastionHost:*** *Verify you're using the private IP address, not the public IP*
> ***HTTP access blocked after NSG changes:*** *Wait 30-60 seconds for NSG rule changes to propagate across Azure infrastructure*
> ***Still stuck?*** *Check Azure Activity Log for deployment errors, and verify all VMs are in the same VNet and subnet*

4.1.5 Summary

You've successfully implemented a secure cloud network architecture which:

- √ Creates isolated network space with Azure Virtual Networks

- ✓ Implements server role separation for security and maintainability
- ✓ Uses Network Security Groups for defense-in-depth security
- ✓ Applies the principle of least privilege to minimize attack surface

> **KEY TAKEAWAY:**
>
> Proper network architecture is essential for security because it creates defense layers that protect your infrastructure even if one layer is compromised. This pattern of using reverse proxies and bastion hosts is industry-standard for production deployments in cloud environments.

4.1.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Try adding a fourth VM and isolating it in a separate subnet with its own NSG*
> - *Research how Azure Application Gateway differs from nginx as a reverse proxy*
> - *Implement SSL/TLS termination on the ReverseProxy using Let's Encrypt*
> - *Add Azure Bastion service as an alternative to your custom bastion host*
> - *Create a custom NSG that allows SSH only from your specific public IP address*
> - *Explore VNet peering to connect multiple virtual networks*

4.1.7 Done! 🎉

Excellent work! You've learned how to build secure network architecture and can now design and implement cloud infrastructure that follows security best practices. This foundation will help you create production-ready systems that protect sensitive workloads and data.

4.2 2. Creating a Virtual Network with Enhanced Security

4.2.1 Goal

Build a secure virtual network infrastructure using Azure Virtual Networks with Application Security Groups and Network Security Groups to control traffic between servers with different roles.

WHAT YOU'LL LEARN:

- How to create and configure Azure Virtual Networks with subnets
- When to use Application Security Groups versus Network Security Groups
- Best practices for implementing defense-in-depth security architecture
- How to automate server configuration using cloud-init

4.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Active Azure subscription with permissions to create resources
- ✓ Basic understanding of networking concepts (IP addresses, ports, protocols)
- ✓ Familiarity with Azure Portal navigation
- ✓ Understanding of cloud-init from previous exercises

4.2.3 Exercise Steps

Overview

1. **Create the Virtual Network**
2. **Configure Application Security Groups**
3. **Configure Network Security Group Rules**
4. **Provision Web Server**
5. **Provision Reverse Proxy**
6. **Provision Bastion Host**
7. **Test Your Implementation**

[image: NetworkOverview — not found]

Step 1: *Create the Virtual Network*

Create the foundational network infrastructure that will host your servers. The Virtual Network provides isolated network space in Azure where resources can communicate securely using private IP addresses.

1. **Navigate to** the Azure Portal at https://portal.azure.com
2. **Search for** "Virtual Networks" in the top search bar
3. **Click** the **+ Create** button

4. **Configure** the virtual network on the Basics tab:
   - **Subscription**: Select your subscription
   - **Resource Group**: Select or create `DemoRG`
   - **Name**: Enter `DemoVNet`
   - **Region**: Select `North Europe` or your preferred region
5. **Click Review + Create** and then **Create**

> **CONCEPT DEEP DIVE**
>
> A Virtual Network (VNet) is the fundamental building block for private networks in Azure. It functions like a traditional network in your datacenter but provides additional benefits of Azure infrastructure such as scale, availability, and isolation. Resources in the same VNet can communicate by default, while traffic between VNets or to the Internet must be explicitly configured.
>
> ⚠ **Common Mistakes**
>
> - Choosing different regions for VNet and VMs increases latency and may incur bandwidth costs
> - Using overlapping IP address ranges prevents VNet peering in the future
> - Forgetting to note the default subnet CIDR range for later configuration
>
> ✓ **Quick check:** Virtual Network appears in your resource group with a default subnet

Step 2: *Configure Application Security Groups*

Create logical groupings for your servers based on their roles. Application Security Groups allow you to define fine-grained network security policies based on workload patterns rather than explicit IP addresses.

1. **Search for** "Application Security Groups" in the Azure Portal

2. **Click + Create** to create the first ASG
3. **Configure** the Reverse Proxy ASG:
   - **Resource Group**: Select `DemoRG`
   - **Name**: Enter `ReverseProxyASG`
   - **Region**: Select the same region as your VNet
4. **Click Review + Create** and then **Create**
5. **Repeat** steps 2-4 to create the Bastion Host ASG:
   - **Name**: Enter `BastionHostASG`
   - Keep all other settings the same

> **CONCEPT DEEP DIVE**
>
> Application Security Groups (ASGs) enable you to group virtual machines by function and define security rules based on those groups. This abstraction means you don't need to manually maintain lists of IP addresses in security rules. When you add a VM to an ASG, it automatically inherits all rules associated with that group. This approach scales better and reduces configuration errors in complex environments.
>
> ⚠ **Common Mistakes**
>
> - Creating ASGs in different regions than the VMs causes association errors
> - Not creating ASGs before NSG rules means you'll need to modify rules later
> - Confusing ASGs with availability sets or VM scale sets
>
> ✓ **Quick check:** Both ASGs appear in your resource group with no associated NICs yet

Step 3: *Configure Network Security Group Rules*

Create and configure firewall rules that control traffic to your servers based on their Application Security Groups. Net-

work Security Groups act as distributed firewalls that filter traffic at the network interface or subnet level.

1. **Search for** "Network Security Groups" in the Azure Portal
2. **Click + Create**
3. **Configure** the NSG:
   - **Name**: Enter `DemoNSG`
   - **Resource Group**: Select `DemoRG`
   - **Region**: Select the same region as your VNet
4. **Click Review + Create** and then **Create**
5. **Navigate to** the newly created NSG and **click Go to resource**
6. **Select Inbound security rules** under Settings
7. **Click + Add** to create the SSH rule
8. **Configure** the SSH rule:
   - **Source**: Select `Service Tag`
   - **Source service tag**: Select `Internet`
   - **Destination**: Select `Application security group`
   - **Destination application security group**: Select `BastionHostASG`
   - **Service**: Select `SSH`
   - **Name**: Enter `Allow-SSH-Bastion`
9. **Click Add**
10. **Click + Add** again to create the HTTP rule
11. **Configure** the HTTP rule:
    - **Source**: Select `Service Tag`
    - **Source service tag**: Select `Internet`
    - **Destination**: Select `Application security group`

- **Destination application security group**: Select `ReverseProxyASG`
- **Service**: Select `HTTP`
- **Name**: Enter `Allow-HTTP-ReverseProxy`

12. **Click Add**
13. **Select Subnets** under Settings
14. **Click + Associate**
15. **Configure** the subnet association:
    - **Virtual network**: Select `DemoVNet`
    - **Subnet**: Select `default`
16. **Click OK**

CONCEPT DEEP DIVE

Network Security Groups provide stateful packet filtering at Layer 4 (Transport layer) of the OSI model. Rules are evaluated by priority number, with lower numbers evaluated first. The combination of NSGs with ASGs creates a powerful security model: NSGs define what traffic is allowed, while ASGs define which servers the rules apply to. This separation of concerns makes security policies more maintainable as your infrastructure grows.

Security Architecture Explanation:

- SSH access is limited to the Bastion Host, creating a single, auditable entry point
- HTTP traffic is only allowed to the Reverse Proxy, not directly to backend servers
- The Web Server has no public IP and no inbound rules from Internet, creating defense-in-depth
- All outbound traffic remains allowed by default for package installation and updates

⚠ **Common Mistakes**

- Attaching NSG to both subnet and NIC creates confusing rule interactions
- Forgetting to associate the NSG with the subnet means rules won't take effect
- Creating rules with same priority causes unpredictable behavior
- Using "Any" source for SSH creates security vulnerabilities

√ **Quick check:** NSG shows two custom inbound rules and is associated with the default subnet

Step 4: *Provision Web Server*

Create the backend web server that hosts your application content. This server runs on a non-standard port and has no

public IP address, making it accessible only through the internal network.

1. **Navigate to** Virtual Machines in the Azure Portal
2. **Click + Create** and select **Azure virtual machine**
3. **Configure** the instance details:
   - **Resource group**: Select `DemoRG`
   - **Virtual machine name**: Enter `WebServer`
   - **Region**: Select the same region as your VNet
   - **Image**: Select `Ubuntu Server 24.04 LTS`
   - **Size**: Select `Standard_B1s`
4. **Configure** administrator account:
   - **Authentication type**: Select `SSH public key`
   - **Username**: Enter `azureuser`
   - **SSH public key source**: Generate new key pair or use existing
5. **Click Next: Disks** then **Next: Networking**
6. **Configure** networking:
   - **Virtual network**: Select `DemoVNet`
   - **Subnet**: Select `default`
   - **Public IP**: Select `None`
   - **NIC network security group**: Select `None`
   - **Application security groups**: Leave empty (no ASG needed for internal-only server)
7. **Click Next: Management** then **Next: Monitoring** then **Next: Advanced**
8. **Add** the following cloud-init script in the Custom data field:

```yaml
#cloud-config
packages:
  - nginx
write_files:
  - path: /var/www/html/index.html
    content: |
      <!DOCTYPE html>
      <html>
      <head>
          <title>Hello World!</title>
      </head>
      <body>
          <h1>Hello World!</h1>
      </body>
      </html>
  - path: /etc/nginx/sites-available/default
    content: |
      server {
        listen 8080 default_server;
        server_name _;
        root /var/www/html;
        index index.html;
      }
runcmd:
  - systemctl restart nginx
```

9. **Click Review + Create** and then **Create**

CONCEPT DEEP DIVE

This server demonstrates the principle of least privilege - it has no public IP address and no ASG assignment because it doesn't need inbound Internet access. The web server listens on port 8080 instead of the standard port 80, illustrating how internal services can use non-standard ports safely. The cloud-init script automates the complete server configuration, making the deployment repeatable and eliminating manual configuration steps.

Why port 8080?

Using a non-standard port for the backend server provides several benefits: it clearly distinguishes internal from external services, prevents accidental direct access even if firewall rules fail, and follows the microservices pattern where backend services use different ports.

⚠ **Common Mistakes**

- Adding a public IP "just in case" defeats the security architecture
- Selecting an NSG at the NIC level conflicts with subnet-level NSG
- Incorrect indentation in cloud-init YAML causes configuration failures
- Forgetting to restart nginx means the new configuration won't take effect

✓ **Quick check:** VM is running, has a private IP address but no public IP, and shows no ASG associations

Step 5: *Provision Reverse Proxy*

Create the reverse proxy server that routes external HTTP traffic to the internal web server. This server acts as a security boundary and load distribution point in your architecture.

1. **Navigate to** Virtual Machines in the Azure Portal
2. **Click + Create** and select **Azure virtual machine**

3. **Configure** the instance details:
   - **Resource group**: Select `DemoRG`
   - **Virtual machine name**: Enter `ReverseProxy`
   - **Region**: Select the same region as your VNet
   - **Image**: Select `Ubuntu Server 24.04 LTS`
   - **Size**: Select `Standard_B1s`
4. **Configure** administrator account:
   - **Authentication type**: Select `SSH public key`
   - **Username**: Enter `azureuser`
   - Use the same SSH key as the Web Server
5. **Click Next: Disks** then **Next: Networking**
6. **Configure** networking:
   - **Virtual network**: Select `DemoVNet`
   - **Subnet**: Select `default`
   - **Public IP**: Select `Create new` or use existing
   - **NIC network security group**: Select `None`
   - **Application security groups**: **Select** `ReverseProxyASG`
7. **Click Next: Management** then **Next: Monitoring** then **Next: Advanced**
8. **Add** the following cloud-init script in the Custom data field:

```
#cloud-config
packages:
  - nginx
write_files:
  - path: /etc/nginx/sites-available/default
    content: |
      server {
        listen 80;
        location / {
          proxy_pass http://web-
        server.internal.cloudapp.net:8080/;
          proxy_set_header Host $host;
          proxy_set_header X-Real-IP $remote_addr;
          proxy_set_header X-Forwarded-For
        $proxy_add_x_forwarded_for;
        }
      }
runcmd:
  - systemctl restart nginx
```

9. **Click Review + Create** and then **Create**

CONCEPT DEEP DIVE

A reverse proxy serves as an intermediary that forwards client requests to backend servers. This architecture provides several production benefits: it hides the internal server topology, enables SSL termination at a single point, allows for load balancing across multiple backends, and provides a centralized location for request logging and caching.

Azure Internal DNS:

The configuration uses `web-server.internal.cloudapp.net` which is Azure's internal DNS service. Azure automatically creates DNS records for VMs in the format `vmname.internal.cloudapp.net` within the same VNet. This means you don't need to hardcode IP addresses, making your infrastructure more resilient to IP changes.

Nginx Proxy Headers:

The proxy headers preserve client information through the proxy chain. `X-Real-IP` contains the original client IP, while `X-Forwarded-For` maintains the chain of proxies. These headers are essential for logging, security, and application behavior that depends on client information.

⚠ **Common Mistakes**

- Forgetting to attach `ReverseProxyASG` means HTTP traffic will be blocked by NSG rules
- Using the wrong internal hostname causes connection failures to the backend
- Missing the trailing slash in `proxy_pass` can cause incorrect request routing
- Not including proxy headers breaks features that depend on client IP information

√ **Quick check:** VM is running with both public and private IPs and shows `ReverseProxyASG` in the networking section

Step 6: *Provision Bastion Host*

Create the bastion host that provides secure SSH access to internal servers. This server serves as the single entry point for administrative access, following security best practices.

1. **Navigate to** Virtual Machines in the Azure Portal
2. **Click + Create** and select **Azure virtual machine**
3. **Configure** the instance details:
   - **Resource group**: Select `DemoRG`
   - **Virtual machine name**: Enter `BastionHost`
   - **Region**: Select the same region as your VNet
   - **Image**: Select `Ubuntu Server 24.04 LTS`
   - **Size**: Select `Standard_B1s`
4. **Configure** administrator account:
   - **Authentication type**: Select `SSH public key`
   - **Username**: Enter `azureuser`
   - Use the same SSH key as previous servers
5. **Click Next: Disks** then **Next: Networking**
6. **Configure** networking:
   - **Virtual network**: Select `DemoVNet`
   - **Subnet**: Select `default`
   - **Public IP**: Select `Create new` or use existing
   - **NIC network security group**: Select `None`
   - **Application security groups**: **Select** `BastionHostASG`
7. **Click Review + Create** and then **Create** (no cloud-init needed)

CONCEPT DEEP DIVE

A bastion host (also called jump box) is a hardened server that provides access to internal infrastructure. In production environments, bastion hosts would have additional security measures: hardened OS configuration, multi-factor authentication, session recording, and restricted access hours. By routing all administrative access through a single point, you create an auditable security boundary and reduce the attack surface of your infrastructure.

Why no cloud-init?

The bastion host requires no special configuration - it's simply an SSH gateway. Its security comes from its placement in the network architecture and the NSG rules that control access, not from software configuration. This simplicity is by design: fewer installed packages means fewer potential security vulnerabilities.

⚠ **Common Mistakes**

- Forgetting to attach `BastionHostASG` means SSH access will be blocked
- Installing unnecessary software on the bastion increases attack surface
- Using password authentication instead of SSH keys weakens security
- Not monitoring bastion host logs misses potential security incidents

✓ **Quick check:** VM is running with both public and private IPs and shows `BastionHostASG` in the networking section

Step 7: *Test Your Implementation*

Verify that your security architecture works correctly and that all servers can communicate as designed. This system-

atic testing approach validates both the functionality and the security boundaries of your infrastructure.

1. **Verify NSG rule effectiveness:**

```
# From your local machine, test the Web Server (should
        fail - no public IP)
curl http://<WebServer_PublicIP>
# Expected: Connection timeout or DNS resolution failure

# Test Reverse Proxy HTTP access (should succeed)
curl http://<ReverseProxy_PublicIP>
# Expected: "Hello World!" HTML response
```

2. **Test internal DNS resolution:**

   Connect to the Bastion Host:

```
ssh azureuser@<BastionHost_PublicIP>
```

   Test internal name resolution:

```
# Verify Web Server DNS name resolves
nslookup webserver.internal.cloudapp.net
# Expected: Returns the private IP of WebServer

# Test direct access to Web Server
curl http://webserver.internal.cloudapp.net:8080
# Expected: "Hello World!" HTML response
```

3. **Test SSH access restrictions:**

   From your local machine, attempt SSH to Reverse Proxy (should fail):

```
ssh azureuser@<ReverseProxy_PublicIP>
# Expected: Connection timeout or refused
```

From Bastion Host, SSH to other servers (should succeed):

```
# Get private IPs from Azure Portal first
ssh azureuser@<ReverseProxy_PrivateIP>
# Expected: Successful connection
```

4. **Verify Application Security Group associations:**
   - **Navigate to** `ReverseProxyASG` in Azure Portal
   - **Check** that the Reverse Proxy's NIC appears under Associated network interfaces
   - **Navigate to** `BastionHostASG`
   - **Check** that the Bastion Host's NIC appears under Associated network interfaces
5. **Test the complete traffic flow:**
   - **Open** a browser and navigate to `http://<ReverseProxy_PublicIP>`
   - **Verify** you see the "Hello World!" message
   - **Check** that the traffic is being proxied through to the Web Server

√ ***Success indicators:***

- *Reverse Proxy responds to HTTP requests from the Internet*
- *Web Server is accessible only via internal network*
- *SSH works only to Bastion Host from Internet*
- *Internal DNS resolves VM hostnames correctly*
- *Can SSH from Bastion to internal servers using private IPs*
- *Web Server directly inaccessible from Internet*

√ ***Final verification checklist:***

☐ *Virtual Network created with default subnet*

☐ *Two Application Security Groups created and properly named*

☐ *Network Security Group has two custom inbound rules*

☐ *NSG is associated with the default subnet*

☐ *Web Server has no public IP and no ASG*

☐ *Reverse Proxy has public IP and ReverseProxyASG*

☐ *Bastion Host has public IP and BastionHostASG*

☐

- *HTTP request to Reverse Proxy returns "Hello World!"*
- ☐ *Direct HTTP access to Web Server fails*
- ☐ *SSH to Bastion Host succeeds from Internet*
- ☐ *SSH to Reverse Proxy fails from Internet*
- ☐ *Internal DNS resolution works from any VM*

4.2.4 Common Issues

> ***If you encounter problems:***
> ***"Connection timeout" when accessing Reverse Proxy:*** *Check that the NSG rule exists and that the VM is in the ReverseProxyASG. Verify the NSG is associated with the subnet.*
> ***"Unable to resolve host" for internal DNS:*** *Azure internal DNS takes 1-2 minutes to propagate. Wait a few minutes and try again. Ensure VMs are in the same VNet.*
> ***Cloud-init script didn't run:*** *Check* `/var/log/cloud-init-output.log` *on the VM. Common issues include YAML indentation errors or network connectivity problems during package installation.*
> ***SSH to Bastion works but can't reach internal servers:*** *Verify you're using private IP addresses (10.x.x.x) for internal connections, not public IPs. Check that VMs are in the same subnet.*
> ***Reverse Proxy returns 502 Bad Gateway:*** *The backend Web Server may not be responding. SSH to Bastion, then to Web Server, and check nginx status with* `sudo systemctl status nginx`*.*
> ***All HTTP requests fail:*** *Verify the NSG is associated with the subnet, not just existing as a standalone resource. Check NSG effective rules on the VM's network interface.*
> ***Still stuck?*** *Check Azure Activity Log for deployment errors. Verify all resources are in the same region. Ensure you're using the correct SSH key for authentication.*

4.2.5 Summary

You've successfully implemented a defense-in-depth network security architecture which:

- ✓ Isolates backend services from direct Internet access
- ✓ Uses Application Security Groups for role-based security policies
- ✓ Implements a bastion host for secure administrative access
- ✓ Demonstrates reverse proxy pattern for traffic routing
- ✓ Leverages Azure internal DNS for service discovery

> **KEY TAKEAWAY:**
>
> Network security is implemented in layers - NSGs provide firewall rules, ASGs provide logical grouping, network topology provides isolation, and the bastion pattern provides controlled access. This defense-in-depth approach means a single security control failure doesn't compromise the entire system. You'll use these patterns whenever building production infrastructure that must protect sensitive workloads.

4.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Try adding a second web server and configure the reverse proxy for load balancing*
> - *Implement Azure Firewall for centralized network traffic inspection*
> - *Add NSG flow logs to monitor and analyze network traffic patterns*
> - *Configure Azure Bastion service as a managed alternative to the bastion host VM*
> - *Add User Defined Routes to force traffic through a network virtual appliance*
> - *Implement VNet peering to connect multiple virtual networks securely*

4.2.7 Done!

Excellent work! You've learned how to design and implement secure network architectures in Azure using multiple layers of security controls. This foundation will help you build production-grade infrastructure that follows cloud security best practices. Understanding these networking and security primitives is essential for any cloud infrastructure project.

EXERCISE 5

Command Line Interface

5.1 3. Virtual Network with Enhanced Security

5.1.1 Goal

Build a complete three-tier network infrastructure on Azure to enable secure communication between application components while protecting them from unauthorized access.

WHAT YOU'LL LEARN:

- How to create virtual networks and subnets for network isolation
- When to use Network Security Groups (NSGs) and Application Security Groups (ASGs) together
- Best practices for implementing defense-in-depth security architecture
- How to automate server configuration using cloud-init

5.1.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Azure CLI installed and configured on your machine
- ✓ An active Azure subscription
- ✓ Basic understanding of IP addresses and network security concepts
- ✓ SSH key pair generated (or ability to generate one)

5.1.3 Exercise Steps

Overview

1. **Create Foundation Resources**
2. **Configure Network Security**
3. **Provision Virtual Machines**
4. **Attach Security Groups to Network Interfaces**
5. **Test Your Implementation**

Step 1: *Create Foundation Resources*

Establish the basic Azure infrastructure that will host your network and virtual machines. This includes a resource

group to organize resources and a virtual network to enable communication between components.

1. **Create** a resource group to contain all resources:

```
az group create --name DemoRG --location northeurope
```

2. **Create** a virtual network with a default subnet:

```
az network vnet create \
  --resource-group DemoRG \
  --name DemoVNet \
  --address-prefix 10.0.0.0/16 \
  --subnet-name default \
  --subnet-prefix 10.0.0.0/24
```

CONCEPT DEEP DIVE

Resource groups are logical containers that group related Azure resources for management and billing. Virtual networks (VNets) provide isolated network environments in Azure, similar to traditional on-premises networks. The address prefix `10.0.0.0/16` provides 65,536 IP addresses for the entire VNet, while the subnet prefix `10.0.0.0/24` allocates 256 addresses for the default subnet.

The CIDR notation `/16` and `/24` determines the network size. A `/16` network has 16 fixed bits and 16 variable bits (2^16 = 65,536 addresses). A `/24` network has 24 fixed bits and 8 variable bits (2^8 = 256 addresses).

⚠ **Common Mistakes**

- Using overlapping address ranges with existing networks causes routing conflicts
- Choosing too small a subnet (like `/28`) limits growth to 16 addresses
- Forgetting that Azure reserves 5 addresses in each subnet for internal use

✓ **Quick check:** Run `az network vnet list --resource-group DemoRG` to verify the VNet was created

Step 2: *Configure Network Security*

Implement a defense-in-depth security model using both Network Security Groups and Application Security Groups. This layered approach provides fine-grained control over network traffic to protect your infrastructure.

1. **Create** the Application Security Groups for logical grouping:

```
az network asg create \
  --resource-group DemoRG \
  --name ReverseProxyASG

az network asg create \
  --resource-group DemoRG \
  --name BastionHostASG
```

2. **Create** the Network Security Group:

```
az network nsg create \
  --resource-group DemoRG \
  --name DemoNSG
```

3. **Add** the SSH access rule for the bastion host:

```
az network nsg rule create \
  --resource-group DemoRG \
  --nsg-name DemoNSG \
  --name AllowSSH \
  --priority 1000 \
  --access Allow \
  --protocol Tcp \
  --direction Inbound \
  --source-address-prefixes Internet \
  --source-port-ranges "*" \
  --destination-asg BastionHostASG \
  --destination-port-ranges 22
```

4. **Add** the HTTP access rule for the reverse proxy:

```
az network nsg rule create \
  --resource-group DemoRG \
  --nsg-name DemoNSG \
  --name AllowHTTP \
  --priority 2000 \
  --access Allow \
  --protocol Tcp \
  --direction Inbound \
  --source-address-prefixes Internet \
  --source-port-ranges "*" \
  --destination-asg ReverseProxyASG \
  --destination-port-ranges 80
```

5. **Associate** the NSG with the subnet:

```
az network vnet subnet update \
  --resource-group DemoRG \
  --vnet-name DemoVNet \
  --name default \
  --network-security-group DemoNSG
```

CONCEPT DEEP DIVE

Application Security Groups (ASGs) let you group virtual machines by their role in your application architecture, rather than by IP address. This makes security rules more maintainable because you can add or remove VMs from groups without updating firewall rules.

Network Security Groups (NSGs) act as virtual firewalls that control inbound and outbound traffic. Rules are evaluated in priority order (lowest number first), and the first matching rule is applied. Once a rule matches, evaluation stops.

By attaching the NSG to the subnet, all resources in that subnet inherit the rules. This is more efficient than attaching NSGs to individual network interfaces, especially in larger deployments.

Defense-in-depth architecture: The web server has no public IP and can only be reached through the reverse proxy. The reverse proxy accepts HTTP traffic from the internet but can't be accessed via SSH. Only the bastion host accepts SSH connections, creating a secure entry point for administration.

⚠ **Common Mistakes**

- Setting the same priority for multiple rules causes conflicts
- Using `*` for destination port ranges is too permissive for production
- Forgetting to associate the NSG with the subnet means rules won't be enforced
- Creating ASGs without actually attaching them to VM network interfaces (Step 4)

✓ **Quick check:** Run `az network nsg rule list --resource-group DemoRG --nsg-name DemoNSG --output table` to verify both rules exist

Step 3: *Provision Virtual Machines*

Deploy the three servers that make up your infrastructure: a web server to host content, a reverse proxy to handle external requests, and a bastion host for secure administrative access.

1. **Create** the web server configuration file named `web_server_config.yaml`:

   > `web_server_config.yaml`

```yaml
#cloud-config
packages:
  - nginx
write_files:
  - path: /var/www/html/index.html
    content: |
      <!DOCTYPE html>
      <html>
      <head>
          <title>Hello World!</title>
      </head>
      <body>
          <h1>Hello World!</h1>
      </body>
      </html>
  - path: /etc/nginx/sites-available/default
    content: |
      server {
        listen 8080 default_server;
        server_name _;
        root /var/www/html;
        index index.html;
      }
runcmd:
  - systemctl restart nginx
```

2. **Provision** the web server virtual machine:

```
az vm create \
  --resource-group DemoRG \
  --name WebServer \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --admin-username azureuser \
  --vnet-name DemoVNet \
  --subnet default \
  --nsg "" \
  --public-ip-address "" \
  --generate-ssh-keys \
  --custom-data @web_server_config.yaml
```

3. **Create** the reverse proxy configuration file named `reverse_proxy_config.yaml`:

> `reverse_proxy_config.yaml`

```
#cloud-config
packages:
  - nginx
write_files:
  - path: /etc/nginx/sites-available/default
    content: |
      server {
        listen 80;
        location / {
          proxy_pass http://web-
        server.internal.cloudapp.net:8080/;
          proxy_set_header Host $host;
          proxy_set_header X-Real-IP $remote_addr;
          proxy_set_header X-Forwarded-For
        $proxy_add_x_forwarded_for;
        }
      }
runcmd:
  - systemctl restart nginx
```

4. **Provision** the reverse proxy virtual machine:

```
az vm create \
  --resource-group DemoRG \
  --name ReverseProxy \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --admin-username azureuser \
  --vnet-name DemoVNet \
  --subnet default \
  --nsg "" \
  --generate-ssh-keys \
  --custom-data @reverse_proxy_config.yaml
```

5. **Provision** the bastion host virtual machine:

```
az vm create \
  --resource-group DemoRG \
  --name BastionHost \
  --image Ubuntu2204 \
  --size Standard_B1s \
  --admin-username azureuser \
  --vnet-name DemoVNet \
  --subnet default \
  --nsg "" \
  --generate-ssh-keys
```

CONCEPT DEEP DIVE

Cloud-init is an industry-standard tool for cloud instance initialization. The YAML configuration files define packages to install, files to create, and commands to run during first boot. This automation ensures consistent server configuration and eliminates manual setup steps.

The web server listens on port 8080 (non-standard) because it's an internal service not exposed to the internet. The reverse proxy accepts traffic on port 80 (standard HTTP) and forwards requests to the web server's internal address.

Using Azure's internal DNS (`webserver.internal.cloudapp.net`) provides name resolution within the VNet without requiring static IP addresses. This makes the infrastructure more maintainable.

The `--nsg ""` parameter prevents Azure from automatically creating network interface-level NSGs, since we're using a subnet-level NSG instead. The `--public-ip-address ""` parameter ensures the web server has no public IP, making it only accessible internally.

The `--generate-ssh-keys` parameter creates SSH key pairs automatically if they don't exist at `~/.ssh/id_rsa`. This is convenient for development but in production you should use explicitly managed keys.

⚠ **Common Mistakes**

- Forgetting the `@` symbol in `@web_server_config.yaml` causes Azure CLI to interpret the filename as configuration text
- Using `http://WebServer:8080` in the proxy config fails because the hostname must match Azure's internal DNS format
- Missing `systemctl restart nginx` in cloud-init means configuration changes won't take effect
- Creating VMs before the VNet exists results in creation failures

> ✓ **Quick check:** Run `az vm list --resource-group DemoRG --output table` to verify all three VMs are running

Step 4: *Attach Security Groups to Network Interfaces*

Connect the Application Security Groups to the network interfaces of your virtual machines to enforce the security rules. This crucial step activates the NSG rules you created earlier.

1. **Retrieve** and store the reverse proxy's NIC information:

```
REVERSE_PROXY_NIC_ID=$(az vm show \
  --resource-group DemoRG \
  --name ReverseProxy \
  --query 'networkProfile.networkInterfaces[0].id' \
  --output tsv)

REVERSE_PROXY_NIC_NAME=$(basename $REVERSE_PROXY_NIC_ID)

REVERSE_PROXY_NIC_IP_CONFIG=$(az network nic show \
  --resource-group DemoRG \
  --name $REVERSE_PROXY_NIC_NAME \
  --query 'ipConfigurations[0].name' \
  --output tsv)
```

2. **Attach** the ReverseProxyASG to the reverse proxy's NIC:

```
az network nic ip-config update \
  --resource-group DemoRG \
  --nic-name $REVERSE_PROXY_NIC_NAME \
  --name $REVERSE_PROXY_NIC_IP_CONFIG \
  --application-security-groups ReverseProxyASG
```

3. **Retrieve** and store the bastion host's NIC information:

```
BASTION_HOST_NIC_ID=$(az vm show \
  --resource-group DemoRG \
  --name BastionHost \
  --query 'networkProfile.networkInterfaces[0].id' \
  --output tsv)

BASTION_HOST_NIC_NAME=$(basename $BASTION_HOST_NIC_ID)

BASTION_HOST_NIC_IP_CONFIG=$(az network nic show \
  --resource-group DemoRG \
  --name $BASTION_HOST_NIC_NAME \
  --query 'ipConfigurations[0].name' \
  --output tsv)
```

4. **Attach** the BastionHostASG to the bastion host's NIC:

```
az network nic ip-config update \
  --resource-group DemoRG \
  --nic-name $BASTION_HOST_NIC_NAME \
  --name $BASTION_HOST_NIC_IP_CONFIG \
  --application-security-groups BastionHostASG
```

CONCEPT DEEP DIVE

Network Interface Cards (NICs) are the virtual network adapters attached to VMs. Each NIC has one or more IP configurations that define its network settings. Attaching ASGs to these IP configurations tells Azure which security rules apply to that VM.

The `--query` parameter uses JMESPath syntax to extract specific fields from JSON output. This enables shell scripting by capturing resource IDs and names in variables.

The `basename` command extracts just the filename from a full resource path. For example, it converts `/subscriptions/.../networkInterfaces/ReverseProxyVMNIC` to just `ReverseProxyVMNIC`.

Azure allows multiple IP configurations per NIC (for multi-homing scenarios), which is why we query `ipConfigurations[0]` to get the first (and typically only) configuration.

Why this matters: Until you complete this step, the NSG rules targeting ASGs have no effect. The VMs are essentially unprotected by your custom rules. This step activates your security architecture.

⚠ **Common Mistakes**

- Forgetting the `$` before variable names causes literal strings to be used instead of values
- Running these commands before VMs are fully provisioned results in "resource not found" errors
- Attaching ASGs to the wrong NIC associates security rules with the wrong server
- Not storing the NIC name correctly causes the update command to fail

√ **Quick check:** Run `az network nic show --resource-group DemoRG --name $REVERSE_PROXY_NIC_NAME --query 'ipConfigurations[0].applicationSecurityGroups'` to verify the ASG is attached

Step 5: *Test Your Implementation*

Verify that your infrastructure is working correctly and that the security rules are properly enforced. This systematic testing ensures all components are configured properly and communicate as designed.

1. **Retrieve** the reverse proxy's public IP address:

```
REVERSE_PROXY_IP=$(az vm show \
  --resource-group DemoRG \
  --name ReverseProxy \
  --show-details \
  --query 'publicIps' \
  --output tsv)

echo "Reverse Proxy IP: $REVERSE_PROXY_IP"
```

2. **Test** HTTP access to the reverse proxy:

   Open your browser and navigate to `http://<REVERSE_PROXY_IP>` or use curl:

```
curl http://$REVERSE_PROXY_IP
```

   You should see the "Hello World!" page served by the web server through the reverse proxy.

3. **Retrieve** the bastion host's public IP address:

```
BASTION_IP=$(az vm show \
  --resource-group DemoRG \
  --name BastionHost \
  --show-details \
  --query 'publicIps' \
  --output tsv)

echo "Bastion Host IP: $BASTION_IP"
```

4. **Test** SSH access to the bastion host:

```
ssh azureuser@$BASTION_IP
```

   You should successfully connect to the bastion host.

5. **Verify** security isolation by testing blocked access:

   Try to SSH directly to the web server (this should fail since it has no public IP):

```
# This command will fail - web server has no public IP
az vm show \
  --resource-group DemoRG \
  --name WebServer \
  --show-details \
  --query 'publicIps'
```

   Try to SSH to the reverse proxy (this should be blocked by NSG rules):

```
# This connection will timeout - SSH is blocked by NSG
ssh azureuser@$REVERSE_PROXY_IP
```

6. **Test** internal connectivity from the bastion host:

   SSH into the bastion host, then test connectivity to internal servers:

```
# From the bastion host:
curl http://webserver.internal.cloudapp.net:8080
curl http://reverseproxy.internal.cloudapp.net
```

√ ***Success indicators:***

- *The reverse proxy serves the "Hello World!" page when accessed via HTTP*
- *You can SSH into the bastion host from your local machine*
- *SSH to the reverse proxy is blocked (connection timeout)*
- *The web server has no public IP address*
- *Internal communication works from the bastion host to both servers*
- *NSG rules show correct hit counts when you view them in the portal*

√ ***Final verification checklist:***

- ☐ *Virtual network and subnet created successfully*
- ☐ *NSG rules allow SSH to bastion and HTTP to reverse proxy only*
- ☐ *ASGs attached to the correct VM network interfaces*
- ☐ *All three VMs provisioned and running*
- ☐ *Web server accessible only through reverse proxy*
- ☐ *Bastion host accessible via SSH from internet*
- ☐ *Internal DNS resolution works between VMs*

5.1.4 Common Issues

> ***If you encounter problems:***
> ***"Resource not found" when attaching ASGs:*** *Wait 1-2 minutes after VM creation for all resources to be fully provisioned, then retry*
> ***Connection timeout to reverse proxy:*** *Verify the ASG is attached to the NIC using* `az network nic show`*. Check NSG rule priority and ensure the rule targets the correct ASG*
> ***"Hello World!" page doesn't load:*** *SSH into the reverse proxy and check nginx status with* `sudo systemctl status nginx`*. Verify the web server is reachable with* `curl http://web-server.internal.cloudapp.net:8080`
> ***Can't SSH to bastion host:*** *Check that the NSG rule allows your source IP address (it should allow "Internet" source). Verify your SSH key is correctly configured*
> ***Internal DNS not working:*** *Ensure all VMs are in the same VNet. Azure's internal DNS only works within VNets, not across different VNets without peering*
> ***Variable commands fail:*** *Make sure you run all commands in sequence within the same terminal session. Variables are session-specific and don't persist across terminal restarts*
> ***Still stuck?*** *Run* `az network watcher show-topology --resource-group DemoRG` *to visualize your network architecture and identify configuration issues*

5.1.5 Summary

You've successfully built a production-ready three-tier network architecture which:

- √ Implements defense-in-depth security with NSGs and ASGs
- √ Isolates internal services from direct internet access
- √ Provides secure administrative access through a bastion host
- √ Uses cloud-init for automated server configuration

> **KEY TAKEAWAY:**
>
> Application Security Groups combined with Network Security Groups create flexible, maintainable security policies based on application roles rather than IP addresses. This architecture pattern is essential in production environments where servers are frequently added, removed, or replaced. The bastion host pattern provides secure administrative access while keeping production systems isolated from the internet.

5.1.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Try adding a fourth VM to the WebServerASG and update the reverse proxy to load balance between multiple web servers*
> - *Implement Azure Bastion (a managed PaaS solution) instead of a self-managed bastion host for enhanced security*
> - *Add HTTPS support to the reverse proxy using Let's Encrypt certificates*
> - *Create a separate management subnet with its own NSG for administrative traffic*
> - *Implement network flow logs to monitor and analyze traffic patterns*
> - *Add a Web Application Firewall (WAF) in front of the reverse proxy for additional protection*

5.1.7 Cleanup

> ***To avoid ongoing charges, delete all resources:***

```
az group delete --name DemoRG --yes --no-wait
```

> *This command deletes the resource group and all contained resources (VMs, NICs, NSGs, ASGs, VNet, and public IPs). The `--no-wait` flag allows the command to return immediately while deletion continues in the background.*

5.1.8 Done! 🎉

Excellent work! You’ve learned how to design and implement secure network architectures on Azure using infrastructure as code. You can now build multi-tier applications with proper network isolation and security controls. This foundation will help you deploy production-grade applications in the cloud.

PART III

Deployment

PART III · DEPLOYMENT

EXERCISE 6

CI/CD to Azure Container Apps

6.1 First Pipeline: Build and Push to Docker Hub

6.2 First Pipeline: Build and Push to Docker Hub

6.2.1 Goal

Stand up the smallest possible CI/CD pipeline that takes a fresh ASP.NET Core MVC app, packages it into a Docker image, pushes that image to Docker Hub on every commit to `main`, and runs it on Azure Container Apps. The deploy step

is intentionally manual today — you'll wire up the missing automation in the next exercise.

WHAT YOU'LL LEARN:

- How a GitHub Actions workflow file describes a build pipeline as a YAML recipe
- How GitHub Secrets let the runner authenticate to external services without leaking credentials
- Why a Personal Access Token replaces your Docker Hub password for automated logins
- How a multi-stage Dockerfile produces a small, secure runtime image
- How Azure Container Apps pulls a public image and serves it at a stable FQDN

6.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ .NET 10 SDK installed (`dotnet --version` reports a `10.*` version)
- ✓ Docker Desktop running locally (`docker --version` works)
- ✓ A free Docker Hub account (https://hub.docker.com)
- ✓ The GitHub CLI installed and authenticated (`gh auth status` reports logged in)
- ✓ An Azure subscription where you can create resource groups (the free tier is enough)
- ✓ Git installed and a working terminal in an empty folder for this exercise

6.2.3 Exercise Steps

Overview

1. **Scaffold** a fresh ASP.NET Core MVC project
2. **Add a multi-stage Dockerfile** and a `.dockerignore`
3. **Surface the build SHA** on the homepage so you can see the new revision land
4. **Build and run** the container locally to confirm it works
5. **Initialize Git** and push to a new GitHub repository
6. **Generate a Docker Hub access token**
7. **Add the token** as a GitHub Actions secret
8. **Write the GitHub Actions workflow**
9. **Trigger the workflow** and confirm the image appears on Docker Hub
10. **Create the Azure Container App** from the Docker Hub image
11. **Open the FQDN** and verify the live page
12. **Push a change** and manually update the Container App revision
13. **Test Your Implementation**

Step 1: *Scaffold a fresh ASP.NET Core MVC project*

Start from the default MVC template. The scaffold gives you a working app — a home controller, a shared layout, and Bootstrap pre-wired — without distractions. The CI/CD work in this exercise is independent of the application itself; using the default template means the only thing that can break is the pipeline.

1. **Create** the project from the MVC template:

```
dotnet new mvc -o CloudCi --framework net10.0
cd CloudCi
```

2. **Verify** it builds and runs locally:

```
dotnet run --launch-profile http
```

Opening `http://localhost:5000` (or whatever port the launch profile reports) should show the default MVC welcome page. Stop the server with `Ctrl+C`.

> **QUICK CHECK:**
> The project builds and the welcome page renders.

Step 2: *Add a multi-stage Dockerfile and a .dockerignore*

The container is the unit of deployment for the rest of this chapter. A multi-stage Dockerfile uses one image to compile the app and a different, much smaller image to run it. The build stage pulls in the .NET SDK (well over a gigabyte); the runtime stage carries only the ASP.NET runtime, so the final image you ship is small and contains no compilers or build tools.

1. **Create** a new file at the project root:

> `CloudCi/Dockerfile`

```
# Build stage — has the SDK, compilers, NuGet, etc.
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# Copy csproj first so layer caching kicks in on
        dependency changes.
COPY *.csproj ./
RUN dotnet restore

# Copy the rest of the source and publish a release
        build.
COPY . ./
RUN dotnet publish -c Release -o /app/publish --no-
        restore

# Runtime stage — only the ASP.NET runtime, no build
        tools.
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish ./

# Bake the build commit SHA into the image at build
        time.
ARG BUILD_SHA=local
ENV BUILD_SHA=$BUILD_SHA

# Listen on :8080 — Container Apps maps this to the
        public ingress.
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080

ENTRYPOINT ["dotnet", "CloudCi.dll"]
```

2. **Create** a `.dockerignore` at the project root so the build context stays small:

   > `CloudCi/.dockerignore`

```
bin/
obj/
.vs/
.vscode/
.idea/
.git/
.gitignore
*.user
Dockerfile
.dockerignore
```

CONCEPT DEEP DIVE

Why a multi-stage Dockerfile? The .NET SDK image is over a gigabyte and ships with compilers, NuGet caches, and build tools. None of that is useful — or safe — at runtime. The `aspnet` image weighs in at a few hundred megabytes and contains only the runtime. Multi-stage builds let you do the heavy lifting in the first stage and then `COPY --from=build` only the published output into a clean second stage, so the final image is small and has a much smaller attack surface.

⚠ **Common Mistakes**

- Forgetting `EXPOSE 8080` and `ASPNETCORE_URLS=http://+:8080` means the app listens on `localhost:5000` inside the container and the port mapping silently does nothing.
- Without a `.dockerignore`, the build context includes `bin/` and `obj/` from your local machine, which can confuse layer caching and bloat the image.

✓ **Quick check:** `Dockerfile` and `.dockerignore` exist in the project root.

Step 3: *Surface the build SHA on the homepage*

You need a visible cue that proves the running container is the one your pipeline just built. The simplest cue is to display

the Git commit SHA on the homepage. The Dockerfile already declared `BUILD_SHA` as a build argument that becomes a runtime environment variable; now read that variable in the view.

1. **Open** the existing `Views/Home/Index.cshtml`
2. **Replace** the file contents with:

> `CloudCi/Views/Home/Index.cshtml`

```
@{
    ViewData["Title"] = "Home";
    var buildSha = Environ-
        ment.GetEnvironmentVariable("BUILD_SHA") ??
        "local";
    var hostName = Environment.MachineName;
}

<div class="text-center">
    <h1 class="display-4">Welcome</h1>
    <p>Hello from CloudCi.</p>

    <div class="mt-4">
        <span class="badge bg-secondary" data-
        testid="build-sha">
            build: @buildSha
        </span>
        <span class="badge bg-secondary" data-
        testid="host-name">
            host: @hostName
        </span>
    </div>
</div>
```

> **CONCEPT DEEP DIVE**
>
> What does `--build-arg` do? Arguments declared with `ARG` in the Dockerfile are available only during `docker build`; they are not part of the running image unless you copy them into an `ENV`. The line `ENV BUILD_SHA=$BUILD_SHA` pins the build-time value into the runtime environment, so the running container can read it like any other environment variable. The workflow will pass `--build-arg BUILD_SHA=${{ github.sha }}` so each image carries the commit it was built from.
>
> ✓ **Quick check:** Running `dotnet run` and visiting the homepage shows two badges — `build: local` and `host: <your machine>`.

Step 4: *Build and run the container locally*

Before you push anything to the cloud, prove the image actually works on your laptop. This is a tight feedback loop: a broken Dockerfile here is much cheaper to debug than a broken image discovered in Container Apps.

1. **Build** the image, passing a placeholder commit SHA:

```
docker build --build-arg BUILD_SHA=dev-local -t
        cloudci:local .
```

2. **Run** the container, mapping port 8080 on your host to port 8080 inside the container:

```
docker run --rm -p 8080:8080 cloudci:local
```

3. **Browse to** `http://localhost:8080` and confirm the homepage renders with the badge `build: dev-local`.
4. **Stop** the container with `Ctrl+C`.

COMMON MISTAKES

- If `docker run` exits immediately, check the build output for compilation errors and confirm `EXPOSE 8080` and `ASPNETCORE_URLS` are set in the Dockerfile.
- If the page loads but the badge says `build: local`, you forgot the `--build-arg`. Rebuild.

√ **Quick check:** `http://localhost:8080` renders the homepage with the `build: dev-local` badge.

Step 5: *Initialize Git and push to a new GitHub repository*

The pipeline triggers on `push` to the `main` branch, so you need a remote first. Use the GitHub CLI to create the repository and push your initial commit in one shot.

1. **Initialize** Git and make the first commit:

```
git init -b main
git add .
git commit -m "Initial scaffold with Dockerfile"
```

2. **Create** a new public GitHub repository from the current folder and push:

```
gh repo create --public --source=. --remote=origin --
        push
```

3. **Confirm** the push by visiting the repository URL the CLI prints, or with:

```
gh repo view --web
```

> **QUICK CHECK:**
> The repository exists on GitHub with your initial commit visible.

Step 6: *Generate a Docker Hub access token*

A Personal Access Token (PAT) is what your GitHub Actions runner will use to authenticate to Docker Hub. Tokens are scoped — you can grant just the permissions the pipeline needs — and revocable, so leaking one is not the same as leaking your account password.

1. **Sign in** at https://hub.docker.com/ with your account.
2. **Open** Account Settings: click your avatar (top right) → **Account settings** → **Personal access tokens**.
3. **Click Generate new token**.
4. **Fill in** the form:
   - **Description:** `cloudci-github-actions`
   - **Access permissions: Read, Write, Delete**
   - **Expiration:** the shortest period that covers this course (typically 60 or 90 days)
5. **Click Generate** and **copy the token**. You will not see it again — paste it into a temporary scratch file or directly into the next step.

CONCEPT DEEP DIVE

Why a PAT instead of your Docker Hub password? Tokens carry their own audit trail and can be revoked individually, so you can rotate the GitHub Actions credential without touching anything else. They are also scoped — a leaked Read/Write/Delete token can push to your registries but cannot, for example, change your account email or delete your account. Treat tokens as you would a SSH key: one per consumer, never reused across services.

⚠ **Common Mistakes**

- Confusing your Docker Hub **username** with your **email address**. Docker login takes the username (the one in your profile URL), not the email.
- Closing the dialog before copying the token. If that happens, delete the token and generate a new one.

Step 7: *Add the token as a GitHub Actions secret*

GitHub Secrets are encrypted environment variables that are only injected into workflow runs. The runner can read them, but they are masked in logs and unavailable to anyone reading the repository or pull requests from forks. Add both your Docker Hub username and the token you just generated.

1. **Set** the username secret. Replace the placeholder with your real Docker Hub username:

```
gh secret set DOCKERHUB_USERNAME --body "<your-docker-
        hub-username>"
```

2. **Set** the token secret. The CLI will prompt you to paste it:

```
gh secret set DOCKERHUB_TOKEN
```

Paste the PAT and press Enter. The CLI does not echo it back.

3. **Verify** both secrets exist:

```
gh secret list
```

You should see `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`.

> **COMMON MISTAKES**
>
> - Pushing the workflow file before adding the secrets — the run will fire and fail authentication. Add the secrets first, then write the workflow.
> - Pasting the PAT into the workflow YAML directly. Anything in YAML is in Git history forever.

Step 8: *Write the GitHub Actions workflow*

The workflow is a YAML recipe stored in `.github/workflows/`. GitHub Actions watches that folder and runs whatever it finds there in response to the configured trigger. This first version does the bare minimum: check out the code, log in to Docker Hub, build the image, and push it under two tags — the immutable commit SHA and the mutable `latest`.

1. **Create** the workflow folder and file:

```
mkdir -p .github/workflows
```

2. **Add** the workflow file:

> `CloudCi/.github/workflows/ci.yml`

```yaml
name: ci

on:
  push:
    branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Check out the repository
        uses: actions/checkout@v4

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          build-args: |
            BUILD_SHA=${{ github.sha }}
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/
        cloudci:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/cloudci:$
        {{ github.sha }}
```

3. **Commit and push** the workflow:

```bash
git add .github/workflows/ci.yml
git commit -m "Add CI workflow that pushes to Docker
        Hub"
git push
```

CONCEPT DEEP DIVE

Why two tags? The `:latest` tag is mutable — it always points at the most recent build. That is convenient for the Container App's "always pull the newest" mode, but useless for forensics: there is no record of which commit produced the image currently running in production. The immutable `:${{ github.sha }}` tag pins each image to the exact commit it was built from, so rolling back is just a question of pointing the deployment at an older SHA tag.

⚠ **Common Mistakes**

- Misindented YAML. Use a real editor with YAML support; one stray space breaks the file silently and the workflow run will refuse to start.
- Forgetting `push: true` on `docker/build-push-action`. The default is to build only — the image will be made and immediately discarded.

Step 9: *Trigger the workflow and confirm the image on Docker Hub*

The push you just did already triggered a run. Watch it complete and verify the resulting image lands in your Docker Hub repository.

1. **Watch** the most recent run from the terminal:

   ```
   gh run watch
   ```

   Or open the **Actions** tab on the repository in a browser and click into the latest run.

2. **Confirm** the run finished green. The Build and push image step should report two tags pushed.
3. **Visit** your Docker Hub repository in a browser:

```
https://hub.docker.com/r/<your-dockerhub-username>/
cloudci/tags
```

You should see two tags: `latest` and a long hex tag matching your commit SHA.

> **COMMON MISTAKES**
>
> - The image does not appear and the run reports `denied: requested access to the resource is denied`. Check that the `DOCKERHUB_USERNAME` secret matches the namespace in the tag (it must — the workflow uses one to build the other).
> - The repository on Docker Hub is created on first push and defaults to **public** for free accounts. If you set the namespace to one you do not own, the push fails. Always use your own username.

Step 10: *Create the Azure Container App from the Docker Hub image*

Container Apps is Azure's serverless container platform. Give it a public image reference, an ingress port, and a scale rule, and it produces a stable FQDN that serves the image. There are several ways to create one — this exercise uses the Portal so you can see every knob the platform exposes.

1. **Sign in** at https://portal.azure.com.
2. **Search** for **Container Apps** in the top bar and click **+ Create**.
3. **Basics tab — Project details:**
   - **Subscription:** your subscription
   - **Resource group:** click **Create new**, name it `rg-cicd-week4`
   - **Container app name:** `ca-cicd-week4`

- **Region:** `North Europe`

4. **Basics tab — Container Apps Environment:**
   - Click **Create new** next to the Environment dropdown
   - **Environment name:** `cae-cicd-week4`
   - Leave the rest at defaults and click **Create**
5. **Click Next: Container >**.
6. **Container tab:**
   - **Uncheck Use quickstart image**
   - **Name:** `cloudci`
   - **Image source: Docker Hub or other registries**
   - **Image type: Public**
   - **Registry login server:** `docker.io`
   - **Image and tag:** `<your-dockerhub-username>/cloudci:latest`
   - Leave CPU and memory at defaults
7. **Click Next: Ingress >**.
8. **Ingress tab:**
   - **Ingress: Enabled**
   - **Ingress traffic: Accepting traffic from anywhere**
   - **Ingress type: HTTP**
   - **Target port:** `8080`
9. **Click Review + create**, then **Create**. Provisioning takes a couple of minutes.
10. **When deployment finishes**, click **Go to resource** and copy the **Application Url** from the Overview page. That is the FQDN your Container App is serving on.

COMMON MISTAKES

- Leaving **Target port** at the default 80. The container listens on 8080, so any other value sends traffic to a closed socket and produces a 502 in the browser.
- Picking a region that does not yet have Container Apps capacity. North Europe and West Europe are reliable choices for this course.
- Forgetting to switch **Image source** away from the quick-start image. The Microsoft sample image will provision and serve happily, but it is not your image.

Step 11: *Open the FQDN in a browser*

This is the moment that turns CI/CD from theory into something tangible.

1. **Browse to** the Application Url you copied in the previous step.
2. **Confirm** the homepage renders.
3. **Confirm** the `build:` badge shows the commit SHA from your most recent push (a long hex string), not `local` or `dev-local`. That proves the image running in Azure is the one your pipeline built.
4. **Confirm** the `host:` badge shows a generated container name (something like `ca-cicd-week4--xxxxxxx`). That is the replica's hostname inside the Container Apps environment.

QUICK CHECK:

The page loads and the `build:` badge matches the SHA of the commit shown at `gh repo view --web` → latest commit.

Step 12: *Push a change and manually update the Container App*

You now have a working build pipeline and a running app, but no automated deploy. Make a code change, push it, watch the new image appear on Docker Hub, and then — by hand — point the Container App at the new tag. This is the step that should feel awkward; the next exercise removes it.

1. **Edit** the homepage greeting:

   > `CloudCi/Views/Home/Index.cshtml`

   ```
   <h1 class="display-4">Welcome (v2)</h1>
   <p>Hello again from CloudCi.</p>
   ```

2. **Commit and push:**

   ```
   git add Views/Home/Index.cshtml
   git commit -m "Update homepage greeting to v2"
   git push
   ```

3. **Watch** the new workflow run with `gh run watch` and confirm it ends green.
4. **Confirm** a new SHA tag appeared at `https://hub.docker.com/r/<your-dockerhub-username>/cloudci/tags`.
5. **Refresh** the Container App's Application Url. You will most likely still see the old greeting — Container Apps caches the resolved image digest of `:latest` and does not re-pull on its own.
6. **Force** the new image to roll out via the Portal:
   - Open the Container App **ca-cicd-week4** in the Portal
   - In the left menu, click **Revisions and replicas**
   - Click **Create new revision**

- Click into the **cloudci** container row, set the **Image and tag** to the new SHA tag (or keep `:latest` and Azure will re-resolve the digest)
- Click **Save** at the container level, then **Create** at the revision level

7. **Refresh** the Application Url again. The new greeting **Welcome (v2)** appears.

> **CONCEPT DEEP DIVE**
>
> Why is this not real CD yet? The build half is automated — every commit produces an image. The deploy half is a Portal click. Until something tells Container Apps to roll out a new revision, the production environment keeps serving the old image regardless of what is on Docker Hub. The next exercise replaces that click with another GitHub Actions step that calls the Azure CLI; it also switches from the public Docker Hub registry to a private Azure Container Registry, since most production images are not something you want the whole internet to be able to pull.
>
> ⚠ **Common Mistakes**
>
> - Refreshing the FQDN immediately after the workflow finishes and concluding the deploy "worked." The page still shows the old greeting; Container Apps did not re-pull. The point of this step is to feel that gap, not paper over it.
> - Creating a new revision but leaving the image tag pointing at the old SHA. Always confirm the tag in the revision editor matches the latest run.

Step 13: *Test Your Implementation*

Walk through the full pipeline once more from a clean slate to make sure every piece works end to end.

1. **Make** a third visible code change in `Views/Home/Index.cshtml` — for example change the `<p>` text to `Pipeline run #3.`
2. **Commit and push:**

```
git add Views/Home/Index.cshtml
git commit -m "Pipeline run 3"
git push
```

3. **Watch** the workflow:

```
gh run watch
```

4. **Verify** the Docker Hub repository at `https://hub.docker.com/r/<your-dockerhub-username>/cloudci/tags` lists a fresh SHA tag matching `git rev-parse HEAD`.
5. **Create** a new revision in the Container App pointing at the new SHA (or `:latest`).
6. **Browse** to the Application Url and verify the new copy is live.
7. **Stop** any local containers you no longer need:

```
docker ps
docker stop <container-id>
```

√ ***Success indicators:***

- *Every* `git push` *to* `main` *produces a green Actions run that ends in two new Docker Hub tags*
- *The build SHA shown in the page badge matches the Git commit you pushed*
- *Updating the Container App revision rolls the running app forward*
- *No credentials appear in the workflow file, the repository, or the logs*

√ ***Final verification checklist:***

- [ ] `Dockerfile` *and* `.dockerignore` *exist at the project root*
- [ ] `Views/Home/Index.cshtml` *displays the* `BUILD_SHA` *badge*
- [ ] *GitHub repository contains* `.github/workflows/ci.yml`
- [ ] `DOCKERHUB_USERNAME` *and* `DOCKERHUB_TOKEN` *exist in* `gh secret list`
- [ ] *Docker Hub repository* `<your-dockerhub-username>/cloudci` *shows* `latest` *plus one tag per commit*
- [ ] *Azure Container App is running and serving the latest image at its FQDN*
- [ ]

> *Manually creating a new revision in the Portal rolls the homepage forward*

6.2.4 Common Issues

> ***If you encounter problems:***
> **`denied: requested access to the resource is denied`** ***in the Actions log:*** *Either the* `DOCKERHUB_USERNAME` *does not match the image namespace in the tag, or the PAT was revoked or has expired. Regenerate the token, update the secret, and re-run.*
> ***Workflow run never starts:*** *YAML indentation is almost always the culprit. Open the file in an editor with YAML support and look for tabs (use spaces only) or misaligned* `with:` *blocks.*
> ***Container App returns 502 Bad Gateway:*** *The target port is wrong. Open the Container App → Ingress and confirm* ***Target port*** *is* `8080`*, not the default* `80`*.*
> ***Container App returns 404 on the FQDN:*** *Ingress is disabled, or the revision is unhealthy. Check Revisions and replicas — if a revision is showing 0 replicas, click into it and read the system logs.*
> ***The page renders but the badge still says*** **`build: local`***:* *The* `--build-arg` *is not being passed. Check the* `build-args` *block in the workflow has* `BUILD_SHA=${{ github.sha }}` *on its own line and that the indentation matches.*
> ***Still stuck?*** *Re-run the workflow from the Actions tab with* ***Re-run jobs → Enable debug logging****. The verbose output usually points straight at the problem.*

6.2.5 Summary

You've built the smallest CI/CD pipeline that does something useful:

- ✓ A multi-stage Dockerfile produces a small runtime image with the build commit baked in
- ✓ A GitHub Actions workflow logs in to Docker Hub with a scoped token and pushes the image on every commit
- ✓ Azure Container Apps serves that image at a stable public FQDN
- ✓ The build SHA badge on the homepage gives you a one-glance answer to "is the new code live yet?"

> **KEY TAKEAWAY:**
>
> A pipeline is just a script that runs on someone else's computer with credentials you trusted it with. The interesting design decisions are about which credentials to hand over, how to scope them, and where the manual steps remain. You found one of those manual steps in the deploy half — that is the gap the next exercise closes.

6.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add a second job to the workflow that runs* `dotnet test` *before the image is built, so a failing test blocks the push*
> - *Replace the* `:latest` *tag with a Git tag-driven release scheme (*`v1.0.0`*,* `v1.0.1`*) and configure the Container App to follow a specific tag*
> - *Read the Docker Hub rate-limit policy for anonymous pulls — Container Apps will hit it eventually if you scale out*
> - *Inspect the final image's contents with* `docker run --rm -it cloudci:local sh` *and see what is actually shipped*

6.2.7 Done!

The pipeline builds and the app runs. Next exercise: switch to a private Azure Container Registry and let the workflow trigger the deploy itself, so a `git push` to `main` is all it takes to roll a new revision into production.

6.3 Private Registry and Automated Deployment

6.4 Private Registry and Automated Deployment

6.4.1 Goal

In the previous exercise you built a pipeline that pushes images to Docker Hub and an Azure Container App that pulls

them. That gets you a build, but every release still needs a human to run `az containerapp update`, and your image is sitting in a public registry where anyone in the world can pull it. In this exercise you'll move the image into a **private** registry (Azure Container Registry), let the pipeline **deploy** the new image for you, and add a **smoke test** so a broken release fails the build instead of silently going live.

The new picture looks like this: GitHub Actions logs into Azure with a **service principal**, pushes the image into ACR, then issues `az containerapp update` to roll the Container App forward. The Container App itself uses its own **managed identity** to pull from the registry. A second job in the workflow then curls the public FQDN and fails the build if the response isn't `200`.

WHAT YOU'LL LEARN:

- Why a private registry matters for production and how ACR fits into a CI/CD flow
- The "robot account" pattern: service principals for non-human auth from outside Azure
- How `azure/login@v2` consumes service principal JSON to authorize `az` commands in GitHub Actions
- How to wire an existing Container App to pull from a private registry using a system-assigned managed identity
- How to deploy a new image from the pipeline with `az containerapp update`
- Why a smoke test is the missing piece between "image pushed" and "deployment succeeded"

6.4.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- √ A working pipeline from the previous exercise that builds and pushes to Docker Hub
- √ A live Azure Container App pulling from that public Docker Hub image
- √ The Azure CLI signed in (`az account show` returns your subscription)
- √ The GitHub CLI signed in (`gh auth status` reports a logged-in user)
- √ Owner or User Access Administrator rights on the resource group `rg-cicd-week4` (you need to create role assignments)
- √ `jq` installed locally (used to read fields out of JSON output; install with `brew install jq` on macOS or `sudo apt install jq` on Ubuntu)

6.4.3 Exercise Steps

Overview

1. **Create the Azure Container Registry**
2. **Create a service principal for the pipeline**
3. **Store the service principal JSON as a GitHub secret**
4. **Update the workflow to push to ACR**
5. **Trigger the pipeline and verify the image lands in ACR**
6. **Reconfigure the Container App to pull from ACR**
7. **Confirm the app still serves from the new registry**
8. **Add an automated deploy step to the workflow**
9. **Push a UI change and watch it deploy automatically**
10. **Add a smoke-test job to the workflow**
11. **Break the deploy briefly to see the smoke test fail**

12. **Test Your Implementation**

Step 1: *Create the Azure Container Registry*

The registry is the private bucket your pipeline pushes images into and your Container App pulls them out of. ACR is a managed service — you don't run it, you just point Azure resources at it. The Basic SKU is enough for coursework: it's the cheapest tier and the feature set is identical for what we need (push, pull, role-based access).

1. **Make sure** the resource group from the previous exercise exists. The command is idempotent — if it's already there, nothing happens:

```
az group create -n rg-cicd-week4 -l northeurope
```

2. **Choose** a globally unique ACR name. ACR names must be 5–50 characters, lowercase letters and digits only, and unique across all of Azure. A safe pattern is `acr` plus your initials plus four random digits, for example `acrlap4827`:

```
ACR_NAME=acrlap4827
```

3. **Create** the registry:

```
az acr create -n $ACR_NAME -g rg-cicd-week4 --sku Basic
```

4. **Capture** its full resource ID — you'll need it for role assignments in the next step:

```
ACR_ID=$(az acr show -n $ACR_NAME -g rg-cicd-week4 --
         query id -o tsv)
echo $ACR_ID
```

CONCEPT DEEP DIVE

A registry is just an OCI-compliant API in front of blob storage. Docker Hub is one; GitHub Container Registry is another; ACR is Azure's. The reason to prefer ACR for production is integration: ACR understands Azure RBAC, supports managed identities for pulling, and lives in the same network and billing context as the workload that consumes it. For a public open-source image, Docker Hub is fine. For a closed-source production image, you want the registry to enforce who can read it.

⚠ **Common Mistakes**

- The registry name must be globally unique. If `az acr create` fails with `already in use`, pick another name.
- Hyphens and underscores are not allowed — only lowercase letters and digits.

✓ **Quick check:** `az acr show -n $ACR_NAME -g rg-cicd-week4` returns JSON describing the registry.

Step 2: *Create a service principal for the pipeline*

A **service principal** is an Azure-managed account that lives outside Azure — exactly what GitHub Actions needs in order to call `az` commands without a human typing a password. You create it once, hand the credentials to GitHub as a secret, and from then on every workflow run authenticates with it. The trick is to scope it tightly: it should be allowed to push images to *your* ACR and update *your* Container App, and nothing else.

1. **Create** the service principal with permission to push to your registry. The `--json-auth` flag formats the output exactly the way the `azure/login@v2` action expects:

```
az ad sp create-for-rbac \
  --name "github-cicd-sp" \
  --role AcrPush \
  --scopes $ACR_ID \
  --json-auth > creds.json
```

2. **Look at** the file (you'll delete it in the next step). It contains five fields — `clientId`, `clientSecret`, `subscriptionId`, `tenantId`, and a couple of endpoint URLs:

```
cat creds.json
```

3. **Capture** the `clientId` and the resource ID of your Container App — you need both for the second role assignment:

```
APP_ID=$(jq -r .clientId creds.json)
CA_ID=$(az containerapp show -g rg-cicd-week4 -n <ca-
        name> --query id -o tsv)
```

   Replace `<ca-name>` with the Container App name you used in the previous exercise.

4. **Grant** the service principal permission to update the Container App:

```
az role assignment create \
  --assignee "$APP_ID" \
  --role "Container Apps Contributor" \
  --scope $CA_ID
```

CONCEPT DEEP DIVE

The service principal needs **two** roles because it does two different things: `AcrPush` lets it push images to the registry, and `Container Apps Contributor` lets it issue `az containerapp update`. Granting both at the smallest possible scope (this one ACR, this one Container App) is the principle of least privilege. If you scoped `AcrPush` to the subscription instead, this principal could push to *every* registry in your subscription — not what you want.

⚠ **Common Mistakes**

- Granting `Contributor` on the resource group is overkill. Stick with the narrow built-in roles.
- The `clientSecret` in `creds.json` is a long-lived password. Treat it like one.

✓ **Quick check:** `az role assignment list --assignee $APP_ID -o table` lists two assignments — one on the ACR, one on the Container App.

Step 3: *Store the service principal JSON as a GitHub secret*

`creds.json` is a credential. It does not belong in your repo, your shell history, or your screenshots. The right place for it is GitHub Actions secrets, where it's encrypted at rest and only injected as an environment variable at workflow runtime.

1. **Set** the secret directly from the file using the GitHub CLI:

```
gh secret set AZURE_CREDENTIALS < creds.json
```

2. **Delete** the local file immediately:

```
rm creds.json
```

3. **Verify** the secret is registered:

```
gh secret list
```

You should see `AZURE_CREDENTIALS` in the output. (You can never read its value back — that's by design.)

> **CONCEPT DEEP DIVE**
>
> GitHub Actions secrets are encrypted with the repository's public key the moment you set them. Workflow runs see them as environment variables only inside steps that explicitly reference `${{ secrets.NAME }}`. They're masked in logs, but a malicious workflow could still print them — secrets are a deterrent against accidents, not against a hostile pull request.
>
> ⚠ **Common Mistakes**
>
> - Pasting the JSON into the GitHub web UI works but is hard to get right (newlines matter). The `gh secret set ... < file` form is reliable.
> - Forgetting to delete the local file is the single most common way these credentials leak. Get used to deleting it as part of the same command: `gh secret set AZURE_CREDENTIALS < creds.json && rm creds.json`.

Step 4: *Update the workflow to push to ACR*

The build step doesn't change — Docker doesn't care which registry it's pushing to. What changes is the **login** step (you authenticate to Azure, then have Docker authenticate to ACR through it) and the **image tag** (`<acr-name>.azurecr.io/<repo>:<tag>` instead of `docker.io/<user>/<repo>:<tag>`).

1. **Open** the workflow file from the previous exercise:

> `.github/workflows/ci.yml`

2. **Replace** the Docker Hub login and build steps with the ACR variants. Set `ACR_NAME` and `IMAGE_NAME` at the top of the job so you can reuse them:

> `.github/workflows/ci.yml`

```yaml
name: Build and deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

env:
  ACR_NAME: acrlap4827
  IMAGE_NAME: cloudci
  RESOURCE_GROUP: rg-cicd-week4
  CONTAINER_APP: ca-cicd-week4

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Check out source
        uses: actions/checkout@v4

      - name: Log in to Azure
        uses: azure/login@v2
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Log in to ACR
        run: az acr login --name $ACR_NAME

      - name: Build and push image
        run: |
          docker build \
            --build-arg BUILD_SHA=${{ github.sha }} \
            -t $ACR_NAME.azurecr.io/$IMAGE_NAME:$
        {{ github.sha }} \
            -t $ACR_NAME.azurecr.io/$IMAGE_NAME:latest \
            .
          docker push $ACR_NAME.azurecr.io/$IMAGE_NAME:$
        {{ github.sha }}
          docker push $ACR_NAME.azurecr.io/
        $IMAGE_NAME:latest
```

3. **Replace** `acrlap4827` and `ca-cicd-week4` with your actual ACR and Container App names.
4. **Remove** the `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` secrets from the repo — you don't need them anymore:

```
gh secret delete DOCKERHUB_USERNAME
gh secret delete DOCKERHUB_TOKEN
```

> **CONCEPT DEEP DIVE**
>
> Tagging the image with **both** `${{ github.sha }}` and `latest` is the conventional pattern. The SHA tag is immutable: `acrlap4827.azurecr.io/cloudci:a1b2c3d` will always point to the exact image built from commit `a1b2c3d`. The `latest` tag is a moving pointer to whatever was pushed most recently. You'll deploy by SHA below, because that's the only way to roll back without rebuilding.
>
> ⚠ **Common Mistakes**
>
> - `azure/login@v2` will fail with `Login failed` if the secret JSON is malformed. The `--json-auth` flag in step 2 is what produced the right shape.
> - `az acr login` only works *after* `azure/login` — it reuses the Azure session.

Step 5: *Trigger the pipeline and verify the image lands in ACR*

The first run of the new workflow proves three things at once: the service principal works, ACR is reachable from GitHub's runners, and your image makes it across.

1. **Commit and push** the workflow change:

```
git add .github/workflows/ci.yml
git commit -m "Switch CI from Docker Hub to ACR"
git push
```

2. **Watch** the run from the terminal:

```
gh run watch
```

   The job should finish green in a couple of minutes.

3. **Confirm** the image is in ACR:

```
az acr repository show-tags -n $ACR_NAME --repository
        cloudci -o table
```

   You should see two tags: `latest` and the commit SHA.

> **QUICK CHECK:**
> Both tags appear in `az acr repository show-tags`. The Container App is still running the old Docker Hub image — that's expected; you fix it in the next step.

Step 6: *Reconfigure the Container App to pull from ACR*

The pipeline can now push to a private registry, but the Container App still has its `image:` field pointing at Docker Hub. Worse, even if you change the field, the Container App can't pull from a private ACR without credentials. The right way to give it those credentials is **not** to share the service principal — that's GitHub's account, not the Container App's. Each resource gets its own identity.

The cleanest pattern is a **system-assigned managed identity** on the Container App: an Azure-managed account whose lifecycle is tied to the resource itself. When the Container App is deleted, the identity disappears with it. You grant that identity `AcrPull` on the registry and tell the Container App to use it.

1. **Enable** a system-assigned managed identity on the Container App:

```
az containerapp identity assign \
  -g rg-cicd-week4 \
  -n $CONTAINER_APP \
  --system-assigned
```

Replace `$CONTAINER_APP` with your Container App name (or set it: `CONTAINER_APP=ca-cicd-week4`).

2. **Capture** the identity's principal ID:

```
PRINCIPAL_ID=$(az containerapp show \
  -g rg-cicd-week4 -n $CONTAINER_APP \
  --query identity.principalId -o tsv)
```

3. **Grant** the identity `AcrPull` on the registry:

```
az role assignment create \
  --assignee "$PRINCIPAL_ID" \
  --role AcrPull \
  --scope $ACR_ID
```

4. **Tell** the Container App to use that identity when pulling from ACR:

```
az containerapp registry set \
  -g rg-cicd-week4 \
  -n $CONTAINER_APP \
  --server $ACR_NAME.azurecr.io \
  --identity system
```

5. **Update** the Container App's image to the one you just pushed:

```
az containerapp update \
  -g rg-cicd-week4 \
  -n $CONTAINER_APP \
  --image $ACR_NAME.azurecr.io/cloudci:latest
```

CONCEPT DEEP DIVE

There are two non-human accounts in this exercise and they do different jobs. The **service principal** lives outside Azure and is used by GitHub to push images and update the Container App. The **managed identity** lives on the Container App resource and is used by the Container App to pull images. Mixing them up — for example, giving the service principal `AcrPull` and using it as the registry identity — works but is conceptually wrong: you'd be sharing GitHub's credential with an Azure resource that has its own.

⚠ **Common Mistakes**

- Skipping the `AcrPull` role assignment: the Container App will fail to pull and you'll see `ImagePullBackOff` or a perpetual "activating" status.
- Forgetting `--identity system` on `containerapp registry set`: the Container App will keep using whatever username/password it was using before (often nothing, if you started from a public Docker Hub image).

√ **Quick check:** `az containerapp show -g rg-cicd-week4 -n $CONTAINER_APP --query properties.template.containers[0].image` reports the ACR image path.

Step 7: *Confirm the app still serves from the new registry*

Before adding more automation, verify the manual cutover worked. The user-facing behaviour shouldn't have changed at all.

1. **Get** the FQDN of the Container App:

```
FQDN=$(az containerapp show \
  -g rg-cicd-week4 -n $CONTAINER_APP \
  --query properties.configuration.ingress.fqdn -o tsv)
echo https://$FQDN
```

2. **Curl** the home page and confirm a `200`:

```
curl -I https://$FQDN/
```

3. **Open** the URL in a browser. The app should look identical to before — same SHA badge, same content.

> **QUICK CHECK:**
> `curl -I` returns `HTTP/2 200`. The app loads in the browser.

Step 8: *Add an automated deploy step to the workflow*

Right now your pipeline pushes a new image but doesn't tell the Container App to use it. Adding one more step closes the loop. The step issues `az containerapp update` with the SHA-tagged image — the same SHA that was just built — so the version that's live is unambiguous.

1. **Open** the workflow file again:

   > `.github/workflows/ci.yml`

2. **Add** a new step after the build/push step:

   > `.github/workflows/ci.yml`

```
      - name: Deploy to Container App
        run: |
          az containerapp update \
            -g $RESOURCE_GROUP \
            -n $CONTAINER_APP \
            --image $ACR_NAME.azurecr.io/$IMAGE_NAME:$
        {{ github.sha }}
```

3. **Commit and push:**

```
git add .github/workflows/ci.yml
git commit -m "Auto-deploy to Container App after push"
git push
```

CONCEPT DEEP DIVE

Deploying by SHA, not by `:latest`, is what makes this pipeline auditable. If something goes wrong in production at 14:32, you can read the GitHub Actions log, find which commit was built at 14:30, and roll back by issuing `az containerapp update --image ...:<previous-sha>`. With `:latest` you'd have no idea which image is actually running.

⚠ **Common Mistakes**

- Deploying with `--image ...:latest`: works, but defeats traceability and makes rollback impossible without rebuild.
- Forgetting that `az containerapp update` is asynchronous: the command returns when Azure accepts the request, not when the new revision is healthy. The smoke test in step 10 covers that gap.

Step 9: *Push a UI change and watch the pipeline deploy automatically*

This is the moment the pipeline pays off. From now on, every commit on `main` flows all the way to production without you typing `az` once.

1. **Edit** the home page heading. Open `Views/Home/Index.cshtml` and change something visible — for example the welcome heading.
2. **Commit and push:**

```
git add Views/Home/Index.cshtml
git commit -m "Tweak home page heading"
git push
```

3. **Watch** the pipeline:

```
gh run watch
```

4. **Refresh** the FQDN in your browser. Your change is live.

> **QUICK CHECK:**
> The new heading appears at `https://$FQDN/` within a minute or two of the workflow finishing.

Step 10: *Add a smoke-test job to the workflow*

`az containerapp update` returning success doesn't mean the new revision is actually serving traffic. Maybe the image starts but throws on the first request. Maybe a config value is missing. A **smoke test** is the cheapest possible safety net: hit the public URL after deploy, fail the pipeline if it doesn't return `200`. That single check catches a surprising fraction of bad releases.

1. **Open** the workflow file:

> `.github/workflows/ci.yml`

2. **Add** a second job below the existing `deploy` job. Use `needs: deploy` to make it wait. Replace `<your-fqdn>` with the FQDN you captured in step 7:

> `.github/workflows/ci.yml`

```yaml
  smoke-test:
    runs-on: ubuntu-latest
    needs: deploy
    steps:
      - name: Curl the home page
        run: |
          curl --fail \
            --max-time 30 \
            --retry 5 \
            --retry-delay 5 \
            --retry-all-errors \
            https://<your-fqdn>/
```

3. **Commit and push:**

```bash
git add .github/workflows/ci.yml
git commit -m "Add smoke test after deploy"
git push
```

4. **Watch** the run. Both jobs should turn green:

```bash
gh run watch
```

CONCEPT DEEP DIVE

`--fail` makes `curl` return a non-zero exit code on any HTTP error (4xx, 5xx) — without it, `curl` happily prints the error page and exits `0`. `--retry-all-errors` plus `--retry-delay 5` gives the new revision a fair chance to come up: Container Apps can take 30–60 seconds to swap revisions in production. `needs: deploy` is the GitHub Actions way to express "don't run this until the deploy job succeeds" — if deploy fails, smoke-test is skipped, and the workflow as a whole is marked failed.

⚠ **Common Mistakes**

- Calling `curl https://...` without `--fail`: a 500 response will pass.
- Forgetting `--max-time`: a hung server will spin the runner forever.
- Hard-coding the FQDN here is fine for one Container App. For multiple environments, pass it via a workflow output or `vars`.

Step 11: *Break the deploy briefly to see the smoke test fail*

A safety net you've never seen catch anything is one you don't trust. Force a failure on purpose, watch the smoke test fire, then restore.

1. **Edit** the workflow file. In the `Deploy to Container App` step, change the image tag from `${{ github.sha }}` to a tag that doesn't exist, e.g. `does-not-exist`:

> `.github/workflows/ci.yml`

```
      - name: Deploy to Container App
        run: |
          az containerapp update \
            -g $RESOURCE_GROUP \
            -n $CONTAINER_APP \
            --image $ACR_NAME.azurecr.io/
        $IMAGE_NAME:does-not-exist
```

2. **Commit and push.** Watch the run:

```
git add .github/workflows/ci.yml
git commit -m "Intentionally break deploy to test smoke
        gate"
git push
gh run watch
```

3. **Observe** what happens. The `deploy` job may itself fail (the image isn't in the registry), or it may succeed but the new revision will fail to start. Either way, the `smoke-test` job either fails (curl gets a non-200) or is skipped (deploy didn't succeed). The workflow as a whole turns red.
4. **Restore** the working tag and push again:

> `.github/workflows/ci.yml`

```
      - name: Deploy to Container App
        run: |
          az containerapp update \
            -g $RESOURCE_GROUP \
            -n $CONTAINER_APP \
            --image $ACR_NAME.azurecr.io/$IMAGE_NAME:$
        {{ github.sha }}
```

```
git add .github/workflows/ci.yml
git commit -m "Restore working deploy"
git push
```

> **QUICK CHECK:**
> The deliberately-broken run is red on GitHub's run list; the recovery run is green. The Container App keeps serving the previous good revision throughout — broken deploys do not bring the site down.

Step 12: *Test Your Implementation*

Step back and verify the whole pipeline behaves the way you expect.

1. **Run** the workflow once more by pushing a trivial change (for example, edit a comment in `Views/Home/Index.cshtml`):

```
git commit --allow-empty -m "Trigger pipeline"
git push
gh run watch
```

2. **Confirm** both jobs ran:
   - `deploy` is green
   - `smoke-test` is green
3. **Inspect** ACR. The new SHA should be present:

```
az acr repository show-tags -n $ACR_NAME --repository
        cloudci -o table
```

4. **Inspect** the running Container App. The image should match the SHA you just pushed:

```
az containerapp show \
  -g rg-cicd-week4 -n $CONTAINER_APP \
  --query properties.template.containers[0].image -o tsv
```

5. **Visit** `https://$FQDN/` — the app responds and (if your home page shows the SHA badge from the previous exercise) the badge matches the new commit.
6. **Reflect** on the secret you stored. `AZURE_CREDENTIALS` is a JSON blob containing a long-lived `clientSecret` — a password, effectively. Anyone who exfiltrates it (a compromised dependency in a workflow, a copy-pasted log) can push images to your registry and update your Container App until you rotate it. The next exercise removes that risk by replacing the long-lived secret with short-lived OIDC tokens that GitHub mints on every run.

√ ***Success indicators:***

- *The pipeline pushes to ACR (not Docker Hub) and the registry contains both* `latest` *and a SHA tag for the latest commit*
- *The Container App pulls from ACR using its system-assigned managed identity*
- *Pushing to* `main` *deploys automatically — no manual* `az containerapp update`
- *The smoke test fails the pipeline when the deploy is broken*
- *The home page reflects the latest commit's UI changes within minutes of* `git push`

√ ***Final verification checklist:***

- [ ] *ACR exists and holds at least two tags*
- [ ] *Service principal has only* `AcrPush` *on the registry and* `Container Apps Contributor` *on the Container App*
- [ ] `AZURE_CREDENTIALS` *is set as a GitHub secret;* `creds.json` *is not on disk*
- [ ] *Container App has system-assigned managed identity with* `AcrPull` *on the registry*
- [ ] *Workflow has a deploy step that uses the SHA tag*
- [ ] *Workflow has a* `smoke-test` *job with* `needs: deploy`

> ☐
>
> *A green pipeline run exists with both jobs passing*

6.4.4 Common Issues

> ***If you encounter problems:***
> **`azure/login@v2` *fails with "Login failed":*** *The* `AZURE_CREDENTIALS` *secret is malformed. Re-run* `az ad sp create-for-rbac --json-auth` *and re-set the secret with* `gh secret set AZURE_CREDENTIALS < creds.json`*.*
> **`az acr login` *fails with "unauthorized":*** *The service principal lacks* `AcrPush` *on the registry. Confirm with* `az role assignment list --assignee $APP_ID --scope $ACR_ID`*.*
> **`az containerapp update` *succeeds but the app shows the old content:*** *Container Apps swap revisions asynchronously. Wait a minute and refresh. If the new revision is unhealthy, the platform keeps the old one serving traffic — check* `az containerapp revision list` *for one with* `Failed` *provisioning state.*
> ***Container App shows "ImagePullBackOff" or stays in "Activating":*** *The Container App's managed identity doesn't have* `AcrPull` *on the registry, or the* `--identity system` *flag wasn't used on* `az containerapp registry set`*. Re-run both commands from step 6.*
> ***Smoke test fails with "curl: (28) Operation timed out":*** *The Container App's new revision didn't come up in time. Increase* `--max-time`*, or check the app's logs with* `az containerapp logs show -g rg-cicd-week4 -n $CONTAINER_APP --follow`*.*
> ***Still stuck?*** *Re-read the role assignments in step 2 and step 6. Most failures in this exercise are RBAC mismatches: the principal isn't who you*

> *think it is, or it doesn't have the role you think it has, or it has it on the wrong scope.*

6.4.5 Summary

You replaced a public registry with a private one, gave GitHub Actions a scoped identity to push and deploy, gave the Container App its own identity to pull, and put a smoke test in front of the live URL. The pipeline now does end-to-end what it should: every push results in a tagged image in ACR, a Container App revision running that exact image, and an automated check that the app is actually serving.

- √ Private registry (ACR) instead of public Docker Hub
- √ Service principal for non-human auth from outside Azure
- √ System-assigned managed identity for the Container App's pull
- √ Auto-deploy step using the immutable SHA tag
- √ Smoke test gating the pipeline on real HTTP behaviour

> **KEY TAKEAWAY:**
>
> A pipeline that pushes an image is half a pipeline. The other half is deploying that image and proving the deploy worked. Until both halves are automated, "shipping to production" is still a manual ritual — and it stays unreliable as long as humans are in the loop for the routine path.

6.4.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add a manual approval gate before the deploy job (GitHub Environments) so production releases require a click*
> - *Read the Container App revision logs after a deploy and surface them as a workflow artifact*
> - *Replace* `:latest` *everywhere with the SHA tag and observe how rollback becomes a one-line command*
> - *Investigate* `az containerapp revision copy` *for blue/green deploys without downtime*

6.4.7 Done!

The pipeline now ships code on its own. There's still one weak link: the `AZURE_CREDENTIALS` secret is a long-lived password sitting in your repository's secret store. The next exercise removes it entirely by replacing it with **OpenID Connect federation** — short-lived tokens minted per workflow run, tied to your specific repository and branch.

6.5 Passwordless Deployment with OIDC Federation

6.6 Passwordless Deployment with OIDC Federation

6.6.1 Goal

In the previous exercise you stored a service principal's client secret as `AZURE_CREDENTIALS` in GitHub. The pipeline worked, but the secret was a long-lived password sitting in two places (Azure and GitHub) with no automatic rotation and no scoping beyond "whoever holds it can deploy." In this exercise you'll replace that password with a *trust relationship* — and the pipeline keeps working.

You will configure **OpenID Connect (OIDC) federation** between GitHub Actions and Microsoft Entra ID. On every workflow run, GitHub mints a short-lived token describing the run (which repo, which branch, which event). Azure trusts that token because of a federated credential you registered up front. No password ever leaves Azure; no password ever sits in GitHub.

WHAT YOU'LL LEARN:

- How OIDC federation replaces a long-lived secret with a short-lived token exchange
- The difference between an Entra **app registration**, a **service principal**, and a **federated credential**
- How the `subject` claim scopes who is allowed to authenticate as your service principal
- Why GitHub workflows need `permissions: id-token: write` to participate
- How `azure/login@v2` switches from secret-based to federated authentication

6.6.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed the previous exercise: a working pipeline that uses `AZURE_CREDENTIALS` to push to ACR and update a Container App on every push to `main`
- ✓ Owner (or User Access Administrator) on the subscription, **and** permission to create app registrations in Entra ID
- ✓ `az` CLI logged in (`az login`) and `gh` CLI logged in (`gh auth status`)
- ✓ Your existing ACR name, Container App name, and resource group name handy
- ✓ Your GitHub repository slug in the form `<your-username>/<your-repo>`

6.6.3 Exercise Steps

Overview

1. **Confirm your Azure permissions**

2. **Create the Entra app registration**
3. **Create a service principal for the app**
4. **Assign the deployment roles to the new identity**
5. **Create the federated credential for `main`**
6. **Add the three new GitHub secrets (no client secret)**
7. **Grant the workflow `id-token: write` permission**
8. **Switch `azure/login@v2` to federated authentication**
9. **Push and watch the pipeline deploy with no password**
10. **Delete the old `AZURE_CREDENTIALS` secret**
11. **Inspect the trust relationship**
12. **Reflect on subject-claim scope**
13. **Test Your Implementation**

Step 1: *Confirm your Azure permissions*

Federation requires you to create an Entra app registration and assign roles on Azure resources. Both privileges are tightly held in real organisations — verifying you have them now saves you a confusing error halfway through.

1. **List your role assignments:**

```
az role assignment list \
  --assignee "$(az ad signed-in-user show --query id -o
        tsv)" \
  --output table
```

2. **Confirm** at least one of the following appears:
   - `Owner` on the subscription (or on your resource group)
   - `User Access Administrator` on the subscription (or on your resource group)
3. **Verify** you can create app registrations. In the Entra admin centre this is governed by the **Application Developer** role or by the tenant-wide "Users can register applications"

toggle. If `az ad app create` later fails with `Insufficient privileges`, that's the toggle you're missing.

> **CONCEPT DEEP DIVE**
>
> Two privilege boundaries are at play. Creating app registrations is a tenant-level Entra ID permission. Assigning roles to the resulting identity is a subscription-level Azure RBAC permission. These are separate systems and a user often has one without the other. In a corporate tenant, ask a directory admin to grant Application Developer; ask a subscription owner to grant you a scoped role assignment, ideally only on the resource group you'll deploy into.
>
> ✓ **Quick check:** The role list includes Owner or User Access Administrator on the scope you intend to deploy to.

Step 2: *Create the Entra app registration*

An **app registration** is the *definition* of an identity in Entra ID — a globally unique application object describing "an application that wants to authenticate." It does not, by itself, have any permissions in any Azure subscription. Think of it as the blueprint.

1. **Create** the app registration:

```
az ad app create \
  --display-name "github-cicd-oidc"
```

2. **Capture** the `appId` value from the output. You will use it many times. Store it in a shell variable:

```
export APP_ID="<paste appId here>"
echo "$APP_ID"
```

CONCEPT DEEP DIVE

The naming convention `<purpose>-<auth-type>` (here, `github-cicd-oidc`) makes it obvious in the Entra portal why the registration exists and how it authenticates. Production setups typically split per environment — `github-cicd-oidc-dev`, `-staging`, `-prod` — each with its own role assignments and federated credentials. For this lab, one registration is enough.

⚠ **Common Mistakes**

- The `appId` returned here is **not** the same as the registration's `id` (object ID). The `appId` is the *client ID* used by tokens; the `id` is the directory object identifier. The federated-credential commands and the GitHub workflow both use `appId`.

√ **Quick check:** `az ad app show --id "$APP_ID" --query displayName -o tsv` prints `github-cicd-oidc`.

Step 3: *Create a service principal for the app*

The app registration is the blueprint; the **service principal** is the instance you actually grant permissions to in your subscription. Without a service principal, role assignments have nothing to attach to. One app registration produces one service principal per tenant.

1. **Create** the service principal:

```
az ad sp create --id "$APP_ID"
```

2. **Note** that the output contains both `id` (the SP's object ID, used for some directory operations) and `appId` (the same client ID you already have). Role assignments below use `appId`.

> **CONCEPT DEEP DIVE**
>
> The split between app registration and service principal is one of Entra ID's most confusing details. The mental model: the app registration is published once in your home tenant; if a multi-tenant app is consented to in a partner tenant, *that* tenant gets its own service principal pointing at your app registration. For single-tenant CI/CD the distinction is mostly bookkeeping, but you can't skip the `az ad sp create` step — without it, role assignments fail with "principal not found."
>
> ✓ **Quick check:** `az ad sp show --id "$APP_ID" --query appId -o tsv` returns the same value as `$APP_ID`.

Step 4: *Assign the deployment roles to the new identity*

The service principal needs the same two roles your old `AZURE_CREDENTIALS` SP had: `AcrPush` to push images to ACR, and `Container Apps Contributor` to update the Container App. Same scopes, same roles — different identity.

1. **Capture** the resource IDs you'll need:

```
export ACR_ID=$(az acr show \
  --name "<your-acr-name>" \
  --query id -o tsv)

export CA_ID=$(az containerapp show \
  --name "<your-container-app-name>" \
  --resource-group "<your-resource-group>" \
  --query id -o tsv)
```

2. **Grant** push access to the registry:

```
az role assignment create \
  --assignee "$APP_ID" \
  --role AcrPush \
  --scope "$ACR_ID"
```

3. **Grant** Container App update access:

```
az role assignment create \
  --assignee "$APP_ID" \
  --role "Container Apps Contributor" \
  --scope "$CA_ID"
```

Role assignments take a few seconds to propagate. If a subsequent step fails with `Principal does not exist in the directory`, wait ~10 seconds and retry — Entra ID is still replicating the new principal.

CONCEPT DEEP DIVE

Note how `--scope` is a specific resource ID, not a subscription. This is **least privilege** in practice: the SP can push to *this* registry and update *this* Container App and nothing else. If you ever need to add a second app or registry, add another role assignment with that resource's scope rather than widening this one.

⚠ **Common Mistakes**

- Assigning `Contributor` at subscription scope works but gives the SP authority to delete every resource in your subscription. The pipeline only needs the two roles above.
- Role assignments take **a few seconds to propagate**. If the next step fails immediately with an authorization error, retry once after 10 seconds.

√ **Quick check:** `az role assignment list --assignee "$APP_ID" --all -o table` lists both `AcrPush` and `Container Apps Contributor` with the expected scopes.

Step 5: *Create the federated credential for* `main`

So far you have an identity and you have permissions. What you don't have is a way to authenticate as that identity. With a service principal secret you'd add a password and ship it to GitHub. With OIDC, you add a **federated credential** — a trust rule that says "if a token from issuer X has claims matching pattern Y, treat the bearer as this app."

1. **Create** the federated credential. Replace `<your-username>/<your-repo>` with your repository slug:

```
az ad app federated-credential create \
  --id "$APP_ID" \
  --parameters '{
    "name": "main-branch",
    "issuer": "https://
        token.actions.githubusercontent.com",
    "subject": "repo:<your-username>/<your-
        repo>:ref:refs/heads/main",
    "audiences": ["api://AzureADTokenExchange"],
    "description": "GitHub Actions, main branch only"
  }'
```

2. **Verify** it exists:

```
az ad app federated-credential list \
  --id "$APP_ID" \
  -o table
```

CONCEPT DEEP DIVE

Each field of the federated credential pins down one half of the trust handshake.

- `issuer` — only tokens signed by GitHub Actions' OIDC issuer are accepted. Azure validates the JWT signature against GitHub's published JWKS, so you cannot forge one.
- `audiences` — the token must declare `api://AzureAD-TokenExchange` as its intended audience. The `azure/login` action sets this for you.
- `subject` — an exact-match string GitHub embeds in every token it issues from your workflow. The format `repo:org/name:ref:refs/heads/main` means "from the repo `org/name`, on a workflow run for the ref `refs/heads/main`." Anything else — a different branch, a pull request, a tag — produces a different subject and is silently rejected.

⚠ **Common Mistakes**

- Typing `head` instead of `heads` (it is `refs/heads/main`, the Git ref form) — silent auth failure.
- Wrapping `<your-username>/<your-repo>` in `<>` literally — the angle brackets are placeholders, remove them.
- Creating the federated credential on the wrong app registration. The roles you assigned in Step 4 are tied to `$APP_ID`; the federated credential must be on the same `$APP_ID`.

√ **Quick check:** The list shows one entry named `main-branch` with the subject `repo:<your-username>/<your-repo>:ref:refs/heads/main`.

Step 6: *Add the three new GitHub secrets (no client secret)*

`azure/login@v2` in federated mode needs three pieces of information to ask Azure for a token: which app to

authenticate as (`client-id`), which directory (`tenant-id`), and which subscription to operate against (`subscription-id`). None of these are secrets in the cryptographic sense — they're identifiers — but storing them as GitHub secrets keeps your repo's `.yaml` reusable across forks and clones.

1. **Read** the three values:

```
echo "CLIENT_ID = $APP_ID"
echo "TENANT_ID = $(az account show --query tenantId -o
        tsv)"
echo "SUBSCRIPTION_ID = $(az account show --query id -o
        tsv)"
```

2. **Set** them as repository secrets:

```
gh secret set AZURE_CLIENT_ID \
  --body "$APP_ID"

gh secret set AZURE_TENANT_ID \
  --body "$(az account show --query tenantId -o tsv)"

gh secret set AZURE_SUBSCRIPTION_ID \
  --body "$(az account show --query id -o tsv)"
```

3. **Confirm** they exist:

```
gh secret list
```

You should see all three new names alongside the old `AZURE_CREDENTIALS` (which you'll delete in Step 10).

> **CONCEPT DEEP DIVE**
>
> Notice what is *not* in this list: a client secret. The whole point of OIDC federation is that the secret material — the cryptographic key proving "I am this app" — never leaves Azure. GitHub holds only the app's *name* (its client ID). The proof of identity is freshly minted by GitHub on every run as a short-lived JWT.
>
> ✓ **Quick check:** `gh secret list` shows `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, and `AZURE_SUBSCRIPTION_ID`.

Step 7: *Grant the workflow* `id-token: write` *permission*

GitHub treats issuing OIDC tokens as a security-sensitive operation. By default, no workflow can request one — you must opt in explicitly with a `permissions:` block. This prevents a compromised dependency or a careless `pull_request` workflow from exfiltrating your cloud identity.

1. **Open** your workflow file at `.github/workflows/ci.yml` (or whatever you named it in the previous exercise).
2. **Add** a `permissions` block at the job level. The block goes immediately under the job's `runs-on:` line:

> `.github/workflows/ci.yml`

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4
      # ...remaining steps unchanged for now
```

CONCEPT DEEP DIVE

The two permissions do different things.

- `id-token: write` lets the runner request a fresh OIDC JWT from GitHub's token service. Without it, the `azure/login` step fails with `Error: Failed to retrieve OIDC token`.
- `contents: read` is the default for most workflows but is dropped to nothing once you specify *any* `permissions:` block. Re-adding it keeps `actions/checkout@v4` working.

You can also place `permissions:` at the top of the file (workflow level) instead of inside `jobs.deploy:`. Job-level scoping is the safer default — only this job, not other jobs in the same workflow, can mint tokens.

⚠ **Common Mistakes**

- Adding `permissions:` only at the workflow level but no `id-token: write` — `azure/login` will fail.
- Forgetting `contents: read` after introducing `permissions:` — `actions/checkout@v4` fails with "Resource not accessible by integration."

√ **Quick check:** YAML parses without error. The block is indented inside `jobs.deploy:` at the same level as `steps:`.

Step 8: *Switch* `azure/login@v2` *to federated authentication*

This is the only line of code in the workflow that actually changes. The `azure/login@v2` action accepts two mutually exclusive modes: secret-based (`creds:`) and federated (`client-id` / `tenant-id` / `subscription-id`). You're swapping one for the other.

1. **Locate** the existing `azure/login@v2` step. It currently looks like this:

> `.github/workflows/ci.yml` *(before)*

```yaml
      - name: Sign in to Azure
        uses: azure/login@v2
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}
```

2. **Replace** it with the federated form:

> `.github/workflows/ci.yml` *(after)*

```yaml
      - name: Sign in to Azure
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: $
        {{ secrets.AZURE_SUBSCRIPTION_ID }}
```

Note: no `creds:` line. The action infers federated mode from the *absence* of `creds:` and the presence of the three explicit IDs.

CONCEPT DEEP DIVE

Internally, `azure/login` now does this on every run: it asks the GitHub runner to mint an OIDC JWT (using the `id-token: write` permission you granted in Step 7), then sends that JWT to Entra ID's `/oauth2/token` endpoint with grant type `urn:ietf:params:oauth:grant-type:token-exchange`. Entra ID validates the JWT against the federated credential you created in Step 5, finds a match, and returns a *different* short-lived access token scoped to the service principal. The rest of the workflow uses that access token. The whole exchange takes milliseconds and leaves no persistent secret behind.

⚠ **Common Mistakes**

- Leaving the `creds:` line *and* adding the three IDs — `azure/login` complains about ambiguous configuration.
- Using the wrong secret name (`AZURE_CLIENT_SECRET` from a half-finished tutorial) — there is no client secret in this flow.

√ **Quick check:** The step references three secrets and no longer mentions `creds` or `AZURE_CREDENTIALS`.

Step 9: *Push and watch the pipeline deploy with no password*

The behaviour change is invisible — a successful run looks just like the previous exercise's run. What's *different* is what's in motion: the runner mints a JWT, Azure exchanges it for a token, the token authorises the push to ACR and the update to the Container App.

1. **Commit and push** the workflow change:

```
git add .github/workflows/ci.yml
git commit -m "Switch to OIDC federation for Azure auth"
git push
```

2. **Watch** the run:

```
gh run watch
```

3. **Open** the run logs and expand the **Sign in to Azure** step. You should see a line like:

```
Federated identity credential resolved.
Login successful.
```

   No `creds` value is logged because there isn't one.

4. **Verify** the deployment landed: hit your Container App's FQDN. The new image you just pushed should be serving requests.

> **QUICK CHECK:**
> The workflow run is green, the **Sign in to Azure** step shows the federated-resolution log line, and your app responds at its FQDN.

Step 10: *Delete the old* `AZURE_CREDENTIALS` *secret*

Until you delete it, the old long-lived password is still sitting in GitHub. The whole point of this exercise is to have *no* password — so prove that the new flow doesn't depend on the old secret by removing it and pushing one more change.

1. **Delete** the secret:

```
gh secret delete AZURE_CREDENTIALS
```

2. **Confirm** it is gone:

```
gh secret list
```

Only `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, and `AZURE_SUBSCRIPTION_ID` should remain.

3. **Trigger** another pipeline run by making any small change (for example, edit the homepage text in your MVC app) and pushing:

```
git commit -am "Confirm pipeline runs without AZURE_CRE-
        DENTIALS"
git push
gh run watch
```

The run should pass. The pipeline now has no long-lived secret backing it.

> **COMMON MISTAKES**
>
> - Deleting the secret *before* pushing the workflow change in Step 8 — the old `creds: ${{ secrets.AZURE_CREDENTIALS }}` line would resolve to an empty string and fail confusingly. Always change the workflow first, verify it works, then remove the old secret.

Step 11: *Inspect the trust relationship*

The federated credential is the *only* thing standing between an arbitrary GitHub workflow and your Azure subscription. Worth looking at it directly so you understand what changing it would change.

1. **List** the credentials on your app registration:

```
az ad app federated-credential list \
  --id "$APP_ID" \
  -o table
```

2. **Show** the full subject of the one credential you have:

```
az ad app federated-credential list \
  --id "$APP_ID" \
  --query "[].{name:name, subject:subject,
        issuer:issuer}" \
  -o json
```

3. **Demonstrate** the subject's exact-match nature. Try this:

```
git checkout -b experiment-branch
git commit --allow-empty -m "Try to deploy from a
        feature branch"
git push -u origin experiment-branch
```

The push triggers no deployment because your workflow is configured for `on: push: branches: [main]` — that's the first layer of isolation. To see the auth layer also reject the branch, force a run from `experiment-branch` using the `workflow_dispatch` trigger you added in the previous exercise:

```
gh workflow run ci.yml --ref experiment-branch
gh run watch
```

The run starts, but it fails at the **Sign in to Azure** step:

```
AADSTS70021: No matching federated identity record found
for presented assertion.
```

The token GitHub mints for `experiment-branch` carries the subject `repo:org/name:ref:refs/heads/experiment-branch`, which does not match the federated credential's

`repo:org/name:ref:refs/heads/main`. Azure refuses to issue a token.

4. **Clean up** the experiment branch:

```
git checkout main
git push origin --delete experiment-branch
git branch -D experiment-branch
```

> **CONCEPT DEEP DIVE**
>
> Real production setups exploit this exact-match behaviour deliberately. A common pattern: one app registration per environment, each with its own federated credential, each scoped to a different subject pattern.
>
> - `repo:org/name:environment:prod` — only workflow jobs that target the GitHub `prod` environment can deploy
> - `repo:org/name:pull_request` — a separate, lower-privilege identity used by PR validation jobs
> - `repo:org/name:ref:refs/tags/v*` — a release identity that only authenticates for tag pushes
>
> The principal of least privilege is no longer "give the SP the smallest possible role" — that's still true — but also "give the SP the smallest possible *trigger surface* in GitHub." A leaked role assignment is bad; a leaked secret is worse; a federated credential bound to a single subject is the best of the three.
>
> ✓ **Quick check:** Pushing to `main` deploys; pushing to any other branch does not authenticate.

Step 12: *Reflect on subject-claim scope*

The OIDC concepts you saw in the authentication chapter apply here too: a federated identity, a `subject` claim, a token-exchange flow. The difference is just *who* is authenticat-

ing to *whom*. In a user-to-app login, a person proves to your app that they own a Google account. In machine-to-cloud OIDC, GitHub proves to Azure that this workflow run came from this repo on this branch. Same OAuth 2.0 family, same trust-by-prearrangement, different problem.

The brittleness of the `subject` claim is a feature, not a bug. Walk through these scenarios and predict the behaviour:

1. **You add** an `on: pull_request:` trigger to the workflow so the build runs on PRs as well.

   Result: the build runs, but the **Sign in to Azure** step fails for every PR — the subject is `repo:org/name:pull_request`, which has no matching federated credential. Tests can run; deployment cannot. This is the design.

2. **A teammate** opens a PR from a fork that modifies `.github/workflows/ci.yml` to print `$ {{ secrets.AZURE_CLIENT_ID }}` to the logs.

   Result: even if a maintainer accidentally approves and merges, the leaked client ID alone cannot be used to authenticate. There is no client secret to leak, and the federated credential only trusts tokens *minted by GitHub for runs of this repo on* `main`. The attacker would need to control your `main` branch to make a token Azure trusts.

3. **You move** the repo from `your-username/<repo>` to a GitHub organisation `your-org/<repo>`.

   Result: the subject changes from `repo:your-username/<repo>:...` to `repo:your-org/<repo>:...`. Authentication breaks until you update the federated credential's subject. (This is a real footgun during organisational migrations.)

> **CONCEPT DEEP DIVE**
>
> Production CI/CD with OIDC is rarely "one credential, one branch." A typical setup uses GitHub *environments* (Settings → Environments → Production) and binds the federated credential to `repo:org/name:environment:Production`. The Production environment is configured with required reviewers, so a deployment can only run after a human approval. The subject in the OIDC token reflects the environment, so Azure won't issue a deployment token unless the workflow run is gated by the right reviewers. Authentication and authorization meet in the same claim. That's a subject worth a whole separate exercise.

Step 13: *Test Your Implementation*

Validate end-to-end that the pipeline works without any long-lived secret.

1. **Confirm** GitHub no longer holds the old secret:

   ```
   gh secret list
   ```

   Expected: only `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`.

2. **Confirm** Azure does not hold a client secret for the app:

   ```
   az ad app credential list \
     --id "$APP_ID" \
     -o table
   ```

   Expected: an empty table (no password credentials, no certificate credentials).

3. **Trigger** a deployment by pushing to `main`:

```
git commit --allow-empty -m "Trigger deployment to
        verify OIDC flow"
git push
gh run watch
```

4. **Verify** the **Sign in to Azure** step prints `Federated identity credential resolved.` (or equivalent) and the workflow finishes green.
5. **Verify** the FQDN serves the deployed image:

```
curl -I "https://<your-container-app-fqdn>"
```

   Expected: `HTTP/2 200`.

6. **Verify** branch isolation. Push an empty commit on a different branch, then dispatch the workflow against it and confirm it cannot authenticate:

```
git checkout -b verify-branch-isolation
git commit --allow-empty -m "Should not deploy"
git push -u origin verify-branch-isolation
gh workflow run ci.yml --ref verify-branch-isolation
gh run watch
```

   Expected: the dispatched run fails at **Sign in to Azure** with `AADSTS70021: No matching federated identity record found for presented assertion.` The trust relationship only honours the `main` ref.

7. **Clean up the experiment branch:**

```
git checkout main
git push origin --delete verify-branch-isolation
git branch -D verify-branch-isolation
```

8. **Tear down the cloud resources** when you are completely done with the chapter. The work splits into two homes — the Azure subscription holds the running resources, and Microsoft Entra ID (a tenant-level service) holds the identity used by the pipeline. Deleting one does **not** delete the other.

   First, capture the app registration's `appId` so you can delete it after the resource group:

```
APP_ID=$(az ad app list --display-name "github-cicd-
         oidc" --query "[0].appId" -o tsv)
echo "$APP_ID"
```

   Then delete the resource group (this removes the Container App, the ACR with all its images, the Container Apps environment, the Log Analytics workspace, and every role assignment scoped under the group):

```
az group delete -n rg-cicd-week4 --yes --no-wait
```

   Then delete the Entra app registration. This cascades to the service principal and the federated credential — both are children of the app registration:

```
az ad app delete --id "$APP_ID"
```

   Finally, delete the now-orphaned GitHub secrets:

```
gh secret delete AZURE_CLIENT_ID
gh secret delete AZURE_TENANT_ID
gh secret delete AZURE_SUBSCRIPTION_ID
```

COMMON MISTAKES

- Deleting the resource group is **not enough**. The Entra app registration lives in the tenant, not the subscription, and stays alive (along with the SP and the federated credential) until you delete it explicitly.
- Role assignments scoped to deleted resources become **orphaned** — they linger in the directory until garbage collection, but their scope path no longer resolves. Harmless, but they show up as "Identity not found" entries in `az role assignment list` for a while.
- Entra ID **soft-deletes** app registrations for 30 days. They sit under **Microsoft Entra ID → App registrations → Deleted applications** and can be restored. To purge immediately, click into the deleted app there and choose **Delete permanently**.
- Deleting the resource group also takes the auto-created Log Analytics workspace with it. If you want to keep your logs (you usually don't, for coursework), export them first.

√ **Success indicators:**

- The pipeline runs green on every push to `main` with no `AZURE_CREDENTIALS` secret in the repo
- The **Sign in to Azure** step reports a federated token exchange, not a credential blob
- `az ad app credential list` is empty for the app registration — the SP has no password
- Pushes to non-`main` branches either do not deploy or fail authentication
- The Container App FQDN serves the latest image

√ **Final verification checklist:**

- [ ] Entra app registration `github-cicd-oidc` exists
- [ ]

- [ ] Service principal exists for that app, with `AcrPush` and `Container Apps Contributor` role assignments scoped to your specific resources
- [ ] One federated credential is configured, with subject `repo:<your-username>/<your-repo>:ref:refs/heads/main`
- [ ] Three GitHub secrets present (`AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`); old `AZURE_CREDENTIALS` deleted
- [ ] Workflow has `permissions: id-token: write` and `contents: read` at the job level
- [ ] `azure/login@v2` uses `client-id` / `tenant-id` / `subscription-id`, not `creds`
- [ ] Pipeline deploys successfully on push to `main`

6.6.4 Common Issues

> ***If you encounter problems:***
> **`Error: Failed to retrieve OIDC token`***:* *The workflow is missing* `permissions: id-token: write`*. Add it at the job (or workflow) level and re-run.*
> **`AADSTS70021: No matching federated identity record found for presented assertion`***:* *The* `subject` *in the GitHub-minted token doesn't match any federated credential on your app registration. Compare exactly:* `repo:<exact-org-or-user>/<exact-repo>:ref:refs/heads/main`*.*
> *Watch out for* `head` *vs* `heads`*, missing colons, and angle brackets left in from the placeholder.*
> **`AADSTS700016: Application with identifier '...' was not found`***:* `AZURE_CLIENT_ID` *is wrong or refers to a deleted app registration. Re-fetch it:* `az ad app show --id "<app-name>" --query appId -o tsv`*.*
> **`Insufficient privileges to complete the operation`** ***on*** **`az ad app create`***: Your tenant does not let regular users register applications. Ask a directory admin for the* ***Application Developer*** *role or to register the app on your behalf.*
> **`AuthorizationFailed`** ***when pushing to ACR:*** *Role assignment from Step 4 didn't propagate, or* `--scope` *was wrong. Run* `az role assignment list --assignee "$APP_ID" --all -o table` *and confirm both* `AcrPush` *and* `Container Apps Contributor` *are listed at the expected scopes. Wait 30 seconds and retry.*

> ***Pipeline still uses the old SP:*** *GitHub may have cached the old run definition. Confirm* `gh secret list` *no longer shows* `AZURE_CREDENTIALS` *and that the workflow YAML on* `main` *is the new version.*
> ***Still stuck?*** *Re-run the* ***Sign in to Azure*** *step in debug mode by adding* `ACTIONS_STEP_DEBUG: true` *as a repo secret and re-running the workflow. The debug logs will show the exact subject the runner sent and the exact response from Entra ID.*

6.6.5 Summary

You replaced a long-lived service-principal secret with a federated trust relationship between GitHub Actions and Microsoft Entra ID. The pipeline still pushes to ACR and updates the Container App, but no password lives in GitHub and no password lives in Azure.

- ✓ The Entra app registration is the identity definition; the service principal is its instance in your subscription
- ✓ Role assignments (`AcrPush`, `Container Apps Contributor`) are attached to the service principal, scoped to specific resources
- ✓ The federated credential's `subject` claim binds authentication to one repo on one branch
- ✓ `permissions: id-token: write` opts the workflow into requesting a JWT from GitHub
- ✓ `azure/login@v2` exchanges that JWT for an Azure access token using OIDC token exchange

> **KEY TAKEAWAY:**
>
> Long-lived secrets in CI/CD are a deployment problem dressed up as a security problem. Federated identity makes the security problem disappear (no secret to leak, rotate, or audit) and simultaneously makes the deployment problem disappear (no rotation runbook). The cost is a one-time setup and a `subject` claim that you must keep aligned with how your repository is structured.

6.6.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add a second federated credential with subject* `repo:<your-username>/<your-repo>:environment:Production` *and gate deployments behind a GitHub Environment with required reviewers.*
> - *Split into two app registrations: one for build (*`AcrPush` *only) and one for deploy (*`Container Apps Contributor` *only). Use job-scoped* `azure/login` *steps to authenticate as each in turn.*
> - *Decode an actual OIDC token from a real workflow run. Add a step that runs* `curl -H "Authorization: Bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=api://AzureADTokenExchange"` *and pipe the JWT through* `jwt.io` *to inspect every claim.*
> - *Repeat the federation setup against AWS (IAM OIDC provider for* `token.actions.githubusercontent.com`*) or GCP (Workload Identity Federation). The vocabulary differs but the mental model is identical.*

6.6.7 Done!

The pipeline runs with no password. The next time someone asks “where is the deploy secret stored?” the honest answer is “nowhere — it doesn’t exist.” That’s the point.

PART III · DEPLOYMENT

EXERCISE 7

Logging and Monitoring

7.1 Structured Logging with ILogger

7.2 Structured Logging with ILogger<T>

7.2.1 Goal

Your `CloudCi` MVC app already runs end-to-end: a push to `main` builds an image, ships it to ACR, and rolls a new revision into Azure Container Apps. The homepage renders two badges — `build: <SHA>` and `host: <hostname>` — that prove the right image is serving. What it does *not* do is tell you anything about what happens between requests.

Right now the only log output is whatever ASP.NET Core happens to emit on its own: HTTP request lines from the hosting layer, a banner at startup, the occasional warning when a configuration value is missing. None of it is yours. None of it carries application meaning. If a user reports "the page rendered without my hostname yesterday afternoon," there is no log line to look for, because no log line was ever written.

In this exercise you will make the logging *deliberate*. You will inject `ILogger<HomeController>`, write log lines with **semantic message templates** (placeholders, not interpolation), and tune which lines are emitted through `appsettings.json`. You will run the app under `dotnet run` and inside `docker run` and confirm that the same structured fields flow through both.

WHAT YOU'LL LEARN:

- How `ILogger<T>` is registered and injected by the ASP.NET Core hosting model
- Why message templates (`"... {Field} ..."`) preserve structure that string interpolation throws away
- How log-level filtering works per category, and how `appsettings.json` controls it
- How the simple and JSON console formatters differ in their *output*, while preserving the same *fields*
- How container logs surface the exact same structured data as `dotnet run`

7.2.2 Prerequisites

> **BEFORE STARTING, ENSURE YOU HAVE:**
>
> - √ The `CloudCi` MVC app from the previous chapter, deployed to Azure Container Apps via the OIDC pipeline
> - √ The homepage renders two badges — `build: <SHA>` and `host: <hostname>` — both locally and in Azure
> - √ .NET 10 SDK and Docker Desktop (or another OCI runtime) installed locally
> - √ A terminal in the `CloudCi/` project root, with `git status` clean
> - √ No outstanding work in your `Dockerfile` or `Program.cs` from the last exercise

7.2.3 Exercise Steps

Overview

1. **Open the project and confirm the starting state**
2. **Observe what ASP.NET Core logs by default**
3. **Inject ILogger<HomeController> into the controller**
4. **Add a structured Information log line in Index()**
5. **Add a Warning when BUILD_SHA is missing**
6. **Run locally and inspect the captured fields**
7. **Tune log levels per category via appsettings.json**
8. **Demonstrate category filtering by raising the threshold**
9. **Switch the console formatter to JSON in Development**
10. **Build the container and run it locally**
11. **Inspect the container's logs and reflect on formatters vs fields**
12. **Test Your Implementation**

Step 1: *Open the project and confirm the starting state*

Before changing anything, prove that the app you finished the previous chapter with still runs. The exercise builds on that exact state — the same `Program.cs`, the same `HomeController.cs`, the same `_Layout.cshtml` with the two badges. If the homepage doesn't render the badges now, no amount of logging work will fix it.

1. **Navigate** to the project root:

```
cd CloudCi
```

2. **Run** the app in the foreground:

```
dotnet run
```

3. **Open** the URL the host prints (typically `<http://localhost:5000>` or a similar 5xxx port).
4. **Confirm** that the homepage renders two badges in the layout — `build: local` (or whatever the fallback is) and `host: <your-machine-name>`.
5. **Stop** the app with `Ctrl+C` once verified.

> **QUICK CHECK:**
> The homepage renders both badges. `git status` shows no pending changes.

Step 2: *Observe what ASP.NET Core logs by default*

The framework already logs. The question is *what*. Knowing the baseline matters because everything you add in the next steps must add **application** signal, not duplicate the framework's own.

1. **Run** the app again and watch the terminal carefully:

```
dotnet run
```

2. **Make** a single request from another terminal:

```
curl -i http://localhost:5000/
```

3. **Read** the output in the first terminal. You will see lines like:

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.AspNetCore.Hosting.Diagnostics[1]
      Request starting HTTP/1.1 GET http://local-
host:5000/ - - -
info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
      Request finished HTTP/1.1 GET http://local-
host:5000/ - 200 - text/html;...
```

4. **Stop** the app with `Ctrl+C`.

> **CONCEPT DEEP DIVE**
>
> Each line follows the same shape: `level: Category[EventId]` on one line, then the message indented underneath. The category is the fully-qualified type name of whoever called the logger. `Microsoft.Hosting.Lifetime` and `Microsoft.AspNetCore.Hosting.Diagnostics` are framework categories — you did not write any of those log lines. They come from the host and the request pipeline.
>
> What is **not** in this output: anything specific to your `HomeController`, anything mentioning the build SHA, anything you could grep for if a user reported a bug at 14:32 yesterday. The framework gives you protocol-level signal for free; the application-level signal is yours to add.
>
> √ **Quick check:** You see at least one `Microsoft.Hosting.Lifetime` line and one `Microsoft.AspNetCore.Hosting.Diagnostics` line per request, and zero lines with `CloudCi` in the category.

Step 3: *Inject ILogger<HomeController> into the controller*

`ILogger<T>` is the canonical logging abstraction in ASP.NET Core. The hosting model registers it as a generic service: ask for `ILogger<HomeController>` in a constructor, and the DI container hands you a logger whose **category** is automatically `CloudCi.Controllers.HomeController` — exactly the string `appsettings.json` will use to filter levels later.

1. **Open** the existing controller at `Controllers/HomeController.cs`.
2. **Locate** the class declaration. The default `dotnet new mvc` template produces something close to this — your `Index()` already pulls `BUILD_SHA` and `HOSTNAME` from the environment to render the two badges, so keep that

logic and just add a logger field and constructor parameter:

> `Controllers/HomeController.cs` *(before)*

```
using Microsoft.AspNetCore.Mvc;

namespace CloudCi.Controllers;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        var buildSha = Environment.GetEnvironmentVari-
        able("BUILD_SHA") ?? "local";
        var hostName = Environment.MachineName;
        ViewData["BuildSha"] = buildSha;
        ViewData["HostName"] = hostName;
        return View();
    }
}
```

3. **Replace** the class with the version that takes an injected logger:

> `Controllers/HomeController.cs` *(after)*

```
using Microsoft.AspNetCore.Mvc;

namespace CloudCi.Controllers;

public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController>
        logger)
    {
        _logger = logger;
    }

    public IActionResult Index()
    {
        var buildSha = Environment.GetEnvironmentVari-
        able("BUILD_SHA") ?? "local";
        var hostName = Environment.MachineName;
        ViewData["BuildSha"] = buildSha;
        ViewData["HostName"] = hostName;
        return View();
    }
}
```

4. **Save** the file. Do not run yet — the next step adds the actual log line.

CONCEPT DEEP DIVE

No service registration is needed. The default ASP.NET Core host calls `AddLogging()` internally as part of `WebApplication.CreateBuilder(args)`, which registers the open generic `ILogger<>` against `Logger<>`. Every controller in your app can add an `ILogger<TController>` parameter to its constructor and receive a working logger immediately.

The category embedded in `ILogger<HomeController>` is the string form of `typeof(HomeController).FullName` — i.e. `CloudCi.Controllers.HomeController`. That string is what `appsettings.json` matches against in Step 7. Categories are hierarchical: a filter on `CloudCi` covers everything in your project; a filter on `CloudCi.Controllers.HomeController` covers only this controller.

⚠ **Common Mistakes**

- Declaring `_logger` but forgetting the constructor parameter, then trying to call `_logger.LogInformation(...)`, throws `NullReferenceException` at the first request.
- Using `ILogger` (non-generic) works at runtime but loses the category — log lines come out under `Microsoft.Extensions.Logging.Logger`, which makes per-category filtering impossible.
- Newing up `Logger<HomeController>` by hand defeats the DI system and produces a logger with no providers attached. Always inject.

✓ **Quick check:** `dotnet build` succeeds with no warnings about unused fields.

Step 4: *Add a structured Information log line in Index()*

The first real log line. The whole point of structured logging is that this line should carry **named fields**, not a frozen string.

A sink (Log Analytics, Application Insights, Seq, Elasticsearch) can later filter on `HostName == "abc-123"` or aggregate by `BuildSha`. That is only possible if the original log call expressed those values as separate placeholders.

1. **Open** `Controllers/HomeController.cs` again.
2. **Add** a log line at the top of `Index()`, before assigning to `ViewData`. The placeholders `{HostName}` and `{BuildSha}` are *names*, not C# variable references — the values come from the trailing arguments:

   > `Controllers/HomeController.cs`

```
public IActionResult Index()
{
    var buildSha = Environ-
        ment.GetEnvironmentVariable("BUILD_SHA") ??
        "local";
    var hostName = Environment.MachineName;

    _logger.LogInformation(
        "Home page rendered for {HostName} build {Build-
        Sha}",
        hostName,
        buildSha);

    ViewData["BuildSha"] = buildSha;
    ViewData["HostName"] = hostName;
    return View();
}
```

3. **Save** the file.

CONCEPT DEEP DIVE — WHAT MAKES A LOG LINE "STRUCTURED"

A traditional log line is a single string: `"Home page rendered for abc-123 build a1b2c3"`. Once written, the host name and the SHA are dissolved into one opaque blob. To answer "show me all renders for `abc-123`" downstream, you have to parse the message back apart with a regex.

A *structured* log line carries the values separately:

```
Message: "Home page rendered for {HostName} build
{BuildSha}"
Fields:  HostName = "abc-123", BuildSha = "a1b2c3"
```

The console formatter still prints `"Home page rendered for abc-123 build a1b2c3"` for human eyes, but the underlying `LogRecord` keeps `HostName` and `BuildSha` as discrete properties. A sink that understands structure (Log Analytics with the right pipeline, Application Insights, Serilog) stores them as columns. A sink that doesn't (a plain text file) flattens them — but the placeholders make re-extraction trivial.

⚠ **Common Mistakes**

- Writing `_logger.LogInformation($"Home page rendered for {hostName} build {buildSha}")` is *string interpolation* — C# builds the final string at the call site, the placeholders never reach the logging pipeline, and the structure is destroyed before the log call begins. The compiler does not warn about this; the analyzer rule `CA2254` does, if enabled.
- Inverting the placeholder order (`"... {BuildSha} ... {HostName}"`) without inverting the trailing arguments produces a silently wrong log line. Match positionally.
- Using `string.Format(...)` and passing the result to `LogInformation` collapses the structure exactly like interpolation.

> ✓ **Quick check:** The placeholders use PascalCase names (`{HostName}`, `{BuildSha}`), and the trailing arguments line up with them positionally.

Step 5: *Add a Warning when BUILD_SHA is missing*

A Warning is for an event that is not an error but is worth noticing. "We rendered the homepage with the literal string `local` because no build SHA was set" is a perfect example: the app works, but in production this almost certainly means the deployment forgot to inject the env var.

1. **Open** `Controllers/HomeController.cs`.
2. **Replace** the body of `Index()` with the version that distinguishes the missing-env-var case:

> `Controllers/HomeController.cs`

```
public IActionResult Index()
{
    var buildShaEnv = Environ-
        ment.GetEnvironmentVariable("BUILD_SHA");
    var buildSha = buildShaEnv ?? "local";
    var hostName = Environment.MachineName;

    if (buildShaEnv is null)
    {
        _logger.LogWarning(
            "BUILD_SHA environment variable is not set;
        falling back to {Fallback}",
            buildSha);
    }

    _logger.LogInformation(
        "Home page rendered for {HostName} build {Build-
        Sha}",
        hostName,
        buildSha);

    ViewData["BuildSha"] = buildSha;
    ViewData["HostName"] = hostName;
    return View();
}
```

3. **Save** the file.

CONCEPT DEEP DIVE — THE LOG-LEVEL HIERARCHY

ASP.NET Core uses six levels, ordered by severity:

- `Trace` — extremely detailed; almost never in production.
- `Debug` — diagnostic detail useful while developing.
- `Information` — normal application flow worth recording.
- `Warning` — something unexpected, but the app is still functioning.
- `Error` — a request or operation failed.
- `Critical` — the application is in an unrecoverable state.

A category's effective minimum level is the highest of: the framework default (`Information`), the value in `appsettings.json`, and the value in `appsettings.{Environment}.json`. Lines below the threshold are *cheap to ignore* — the logging infrastructure short-circuits before formatting placeholders, so a disabled `LogDebug` call costs roughly the price of a virtual call and a comparison. You can leave Debug lines in production code with no measurable cost.

✓ **Quick check:** The Warning is conditional on `buildShaEnv is null`; the Information line runs unconditionally.

Step 6: *Run locally and inspect the captured fields*

Time to see the new log lines in action. The default console formatter prints them as plain text, but the structure is already there underneath — Step 9 will make the structure visible by switching formatters.

1. **Make sure** `BUILD_SHA` is *not* set in your shell. On macOS / Linux:

```
unset BUILD_SHA
```

2. **Run** the app:

```
dotnet run
```

3. **Hit** the homepage from another terminal:

```
curl -s http://localhost:5000/ > /dev/null
```

4. **Read** the log output. Among the framework lines you will now see two new ones, both under the category `CloudCi.Controllers.HomeController`:

```
warn: CloudCi.Controllers.HomeController[0]
      BUILD_SHA environment variable is not set; falling
back to local
info: CloudCi.Controllers.HomeController[0]
      Home page rendered for your-machine-name build
local
```

5. **Stop** the app with `Ctrl+C`.
6. **Set** `BUILD_SHA` and re-run, confirming the Warning disappears:

```
BUILD_SHA=manual-test dotnet run
```

   Hit `/` once more. The Information line now reports `build manual-test`; no Warning is emitted.
7. **Stop** the app.

> **QUICK CHECK:**
> Without `BUILD_SHA`, you see one Warning and one Information per request from your category. With `BUILD_SHA` set, you see only the Information line.

Step 7: *Tune log levels per category via appsettings.json*

Until now you have been at the mercy of the default level (`Information`). To control which categories speak and which stay silent, ASP.NET Core reads the `Logging:LogLevel` section of configuration. Adding a Debug line and a per-category override is the cleanest way to feel how the filter works.

1. **Open** `Controllers/HomeController.cs` and **add** a Debug line just before the Information line:

> `Controllers/HomeController.cs`

```
_logger.LogDebug("Resolved hostName={HostName} build-
        Sha={BuildSha}", hostName, buildSha);

_logger.LogInformation(
    "Home page rendered for {HostName} build {Build-
        Sha}",
    hostName,
    buildSha);
```

2. **Open** `appsettings.json` in the project root. The default file looks like this:

> `appsettings.json` *(before)*

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

3. **Add** a category-specific entry for your controller:

> `appsettings.json` *(after)*

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "CloudCi.Controllers.HomeController": "Debug"
    }
  },
  "AllowedHosts": "*"
}
```

4. **Run** the app and hit `/`:

```
dotnet run
```

5. **Confirm** all three lines now appear for your category — the Debug, the Warning (if `BUILD_SHA` is unset), and the Information. The framework categories still emit only Warning and above, because their entry (`Microsoft.AspNetCore: Warning`) takes precedence over `Default: Information`.
6. **Stop** the app.

CONCEPT DEEP DIVE — CATEGORY FILTERING

The filter resolution walks **most-specific to least-specific**. For a log call from `CloudCi.Controllers.HomeController`, ASP.NET Core checks:

1. `CloudCi.Controllers.HomeController` — exact match
2. `CloudCi.Controllers` — prefix match
3. `CloudCi` — prefix match
4. `Default` — fallback

The first match wins. This means a single line `"CloudCi": "Debug"` would turn on Debug for every category in your project; a single line `"CloudCi.Controllers.HomeController": "Warning"` mutes Information and Debug for that one type only.

⚠ **Common Mistakes**

- Editing `appsettings.json` while the app is running has no effect — the default host **does** reload configuration, but the logger filter is cached at startup unless you opt into `IOptionsMonitor`. Restart the app to be sure.
- Putting the override in `appsettings.Development.json` and then running the app with `ASPNETCORE_ENVIRONMENT=Production` (e.g. inside the container) means the override silently disappears. Step 11 returns to this point.
- Using kebab-case (`cloud-ci.controllers...`) — category matching is case-insensitive but punctuation must match. Use the exact namespace.

√ **Quick check:** Your category emits Debug, Warning, and Information. Framework categories still emit only Warning.

Step 8: *Demonstrate category filtering by raising the threshold*

The fastest way to feel that the filter is doing real work is to crank it up to `Warning` and watch the Information and Debug lines disappear.

1. **Edit** `appsettings.json`:

   > `appsettings.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "CloudCi.Controllers.HomeController": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

2. **Restart** the app:

```
dotnet run
```

3. **Hit** `/` once. Confirm that:
   - the Debug line is gone,
   - the Information line is gone,
   - the Warning line still fires (if `BUILD_SHA` is unset).
4. **Stop** the app.
5. **Set** the override back to `Information` for the rest of the exercise — the demonstrations from this point on assume Information is on:

   > `appsettings.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "CloudCi.Controllers.HomeController": "Informa-
        tion"
    }
  },
  "AllowedHosts": "*"
}
```

> **QUICK CHECK:**
> With the threshold at Warning, only Warnings (and Errors and Critical) flow. With it back at Information, the Information line returns; Debug stays muted.

Step 9: *Switch the console formatter to JSON in Development*

The default console formatter prints lines for humans. The JSON formatter prints lines for machines — one JSON object per log entry, with the message template and the structured fields preserved as separate keys. This matters because the next exercise (the one that pivots to centralised logs) needs the fields, not the prose.

1. **Open** `appsettings.Development.json` (the file `dotnet new mvc` already created):

> `appsettings.Development.json` *(before)*

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

2. **Add** a `Console` section that selects the JSON formatter:

> `appsettings.Development.json` *(after)*

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    },
    "Console": {
      "FormatterName": "json"
    }
  }
}
```

3. **Run** the app and hit `/`:

```
dotnet run
```

4. **Inspect** the new output. Each line is now a JSON object on its own line:

```
{"EventId":0,"LogLevel":"Information","Category":"CloudCi.Controllers
        sage":"Home page rendered for your-machine-name
        build local","State":{"Message":"Home page
        rendered for your-machine-name build
        local","HostName":"your-machine-
        name","BuildSha":"local","{OriginalFormat}":"Home
        page rendered for {HostName} build {BuildSha}"}}
```

The `State` object is what makes this useful. `HostName` and `BuildSha` are real keys; `{OriginalFormat}` carries the message template; the rendered message is also there for human reading.

5. **Stop** the app.

> **CONCEPT DEEP DIVE — WHY MESSAGE TEMPLATES BEAT STRING INTERPOLATION**
>
> The JSON above is the proof. If you had written `_logger.LogInformation($"Home page rendered for {hostName} build {buildSha}")`, the `State` object would contain only `Message` and `{OriginalFormat}` — both equal to the already-formatted string. There would be no `HostName` key, no `BuildSha` key. A downstream sink looking for "all renders where `HostName == 'abc-123'`" would have to regex it back out of the message.
>
> Message templates promise the sink that the same call site will always emit the same fields with the same names. That promise is what makes a log message *queryable* instead of *searchable*.
>
> √ **Quick check:** The console emits one JSON object per log call. The object contains a `State` block with `HostName` and `BuildSha` as discrete keys.

Step 10: *Build the container and run it locally*

Now run the same code through the container build that ships to Azure. The point is to confirm that the structured log lines appear in `docker logs` exactly as they do under

`dotnet run` — this is the bridge to centralised logging in the next exercise.

1. **Build** the image. The `BUILD_SHA` build arg is the same one your existing `Dockerfile` (from the previous chapter) bakes into the image:

```
docker build \
  --build-arg BUILD_SHA=local-dev \
  -t cloudci:local .
```

2. **Run** the container, mapping its 8080 port to your host:

```
docker run --rm -p 8080:8080 --name cloudci-test
        cloudci:local
```

3. **From another terminal**, hit the homepage:

```
curl -s http://localhost:8080/ > /dev/null
```

4. **Watch** the first terminal. You should see:
   - the framework's own Information lines about request start/finish,
   - your `CloudCi.Controllers.HomeController` Information line (`build local-dev`).

   You will *not* see the Warning, because `BUILD_SHA` is set (to `local-dev`) inside the image at build time.
5. **Stop** the container with `Ctrl+C`.

COMMON MISTAKES

- Forgetting `--build-arg BUILD_SHA=...` produces an image whose `BUILD_SHA` is empty. The Warning fires on every request and the badge displays `local`. Useful for triggering the Warning path on purpose; surprising if you didn't mean to.
- Mapping the wrong port. The Dockerfile from the previous chapter has the app listening on 8080 inside the container; if you used `-p 5000:8080` instead, hit `<http://localhost:5000>` accordingly.
- Forgetting `--rm` leaves stopped containers piling up under `docker ps -a`. Harmless, but tidier to add it.

√ **Quick check:** `docker run` produces the same `CloudCi.Controllers.HomeController` Information line as `dotnet run` did.

Step 11: *Inspect the container's logs and reflect on formatters vs fields*

A subtle but important detail: inside the container, `ASPNETCORE_ENVIRONMENT` defaults to `Production`, so `appsettings.Development.json` is **not** loaded. Your JSON formatter override therefore does not take effect — the simple console formatter prints the lines as text. To prove that the *fields* are still there even when the *formatter* is plain, run the same image with the Development environment forced on.

1. **Run** the container as Production (the default). In one terminal:

```
docker run --rm -d -p 8080:8080 --name cloudci-prod
        cloudci:local
```

2. **Hit** the page and dump the logs from a second terminal:

```
curl -s http://localhost:8080/ > /dev/null
docker logs cloudci-prod
```

The Information line appears as plain text — no JSON, no structure visible to the eye.

3. **Stop** the container:

```
docker stop cloudci-prod
```

4. **Run** it again, this time forcing Development so the JSON formatter kicks in:

```
docker run --rm -d -p 8080:8080 \
  -e ASPNETCORE_ENVIRONMENT=Development \
  --name cloudci-dev cloudci:local
```

5. **Hit** the page and dump the logs:

```
curl -s http://localhost:8080/ > /dev/null
docker logs cloudci-dev
```

The Information line is now a JSON object, with `HostName` and `BuildSha` as keys exactly as in Step 9.

6. **Stop** the container:

```
docker stop cloudci-dev
```

CONCEPT DEEP DIVE — FORMATTERS VS FIELDS

The formatter decides what bytes show up on stdout. The fields are part of the `LogRecord` regardless. When you ship to a centralised sink that *parses* container stdout (Log Analytics, Loki, Fluent Bit), one of two things happens:

- the sink already speaks JSON — every field becomes a column automatically;
- the sink reads plain text — every field is still recoverable, but only if a parser at the sink rewrites the message back into structure (regex, Grok, ingestion-time KQL).

JSON-on-stdout is the easy path; plain-text-on-stdout is the cheap path. Both are common in production. What matters is that you wrote the log call with placeholders, because that is what makes either path *possible*.

⚠ **Common Mistakes**

- Assuming `appsettings.Development.json` applies inside the container. It does not, unless you set `ASPNETCORE_ENVIRONMENT=Development`.
- Putting the JSON formatter into `appsettings.json` (production-eligible) without thinking through the implications. JSON-on-stdout is a deliberate operational choice — make it consciously per environment.

√ **Quick check:** With `ASPNETCORE_ENVIRONMENT=Production` (the default), `docker logs` shows plain text. With `ASPNETCORE_ENVIRONMENT=Development`, `docker logs` shows JSON. The same fields are present in both — the formatter is the only difference.

Step 12: *Test Your Implementation*

Walk through the full chain end-to-end and confirm every layer behaves as designed.

1. **Verify** the app runs locally and emits your structured Information line on every request:

   ```
   dotnet run
   ```

   In another terminal:

   ```
   curl -s http://localhost:5000/ > /dev/null
   ```

   Expected in the first terminal: one `info: CloudCi.Controllers.HomeController` line per request, mentioning both the host name and the build SHA.
2. **Stop** the app.
3. **Verify** category filtering. Set the category to `Warning` in `appsettings.json`, restart, hit `/`, and confirm the Information line is suppressed while a Warning (when `BUILD_SHA` is unset) still flows. Restore the override to `Information` afterwards.
4. **Verify** the JSON formatter under Development. Re-run `dotnet run`, hit `/`, and confirm one log line per request comes out as a JSON object with `HostName` and `BuildSha` as discrete keys.
5. **Verify** the container path:

```
docker build --build-arg BUILD_SHA=local-dev -t
        cloudci:local .
docker run --rm -d -p 8080:8080 --name cloudci-test
        cloudci:local
curl -s http://localhost:8080/ > /dev/null
docker logs cloudci-test
docker stop cloudci-test
```

Expected: the same `CloudCi.Controllers.HomeController` Information line, in plain text (Production), mentioning `build local-dev`.

6. **Verify** structure survives in the container. Repeat with `-e ASPNETCORE_ENVIRONMENT=Development` and confirm the line is JSON with `HostName` and `BuildSha` as keys.

√ ***Success indicators:***

- *Every page render produces an Information log line under the* `CloudCi.Controllers.HomeController` *category.*
- *When* `BUILD_SHA` *is missing, a Warning is also emitted on the same render.*
- *Setting the category override to* `Warning` *in* `appsettings.json` *silences Debug and Information for that category only; framework categories are unaffected.*
- *The JSON formatter (in Development) emits one JSON object per log line, with* `HostName` *and* `BuildSha` *as discrete fields.*
- `docker logs` *shows the same lines as* `dotnet run` *— plain text by default, JSON if Development is forced.*

√ ***Final verification checklist:***

- [ ] `HomeController` *takes* `ILogger<HomeController>` *via constructor injection*
- [ ] *The Information log call uses placeholders (*`{HostName}`*,* `{BuildSha}`*), not interpolation*
- [ ] *A Warning fires when* `BUILD_SHA` *is unset*
- [ ] *A Debug line exists and is gated by the per-category override*

- [ ] *`appsettings.json` has a `CloudCi.Controllers.HomeController` entry under `Logging:LogLevel`*
- [ ] *`appsettings.Development.json` selects the JSON console formatter*
- [ ] *`dotnet run` and `docker run` both emit the new log lines*
- [ ] *The structured fields are visible (as JSON keys) when the formatter is JSON*

7.2.4 Common Issues

> ***If you encounter problems:***
> **`NullReferenceException` on `_logger.LogInformation(...)`:** *You declared `_logger` but forgot to assign it in the constructor — the field is null. Confirm `HomeController` has the `ILogger<HomeController> logger` parameter and assigns `_logger = logger`.*
> ***No `CloudCi.Controllers.HomeController` lines in the output, only framework lines:*** *Either the controller's `Index()` is not actually being hit (check the URL and port), or the category is filtered below Information. Hit `/` directly with `curl`, then check the `Logging:LogLevel` section in the config that matches your environment.*
> ***The JSON formatter is configured but plain text still appears:*** *Almost always one of two things. (1) The JSON setting is in `appsettings.Development.json` but the running process is in Production — set `ASPNETCORE_ENVIRONMENT=Development` or move the setting. (2) You edited the file but did not restart the app — restart.*
> ***The Information message is one big string with no structured fields in the JSON:*** *You used string interpolation (`$"... {hostName} ..."`) instead of a message template (`"... {HostName} ..."`). Replace the `$` interpolation with placeholders and pass the values as trailing arguments.*
> ***Setting `LogLevel:CloudCi.Controllers.HomeController`***

> ***to `Warning` does not silence the Information line:*** *You edited the wrong file. Check whether the running process loaded `appsettings.json` or `appsettings.Development.json` — the Development file overrides the base file when the environment is Development. Edit the one that applies, or edit both.*
> ***`docker logs` is empty:*** *The container is logging to a different stream, or it crashed. Run without `-d` so the logs come to your terminal directly, and confirm the app started.*
> ***`docker build` complains it cannot find the Dockerfile:*** *Run the build from the directory that contains the `Dockerfile` (the project root), or pass `-f path/to/Dockerfile` explicitly.*
> ***Still stuck?*** *Add `_logger.LogInformation("Hello from HomeController constructor");` to the constructor itself — if even that line is missing from the output, the controller isn't being instantiated and the problem is upstream of logging.*

7.2.5 Summary

You moved the app from "whatever the framework happens to log" to "exactly the events I care about, with exactly the fields a downstream tool will need." Specifically:

- ✓ `ILogger<HomeController>` is injected by the hosting model with no extra registration.
- ✓ Log calls use semantic message templates, so `HostName` and `BuildSha` are first-class fields, not characters in a string.

- ✓ Per-category filtering through `appsettings.json` lets you raise or lower the volume of any single type without touching others.
- ✓ The console formatter is a presentation choice; the structure exists regardless of formatter.
- ✓ Container stdout carries the same fields as local stdout — what changes is the formatter, and that is itself a configuration concern.

> **KEY TAKEAWAY:**
>
> Structured logging is not a feature of `ILogger<T>` — it is a *discipline*. The discipline is: every log call uses a constant message template with named placeholders, and never string interpolation. Followed consistently, that one rule turns your logs from text you grep into data you query. Everything that follows in monitoring — centralised log stores, dashboards, alerts — depends on that data being structured at the source.

7.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Read the Microsoft.Extensions.Logging documentation to see the full provider model — Console, Debug, EventSource, EventLog, plus third-party sinks. The same `ILogger<T>` calls flow to all configured providers.*
> - *Add Serilog as a provider. The mental model is identical (templates, placeholders, sinks), but Serilog ships richer enrichers (machine name, thread ID, correlation IDs) and a much larger set of sinks (Seq, Elasticsearch, Splunk, files with rolling).*
> - *Add `LoggerMessage`-source-generated logging (`[LoggerMessage(...)]`) for high-frequency log calls. The compiler emits an allocation-free wrapper, useful when a log statement runs on every request.*
> - *Configure a scope with `_logger.BeginScope(...)` around an HTTP request so every log line in that request automatically carries a correlation ID.*
> - *In the next exercise the Container App's stdout becomes queryable in a Log Analytics workspace — the structured fields you set up here are exactly what makes that query meaningful.*

7.2.7 Done!

The app is no longer logging by accident. Every line that comes out of your code is one you wrote on purpose, with named fields you can search on, at a level you can tune

without redeploying. That is the foundation everything else in this chapter rests on.

In the next exercise the Container App's stdout becomes queryable in a Log Analytics workspace; the `HostName` and `BuildSha` fields you established here are what makes that query meaningful — without them, the centralised store would just hold a pile of free-text strings.

7.3 Container Logs to Log Analytics

7.4 Container Logs to Log Analytics

7.4.1 Goal

In the previous exercise you replaced `Console.WriteLine` with `ILogger<HomeController>` and a pair of semantic message templates — `{HostName}` and `{BuildSha}`. The lines now leave the application as structured records, but they still vanish into the container's stdout the moment a replica restarts or scales out. `docker logs` and `az containerapp logs show --follow` both work for an interactive look at *one* replica, but neither lets you ask "what happened across the fleet between 14:30 and 14:45?"

In this exercise you'll connect those structured log lines to a centralised, queryable store. You'll discover the **Log Analytics workspace** that Azure auto-created when you provisioned the Container Apps environment in the previous chapter, learn enough **Kusto Query Language (KQL)** to filter and project log records, attach a request-scoped **correlation ID** so every log line from a single HTTP request can be tied together, and scale the app to two replicas to see log lines from both in one query.

WHAT YOU'LL LEARN:

- How a Container Apps environment streams stdout into a Log Analytics workspace, and what tables that produces
- Enough KQL to filter, project, and aggregate log records over a time window
- The difference between the legacy `_CL` tables and the modern Container Apps-managed tables
- How `ILogger.BeginScope` attaches structured fields to *every* subsequent log call inside a request without changing each call site
- Why centralised logs beat per-replica `docker logs` the moment you have more than one replica

7.4.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed the previous exercise: `HomeController` uses `ILogger<HomeController>` with the message template `"Home page rendered for {HostName} build {BuildSha}"`
- ✓ A working CI/CD pipeline that deploys to the Container App `ca-cicd-week4` in resource group `rg-cicd-week4` (region `northeurope`)
- ✓ Azure CLI signed in (`az account show` returns your subscription)
- ✓ GitHub CLI signed in (`gh auth status`)
- ✓ Familiarity with the Azure portal — most of this exercise lives in the **Logs** blade

7.4.3 Exercise Steps

Overview

1. **Push the structured-logging changes and confirm a green deploy**
2. **Locate the Log Analytics workspace**
3. **Understand the workspace–environment–table relationship**
4. **Discover which log table schema your environment uses**
5. **Generate traffic to populate the workspace**
6. **Run your first KQL query — find log lines from the Container App**
7. **Filter to the structured "Home page rendered" line**
8. **Add a request-scoped correlation ID middleware**
9. **Update `HomeController.Index()` to log inside the scope**
10. **Push, deploy, and query by correlation ID**
11. **Scale the app to two replicas and query across both**
12. **Save the query for reuse**
13. **Scale back to a single replica**
14. **Test Your Implementation**

Step 1: *Push the structured-logging changes and confirm a green deploy*

Before you can query anything, the new logging code from the previous exercise needs to be running in the cloud. The pipeline handles the build and deploy — your only job is to push and verify.

1. **Commit and push** the structured-logging changes from the previous exercise:

```
git add Controllers/HomeController.cs
git commit -m "Use ILogger with structured fields"
git push
```

2. **Watch** the pipeline finish:

```
gh run list --limit 1
gh run watch
```

3. **Confirm** the new revision is serving requests. Capture the FQDN and curl it:

```
FQDN=$(az containerapp show \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  --query properties.configuration.ingress.fqdn -o tsv)
curl -I "https://$FQDN/"
```

Expected: `HTTP/2 200`.

> **QUICK CHECK:**
> The most recent workflow run is green, both jobs (`deploy` and `smoke-test`) passed, and the FQDN responds with `200`.

Step 2: *Locate the Log Analytics workspace*

When you provisioned the Container Apps environment in the previous chapter, Azure quietly created a **Log Analytics workspace** to hold its logs. The workspace name was auto-generated and follows the pattern `workspace-rgcicd-`

`week4XXXX`, where `XXXX` is a random suffix unique to your subscription. There are two equivalent ways to find it.

1. **Portal path.** Open the resource group `rg-cicd-week4` in the portal. Click the Container Apps environment named `cae-cicd-week4`, then choose **Logs** from the left-hand blade. The workspace name is displayed in the page header — note it down.
2. **CLI path.** From your terminal:

   ```
   az monitor log-analytics workspace list \
     -g rg-cicd-week4 \
     -o table
   ```

   You should see one workspace whose name begins with `workspace-rgcicdweek4`.
3. **Capture** the workspace name into a shell variable for the queries below:

   ```
   WS=$(az monitor log-analytics workspace list \
     -g rg-cicd-week4 \
     --query '[0].name' -o tsv)
   echo "$WS"
   ```

CONCEPT DEEP DIVE

The Container Apps environment is the network and observability boundary inside which one or more Container Apps run. When the environment was created, Azure auto-provisioned a Log Analytics workspace and configured the environment to stream stdout/stderr from every container into it. You did not write code to make this happen — it is the platform default. With `az containerapp env create --logs-destination none` you get no workspace, and console output disappears once it scrolls off the per-replica buffer.

⚠ **Common Mistakes**

- More than one workspace in your subscription? Filter by resource group (`-g rg-cicd-week4`) so you don't grab the wrong one.
- The `[0]` JMESPath index assumes a single workspace in the group. If multiple match, pick the name explicitly.

✓ **Quick check:** `echo "$WS"` prints a name like `workspace-rgcicdweek4ab12`.

Step 3: *Understand the workspace–environment–table relationship*

A **Log Analytics workspace** is a multi-tenant store: many Azure services can stream data into the same workspace, each into its own **table**. A **Container Apps environment** is one such producer. Inside the workspace, every Container App in the environment writes to the same console-log table — there is no per-app table. To filter to one app, you add a `where ContainerAppName_s == "..."` clause.

There are two table schemas in the wild, and you are about to find out which one your environment uses.

> **CONCEPT DEEP DIVE**
>
> The legacy schema is `ContainerAppConsoleLogs_CL` — the `_CL` suffix stands for "Custom Log." The newer schema, `ContainerAppConsoleLogs` (no suffix), is part of a managed-table family Microsoft introduced as Container Apps moved out of preview. New environments default to the modern schema; older environments may still emit the legacy one. The columns differ — `Log_s` versus `Log`, `_s` / `_d` suffixes versus plain names — so a query written for one schema fails on the other with `failed to resolve table or column expression`. The `_s` suffix means "string"; `_d` means "double"; `_b` means "boolean"; `_t` means "datetime"; `_g` means "guid."

Step 4: *Discover which log table schema your environment uses*

Ask the workspace what tables it has and how many rows are in each. This is also a first taste of KQL — a pipeline language where each `|` transforms the table flowing through it.

1. **Open** the Logs blade in the portal (Container Apps environment → **Logs**).
2. **Paste** the following query and click **Run**:

```
search ""
| summarize count() by Type
| top 20 by count_
```

3. **Read** the result. Look for one of these:
   - `ContainerAppConsoleLogs_CL` — legacy schema
   - `ContainerAppConsoleLogs` — modern schema

   The queries below are written against the legacy `_CL` schema. If yours is modern, drop the `_CL` and the `_s` / `_d` column suffixes.

CONCEPT DEEP DIVE

`search ""` is "across all tables" — expensive, never run without a downstream `summarize` or `take`. KQL renames `count()` to `count_` (note the trailing underscore) automatically. The pattern `source | filter | aggregate | sort | take` is how every KQL query is built.

⚠ **Common Mistakes**

- Writing `top 20 by count` (no underscore) silently sorts on a non-existent column.
- If neither table shows up, the app has not yet emitted logs. Hit the FQDN and wait.

✓ **Quick check:** The result includes `ContainerAppConsoleLogs_CL` or `ContainerAppConsoleLogs` with a non-zero row count.

Step 5: *Generate traffic to populate the workspace*

Log Analytics ingests data in batches — there is a one to three minute delay between when the application writes a line and when KQL can query it. Send a small burst of traffic now so the rows are in flight while you read on.

1. **Hit** the home page twenty times in a tight loop:

```
for i in {1..20}; do curl -s "https://$FQDN/" >/dev/
        null; done
```

2. **Note the time.** Ingestion takes one to three minutes. If the first query returns zero rows, wait sixty seconds and rerun it.

CONCEPT DEEP DIVE

Container Apps batches log lines per replica and ships them to Log Analytics on a short interval; Log Analytics in turn batches records into the search index every few seconds to a minute. The combined delay is rarely under thirty seconds and rarely over three minutes for a healthy environment. For real-time-ish observability you would add a metrics pipeline; for log queries the latency is the price of a queryable history.

⚠ **Common Mistakes**

- Running the KQL query immediately after a single curl and concluding "logs aren't working." They are — they just have not arrived yet.
- Generating thousands of requests in a runaway loop. Twenty is plenty.

Step 6: *Run your first KQL query — find log lines from the Container App*

Now write the canonical query: "show me console-log records from `ca-cicd-week4` in the last thirty minutes." This is the query you will tweak over and over once the system is in production — it is the single most useful KQL pattern for Container Apps.

1. **In the Logs blade**, paste the following query and click **Run**:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(30m)
| where ContainerAppName_s == "ca-cicd-week4"
| project TimeGenerated, RevisionName_s, ContainerGroup-
Name_s, Log_s
| order by TimeGenerated desc
| take 50
```

2. **Read** the result. Each `ILogger` call produces *two* records: a header line with the category and event id (`info: CloudCi.Controllers.HomeController[0]`) and a body line with the formatted message (`Home page rendered for ca-cicd-week4--rev-0000007-7c8d build a1b2c3d`). Both share the same `TimeGenerated`.

CONCEPT DEEP DIVE

The pipeline reads as English. `ContainerAppConsoleLogs_CL` is the source table. `where TimeGenerated > ago(30m)` filters to the last thirty minutes (`ago` returns "now minus N"). `where ContainerAppName_s == "ca-cicd-week4"` narrows to one app — without it, you'd see logs from every Container App in the environment. `project` selects columns (like SQL `SELECT`); `order by` sorts; `take` limits. KQL operators always read top-to-bottom, each transforming the table flowing through it. The `_s` suffixes are the legacy-schema fingerprint; on the modern schema the column is `Log` and the filter becomes `where ContainerAppName == "ca-cicd-week4"`.

⚠ **Common Mistakes**

- Forgetting `where ContainerAppName_s == "..."` and getting flooded by other apps. Always filter to one app first.
- Querying `_CL` on a modern-schema environment. Drop the suffix.
- Running the query before ingestion catches up. Wait, then rerun.

√ **Quick check:** The result has at least 20 rows from the curl burst, all with `ContainerAppName_s == "ca-cicd-week4"`.

Step 7: *Filter to the structured "Home page rendered" line*

The `ILogger` formatter renders your message template into the human-readable line in `Log_s`. The structured fields — `HostName`, `BuildSha` — are *in* the text but are not their own columns in the legacy schema, because the default console formatter writes plain text rather than JSON.

1. **Run** this query in the Logs blade:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(30m)
| where ContainerAppName_s == "ca-cicd-week4"
| where Log_s has "Home page rendered"
| project TimeGenerated, ContainerGroupName_s, Log_s
| order by TimeGenerated desc
```

2. **Inspect** the rows. The `Log_s` column contains the rendered message, e.g. `Home page rendered for ca-cicd-week4--rev-0000007-7c8d build a1b2c3d`.
3. **Pull** the `HostName` back out with KQL's `extract` function — for example, count requests per replica:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(30m)
| where ContainerAppName_s == "ca-cicd-week4"
| where Log_s has "Home page rendered"
| extend HostName = extract(@"for (\S+)", 1, Log_s)
| summarize requests = count() by HostName
```

CONCEPT DEEP DIVE

The `has` operator is KQL's tokenized substring match — faster than `contains` because the indexer pre-tokenizes the column. Use `has` for whole words or phrases; `contains` only for partial-word matches. The previous exercise already showed the JSON console formatter at work locally — that formatter only fires under `ASPNETCORE_ENVIRONMENT=Development`, while the running container in Azure defaults to Production and uses the simple text formatter. That is why `Log_s` arrives here as text and not as JSON. If you want first-class structured fields in KQL, set `ASPNETCORE_ENVIRONMENT=Development` on the Container App (or wire the JSON formatter unconditionally in `Program.cs`), then `parse_json(Log_s)` projects the embedded fields.

⚠ **Common Mistakes**

- Treating `_s` columns as JSON. They are strings even if the content is JSON-shaped.
- Using `==` for a partial string. `where Log_s == "Home page rendered"` matches nothing — use `has`.

✓ **Quick check:** The query returns rows whose `Log_s` starts with `Home page rendered for`.

Step 8: *Add a request-scoped correlation ID middleware*

Right now you can find every log line for `ca-cicd-week4`, and every "Home page rendered" line. What you cannot do is take *one* log line and find the *other* log lines from the same HTTP request. In a single-page render this hardly matters; in a real application that calls a database, an external API, and writes its own diagnostics, it matters enormously.

The fix is a small middleware that generates a `Guid` per request and attaches it to a logging scope. `ILogger.BeginScope` adds key-value fields to *every* subsequent

log call inside the scope — including library calls you did not write.

1. **Open** `Program.cs` in the project root.
2. **Add** the following middleware *before* `app.MapController-Route(...)`:

   > `Program.cs`

   ```
   // Generates a per-request correlation ID and pushes it
           onto the
   // logger scope so every log call inside the request
           carries it.
   app.Use(async (context, next) =>
   {
       var correlationId = Guid.NewGuid().ToString("N");
       context.Items["CorrelationId"] = correlationId;

       var logger = context.RequestServices
           .GetRequiredService<ILoggerFactory>()
           .CreateLogger("CorrelationIdMiddleware");

       using (logger.BeginScope(new Dictionary<string,
           object>
       {
           ["RequestId"] = correlationId
       }))
       {
           await next();
       }
   });
   ```

3. **Verify** locally that the app still starts:

   ```
   dotnet run
   ```

   Press Ctrl+C once you see the `Now listening on:` line — you do not need to hit any pages locally for this step.

CONCEPT DEEP DIVE

`ILogger.BeginScope` is the right tool for "attach this field to every log call in this region of code." Internally the scope is a stack — when you call `_logger.LogInformation(...)`, the logger walks the stack and merges all active scope fields into the structured payload. Default console formatters render scopes in parentheses; the JSON formatter emits them as JSON object members. The alternative — passing `correlationId` into every method signature — works but rots. Diagnostics libraries (Serilog, OpenTelemetry, Application Insights) all read scope state automatically.

⚠ **Common Mistakes**

- Forgetting `using` on `BeginScope`. Without it, the scope leaks across requests and every line gets the same wrong ID.
- Putting the middleware *after* `app.MapControllerRoute(...)`. Middleware runs in registration order; anything past `MapControllerRoute` is unreachable for controller log calls.

✓ **Quick check:** `dotnet run` starts cleanly and the app responds at `http://localhost:5000/`.

Step 9: *Update* `HomeController.Index()` *to log inside the scope*

The middleware sets the scope on every request before the controller runs. Your existing `_logger.LogInformation(...)` call already runs *inside* that scope — no code change needed. Verify the scope is attached locally.

1. **Confirm** the existing log line in `Controllers/HomeController.cs` is still the one from the previous exercise — `_logger.LogInformation("Home page rendered for {HostName} build {BuildSha}", hostName, buildSha);`.

2. **Run locally:**

```
dotnet run
```

   Open `http://localhost:5000/`. The console line should now include `RequestId` in the parenthesised scope, like `RequestId:7c8d...`. Press Ctrl+C.

> **QUICK CHECK:**
> The local console shows `RequestId:` in the scope output for at least one request.

Step 10: *Push, deploy, and query by correlation ID*

Ship the middleware to Azure and watch the correlation ID surface in Log Analytics.

1. **Commit and push:**

```
git add Program.cs
git commit -m "Add request-scoped correlation ID via
        ILogger.BeginScope"
git push
gh run watch
```

2. **Wait** for the smoke test to pass, then generate a fresh burst of traffic and note the time:

```
for i in {1..10}; do curl -s "https://$FQDN/" >/dev/
        null; done
```

3. **Wait** sixty to ninety seconds for ingestion.
4. **Run** this KQL query in the Logs blade:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(15m)
| where ContainerAppName_s == "ca-cicd-week4"
| where Log_s has "RequestId"
| project TimeGenerated, Log_s
| order by TimeGenerated desc
| take 30
```

5. **Inspect** a row. The `Log_s` text now contains a fragment like `RequestId:abc123...` — the correlation ID generated by the middleware. Pick one ID, then run a second query (`where Log_s has "abc123"`) to find every line from that single request. For a page render that's a handful of lines; in a real app touching a database, an HTTP client, and a queue, the same query surfaces dozens of lines that together tell the story of one user's interaction.

CONCEPT DEEP DIVE

If you switched the running container's environment to Development (so the JSON formatter from the previous exercise actually fires in production), `Log_s` arrives as JSON and you can pivot more cleanly:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(15m)
| where ContainerAppName_s == "ca-cicd-week4"
| extend payload = parse_json(Log_s)
| extend RequestId =
tostring(payload.Scopes[0].RequestId)
| where isnotempty(RequestId)
| summarize lineCount = count() by RequestId
| order by lineCount desc
```

JSON formatting is the production-grade choice once logs are routinely consumed by KQL. For coursework the text formatter is fine.

√ **Quick check:** A query for one specific `RequestId` returns at least two rows that share that ID.

Step 11: *Scale the app to two replicas and query across both*

Centralised logs earn their keep when there is more than one replica running. With one, `az containerapp logs show --follow` is enough. With two, you immediately need to ask "is the slow request hitting replica A or replica B?" — and the answer lives in `ContainerGroupName_s`.

1. **Pin** the Container App to two replicas. `--min-replicas 2 --max-replicas 2` fixes the count so you do not wait for autoscale:

```
az containerapp update \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  --min-replicas 2 \
  --max-replicas 2
```

2. **Wait** for both replicas. The command returns once the revision is accepted; the platform brings up the second replica over the next minute. Confirm:

```
az containerapp replica list \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  -o table
```

   Expected: two rows.

3. **Generate** traffic so both replicas get hit. The ingress load-balances roughly round-robin:

```
for i in {1..40}; do curl -s "https://$FQDN/" >/dev/
        null; done
```

4. **Wait** ninety seconds, then run this query — it groups "Home page rendered" lines by replica:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(10m)
| where ContainerAppName_s == "ca-cicd-week4"
| where Log_s has "Home page rendered"
| summarize requests = count() by ContainerGroupName_s
| order by requests desc
```

   You should see two rows — one per replica — each with roughly half the total. This is the moment centralised logs prove their worth: a single query, both replicas visible at

once. The same answer with `docker logs` would require knowing both replica names and tailing two terminals side by side.

> **CONCEPT DEEP DIVE**
>
> `ContainerGroupName_s` includes the revision name as a prefix, so values look like `ca-cicd-week4--rev-0000008-7c8d`. If you push a new image and trigger a new revision, the replica names change — old log lines stay associated with the old revision name forever, which is how you investigate "did the bug start with revision 8 or revision 9?"
>
> ⚠ **Common Mistakes**
>
> - Running `summarize` before the second replica has actually started serving. If only one row appears, wait, generate more traffic, and rerun.
> - Pinning `--min-replicas 2 --max-replicas 2` disables scale-to-zero. You are now paying for two always-on replicas.
>
> ✓ **Quick check:** The result has two `ContainerGroupName_s` rows, each with at least five requests.

Step 12: *Save the query for reuse*

Queries you write often deserve a name. Log Analytics saves them either to your personal account or, with the right RBAC, to the workspace itself.

1. **In the Logs blade**, with the "requests by replica" query in the editor, click **Save** → **Save as Query**.
2. **Name** it `home-page-traffic` with a short description, then **Save**. Reopen later from **Queries** in the left-hand pane.

CONCEPT DEEP DIVE

Saved queries come in two flavours. **User queries** are scoped to your account and follow you across workspaces. **Workspace queries** are shared with anyone who has read access on the workspace and require the `Log Analytics Contributor` role to create. Above saved queries are **functions**, which behave like KQL stored procedures — a `ca_logs(name:string)` function can encapsulate the standard filter prefix so the rest of the query becomes a one-liner.

√ **Quick check:** Reopening **Queries → Saved Queries → home-page-traffic** loads the same KQL back into the editor.

Step 13: *Scale back to a single replica*

Two always-on replicas cost roughly twice as much as one. Scale back before moving on.

1. **Update** to one replica:

```
az containerapp update \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  --min-replicas 1 \
  --max-replicas 1
```

2. **Confirm** only one replica is running:

```
az containerapp replica list \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  -o table
```

Expected: one row.

COMMON MISTAKES

- Leaving the lab in two-replica mode at the end of the exercise. The cost is small per hour but accumulates over a course week. Always scale back.
- Setting `--min-replicas 0` thinking it returns the app to the default. The original setting from the previous chapter was likely `1/1`; setting `0/1` enables scale-to-zero, which has different cold-start behaviour. Stick with `1/1` unless you intentionally want scale-to-zero.

Step 14: *Test Your Implementation*

Walk the full set of behaviours in one pass.

1. **Verify** the workspace and a single replica:

```
echo "$WS"
az containerapp replica list -g rg-cicd-week4 -n ca-
        cicd-week4 -o table
```

   Expected: one workspace name, one replica row.

2. **Generate** traffic and wait sixty seconds:

```
for i in {1..10}; do curl -s "https://$FQDN/" >/dev/
        null; done
sleep 60
```

3. **Run** the canonical query in the Logs blade — expected: at least ten rows:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(15m)
| where ContainerAppName_s == "ca-cicd-week4"
| where Log_s has "Home page rendered"
| project TimeGenerated, ContainerGroupName_s, Log_s
| order by TimeGenerated desc
```

4. **Confirm** correlation IDs — expected: at least five rows with `RequestId:` in `Log_s`:

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(15m)
| where ContainerAppName_s == "ca-cicd-week4"
| where Log_s has "RequestId"
| take 5
```

√ ***Success indicators:***

- *The workspace name is visible from both the portal and* `az monitor log-analytics workspace list`
- *KQL queries return rows for* `ContainerAppName_s == "ca-cicd-week4"` *over a recent time window*
- *The "Home page rendered" line appears with the rendered structured fields in* `Log_s`
- *Each request carries a unique* `RequestId` *correlation ID in the logging scope*
- *Scaling to two replicas produced two* `ContainerGroupName_s` *values in a single query result*
- *The Container App is back to a single replica*

√ ***Final verification checklist:***

- [ ] *Log Analytics workspace name captured in* `$WS`
- [ ] *KQL query against* `ContainerAppConsoleLogs_CL` *(or modern equivalent) returns rows*
- [ ] *"Home page rendered" line is visible with the structured fields rendered into the message text*
- [ ] *Correlation-ID middleware is registered before* `MapControllerRoute` *in* `Program.cs`

- [ ] *Each log line carries a* `RequestId` *value from the scope*
- [ ] `home-page-traffic` *saved query exists in the workspace*
- [ ] *Container App* `ca-cicd-week4` *is back at* `--min-replicas 1 --max-replicas 1`

7.4.4 Common Issues

> ***If you encounter problems:***
> **`'ContainerAppConsoleLogs_CL': failed to resolve table or column expression`***:* *Your environment uses the modern schema. Drop the `_CL` suffix and the `_s`/`_d` column suffixes — `Log_s` becomes `Log`, `ContainerAppName_s` becomes `ContainerAppName`.*
> ***Queries return zero rows after several minutes:*** *Either no traffic reached the app (curl the FQDN and check `HTTP/2 200`), or the time-window excludes your data. Widen with `where TimeGenerated > ago(2h)` and rerun.*
> ***The `RequestId` token never appears in `Log_s`:*** *The middleware was registered after `app.MapControllerRoute(...)`. Move the `app.Use(...)` block above it, redeploy, and re-query.*
> **`az containerapp replica list` *shows only one replica after scaling to two:*** *The platform is still bringing up the second. Wait 60–90 seconds. If still one, check `az containerapp revision list` for any `Failed` revision.*
> **`extract` *returns empty values:*** *The regex did not match. Inspect a single `Log_s` row first, then refine.*
> **`_s` *columns treated as JSON:*** *They are not — legacy `_s` columns are flat strings. Wrap with `parse_json()` only if you switched to the JSON formatter.*
> ***Logs from unrelated apps:*** *Always include `where ContainerAppName_s == "..."`. The unfiltered table is noisy and slow.*

> ***Still stuck?*** *Re-run step 4 to confirm the table name, then re-run step 6's query verbatim. Most KQL failures here are typos.*

7.4.5 Summary

You connected the structured log lines from the previous exercise to a centralised, queryable store. You discovered the auto-provisioned Log Analytics workspace, learned which schema your environment uses, and wrote KQL to filter, project, and aggregate console-log records. You added a request-scoped correlation ID via `ILogger.BeginScope` and scaled to two replicas to see the difference between centralised logs and per-replica `docker logs`.

- ✓ A Log Analytics workspace was already streaming your container's stdout — you only had to learn how to query it
- ✓ KQL's pipeline operators (`where | project | summarize | order by | take`) compose into queries that read like English
- ✓ `ILogger.BeginScope` attaches a correlation ID to every log call inside a request without changing any call site
- ✓ Centralised logs make a multi-replica fleet visible in a single query; `docker logs` does not scale past one container

> **KEY TAKEAWAY:**
>
> The moment a workload runs on more than one replica, per-replica log inspection becomes useless. Centralised logs are not an "advanced" feature — they are the baseline for any production environment.

7.4.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Promote* `home-page-traffic` *to a* ***workspace function*** *so the team can call* `home_page_traffic(30m)` *from any other query.*
> - *Configure a* ***log-based alert rule****: run* `ContainerAppConsoleLogs_CL | where Log_s has "ERROR" | count`*, click* ***New alert rule****, set a threshold of* `> 0` *over five minutes. Log-based alerts catch errors before users complain — though metric-based alerts are usually faster and cheaper.*
> - *Investigate* ***retention and cost tuning.*** *The Basic SKU charges per ingested and per retained gigabyte; default retention is 30 days but can go to two years.*
> - *Promote the JSON console formatter from the previous exercise to fire unconditionally (or set* `ASPNETCORE_ENVIRONMENT=Development` *on the Container App) and rewrite the queries with* `parse_json(Log_s)` *for first-class structured-field access.*
> - *Read the KQL quick reference.*

7.4.7 Done!

You can now query the structured logs across replicas, time windows, and individual requests. That is a real operator superpower — most of the on-call playbook in any cloud-native shop starts with a KQL query.

But notice what you cannot easily ask of these logs. “What was the p95 response time across the last hour?” needs you to parse a number out of a string field and aggregate it. “Show me error rate over the last day correlated with deployment events” needs more than one source pivoted on a shared timeline. Text logs are the wrong shape for those questions — the right shape is **metrics**, sampled at a regular interval, plus **traces**, which stitch correlated spans into a request waterfall. Those live in a different sink, with a different query model and a different cost profile. The next exercise wires that in.

7.5 Application Insights and Distributed Tracing

7.6 Application Insights and Distributed Tracing

7.6.1 Goal

Up to this point your app produces structured `ILogger<T>` lines that flow into the Log Analytics workspace, queryable with KQL by correlation ID. That is the *logs* half of observability — what happened, in order, with the right context attached. What it does not tell you is *how the system performed*: which requests were slow, which dependencies were called, where time was spent, and how many exceptions per minute were thrown across all replicas.

In this exercise you’ll add **Application Insights**, an Application Performance Management (APM) sink layered on top of the same Log Analytics workspace. In workspace-based mode it writes into the workspace you already have — same store, different shape. Requests, dependencies, exceptions, and custom metrics become first-class entities the Azure Portal

already knows how to visualise. You'll see live traffic in **Live Metrics**, follow a request through the **Application Map**, and drill into a deliberately induced exception in the **Failures** blade.

This exercise ends the chapter and the live cloud lab that has run since Week 4. The final step tears down the resource group `rg-cicd-week4` (Container App, ACR, Log Analytics workspace, App Insights, and every role assignment scoped under the group) and deletes the Entra app registration created for OIDC federation. After this exercise nothing you provisioned should still be running.

WHAT YOU'LL LEARN:

- What Application Insights gives you that raw `ILogger` lines do not — requests, dependencies, exceptions, traces, and custom metrics as structured telemetry
- Why workspace-based Application Insights is the right default in 2026 — telemetry lands in the same Log Analytics workspace as your container logs, so KQL queries can join across both
- How to inject a connection string as a Container Apps secret rather than a plain environment variable, and what `secretref:` means at runtime
- How to read three Portal blades — Live Metrics, Application Map, Failures — and what each is for
- How to track a custom metric with `TelemetryClient.GetMetric` and find it in the Metrics blade
- How to tear down both the resource group and the tenant-level Entra app registration so no Azure or Entra clutter is left behind

7.6.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ The previous exercise complete: structured `ILogger<T>` lines flowing into Log Analytics, queryable via KQL with correlation IDs, and the Container App back at one replica
- ✓ The resource group `rg-cicd-week4` still alive, holding ACR, the Container Apps environment, the Container App `ca-cicd-week4`, and the Log Analytics workspace
- ✓ The CI/CD pipeline from the previous chapter still wired up — `git push` deploys to the Container App via OIDC federation
- ✓ The Azure CLI signed in (`az account show` returns your subscription)
- ✓ A local clone of the `CloudCi` ASP.NET Core MVC project where you can edit code and push to `main`

7.6.3 Exercise Steps

Overview

1. **What Application Insights gives you that raw logs do not**
2. **Provision the Application Insights component in workspace-based mode**
3. **Add the SDK to the project**
4. **Wire it up in `Program.cs`**
5. **Inject the connection string as a Container Apps secret**
6. **Reference the secret as an environment variable**
7. **Push and deploy**
8. **Open Live Metrics in the Portal**
9. **Explore the Application Map**
10. **Induce an exception with `/Home/Boom`**

11. **Find the exception in the Failures blade**
12. **Add a custom metric for home-page views**
13. **Test Your Implementation**
14. **Tear down the cloud resources**

Step 1: *What Application Insights gives you that raw logs do not*

Before adding any code, get clear on what Application Insights actually models. The previous exercise gave you `ILogger` lines: free-form structured text with a severity, a category, a timestamp, and a few key/value properties. That is the right shape for *narrative* — "user X did Y at time Z" — but the wrong shape for asking *aggregate* questions like "what is the p95 latency of `/Home/Index` over the last hour?" Application Insights makes those questions cheap by modelling distinct telemetry types from the start.

CONCEPT DEEP DIVE: REQUESTS, DEPENDENCIES, AND EXCEPTIONS

A **request** is one unit of work the application served — for ASP.NET Core, one HTTP request. The SDK auto-instruments this: every request that reaches your middleware pipeline produces a request telemetry item with the URL, the HTTP method, the response code, the duration in milliseconds, and the operation ID. You don't write any code to make this happen. Aggregations like "p95 latency of `/Home/Index`" are queries over this stream.

A **dependency** is one outgoing call your app made to serve a request — a SQL query, an HTTP call to another service, a call to Azure Storage. The SDK auto-instruments common .NET clients (`HttpClient`, `SqlClient`, the Azure SDKs). Each dependency telemetry item is correlated to the parent request via the operation ID. Today your app has none, but as soon as you add a database or call a downstream API, dependencies start appearing — and the Application Map starts having more than one node.

An **exception** is a `System.Exception` that bubbled up far enough to be caught by the framework. The SDK records the type, the message, the stack trace, and any custom properties you attach. Exceptions are correlated to the request that produced them, so you can see "this 500 was caused by this `InvalidOperationException` thrown in this method."

ℹ Concept Deep Dive: Traces, custom metrics, the Application Map, sampling

Trace is overloaded in this world and worth pinning down. In Application Insights, **trace telemetry** means a log line — exactly what `ILogger` produces, captured into the same store. **Distributed tracing**, in the OpenTelemetry sense, means following one logical operation across many services using an operation ID. The two share a name and not much else. This exercise covers the first; the foreshadowing at the end of the chapter is about the second.

> A **custom metric** is a numeric value you track yourself — "how many home-page views per minute," "how big was the uploaded file." The SDK aggregates these client-side (count, sum, min, max, p50/p95/p99 over a one-minute bucket) and ships the aggregate, not the raw values. That is the cheap path. The expensive path is `TrackEvent` with a numeric property, which sends every individual occurrence — fine for low volume, fatal at high volume.
>
> The **Application Map** is one of the screens the Portal builds for you out of the request and dependency streams. Each node is a logical service; each edge is a dependency relationship annotated with p95 latency and failure rate. With one app and no dependencies, your map is one node — the moment you add a database, an HTTP API, or a queue, the map fills out automatically.
>
> The .NET SDK ships at 100% **sampling** by default — every request, dependency, exception, and trace is sent. That is fine for coursework. For high-traffic production services, *adaptive sampling* kicks in once you exceed five telemetry items per second, and the SDK starts shipping a representative subset; aggregations are reweighted server-side so counts and percentiles stay accurate. For this exercise we leave the defaults.

Step 2: *Provision the Application Insights component in workspace-based mode*

Application Insights has two storage modes. The legacy mode stored telemetry in its own private store, separate from any Log Analytics workspace. The modern, workspace-based mode stores telemetry inside the Log Analytics workspace you already have — same store, same retention, same KQL queries, same billing. You'll use the workspace-based mode

so the App Insights data lands in the same workspace as the container logs from the previous exercise.

1. **Capture** the resource ID of the existing Log Analytics workspace inside `rg-cicd-week4`:

```
WS_ID=$(az monitor log-analytics workspace list \
  -g rg-cicd-week4 \
  --query '[0].id' -o tsv)

echo "$WS_ID"
```

   The output is a long path that looks like `/subscriptions/.../resourceGroups/rg-cicd-week4/providers/Microsoft.OperationalInsights/workspaces/<workspace-name>`. If `$WS_ID` is empty, the workspace doesn't exist; revisit the previous exercise.

2. **Create** the Application Insights component, pointing it at the workspace:

```
az monitor app-insights component create \
  --app cloudci-insights \
  -g rg-cicd-week4 \
  --location northeurope \
  --workspace "$WS_ID"
```

3. **Capture** the connection string. This is what the SDK reads at runtime to know where to send telemetry:

```
CONN=$(az monitor app-insights component show \
  --app cloudci-insights \
  -g rg-cicd-week4 \
  --query connectionString -o tsv)

echo "$CONN"
```

The connection string looks like `InstrumentationKey=<guid>;IngestionEndpoint=https://northeurope-1.in.applicationinsights.azure.com/;LiveEndpoint=htt northeurope.livediagnostics.monitor.azure.com/;ApplicationId=<gu` Treat it as sensitive — anyone holding it can write telemetry to your component.

CONCEPT DEEP DIVE

Two artefacts matter here. The **instrumentation key** is the legacy identifier — a single GUID. The **connection string** is the modern form: it bundles the instrumentation key together with the regional ingestion endpoint and the live-metrics endpoint. Always pass the connection string; the SDK accepts a bare instrumentation key for backward compatibility, but then has to guess regional endpoints, which fails in sovereign clouds and in regions with non-default endpoints.

⚠ **Common Mistakes**

- Provisioning Application Insights in *classic* mode (without `--workspace`) splits your telemetry across two stores — App Insights data in the classic backend, container logs in the workspace. KQL queries that try to join across both then fail. The `--workspace` argument is what makes this a single pane of glass.
- Naming the component the same as the Container App (`ca-cicd-week4`) is allowed but confusing. The convention `<app-name>-insights` keeps it obvious.

√ **Quick check:** `az monitor app-insights component show --app cloudci-insights -g rg-cicd-week4 --query workspaceResourceId -o tsv` returns the same workspace path you stored in `$WS_ID`.

Step 3: *Add the SDK to the project*

The Application Insights SDK for ASP.NET Core is a single NuGet package that auto-instruments requests, dependencies, exceptions, and `ILogger` traces. Adding it does not change any behaviour by itself — it adds the *capability* to send telemetry, which is then activated in `Program.cs` and configured by the connection string at runtime.

1. **Open** a terminal in the project root (the directory containing `CloudCi.csproj`).
2. **Add** the SDK package:

```
dotnet add package
        Microsoft.ApplicationInsights.AspNetCore
```

3. **Inspect** the resulting `.csproj`. The `ItemGroup` for package references should now contain the new entry:

> `CloudCi.csproj`

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference
        Include="Microsoft.ApplicationInsights.AspNetCore"
        Version="2.22.0" />

  </ItemGroup>

</Project>
```

The exact version pinned by `dotnet add` will be the latest 2.x at the time you run the command.

> **CONCEPT DEEP DIVE**
>
> There is no separate "App Insights for `ILogger`" package — the moment you call `AddApplicationInsightsTelemetry()` in the next step, an `ILoggerProvider` is registered and your existing `_logger.LogInformation` lines start flowing into App Insights as **trace** telemetry on top of going to stdout.
>
> ✓ **Quick check:** `dotnet build` completes with no errors and no new warnings.

Step 4: *Wire it up in* **`Program.cs`**

A single line registers the SDK with the dependency injection container. By default the SDK reads the connection string from the environment variable `APPLICATIONINSIGHTS_CONNECTION_STRING`. You're going to inject that variable in the next two steps; for now just register the services.

1. **Open** `Program.cs` in the project root.
2. **Add** the call after the existing `builder.Services.AddControllersWithViews()` (or whichever service registration is already there):

> `Program.cs`

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

// Register Application Insights. Reads the connection
        string from the
// APPLICATIONINSIGHTS_CONNECTION_STRING environment
        variable by default.
builder.Services.AddApplicationInsightsTelemetry();

var app = builder.Build();

// ... existing middleware pipeline below, including the
        correlation-ID
//     middleware from the previous exercise — leave it
        in place.
```

Leave the correlation-ID middleware from the previous exercise in place. `AddApplicationInsightsTelemetry()` reads `ILogger` scope state automatically, so the `RequestId` you push onto the scope appears as a custom property on every telemetry item without any extra wiring.

CONCEPT DEEP DIVE

`AddApplicationInsightsTelemetry()` does several things in one call: it registers the `TelemetryClient` (used for custom telemetry in step 12), wires up an `ITelemetryInitializer` that stamps every item with role name and instance id, hooks into the request pipeline to produce request telemetry, and adds an `ILoggerProvider` so log lines become trace telemetry. For ASP.NET Core in Container Apps, this single call is everything you need.

⚠ **Common Mistakes**

- Calling `AddApplicationInsightsTelemetry(connectionString: "...")` with a hard-coded string. This works in development but bakes the connection string into the image, which is exactly what the secret injection in the next step is designed to avoid.
- Adding the call after `var app = builder.Build()`. Service registration must happen on `builder.Services` *before* `Build()`. After `Build()`, the DI container is frozen.

√ **Quick check:** The project still builds. The Container App is not yet sending telemetry — without the connection string env var, the SDK silently does nothing. That changes in Step 6.

Step 5: *Inject the connection string as a Container Apps secret*

The connection string is sensitive: it grants write access to your App Insights component. The wrong way to deliver it to the running container is to set it as a plain environment variable on the Container App. Plain env vars are visible in the output of `az containerapp show`, in the Portal's overview blade, and to anyone with `Reader` role on the resource. That is fine for non-sensitive configuration; it is not fine for credentials. The right way is to store the value as a Container

Apps **secret** and reference the secret from an environment variable.

1. **Set** the connection string as a secret on the Container App. The secret name `appinsights-connstr` is conventional; it must be lowercase and hyphenated:

```
az containerapp secret set \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  --secrets appinsights-connstr="$CONN"
```

2. **Verify** the secret is registered. The CLI lists names but never values — that's by design:

```
az containerapp secret list \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  -o table
```

   You should see one row with name `appinsights-connstr`.

CONCEPT DEEP DIVE

Container Apps secrets are stored encrypted in Azure's control plane and only injected into the container at process start. You cannot read them back via `az containerapp show`; they only leave Azure as values for env vars that explicitly reference them via `secretref:`. This mirrors the Kubernetes `Secret`-mounted-as-env-var pattern intentionally.

⚠ **Common Mistakes**

- Names with uppercase letters or underscores will be rejected. Use `appinsights-connstr`, not `AppInsightsConnStr` or `appinsights_connstr`.
- Setting the same name twice with different values silently overwrites. There is no version history; if you need to rotate, the rotation is destructive.

√ **Quick check:** `az containerapp secret list` shows `appinsights-connstr` and no other unexpected entries.

Step 6: *Reference the secret as an environment variable*

The container reads configuration from environment variables, not from Container Apps secrets directly. To bridge the two, you set an environment variable whose *value* is the literal string `secretref:<secret-name>`. The Container Apps runtime intercepts that prefix and substitutes the actual secret value when the container starts. Inside the container, the env var looks like a normal env var.

1. **Update** the Container App so the env var `APPLICATIONINSIGHTS_CONNECTION_STRING` points at the secret you just stored:

```
az containerapp update \
  -g rg-cicd-week4 \
  -n ca-cicd-week4 \
  --set-env-vars APPLICATIONIN-
        SIGHTS_CONNECTION_STRING=secretref:appinsights-
        connstr
```

2. **Confirm** the env var is wired correctly:

```
az containerapp show \
  -g rg-cicd-week4 -n ca-cicd-week4 \
  --query 'properties.template.containers[0].env' -o
        json
```

You should see an entry like:

```
{
  "name": "APPLICATIONINSIGHTS_CONNECTION_STRING",
  "secretRef": "appinsights-connstr"
}
```

The literal value of the connection string does **not** appear here. That is the whole point.

CONCEPT DEEP DIVE

The `secretref:` prefix is the load-bearing piece. Inside the container the process sees `APPLICATIONINSIGHTS_CONNECTION_STRING=InstrumentationKey=...;IngestionEndpoint=...` — a normal-looking env var. From outside, anyone with `Reader` on the Container App can list env vars but only sees the secret reference; reading the actual secret value requires the separate `Microsoft.App/containerApps/listSecrets/action` permission. Env vars are public, secrets are gated — that separation is what makes the pattern useful.

⚠ **Common Mistakes**

- Setting the env var to the literal connection string (`--set-env-vars APPLICATIONINSIGHTS_CONNECTION_STRING="$CONN"`). This works at runtime but bakes the connection string into the Container App's revision history, where it is enumerable forever via `az containerapp show`. Even though a connection string is not a credential in the strict sense, it grants write access to your App Insights component and should be treated like one.
- Forgetting to set the env var at all. The secret exists but nothing references it, so the SDK gets a null connection string and silently drops every telemetry item.

✓ **Quick check:** The JSON output above shows the env var with `secretRef` (not a `value`). The Container App revision is in `RunningStatus: Running`.

Step 7: *Push and deploy*

The SDK changes from Steps 3 and 4 are still only on your local machine. The pipeline from the previous chapter rebuilds and updates the Container App on every push to

`main`, so commit and push to roll the new SDK into the running container.

1. **Commit and push:**

```
git add CloudCi.csproj Program.cs
git commit -m "Add Application Insights SDK"
git push
gh run watch
```

   The workflow should turn green in a couple of minutes; both `deploy` and `smoke-test` should pass.

2. **Capture** the FQDN and verify the new revision serves traffic:

```
FQDN=$(az containerapp show \
  -g rg-cicd-week4 -n ca-cicd-week4 \
  --query properties.configuration.ingress.fqdn -o tsv)

curl -I "https://$FQDN/"
```

   Expected: `HTTP/2 200`. The first request after a deploy may be slower (cold start) but should succeed.

> **QUICK CHECK:**
> Workflow green, a new revision active with 100% traffic, home page returns `200`.

Step 8: *Open Live Metrics in the Portal*

Live Metrics proves the SDK is wired correctly. Unlike the rest of Application Insights — which has a one-to-three-minute ingestion delay — Live Metrics streams telemetry over

a separate channel with one-second latency. If your container is sending telemetry, you see it within a second.

1. **Open** the Azure Portal at https://portal.azure.com, navigate to Application Insights `cloudci-insights`, and click **Live Metrics** in the left navigation.
2. **Generate** traffic from your terminal:

```
for i in {1..40}; do curl -s "https://$FQDN/" >/dev/
        null; sleep 0.25; done
```

3. **Watch** the view. Within a second you should see incoming requests climbing to ~4/sec, request duration in the tens of milliseconds, sample requests on the right-hand side with URL/response code/duration, and server health counters (CPU and memory of the running replica).

CONCEPT DEEP DIVE

Live Metrics is push-based. The SDK opens an outbound connection to the live-diagnostics endpoint encoded in the connection string and streams a high-frequency summary while the blade is open; closing the blade stops the stream. There is no polling and no historical retention — only the last few minutes. This is why Live Metrics is free on production services: it doesn't go through ingestion.

⚠ **Common Mistakes**

- Live Metrics shows "Not available" — almost always means the Container App can't reach the live-diagnostics endpoint, or the SDK isn't loaded, or the env var isn't set. Re-check the secretref wiring from Step 6.
- Confusing Live Metrics with the Metrics blade. They are different screens. Metrics is for historical charts over your aggregated data. Live Metrics is for *now*.

√ **Quick check:** Live Metrics shows incoming requests spiking when you run the curl loop, and falling to zero when you stop.

Step 9: *Explore the Application Map*

The Application Map is built from the request and dependency streams. With one app and no dependencies it is the most boring possible map: one node. That is fine — opening it now establishes the empty starting state, so the day you add a database call or a downstream HTTP API you immediately recognise the new nodes that appear.

1. **Click Application Map** in the left navigation. Wait a minute or two for the map to draw itself; if it says "No data," re-run the curl loop from Step 8 and refresh.
2. **Inspect** the single node — labelled with the role name (typically the Container App name) — showing request count, failure rate, and average duration over the selected time

range. Click the node to open a side panel with URL breakdowns, slowest requests, and failed requests.

> **CONCEPT DEEP DIVE**
>
> The map fills out automatically as the app gains dependencies. The moment you add an `HttpClient` call to a downstream API, the SDK auto-instruments the call and records dependency telemetry; Application Map draws a second node for the target host and an edge labelled with p95 latency and failure rate. The same happens for SQL queries, Azure Storage calls, and service bus calls. When the course later tackles services calling other services, this map gives you the architecture diagram for free — and it surfaces "this slow request was slow because the database call was slow" because dependency timing is correlated to the parent request's operation ID.
>
> ✓ **Quick check:** The Application Map shows exactly one node with a non-zero request count.

Step 10: *Induce an exception with* `/Home/Boom`

Production exceptions are interesting precisely because they're rare and unpredictable. To see how the Failures blade looks when there's something to look at, you need to throw an exception on purpose. Add a controller action that throws unconditionally; you will hit it a few times, then look at how App Insights presents the result.

1. **Open** `Controllers/HomeController.cs`.
2. **Add** a `Boom` action method to the existing `HomeController` class. Place it next to the existing `Index` action:

> `Controllers/HomeController.cs`

```
public IActionResult Boom()
{
    _logger.LogError("About to throw an exception from /
        Home/Boom");
    throw new InvalidOperationException(
        "Boom! This is a deliberate failure for the
        Failures blade.");
}
```

3. **Commit, push, and hit** the new endpoint once the new revision is active:

```
git add Controllers/HomeController.cs
git commit -m "Add /Home/Boom action that throws"
git push && gh run watch

for i in {1..5}; do curl -s -o /dev/null -w "%
        {http_code}\n" "https://$FQDN/Home/Boom"; done
```

Expected: five `500` lines. The browser would show the standard ASP.NET Core error page.

COMMON MISTAKES

- In Development environment, the `DeveloperExceptionPage` middleware catches the exception, renders a debug page, and the request completes with `500` but the framework treats it as handled. Telemetry still records the exception, but the framing changes. Container Apps run with `ASPNETCORE_ENVIRONMENT=Production` by default, which is what you want here. If you've overridden it to `Development`, the rest of this step still works but the response body looks different.
- Forgetting to wait for the new revision before curling `/Home/Boom`. If the curl returns `200`, the old revision is still serving. Re-check `az containerapp revision list`.

√ **Quick check:** Five `500` responses from `/Home/Boom`. The home page (`/`) still returns `200` — only `/Home/Boom` throws.

Step 11: *Find the exception in the Failures blade*

App Insights ingestion is not instant. The Live Metrics channel is real-time, but everything indexed for query — including Failures — takes one to three minutes to appear. Wait for ingestion, then explore.

1. **Wait** about one minute after the last `curl` to `/Home/Boom`, then navigate to **Failures** in the App Insights left navigation. Set the time range (top right) to the last 30 minutes.
2. **Inspect** the **Operations** tab — one row for `GET Home/Boom` with five failed requests and 100% failure rate. Click the **Exceptions** tab — one row for `System.InvalidOperationException` with five occurrences.
3. **Drill** in by clicking the exception row, then "View N more" or "Examine samples." A side panel opens showing the

exception type, the message, the full stack trace, the request URL, the timestamp, and the operation ID — the same correlation ID you've been using to trace requests through `ILogger`.

> **CONCEPT DEEP DIVE**
>
> What App Insights captures by default for an exception: the type, the message, the stack trace, the request URL, the response code, the operation ID, and any custom properties attached via `TelemetryClient.TrackException` or an `ITelemetryInitializer`. What it does **not** capture: the HTTP request body, request headers, or local variables at the throw site. Omitting the body is deliberate — request bodies often contain sensitive data, and shipping them to telemetry is a compliance risk. To enrich a specific exception you catch it, attach properties, and re-throw, or implement an `ITelemetryProcessor`. Both are deliberate, code-level choices.
>
> ⚠ **Common Mistakes**
>
> - Querying immediately after the first `curl` and seeing nothing — ingestion delay is one to three minutes. Wait, then refresh.
> - Searching by exception *message* instead of *type* — the Failures blade groups by type, so a custom message that varies per request fragments the view. Throw exceptions with stable types and put the variable detail in the message.
>
> √ **Quick check:** Failures blade shows exactly one exception type (`InvalidOperationException`) with five occurrences, all originating from `GET Home/Boom`.

Step 12: *Add a custom metric for home-page views*

Requests, dependencies, and exceptions are auto-instrumented. Custom metrics are not — they are the first telemetry

where you write code. Inject `TelemetryClient`, call `GetMetric("name").TrackValue(value)` wherever the event happens; the SDK aggregates client-side and ships per-minute summaries, cheap even at high volume.

1. **Open** `Controllers/HomeController.cs`, add `using Microsoft.ApplicationInsights;`, then inject `TelemetryClient` alongside the existing `ILogger` and call `GetMetric` from `Index`:

   > `Controllers/HomeController.cs`

   ```
   private readonly ILogger<HomeController> _logger;
   private readonly TelemetryClient _telemetry;

   public HomeController(ILogger<HomeController> logger,
           TelemetryClient telemetry)
   {
       _logger = logger;
       _telemetry = telemetry;
   }

   public IActionResult Index()
   {
       // GetMetric returns a pre-aggregated metric that
           buckets values
       // client-side and ships one-minute summaries. Cheap
           at any volume.
       _telemetry.GetMetric("home-page-
           views").TrackValue(1);

       // ...your existing Index code below
       return View();
   }
   ```

   Leave the `Boom` action from Step 10 unchanged.

2. **Commit, push, and generate traffic** once the new revision is running:

```
git add Controllers/HomeController.cs
git commit -m "Track home-page views as a custom metric"
git push && gh run watch

for i in {1..30}; do curl -s "https://$FQDN/" >/dev/
        null; sleep 0.5; done
```

3. **Wait** ~2 minutes for the per-minute aggregation to ship and ingestion to complete.
4. **Find** the metric in the Portal: Application Insights `cloudci-insights` → **Metrics**. In the picker, choose namespace `azure.applicationinsights`, metric `home-page-views`, aggregation `Sum` or `Count`. Set the time range to the last 30 minutes — you should see a clear bump where the curl loop ran.

CONCEPT DEEP DIVE

`GetMetric` is the cheap path: the SDK keeps an in-memory bucket per metric per one-minute window and ships a single telemetry item with count, sum, min, max, and standard deviation at the end of each window. At a thousand requests per second, you ship one item per minute per metric — not 60,000. The expensive path is `TrackEvent` with a numeric property, which sends one item per occurrence; useful when each occurrence has unique properties you need to keep, but it goes through ingestion, sampling, and storage like any other event.

⚠ **Common Mistakes**

- Newing up `new TelemetryClient()` instead of injecting it. The injected instance has the connection string and `ITelemetryInitializer` chain wired up; a manually constructed one does not, and silently sends to nowhere.
- Picking the wrong namespace in the Metrics picker. Custom metrics live under `azure.applicationinsights` (or the namespace `Custom` depending on Portal version). The default `Microsoft.Insights/Components` namespace shows the platform metrics, not yours.

✓ **Quick check:** The Metrics chart shows a non-zero `home-page-views` count over the last 30 minutes that roughly matches how many times you curled the home page.

Step 13: *Test Your Implementation*

Walk through every signal you've wired up, end to end.

1. **Live Metrics** shows incoming requests within seconds when you `curl` the home page:

```
for i in {1..20}; do curl -s "https://$FQDN/" >/dev/
        null; sleep 0.5; done
```

2. **Application Map** shows exactly one node with non-zero traffic — your app, no dependency edges.
3. **Failures blade** shows `InvalidOperationException` with at least one occurrence after hitting `/Home/Boom` (allow ~1 min for ingestion):

```
curl -s -o /dev/null -w "%{http_code}\n" "https://$FQDN/
        Home/Boom"
```

4. **Metrics blade** plots a non-zero `home-page-views` count over the last 30 minutes.
5. **Secret wiring is correct** — the env var is a `secretRef`, not a literal:

```
az containerapp show -g rg-cicd-week4 -n ca-cicd-week4 \
  --query 'properties.template.containers[0].env[?
        name==`APPLICATIONINSIGHTS_CONNECTION_STRING`]'
        \
  -o json
```

   Expected: an object with `secretRef: "appinsights-connstr"` and **no** `value` field.
6. **Workspace-based mode** — App Insights and container logs share a workspace:

```
az monitor app-insights component show \
  --app cloudci-insights -g rg-cicd-week4 \
  --query workspaceResourceId -o tsv
```

   Expected: the same `$WS_ID` from Step 2.

> √ ***Final verification checklist:***
>
> - [ ] *Application Insights component* `cloudci-insights` *exists in* `rg-cicd-week4` *and is workspace-based*
> - [ ] *Container Apps secret* `appinsights-connstr` *is set; env var references it via* `secretRef:`
> - [ ] *The SDK is registered in* `Program.cs` *via* `AddApplicationInsightsTelemetry()`
> - [ ] `/Home/Boom` *throws* `InvalidOperationException` *and the Failures blade lists it*
> - [ ] `home-page-views` *custom metric appears in the Metrics blade*
> - [ ] *The pipeline still deploys cleanly on push to* `main`

Step 14: *Tear down the cloud resources*

This is the last exercise of the chapter and the last exercise that uses the Week 4 cloud resources. Nothing later in the course needs `rg-cicd-week4`, the Container App, the ACR, the Log Analytics workspace, the Application Insights component, or the Entra app registration created for OIDC federation. Tear all of it down so you finish with no resources running and no orphaned tenant-level identities.

The work splits into two homes — the Azure subscription holds the running resources, and Microsoft Entra ID (a

tenant-level service) holds the identity used by the pipeline. Deleting one does **not** delete the other, exactly as you saw at the end of the previous chapter.

1. **Delete** the resource group. This removes the Container App, the ACR with all its images, the Container Apps environment, the Log Analytics workspace, the Application Insights component (which lives inside the resource group, so the RG delete handles it implicitly), and every role assignment scoped under the group:

```
# 1. Delete the resource group (Container App, ACR, Log
        Analytics, App Insights — all of it).
az group delete -n rg-cicd-week4 --yes --no-wait
```

   `--no-wait` returns immediately and lets the deletion run in the background. The full teardown takes a few minutes (the Container Apps environment is the slowest piece).

2. **Delete** the Entra app registration that was created for OIDC federation in the previous chapter. The app registration lives in the tenant, not in any subscription resource group, so the RG delete above does **not** remove it. Delete it explicitly:

```
# 2. Delete the Entra app registration that was created
        for OIDC federation.
# The app registration lives in the tenant, not in any
        subscription resource group,
# so the RG delete above does NOT remove it. Delete it
        explicitly.
az ad app delete --id
        7c11e4ce-91cd-4ba3-9fce-820669f397fe
```

   Replace the GUID above with your own app's `appId` if it differs — this is the same value you stored as `AZURE_CLIENT_ID` in the previous chapter. You can re-discover it with

`az ad app list --display-name "github-cicd-oidc" --query "[0].appId" -o tsv` if you no longer have it handy.

3. **Verify** the resource group is gone (give it a few minutes if the `--no-wait` deletion is still in progress):

```
az group exists -n rg-cicd-week4
```

   Expected: `false`.

4. **Verify** the Entra app registration is gone:

```
az ad app list --display-name github-cicd-oidc -o tsv
```

   Expected: empty output (no rows).

CONCEPT DEEP DIVE

Why the two-step cleanup matters. The resource group is a subscription-level container — `az group delete` cascades to every Azure resource scoped under it, including the Container App, the ACR, the Log Analytics workspace, the App Insights component, the Container Apps environment, and every role assignment scoped to those resources.

The Entra app registration, however, is a *tenant-level* identity object. It lives in Microsoft Entra ID, not in your Azure subscription, and survives every subscription operation. Without the second `az ad app delete`, the federated identity sticks around as orphaned tenant clutter — visible in the Entra portal with broken role assignments pointing at deleted resources, contributing nothing and confusing future you. Note that Entra ID *soft-deletes* app registrations for 30 days; they're recoverable from **App registrations → Deleted applications** and garbage-collected automatically afterwards.

⚠ **Common Mistakes**

- Stopping after `az group delete`. The Entra app stays alive and accumulates over multiple cohorts as orphaned `github-cicd-oidc` entries.
- Forgetting that the GitHub repo secrets `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, and `AZURE_SUBSCRIPTION_ID` in your repository are now stale. The next workflow run will fail at **Sign in to Azure** because the client ID points at a deleted app. Either delete the secrets via `gh secret delete <name>`, or leave them alone — they're inert without a working app behind them.

✓ **Quick check:** `az group exists -n rg-cicd-week4` returns `false`, `az ad app list --display-name github-cicd-oidc` returns nothing, and your subscription's billing is no longer accruing for any of the Week 4 / Week 5 resources.

7.6.4 Common Issues

> ***If you encounter problems:***
> ***No telemetry anywhere — Live Metrics, Application Map, Metrics all empty:*** *The Container App didn't receive the connection string. Re-check* `az containerapp show ... --query 'properties.template.containers[0].env'` *for* `APPLICATIONINSIGHTS_CONNECTION_STRING` *with a* `secretRef` *field, and* `az containerapp secret list` *for the underlying secret.*
> ***Telemetry works but the connection string is plainly visible in*** **`az containerapp show`***:* *You set the env var as a literal string instead of* `secretref:`*. Remove the env var (*`--remove-env-vars`*), confirm the secret is set, then re-set with the* `secretref:` *form from Step 6.*
> ***Failures blade empty after hitting*** **`/Home/Boom`***:* *Either ingestion delay (wait 1–3 minutes) or* `ASPNETCORE_ENVIRONMENT=Development` *is letting* `DeveloperExceptionPage` *swallow the exception in the dev pipeline. Container Apps default to Production.*
> ***Custom metric*** **`home-page-views`** ***missing from the Metrics blade:*** *Either you used* `new TelemetryClient()` *instead of injecting it (manually constructed clients have no connection string), the namespace picker is wrong (custom metrics live under* `azure.applicationinsights` *or* `Custom`*, not* `Microsoft.Insights/Components`*), or the per-minute aggregate hasn't shipped yet (wait 2–3 minutes).*

> ***Live Metrics says "Not available":*** *The Container App can't reach the live-diagnostics endpoint, the SDK didn't load, or the env var isn't set. Check* `az containerapp logs show -g rg-cicd-week4 -n ca-cicd-week4 --follow` *for SDK errors at startup.*
> `az group delete` ***returns immediately but resources still appear in the Portal:*** *That's* `--no-wait` *working as intended. Re-check with* `az group exists -n rg-cicd-week4`*.*
> ***Still stuck?*** *Verify three things in order: the SDK package in* `CloudCi.csproj`*,* `AddApplicationInsightsTelemetry()` *in* `Program.cs`*, and the Container App's env list. All three must be correct for telemetry to flow.*

7.6.5 Summary

You added Application Insights to a deployed ASP.NET Core app in workspace-based mode, so telemetry lands in the same Log Analytics workspace as your container logs. The connection string is delivered the right way — as a Container Apps secret referenced via `secretref:` — so it never appears in `az containerapp show`. You exercised three Portal blades, each answering a different question: Live Metrics for "is anything happening right now," Application Map for "what is the architecture," Failures for "what is going wrong." One custom metric gives the Metrics blade something of yours to plot.

- ✓ Workspace-based Application Insights — telemetry lives in the same store as container logs
- ✓ Connection string injected as a Container Apps secret, referenced via `secretref:`

- ✓ Auto-instrumented requests, dependencies, and exceptions, no code changes required
- ✓ Custom metric tracked via `TelemetryClient.GetMetric`, aggregated client-side
- ✓ Resource group and Entra app registration both torn down — no orphaned cloud or tenant resources

> **KEY TAKEAWAY:**
>
> Logs answer "what happened, in order." Metrics and traces answer "how is the system performing, in aggregate." A real production observability stack uses both. Application Insights gives you the metrics and traces side as a default-on capability the moment the SDK is registered — you don't need to instrument anything you wouldn't already be writing. The cost of that capability is one NuGet package, one line in `Program.cs`, and one secret on the Container App. The cost of *not* having it is the next outage you investigate from logs alone.

7.6.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Configure an alert rule on the* `home-page-views` *metric or the requests-failed counter (Application Insights → Alerts → New alert rule). Wire it to email or webhook when a threshold is crossed.*
> - *Build a custom Workbook (Application Insights → Workbooks) that combines request rate, p95 latency, and failure count on one page. Workbooks accept KQL against the workspace, so logs and App Insights data sit side by side.*
> - *Read up on the two senses of* ***trace****: trace telemetry in Application Insights (*`ILogger`*lines) versus distributed tracing in the OpenTelemetry sense (one operation crossing service boundaries via the W3C Trace Context header). The vocabulary collision is the single biggest stumbling block in this space.*
> - *Investigate* ***OpenTelemetry*** *as the future direction. The OpenTelemetry .NET SDK with the Azure Monitor exporter writes to the same Application Insights backend, but lets you retarget Datadog, New Relic, or self-hosted Jaeger with only an exporter change.*

7.6.7 Done!

This exercise ends the deployment block of the course. You have a CI/CD pipeline that ships every push to `main`, an OIDC trust relationship that authenticates without a long-lived secret, structured logs flowing into Log Analytics, and now an

APM layer on top of the same workspace that gives you Live Metrics, the Application Map, the Failures blade, and a custom metric. Together those pieces are what people mean when they say “the deployment is observable in production.”

The lab is now torn down. The resource group is gone, the Entra app registration is gone, the GitHub secrets in your repo are stale and inert. There is nothing left running and nothing left billing.

The next thing the course tackles is building services that call other services. The Application Map you saw with one node will start to fill out as soon as you have a downstream API to call — the SDK auto-instrumentation does the work for free, and the same operation ID that ties your `ILogger` lines together also ties the request in service A to the request in service B to the SQL query in the database. That is what distributed tracing buys you, and it is what the rest of the program will spend time exploring.

PART IV

Services and APIs

EXERCISE 8

REST API and DTOs

8.1 REST Controllers, DTOs, and Swagger

8.2 REST Controllers, DTOs, and Swagger

8.2.1 Goal

Stand up a fresh ASP.NET Core Web API called `CloudCiApi` and ship it to Azure Container Apps. The surface is intentionally small — a `Quotes` resource backed by an in-memory store — so the focus stays on the patterns that scale: a controllers-based project layout, a deliberate split between domain entities and wire-format DTOs, and Swagger as the runtime contract document. Once it builds locally, you'll wrap it in a Dockerfile and reuse the OIDC-federated pipeline from

the previous chapter to deploy it, with Application Insights layered on top from the first request.

WHAT YOU'LL LEARN:

- When a controllers-based Web API is the right shape and when minimal APIs are
- Why entity types and DTOs should be separate even before a database exists
- How `[ApiController]` plus attribute routing produces idiomatic 200 / 201 / 404 / 400 responses
- How `CreatedAtAction` builds the `Location` header on POST for free
- The trade-off in leaving Swagger on in production
- How to apply the OIDC-federated pipeline pattern to a brand-new repo with new resource names

8.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- √ .NET 10 SDK installed (`dotnet --version` reports a `10.*` version)
- √ Docker Desktop running, and `docker --version` succeeds
- √ The `az` CLI signed in (`az login`) and the `gh` CLI signed in (`gh auth status`)
- √ Completed the deployment chapter — you've previously built a GitHub Actions pipeline that uses OIDC federation to push to ACR and update a Container App
- √ Completed the logging and monitoring chapter — you've previously injected an Application Insights connection string as a Container Apps secret via `secretref:`
- √ Permission to create Entra app registrations in your tenant and Owner (or User Access Administrator) on the subscription

8.2.3 Exercise Steps

Overview

1. **Scaffold the `CloudCiApi` Web API project**
2. **Create the `Quote` entity**
3. **Create the `IQuoteStore` and seed data**
4. **Create the `QuoteDto` and `CreateQuoteRequest`**
5. **Register the store and Swagger in `Program.cs`**
6. **Build the `QuotesController` with three actions**
7. **Run locally and verify Swagger and the JSON endpoint**
8. **Containerize with a Dockerfile and `.dockerignore`**
9. **Push the project to a fresh GitHub repo**

10. **Set up the OIDC-federated pipeline against new Azure resources**
11. **Add Application Insights in workspace-based mode**
12. **Test Your Implementation**

Step 1: *Scaffold the* `CloudCiApi` *Web API project*

Start from the `webapi` template with `--use-controllers`. The default `dotnet new webapi` template in .NET 8 and later is minimal-API by default; the explicit flag asks for the older controllers-based shape, which is what you want for an API that will grow beyond a few endpoints.

1. **Create** the project folder and scaffold the API:

```
dotnet new webapi -o CloudCiApi --use-controllers --
        framework net10.0
cd CloudCiApi
```

   The `--framework net10.0` argument pins the project to .NET 10 so the `<TargetFramework>` in the `.csproj` matches the `mcr.microsoft.com/dotnet/sdk:10.0` and `aspnet:10.0` base images you'll use in the Dockerfile later in this exercise.

2. **Verify** it builds and runs:

```
dotnet build
dotnet run --launch-profile http
```

   The terminal should report `Now listening on: http://localhost:<port>`. Stop the server with `Ctrl+C` once you've seen it start.

3. **Delete** the demo `WeatherForecast.cs` model and `Controllers/WeatherForecastController.cs`. They're scaf-

fold noise and you're about to replace them with your own resource.

```
rm WeatherForecast.cs Controllers/WeatherForecastCon-
        troller.cs
```

4. **Remove** the built-in OpenAPI package. The .NET 10 template auto-includes `Microsoft.AspNetCore.OpenApi`, which competes with Swashbuckle (the OpenAPI generator this exercise uses). Strip it now so the package set stays consistent.

```
dotnet remove package Microsoft.AspNetCore.OpenApi
```

> **CONCEPT DEEP DIVE: CONTROLLERS VS MINIMAL APIS**
>
> ASP.NET Core ships two shapes for HTTP services. **Minimal APIs** declare endpoints inline in `Program.cs` with `app.MapGet("/path", handler)` — terse, low ceremony, ideal for tiny services where the routing table fits on one screen. **Controllers** declare endpoints as methods on a class decorated with `[ApiController]` and attribute routes — more boilerplate per endpoint, but they pay off the moment you need cross-cutting concerns like model binding, action filters, attribute-based authorization, content negotiation, or per-action OpenAPI metadata. Pick minimal APIs when the entire service is one or two endpoints with no auth, no validation, and no shared filters; pick controllers as soon as any of those become true.
>
> ⚠ **Common Mistakes**
>
> - Forgetting `--use-controllers` and ending up with a minimal-API scaffold whose `Program.cs` has `app.MapGet("/weatherforecast", ...)` inline. Re-run with `--force` to overwrite.
>
> ✓ **Quick check:** `CloudCiApi.csproj` exists, `Controllers/` is empty after the deletion, and `dotnet build` returns no errors.

Step 2: *Create the* `Quote` *entity*

The `Quote` is your domain type — the shape your application code reasons about. There is no database in this chapter; storage comes later. That does not change what the entity is for. Even in memory, the entity captures the *concept* the API is about, with whatever fields the rest of the code needs to do work — including fields you may not want on the wire.

1. **Create** a `Models` directory and add the entity:

> `Models/Quote.cs`

```
namespace CloudCiApi.Models;

public class Quote
{
    public int Id { get; set; }
    public string Author { get; set; } = string.Empty;
    public string Text { get; set; } = string.Empty;
    public DateTimeOffset CreatedAt { get; set; }
}
```

> **CONCEPT DEEP DIVE**
>
> The entity is plain — no attributes, no validation, no JSON serializer hints. Those concerns belong on the DTO. Keeping the entity attribute-free is what lets you later add Entity Framework mappings, persistence-only fields like `RowVersion`, or audit columns like `ModifiedBy`, without those leaking into the wire format.
>
> ✓ **Quick check:** `dotnet build` succeeds. The file lives at `Models/Quote.cs` with namespace `CloudCiApi.Models`.

Step 3: *Create the* `IQuoteStore` *and seed data*

The store is the abstraction the controller depends on. By coding to an interface from day one, you can swap the in-memory implementation for a database-backed one in a later chapter without touching the controller. The seeded quotes are what the API serves out of the box, so a freshly-deployed instance is never empty.

1. **Create** a `Services` directory and add the interface:

   > `Services/IQuoteStore.cs`

```
using CloudCiApi.Models;

namespace CloudCiApi.Services;

public interface IQuoteStore
{
    IReadOnlyList<Quote> GetAll();
    Quote? GetById(int id);
    Quote Add(string author, string text);
}
```

2. **Add** the in-memory implementation:

> `Services/InMemoryQuoteStore.cs`

```
using CloudCiApi.Models;

namespace CloudCiApi.Services;

public class InMemoryQuoteStore : IQuoteStore
{
    private readonly List<Quote> _quotes = new();
    private int _nextId = 1;
    private readonly object _gate = new();

    public InMemoryQuoteStore()
    {
        Add("Edsger W. Dijkstra",
            "Simplicity is prerequisite for reliabil-
        ity.");
        Add("Grace Hopper",
            "The most dangerous phrase in the language
        is 'we've always done it this way.'");
        Add("Alan Kay",
            "The best way to predict the future is to
        invent it.");
        Add("Linus Torvalds",
            "Talk is cheap. Show me the code.");
    }

    public IReadOnlyList<Quote> GetAll()
    {
        lock (_gate)
        {
            return _quotes.ToList();
        }
    }

    public Quote? GetById(int id)
    {
        lock (_gate)
        {
            return _quotes.FirstOrDefault(q => q.Id ==
        id);
        }
    }

    public Quote Add(string author, string text)
```

```
    {
        lock (_gate)
        {
            var quote = new Quote
            {
                Id = _nextId++,
                Author = author,
                Text = text,
                CreatedAt = DateTimeOffset.UtcNow,
            };
            _quotes.Add(quote);
            return quote;
        }
    }
}
```

CONCEPT DEEP DIVE

The store is registered as a **singleton** in the next step — one instance services every request across every thread. That's why mutable state is guarded with a `lock`: without it, two simultaneous POSTs could read `_nextId` at the same instant and both write the same id. A real production app would use a database transaction or `Interlocked.Increment`; the lock is the smallest correct version for an in-memory teaching store.

⚠ **Common Mistakes**

- Registering this as `Scoped` instead of `Singleton`. Each request would get a fresh empty list — the API would respond `[]` to the very next GET after a POST.

√ **Quick check:** Both files compile.

Step 4: *Create the* `QuoteDto` *and* `CreateQuoteRequest`

This is the step where the entity-vs-DTO distinction earns its keep. The `Quote` entity is the *domain* shape — what your application code uses internally. The DTOs are the *wire* shapes — what the API speaks to clients. Keeping them separ-

ate from the start, even when the fields are nearly identical, is what stops the wire contract from changing every time the domain evolves.

1. **Create** a `Dtos` directory and add the output DTO:

   > `Dtos/QuoteDto.cs`

```
namespace CloudCiApi.Dtos;

public class QuoteDto
{
    public int Id { get; set; }
    public string Author { get; set; } = string.Empty;
    public string Text { get; set; } = string.Empty;
    public DateTimeOffset CreatedAt { get; set; }
}
```

2. **Add** the input DTO:

   > `Dtos/CreateQuoteRequest.cs`

```
using System.ComponentModel.DataAnnotations;

namespace CloudCiApi.Dtos;

public class CreateQuoteRequest
{
    [Required]
    [StringLength(100, MinimumLength = 1)]
    public string Author { get; set; } = string.Empty;

    [Required]
    [StringLength(500, MinimumLength = 1)]
    public string Text { get; set; } = string.Empty;
}
```

CONCEPT DEEP DIVE: WHY DTOS EXIST BEFORE THERE'S A DATABASE

A DTO ("Data Transfer Object") is the **wire contract** — the exact shape clients send and receive. The entity is the **domain model** — the shape your code reasons about. Even when both look identical today, three forces will pull them apart:

- **Input is not the same as output.** A client creating a quote should not choose the `Id` (the server assigns it) or `CreatedAt` (the server stamps it). `CreateQuoteRequest` has neither field. The compiler enforces this — there is no way for client JSON to set them.
- **The domain grows fields the wire shouldn't see.** As the entity gains internal-only fields (audit columns, soft-delete flags, EF navigation properties), the DTO stays trimmed to what clients legitimately need. Without the split, you'd either leak internals or break the wire format every time you refactor internals.
- **Validation lives on the wire.** `[Required]` and `[StringLength]` describe what a *client request* must look like. The entity represents already-valid in-memory state. Putting validation on the entity couples request validation and domain invariants, which evolve at different rates.

Entity types are the domain, DTO types are the wire contract, and decoupling them lets each evolve independently. You'll feel the saving the first time you add a database — the entity gains EF attributes the wire never sees.

⚠ **Common Mistakes**

- "But it's the same fields" — yes, today. The point is that the *types* are different, so when they diverge tomorrow the change is local.

> - Reusing one type for both input and output and excluding `Id` with `[JsonIgnore]`. That works at runtime but invites bugs the moment someone removes the attribute.
>
> ✓ **Quick check:** Both DTO files compile in the `CloudCiApi.Dtos` namespace.

Step 5: *Register the store and Swagger in* `Program.cs`

Now wire the store into dependency injection and add Swagger. Swagger is a runtime contract document — it reads the controller's signatures via reflection and produces an OpenAPI JSON file plus a browsable UI. Anyone with the URL can discover the API's shape without seeing your source.

1. **Add** the Swashbuckle package, pinned to 6.6.2:

   ```
   dotnet add package Swashbuckle.AspNetCore --version
           6.6.2
   ```

   The version pin is deliberate. Swashbuckle 10.x (the unpinned latest) consolidates the legacy `Microsoft.OpenApi.Models` namespace into the root `Microsoft.OpenApi` namespace; the security-scheme code in the next two exercises uses the legacy `.Models` namespace, so pinning to 6.6.2 keeps every snippet copy-pasteable.

2. **Replace** the contents of `Program.cs`:

   > `Program.cs`

```
using CloudCiApi.Services;

var builder = WebApplication.CreateBuilder(args);

// Singleton — one in-memory store shared across every
        request.
builder.Services.AddSingleton<IQuoteStore,
        InMemoryQuoteStore>();

builder.Services.AddControllers();

// OpenAPI / Swagger surface.
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Swagger is enabled in BOTH Development AND Production
        for this course.
// See the Concept Deep Dive below for the trade-off.
app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

CONCEPT DEEP DIVE: SWAGGER IN PRODUCTION — USEFUL DEFAULT, REAL TRADE-OFF

The default `dotnet new webapi` template wraps `app.UseSwagger()` and `app.UseSwaggerUI()` in `if (app.Environment.IsDevelopment())`, hiding the docs from anything that's not your laptop. This exercise deliberately removes that gate.

Why leave it on in production: for internal services and learning artifacts, Swagger is the contract. Hiding it forces consumers to read source or scrape the OpenAPI JSON from a build artifact. For this exercise the deployed `/swagger` page is your demo URL — proof that the API is alive.

Why a real public-facing API typically does not: Swagger reveals every route, parameter, and response shape — fine for a known public contract, reconnaissance for an attacker on an internal admin API. The interactive UI is also a chunk of JavaScript you don't strictly need to ship; keeping it means a thing to patch and audit.

The middle ground in real production: ship the OpenAPI JSON at `/swagger/v1/swagger.json` but gate the interactive UI behind auth, or remove the UI from production builds and host the docs separately. For this chapter we leave both on — "Swagger on in production" is a deliberate choice, not the only right answer.

⚠ **Common Mistakes**

- Forgetting `AddEndpointsApiExplorer()` — `AddSwaggerGen()` then has nothing to introspect, and the `/swagger` page renders empty.
- Calling `UseSwagger()` after `MapControllers()` — Swagger middleware must register before the endpoint routing terminates the pipeline.

√ **Quick check:** `dotnet build` succeeds with no warnings.

Step 6: *Build the* `QuotesController` *with three actions*

The controller is the HTTP-facing surface. With `[ApiController]` and `[Route("api/[controller]")]`, ASP.NET Core gives you attribute-routed endpoints, automatic 400 responses on validation failures, and convention-based binding from the request body. Three actions cover the shape of a typical resource: list, get-by-id, and create.

1. **Create** the controller file:

> `Controllers/QuotesController.cs`

```
using CloudCiApi.Dtos;
using CloudCiApi.Models;
using CloudCiApi.Services;
using Microsoft.AspNetCore.Mvc;

namespace CloudCiApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class QuotesController : ControllerBase
{
    private readonly IQuoteStore _store;

    public QuotesController(IQuoteStore store)
    {
        _store = store;
    }

    [HttpGet]
    public ActionResult<IEnumerable<QuoteDto>> GetAll()
    {
        var dtos = _store.GetAll().Select(ToDto);
        return Ok(dtos);
    }

    [HttpGet("{id:int}")]
    public ActionResult<QuoteDto> GetById(int id)
    {
        var quote = _store.GetById(id);
        if (quote is null)
        {
            return NotFound();
        }
        return Ok(ToDto(quote));
    }

    [HttpPost]
    public ActionResult<QuoteDto> Create([FromBody] Cre-
        ateQuoteRequest request)
    {
        var quote = _store.Add(request.Author,
        request.Text);
```

```
        var dto = ToDto(quote);

        // 201 Created with a Location header pointing
        back at GetById.
        return CreatedAtAction(
            nameof(GetById),
            new { id = quote.Id },
            dto);
    }

    private static QuoteDto ToDto(Quote q) => new()
    {
        Id = q.Id,
        Author = q.Author,
        Text = q.Text,
        CreatedAt = q.CreatedAt,
    };
}
```

CONCEPT DEEP DIVE

`[ApiController]` opts the controller into automatic model-state validation (a malformed request body returns `400` with a `ProblemDetails` body before your action runs), into binding source inference (`[FromBody]` is implied for complex types), and into attribute-based routing only.

`CreatedAtAction(nameof(GetById), new { id = quote.Id }, dto)` produces three things in one call: a `201 Created` status, a `Location` response header pointing at the freshly-created resource, and the resource itself in the body. The framework computes the URL from the named action — you never hard-code `"/api/quotes/" + id`, so a route refactor doesn't break clients.

The controller speaks DTOs both ways: `Quote` is converted to `QuoteDto` before going out, and `CreateQuoteRequest` is unpacked into `_store.Add(...)` rather than passed through. The entity never appears in any method signature. That's the discipline from Step 4 paying off.

⚠ **Common Mistakes**

- Returning the entity directly (`return Ok(quote)`). Works today — the JSON serializer produces almost-the-same shape — but the moment the entity gains an internal field, that field leaks. Return the DTO every time.
- Computing the `Location` header by hand. Use `CreatedAtAction` so the URL stays correct under refactoring.
- Forgetting the `:int` route constraint. With it, `/api/quotes/abc` returns `404`; without it, you get a `400` from a binding failure.

√ **Quick check:** `dotnet build` succeeds.

Step 7: *Run locally and verify Swagger and the JSON endpoint*

A quick local round-trip proves all three pieces — the store, the controller, the Swagger pipeline — are wired correctly before you start adding deployment plumbing.

1. **Start** the API:

```
dotnet run --launch-profile http
```

   Note the port the terminal reports (typically `5xxx`); call it `<port>` below.

2. **Open** `http://localhost:<port>/swagger` in a browser. You should see one tag, **Quotes**, expanded to three operations: `GET /api/quotes`, `GET /api/quotes/{id}`, and `POST /api/quotes`.

3. **Hit** the JSON endpoint directly:

```
curl -s http://localhost:<port>/api/quotes | jq
```

   Expected: a JSON array of four objects with `id`, `author`, `text`, and `createdAt` fields.

4. **Try** a `POST` with valid JSON:

```
curl -i -X POST http://localhost:<port>/api/quotes \
  -H "Content-Type: application/json" \
  -d '{"author":"Donald Knuth","text":"Premature optim-
       ization is the root of all evil."}'
```

   Expected: `HTTP/1.1 201 Created`, a `Location: http://localhost:<port>/api/Quotes/5` header, and a body containing the new quote with `id` set to `5` and `createdAt` populated.

5. **Try** a `POST` with a missing field:

```
curl -i -X POST http://localhost:<port>/api/quotes \
  -H "Content-Type: application/json" \
  -d '{"author":"Anonymous"}'
```

Expected: `HTTP/1.1 400 Bad Request` with a `ProblemDetails` body listing `Text` as the validation error. This is `[ApiController]` doing automatic model-state validation.

6. **Stop** the server with `Ctrl+C`.

> **QUICK CHECK:**
> Swagger renders three operations under `Quotes`. `GET /api/quotes` returns four seeded items. `POST` with valid input returns `201` with a `Location` header. `POST` with missing fields returns `400`.

Step 8: *Containerize with a Dockerfile and* `.dockerignore`

The pipeline you'll set up next builds a Docker image and pushes it to ACR — that requires a Dockerfile. Use the same multi-stage `.NET SDK → ASP.NET runtime` pattern from the deployment chapter, listening on port `8080` (Container Apps' default ingress port).

1. **Add** the Dockerfile at the project root:

> `Dockerfile`

```
# Build stage — full SDK, restores and publishes a
        Release build.
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . ./
RUN dotnet publish -c Release -o /app /p:Use-
        AppHost=false

# Runtime stage — slim ASP.NET image, no SDK.
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app ./

# Container Apps' default ingress targets 8080.
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080

ENTRYPOINT ["dotnet", "CloudCiApi.dll"]
```

2. **Add** a `.dockerignore` to keep the build context small:

> `.dockerignore`

```
bin/
obj/
.vs/
.vscode/
.idea/
*.user
.git/
.github/
Dockerfile
.dockerignore
README.md
```

3. **Verify** the image builds locally:

```
docker build -t cloudci-api:local .
docker run --rm -p 8080:8080 cloudci-api:local
```

In another terminal:

```
curl -s http://localhost:8080/api/quotes | jq '. |
        length'
```

Expected: `4`. Stop the container with `Ctrl+C`.

> **CONCEPT DEEP DIVE**
>
> The two-stage build keeps the runtime image small. The build stage uses the full .NET SDK image (~700 MB) but is discarded; only `/app` is copied into the runtime stage's ASP.NET-only image (~220 MB). `ENV ASPNET-CORE_URLS=http://+:8080` makes Kestrel listen on the port Container Apps expects — without it, Kestrel binds to its default `5xxx` and the platform health probe fails.
>
> ✓ **Quick check:** `docker run` serves `/api/quotes` on port 8080 with four seeded entries.

Step 9: *Push the project to a fresh GitHub repo*

The federated credential you create in the next step authenticates from a specific repo on a specific branch — the repo must exist before the credential. Create it now and push.

1. **Initialize** a git repo and make a first commit:

```
git init
git add .
git commit -m "Scaffold CloudCiApi with Quotes control-
        ler and Dockerfile"
```

2. **Create** the GitHub repository and push (substitute your GitHub username for `<your-username>`):

```
gh repo create <your-username>/cloudci-api --public --
        source=. --push
```

COMMON MISTAKES

- Forgetting the federated credential subject will be `repo:<your-username>/cloudci-api:ref:refs/heads/main`. Renaming the repo or moving it to a GitHub organisation changes the subject and breaks authentication.

√ **Quick check:** `gh repo view <your-username>/cloudci-api --web` opens the repo and the source tree matches your local working copy.

Step 10: *Set up the OIDC-federated pipeline against new Azure resources*

You set this up in detail in the deployment chapter — a fresh resource group, an ACR, a Container Apps environment, a Container App, an Entra app registration with two role assignments and a federated credential, and a workflow that signs in via OIDC, pushes the image, and updates the Container App. Here it's a quick recap with new names. If any sub-step feels unfamiliar, jump back to that chapter for the full explanation.

1. **Provision** the Azure resources. Pick a four-character random suffix for the ACR name (it must be globally unique). Substitute it into `<rand>` below:

```
az group create -n rg-api-week6 -l northeurope

az acr create -n acrapi<rand> -g rg-api-week6 \
  --sku Basic --admin-enabled false

az containerapp env create -n cae-api-week6 -g rg-api-
        week6 \
  -l northeurope

az containerapp create \
  -n ca-api-week6 \
  -g rg-api-week6 \
  --environment cae-api-week6 \
  --image mcr.microsoft.com/k8se/quickstart:latest \
  --target-port 8080 \
  --ingress external \
  --min-replicas 1 \
  --max-replicas 1
```

The placeholder `mcr.microsoft.com/k8se/quickstart:latest` image is just so the Container App can be created before your image exists; the pipeline will replace it on the first push.

2. **Create** the Entra app, the service principal, the two role assignments, and the federated credential. **Substitute your GitHub username** for `<your-username>` in the federated subject — leaving the angle brackets in literally is the single most common authentication failure in this exercise.

```
az ad app create --display-name "github-cloudci-api-
        oidc"
export APP_ID="<paste appId from previous output>"
az ad sp create --id "$APP_ID"

ACR_ID=$(az acr show -n acrapi<rand> --query id -o tsv)
CA_ID=$(az containerapp show -n ca-api-week6 -g rg-api-
        week6 --query id -o tsv)

az role assignment create --assignee "$APP_ID" --role
        AcrPush --scope "$ACR_ID"
az role assignment create --assignee "$APP_ID" \
  --role "Container Apps Contributor" --scope "$CA_ID"

az ad app federated-credential create \
  --id "$APP_ID" \
  --parameters '{
    "name": "main-branch",
    "issuer": "https://
        token.actions.githubusercontent.com",
    "subject": "repo:<your-username>/cloudci-
        api:ref:refs/heads/main",
    "audiences": ["api://AzureADTokenExchange"],
    "description": "GitHub Actions, main branch only"
  }'
```

3. **Add** the four GitHub secrets:

```
gh secret set AZURE_CLIENT_ID --body "$APP_ID"
gh secret set AZURE_TENANT_ID --body "$(az account show
        --query tenantId -o tsv)"
gh secret set AZURE_SUBSCRIPTION_ID --body "$
        (az account show --query id -o tsv)"
gh secret set ACR_NAME --body "acrapi<rand>"
```

4. **Add** the workflow at `.github/workflows/ci.yml`:

> `.github/workflows/ci.yml`

```yaml
name: build-and-deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v4

      - name: Sign in to Azure
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: $
        {{ secrets.AZURE_SUBSCRIPTION_ID }}

      - name: Log in to ACR
        run: az acr login --name ${{ secrets.ACR_NAME }}

      - name: Build and push image
        run: |
          IMAGE="${{ secrets.ACR_NAME }}.azurecr.io/
        cloudci-api:${{ github.sha }}"
          docker build -t "$IMAGE" .
          docker push "$IMAGE"
          echo "IMAGE=$IMAGE" >> "$GITHUB_ENV"

      - name: Update Container App
        run: |
          az containerapp update \
            -n ca-api-week6 \
            -g rg-api-week6 \
            --image "$IMAGE"
```

```
      - name: Smoke test
        run: |
          FQDN=$(az containerapp show \
            -n ca-api-week6 -g rg-api-week6 \
            --query properties.configuration.ingress.fqdn -o tsv)
          for i in {1..20}; do
            if curl -fsS "https://$FQDN/api/quotes" >/dev/null; then
              echo "Smoke test passed."
              exit 0
            fi
            sleep 3
          done
          echo "Smoke test failed."
          exit 1
```

5. **Grant** the Container App pull access to ACR. Container Apps uses managed identity to pull images; the simplest path is to enable the system-assigned identity and grant it `AcrPull`:

```
az containerapp identity assign \
  -n ca-api-week6 -g rg-api-week6 \
  --system-assigned

IDENTITY_PRINCIPAL_ID=$(az containerapp show \
  -n ca-api-week6 -g rg-api-week6 \
  --query identity.principalId -o tsv)

az role assignment create \
  --assignee "$IDENTITY_PRINCIPAL_ID" \
  --role AcrPull \
  --scope "$ACR_ID"

az containerapp registry set \
  -n ca-api-week6 -g rg-api-week6 \
  --server acrapi<rand>.azurecr.io \
  --identity system
```

6. **Push** to trigger the first real deployment:

```
git add .github/workflows/ci.yml
git commit -m "Add OIDC-federated build-and-deploy work-
        flow"
git push
gh run watch
```

7. **Verify** the deployed FQDN serves your quotes:

```
FQDN=$(az containerapp show \
  -n ca-api-week6 -g rg-api-week6 \
  --query properties.configuration.ingress.fqdn -o tsv)
echo "https://$FQDN/swagger"
curl -s "https://$FQDN/api/quotes" | jq '. | length'
```

Expected: `4`. Open the printed `https://$FQDN/swagger` URL — you should see the same three operations under `Quotes` you saw locally.

CONCEPT DEEP DIVE

Nothing in this step is conceptually new — it's the same chain of objects from the previous chapter (resource group → ACR → Container Apps environment → Container App → Entra app → service principal → role assignments → federated credential → workflow) with new names. Re-applying a pattern to a new project is when you find out which parts you actually internalised. If any sub-step felt blurry, that's the spot to revisit.

⚠ **Common Mistakes**

- `--target-port` must be `8080` to match the Dockerfile.
- Leaving angle brackets in the federated credential subject literally — substitute your username.
- Skipping the managed-identity / `AcrPull` setup. The pipeline would push the image successfully, but the new revision would fail to pull from ACR.

√ **Quick check:** The workflow run is green, `gh secret list` shows the four secrets, and `https://$FQDN/api/quotes` returns four seeded items.

Step 11: *Add Application Insights in workspace-based mode*

Same pattern as the logging and monitoring chapter — a workspace-based App Insights component, the SDK in the project, the connection string injected as a Container Apps secret referenced via `secretref:`. Only the names change. If any sub-step is fuzzy, revisit that chapter for the full reasoning.

1. **Provision** the App Insights component. The Container Apps environment created an auto-managed Log Analytics workspace inside `rg-api-week6` — point App Insights at it:

```
WS_ID=$(az monitor log-analytics workspace list \
  -g rg-api-week6 \
  --query '[0].id' -o tsv)

az monitor app-insights component create \
  --app cloudci-api-insights \
  -g rg-api-week6 \
  --location northeurope \
  --workspace "$WS_ID"

CONN=$(az monitor app-insights component show \
  --app cloudci-api-insights \
  -g rg-api-week6 \
  --query connectionString -o tsv)
```

2. **Add** the SDK (pinned to 2.22.0) and register it:

```
dotnet add package
        Microsoft.ApplicationInsights.AspNetCore --ver-
        sion 2.22.0
```

The pin is the same one Week 5 used. The unpinned latest is 3.x, which is the new OpenTelemetry-backed line and **throws at host startup** if `APPLICATIONINSIGHTS_CONNECTION_STRING` is not set — fine in production but an annoying paper-cut for `dotnet run` against `localhost`. The 2.x line silently drops telemetry when the connection string is missing, which matches the behaviour every other exercise in this chapter assumes.

In `Program.cs`, add the registration alongside the other service registrations:

> `Program.cs`

```
builder.Services.AddApplicationInsightsTelemetry();
```

3. **Inject** the connection string as a Container Apps secret and bind it to the env var:

```
az containerapp secret set \
  -g rg-api-week6 -n ca-api-week6 \
  --secrets appinsights-connstr="$CONN"

az containerapp update \
  -g rg-api-week6 -n ca-api-week6 \
  --set-env-vars APPLICATIONIN-
        SIGHTS_CONNECTION_STRING=secretref:appinsights-
        connstr
```

4. **Commit and push** the SDK changes:

```
git add CloudCiApi.csproj Program.cs
git commit -m "Add Application Insights SDK"
git push
gh run watch
```

5. **Generate** traffic and look at Live Metrics in the Portal:

```
for i in {1..30}; do curl -s "https://$FQDN/api/quotes"
        >/dev/null; sleep 0.3; done
```

Open Application Insights `cloudci-api-insights` → **Live Metrics** in the Azure Portal. You should see incoming requests within seconds.

> **QUICK CHECK:**
> Live Metrics shows incoming requests when you run the curl loop. `az containerapp show -g rg-api-week6 -n ca-api-week6 --query 'properties.template.containers[0].env'` shows `APPLICATIONINSIGHTS_CONNECTION_STRING` with a `secretRef`, not a literal value.

Step 12: *Test Your Implementation*

End-to-end check against the live deployment.

1. **Capture** the FQDN and open Swagger in a browser. Verify the **Quotes** tag shows `GET /api/quotes`, `GET /api/quotes/{id}`, and `POST /api/quotes`:

```
FQDN=$(az containerapp show \
  -n ca-api-week6 -g rg-api-week6 \
  --query properties.configuration.ingress.fqdn -o tsv)
echo "https://$FQDN/swagger"
```

2. **Exercise** all four code paths:

```
# Happy path: list and get-by-id
curl -s "https://$FQDN/api/quotes" | jq '. |
        length'      # expect 4
curl -i "https://$FQDN/api/quotes/
        1"                      # expect 200
curl -i "https://$FQDN/api/quotes/
        9999"                   # expect 404

# Create
curl -i -X POST "https://$FQDN/api/quotes" \
  -H "Content-Type: application/json" \
  -d '{"author":"Bjarne Stroustrup","text":"Within C++,
        there is a much smaller and cleaner language
        struggling to get out."}'
# expect 201 with Location: https://$FQDN/api/Quotes/5

# Validation failure
curl -i -X POST "https://$FQDN/api/quotes" \
  -H "Content-Type: application/json" \
  -d '{"author":"Anonymous"}'
# expect 400 with ProblemDetails listing Text as the
        missing field
```

The Container App runs one replica, so subsequent GETs from the same FQDN return the new quote — until the replica restarts, when the in-memory store resets.

3. **Confirm** App Insights records the failure. Wait one to three minutes after the bad POST, then in the Azure Portal navigate to Application Insights `cloudci-api-insights` → **Failures**. The `400` shows up — `[ApiController]`'s automatic model-state failures are counted as request failures even though no exception was thrown.

√ ***Success indicators:***

- `https://$FQDN/swagger` *renders the three operations under* `Quotes`
- `GET /api/quotes` *returns the four seeded entries as JSON*
- `GET /api/quotes/9999` *returns* `404`
- `POST /api/quotes` *with valid input returns* `201` *with a* `Location` *header*
- `POST /api/quotes` *with a missing field returns* `400` *with a* `ProblemDetails` *body*
- *Live Metrics shows incoming requests when the API is hit*

√ ***Final verification checklist:***

- [ ] `CloudCiApi` *project scaffolded with* `--use-controllers` *and the demo* `WeatherForecast` *controller removed*
- [ ] `Quote` *entity,* `IQuoteStore` *+* `InMemoryQuoteStore`*, and* `QuoteDto` *+* `CreateQuoteRequest` *all in their conventional folders*
- [ ] `QuotesController` *with* `[ApiController]`*, attribute routing, and three actions returning DTOs*
- [ ] *Swagger registered and middleware enabled in both Development and Production*
- [ ]

> *Dockerfile produces a working image listening on* `:8080`
>
> - [ ] *GitHub repo* `<your-username>/cloudci-api` *pushed and federated against an Entra app* `github-cloudci-api-oidc`
> - [ ] *Workflow runs green on push to* `main`*, deploys to Container App* `ca-api-week6` *in* `rg-api-week6`
> - [ ] *App Insights component* `cloudci-api-insights` *workspace-based, connection string delivered via* `secretref:`

8.2.4 Common Issues

> ***If you encounter problems:***
> ***Swagger page renders empty:*** `AddEndpointsApiExplorer()` *is missing, or* `UseSwagger()` *is registered after* `MapControllers()`*. Swagger middleware must come before endpoint routing.*
> **`POST`** ***returns the new quote but the very next*** **`GET`** ***doesn't include it:*** *The store is registered as* `Scoped` *or* `Transient` *instead of* `Singleton`*. Each request gets a fresh empty list. Confirm* `builder.Services.AddSingleton<IQuoteStore, InMemoryQuoteStore>();`*.*
> **`CreatedAtAction`** ***returns*** **`500`** ***with*** **`No route matches the supplied values`***:* `nameof(GetById)` *doesn't match an action with a* `{id}` *route. Confirm the GET-by-id action is* `[HttpGet("{id:int}")]` *and named* `GetById`*.*
> ***Pipeline run fails at*** **`docker push`** ***with*** **`unauthorized: authentication required`***:* *The* `AcrPush` *role assignment from Step 10 hasn't propagated yet — Entra ID takes a few seconds. Re-run the workflow.*
> ***Container App revision goes*** **`Failed`** ***with*** **`Failed to pull image`***:* *Either the managed identity wasn't granted* `AcrPull`*, or the registry wasn't registered with* `az containerapp registry set`*. Both are required.*
> ***Live Metrics shows "Not available":*** *The connection string env var didn't make it into the container. Re-check the env list for a* `secretRef` *and the secret list for the underlying value.*
> **`AADSTS70021: No matching federated identity record found for presented`**

> **`assertion`**: *The* `subject` *in the token doesn't match the federated credential. Exact form is* `repo:<your-username>/cloudci-api:ref:refs/heads/main` *— watch for* `head` *vs* `heads`*, wrong username, or angle brackets left in literally.*
> ***Still stuck?*** *Re-read the OIDC exercise from the deployment chapter and the App Insights exercise from the logging and monitoring chapter. The patterns are identical; only the names changed.*

8.2.5 Summary

You have a controllers-based ASP.NET Core Web API with a clean entity-vs-DTO split, three idiomatic CRUD-shaped actions, Swagger surfacing the contract, and a multi-stage Docker image. The repo is wired to a fresh OIDC-federated GitHub Actions pipeline that pushes to ACR and updates a Container App on every push to `main`. App Insights is layered on top in workspace-based mode with the connection string delivered as a Container Apps secret.

- √ `dotnet new webapi --use-controllers` produces a project shape that grows beyond a few endpoints
- √ Entity types are the domain; DTOs are the wire contract; keeping them separate from day one pays off when either evolves
- √ `[ApiController]` plus attribute routing gives you 200 / 201 / 404 / 400 responses without extra ceremony, and `CreatedAtAction` builds the `Location` header for free
- √ Swagger in production is a deliberate trade-off — fine for internal services and learning, gated for public-facing APIs

- ✓ The OIDC-federated pipeline pattern from the previous chapter ports cleanly to a new repo with new resource names

> **KEY TAKEAWAY:**
> A Web API is three things stacked on top of each other — a domain, a wire contract, and a deployment pipeline. Treating each as a separate concern (entities for the domain, DTOs for the wire, OIDC federation for the deploy) is what lets each one evolve without breaking the others. You'll feel the saving the first time the domain grows a field, or the wire contract changes, or the cluster moves regions. Each change touches one layer.

8.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add* `[ProducesResponseType(StatusCodes.Status201Created)]` *and friends to each action so Swagger documents the response shapes precisely.*
> - *Replace the hand-written* `ToDto` *mapper with* ***Mapster*** *or* ***AutoMapper*** *and compare the trade-off — magic vs. boilerplate.*
> - *Read the OpenAPI 3.0 JSON at* `https://$FQDN/swagger/v1/swagger.json`*. Notice how the request body schema for* `POST` *is* `CreateQuoteRequest` *while the response body is* `QuoteDto` *— the entity-vs-DTO split surfaces in the contract document.*
> - *Investigate the .NET 9+ built-in* `AddOpenApi()` / `MapOpenApi()` *as an alternative to Swashbuckle, often paired with the standalone Scalar UI.*

8.2.7 Done!

The API runs in the cloud. Anyone with the FQDN can list quotes, fetch one by id, and create new ones — there is no authentication of any kind. That is fine for a learning artifact; it is not fine for anything you want to charge for, rate-limit, or hold accountable. The next exercise closes that gap by gating every request behind a shared API key, the simplest possible authentication scheme that's still better than nothing.

8.3 Securing the API with an API Key Middleware

8.4 Securing the API with an API Key Middleware

8.4.1 Goal

The previous exercise left you with a deployed `CloudCiApi` whose `/api/quotes` endpoint hands a JSON array to anyone on the internet. That is fine as a *shape* but wrong as a posture: a public endpoint with no gate is an abuse vector at minimum, and a data-leakage incident the moment the model behind it stops being seed data. This exercise adds the cheapest possible gate — a shared **API key** the caller must supply in the `X-Api-Key` header — and wires the production key through the same Container Apps secret pattern you used in the logging and monitoring chapter for the App Insights connection string.

The mechanism is a small custom **middleware** that plugs into the pipeline after routing but before the controllers, reads the header, compares it to a value bound from configuration, and either short-circuits with `401 Unauthorized` or `403 Forbidden` or hands off via `await _next(context)`. Locally that value comes from `appsettings.Development.json` (a clearly non-secret placeholder); in production it comes from a Container Apps secret. Swagger UI gets two extra lines so its **Authorize** button keeps the in-browser explorer working after the gate is in place.

This is the second time you see the secret-store pattern, so the *what* is short here. The *why* is longer: an API key proves that *some trusted client holds the key*, not *which* client — that dis-

tinction is what the next exercise replaces with JWT bearer tokens.

WHAT YOU'LL LEARN:

- How to write a custom ASP.NET Core middleware class — `InvokeAsync(HttpContext)`, where it sits in the `Use*` chain
- The 401-vs-403 split per RFC 7235 and why a 401 must include a `WWW-Authenticate` header
- Why a credential goes into a Container Apps **secret** + `secretref:` env var rather than a plain env var
- The convention that maps the env var name `ApiKey__Value` (double underscore) to the configuration key `ApiKey:Value` (colon)
- How to configure Swagger UI's **Authorize** button so the in-browser explorer keeps working after the gate
- What an API key actually proves about the caller, and where it stops being enough

8.4.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ The previous exercise complete: `CloudCiApi` deployed to `ca-api-week6` in `rg-api-week6`, returning a JSON array on `GET /api/quotes` anonymously, with Swagger UI live at `/swagger`
- ✓ The CI/CD pipeline still wired up — `git push` on `<your-username>/cloudci-api` deploys a new revision via OIDC federation
- ✓ App Insights component `cloudci-api-insights` already attached; nothing here changes that wiring
- ✓ The Azure CLI signed in (`az account show` returns your subscription)
- ✓ A local clone of `CloudCiApi` and `openssl` available on your shell

8.4.3 Exercise Steps

Overview

1. **Implement the `ApiKeyMiddleware` class**
2. **Bind the configuration in `Program.cs` and register the middleware**
3. **Add the local placeholder to `appsettings.Development.json`**
4. **Run locally and verify the 401 / 403 / 200 transitions**
5. **Generate the production key locally**
6. **Store the key as a Container Apps secret**
7. **Reference the secret as the `ApiKey__Value` env var**
8. **Wire Swagger UI's Authorize button**
9. **Push and watch the pipeline deploy**
10. **Probe the deployed API from the terminal**

11. **Test Your Implementation**

Step 1: *Implement the* `ApiKeyMiddleware` *class*

Custom middleware in ASP.NET Core is a class with two requirements: a constructor that takes a `RequestDelegate` (the next middleware), and an `InvokeAsync(HttpContext)` method that does the work. The class doesn't implement an interface — the framework discovers the shape by convention when you call `app.UseMiddleware<T>()`. Constructor parameters beyond `RequestDelegate` are resolved from DI at middleware-construction time (once per app), which is fine for a config-bound value like the API key.

1. **Create** a new folder: `Middleware/`
2. **Add** a new file:

> `Middleware/ApiKeyMiddleware.cs`

```
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Configuration;

namespace CloudCiApi.Middleware;

public class ApiKeyMiddleware
{
    // The header callers must send.
    private const string HeaderName = "X-Api-Key";

    // The configuration key bound from appsettings /
        env vars / secrets.
    private const string ConfigKey = "ApiKey:Value";

    private readonly RequestDelegate _next;
    private readonly string? _expectedKey;

    public ApiKeyMiddleware(RequestDelegate next, ICon-
        figuration configuration)
    {
        _next = next;
        _expectedKey = configuration[ConfigKey];
    }

    public async Task InvokeAsync(HttpContext context)
    {
        // No header at all: 401 Unauthorized + WWW-
        Authenticate: ApiKey.
        if (!context.Request.Headers.TryGetValue(Header-
        Name, out var providedKey)
            || string.IsNullOrWhiteSpace(providedKey))
        {
            context.Response.Headers.Append("WWW-Authen-
        ticate", "ApiKey");
            context.Response.StatusCode =
        StatusCodes.Status401Unauthorized;
            await context.Response.WriteAsync("API key
        is missing.");
            return;
        }

        // Header present but doesn't match: 403 Forbid-
        den.
        if (string.IsNullOrEmpty(_expectedKey)
```

```
            || !string.Equals(providedKey, _expec-
        tedKey, StringComparison.Ordinal))
        {
            context.Response.StatusCode =
        StatusCodes.Status403Forbidden;
            await context.Response.WriteAsync("API key
        is invalid.");
            return;
        }

        // Happy path: hand off to the next middleware
        (eventually the controller).
        await _next(context);
    }
}
```

CONCEPT DEEP DIVE: 401 VS 403

RFC 7235 is precise about the split. `401 Unauthorized` is "I don't know who you are" — no credential or an unparseable one — and the response **must** carry a `WWW-Authenticate` header naming the scheme, because that's how the spec tells the client *how* to authenticate. `403 Forbidden` is "I know who you are, and you're not allowed." For an API key the line is mechanical: no header → 401, wrong header → 403.

⚠ **Common Mistakes**

- Forgetting `await _next(context)` on the success path. The request hangs forever — no controller runs and no response is written.
- Using a non-`Ordinal` comparison. API keys are byte-for-byte tokens, not human-readable text — culture-aware comparison can normalize Unicode in ways that accept the wrong key.

√ **Quick check:** The file compiles with `dotnet build` and no warnings.

Step 2: *Bind the configuration in* `Program.cs` *and register the middleware*

The middleware reads `ApiKey:Value` from `IConfiguration` in its constructor — no extra setup needed, since ASP.NET Core's default configuration providers (appsettings, env vars, user secrets) are already in place. The piece you add to `Program.cs` is a single `app.UseMiddleware<T>()` call, **after** `UseRouting()` and **before** `MapControllers()`.

1. **Open** `Program.cs`.
2. **Add** the using at the top:

   > `Program.cs`

   ```
   using CloudCiApi.Middleware;
   ```

3. **Insert** the `UseMiddleware` call between `UseHttpsRedirection()` and `MapControllers()` — keep the Swagger middleware exactly where the previous exercise left it, so `/swagger` stays available in production:

   > `Program.cs`

```
var app = builder.Build();

// Swagger stays on in BOTH Development and Production
        for this course
// (set up in the previous exercise).
app.UseSwagger();
app.UseSwaggerUI();

app.UseHttpsRedirection();

// Gate every endpoint behind the API key.
app.UseMiddleware<ApiKeyMiddleware>();

app.UseAuthorization();
app.MapControllers();

app.Run();
```

> **CONCEPT DEEP DIVE: WHERE THE MIDDLEWARE SITS IN THE PIPELINE**
>
> The pipeline is a chain of `Func<HttpContext, Task>` calls; each `app.Use*()` adds one link. The `WebApplication` host inserts `UseRouting` automatically just before the endpoint terminus when you call `MapControllers()`, so registering your middleware before `MapControllers()` runs it after routing has matched a path to an endpoint. That gives you room to inspect endpoint metadata later (for example, `[AllowAnonymous]` exemptions — that is exactly how built-in authentication works). Putting the call before `MapControllers()` ensures no controller code runs on a rejected request.
>
> The next exercise's auth — `UseAuthentication()` then `UseAuthorization()` — occupies exactly this slot, between routing and the endpoint.
>
> ⚠ **Common Mistakes**
>
> - Putting `UseMiddleware<ApiKeyMiddleware>()` **after** `MapControllers()`. `MapControllers` is a terminal call that builds the endpoint dataset; middleware after it never runs for those routes.
> - Putting it **before** `UseHttpsRedirection()`. Functionally fine for our case, but pushing it past routing keeps the door open for endpoint-aware exemptions later.
>
> ✓ **Quick check:** `dotnet build` succeeds. The middleware class is referenced exactly once in `Program.cs`. The Swagger middleware lines remain un-gated so `/swagger` keeps working in production.

Step 3: *Add the local placeholder to* `appsettings.Development.json`

In development you don't need a real secret — a credential on a `localhost` endpoint accessible only from your laptop is no credential at all. Use a clearly-non-secret placeholder so

the value is self-documenting; if it leaks the next reviewer sees at a glance that nothing was lost. Putting it in `appsettings.Development.json` (committed, but only loaded when the host is in Development) keeps it out of `appsettings.json` so the production image never carries it.

1. **Open** `appsettings.Development.json`.
2. **Add** the `ApiKey` section alongside the existing `Logging` block:

> `appsettings.Development.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ApiKey": {
    "Value": "local-dev-key"
  }
}
```

> **CONCEPT DEEP DIVE: WHERE DEV SECRETS ACTUALLY BELONG**
>
> The strict alternative is `dotnet user-secrets`: a per-developer JSON file outside the repo, identified by a GUID baked into the project file, loaded automatically in Development. That's the right tool when the local credential is real — a connection string to a personal Azure SQL database, an OAuth client secret you can't share. For our placeholder, user-secrets is over-engineering: `local-dev-key` is non-secret by definition since the only thing it grants is access to your own `localhost`. The same question gates both choices — does this value, if read, harm me?
>
> ⚠ **Common Mistakes**
>
> - Putting `ApiKey:Value` in `appsettings.json` (not the Development file). It gets baked into the published image and pushed into the registry, defeating the secret-store mechanism in the next steps.
> - Using a placeholder that looks real (`correcthorsebatterystaple`). Pick something obviously labelled — `local-dev-key` — so future-you doesn't wonder whether it needs rotating.
>
> ✓ **Quick check:** `appsettings.json` has no `ApiKey` section; `appsettings.Development.json` does.

Step 4: *Run locally and verify the 401 / 403 / 200 transitions*

Three calls demonstrate the three branches: anonymous → 401, wrong key → 403, right key → 200. Doing this against `localhost` before pushing means you fix bugs without burning a deploy cycle.

1. **Run the project** from the `CloudCiApi` directory:

```
dotnet run
```

The project starts on `http://localhost:5xxx` (the exact port comes from `launchSettings.json`). Note the URL — call it `$LOCAL` for the rest of this step.

2. **Probe** anonymously (no header). Expect `401`:

```
curl -i "$LOCAL/api/quotes"
```

The response should have `HTTP/1.1 401 Unauthorized` on the status line, a `WWW-Authenticate: ApiKey` header in the headers block, and the body `API key is missing.`.

3. **Probe** with a deliberately wrong key. Expect `403`:

```
curl -i -H "X-Api-Key: not-the-key" "$LOCAL/api/quotes"
```

Status line `HTTP/1.1 403 Forbidden`, body `API key is invalid.`. **No** `WWW-Authenticate` header — that's only for 401.

4. **Probe** with the placeholder. Expect `200` and the JSON array:

```
curl -i -H "X-Api-Key: local-dev-key" "$LOCAL/api/
        quotes"
```

Status line `HTTP/1.1 200 OK`, body the same JSON array of quotes the previous exercise produced.

5. **Stop** the local server with `Ctrl+C`.

> **QUICK CHECK:**
> All three responses match the expected status codes. If any one is wrong, fix the middleware before moving on — debugging local code is cheaper than debugging deployed code.

Step 5: *Generate the production key locally*

Production needs a real key. Generate it on your laptop — `openssl rand -base64 48` produces 48 random bytes (384 bits) of entropy, base64-encoded into a 64-character string. That matches what the next exercise's JWT signing key will use, so the two production secrets in this chapter share one shape.

1. **Generate** the key and capture it in a shell variable:

   ```
   API_KEY=$(openssl rand -base64 48)
   echo "$API_KEY"
   ```

   The output is a string like `kQ8j+7Hf4N...lZtw=` — the exact value differs every time. Treat it as sensitive; copy it into a password manager **once** so you can paste it into curl and into Swagger UI later, then forget the terminal ever held it.

2. **Confirm** the variable is set in this shell session — every subsequent command in this exercise reads `$API_KEY`:

   ```
   [ -n "$API_KEY" ] && echo "API_KEY is set" || echo
           "API_KEY is empty"
   ```

   Expected: `API_KEY is set`.

CONCEPT DEEP DIVE

256 bits is the standard floor for shared-secret API keys — the same strength as symmetric TLS keys. 384 bits is comfortable padding above that. The actual exposure surface for an API key isn't brute force; it's leakage (committed to a repo, logged, screenshotted), so bits past ~128 are mostly insurance. Base64 is wire-friendly — no spaces, no quoting issues, safe in HTTP headers.

⚠ **Common Mistakes**

- Pasting the literal value into the exercise notes, chat, commit message, or `appsettings.json`. The shell variable is the only correct home for it.
- Generating inside Azure Cloud Shell, whose history retention you don't control. Always generate locally.

√ **Quick check:** `echo "$API_KEY" | wc -c` prints `65` (64 base64 chars + newline).

Step 6: *Store the key as a Container Apps secret*

Same shape of command as the App Insights connection string — only the secret name (`api-key`) and the source variable (`$API_KEY`) change. The secret is stored encrypted in Azure's control plane, injected into the container at process start, and never readable via `az containerapp show`.

1. **Set** the secret. The name must be lowercase and hyphenated:

```
az containerapp secret set \
  -g rg-api-week6 \
  -n ca-api-week6 \
  --secrets api-key="$API_KEY"
```

2. **Verify** the secret is registered. The CLI lists names but never values — that's by design:

```
az containerapp secret list \
  -g rg-api-week6 \
  -n ca-api-week6 \
  -o table
```

You should see a row with name `api-key` (in addition to `appinsights-connstr` from the previous chapter).

> **CONCEPT DEEP DIVE: WHY A SECRET STORE, NOT AN ENV VAR?**
>
> Same lesson as the App Insights connection string, different threat. Plain env vars on a Container App are visible in `az containerapp show`, in the Portal's **Containers** blade, and to anyone with `Reader` role — a low-privilege role auditors, on-call engineers, and observability tools are routinely granted. Container Apps secrets sit behind the separate permission `Microsoft.App/containerApps/listSecrets/action`, which Reader does **not** include. The blast radius is smaller.
>
> Compared to App Insights: there, the leaked connection string granted **write** access to telemetry — an attacker pollutes your logs. Here, the API key grants **read** access to every quote. Different threats, same fix.
>
> ⚠ **Common Mistakes**
>
> - Names with uppercase letters or underscores are rejected. `api-key` works; `apiKey`, `api_key`, `ApiKey` all error.
> - Setting the same name twice silently overwrites. There is no version history; rotation is destructive.
>
> ✓ **Quick check:** `az containerapp secret list` shows `api-key` and `appinsights-connstr`.

Step 7: *Reference the secret as the* `ApiKey__Value` *env var*

The container reads configuration from environment variables, not from Container Apps secrets directly. The bridge is an env var whose value is the literal string `secretref:api-key` — Container Apps substitutes the actual value at process start. The env var *name* is the load-bearing piece for ASP.NET Core's configuration system.

1. **Update** the Container App so the env var `ApiKey__Value` points at the secret:

```
az containerapp update \
  -g rg-api-week6 \
  -n ca-api-week6 \
  --set-env-vars ApiKey__Value=secretref:api-key
```

2. **Confirm** the env var is wired correctly:

```
az containerapp show \
  -g rg-api-week6 -n ca-api-week6 \
  --query 'properties.template.containers[0].env' -o
        json
```

Expected: an entry like

```
{
  "name": "ApiKey__Value",
  "secretRef": "api-key"
}
```

The literal value of the API key does **not** appear here. That is the whole point.

CONCEPT DEEP DIVE: WHY `__` (DOUBLE UNDERSCORE)?

ASP.NET Core uses `:` as the configuration hierarchy separator — `ApiKey:Value` means key `Value` inside section `ApiKey`. That works in JSON and command-line args, but Linux env vars can't contain colons. The convention .NET adopted is **double underscore as a colon stand-in**: env var `ApiKey__Value` maps to configuration key `ApiKey:Value`.

The middleware in Step 1 reads `configuration["ApiKey:Value"]`. The same read works in development (nested JSON in `appsettings.Development.json`) and in production (the `ApiKey__Value` env var, populated by Container Apps from the `api-key` secret). One key, three sources, one read.

⚠ **Common Mistakes**

- Naming the env var `ApiKey:Value` instead of `ApiKey__Value`. Linux env vars cannot contain colons.
- Setting the env var to the literal value (`--set-env-vars ApiKey__Value="$API_KEY"`). That bakes the key into the Container App's revision history, where `az containerapp show` enumerates it forever. Use `secretref:api-key`.
- Single underscore (`ApiKey_Value`). Treated as a flat key with no hierarchy, so `configuration["ApiKey:Value"]` returns null.

√ **Quick check:** The JSON output above shows the env var with `secretRef` (not a `value`).

Step 8: *Wire Swagger UI's Authorize button*

With the gate live, Swagger UI's "Try it out" stops working — every in-browser call goes anonymous and gets back 401. Swagger has a built-in fix: describe the API key in the OpenAPI document and Swagger UI renders an **Authorize** button where the user pastes the key once, then attaches the

`X-Api-Key` header to every subsequent call. The wiring is two extras inside the existing `AddSwaggerGen` registration: `AddSecurityDefinition` describes the scheme, `AddSecurityRequirement` says which operations require it (here: all).

1. **Open** `Program.cs`.
2. **Add** the using at the top:

   > `Program.cs`

   ```
   using Microsoft.OpenApi.Models;
   ```

3. **Replace** the existing `builder.Services.AddSwaggerGen()` line with the configured form:

   > `Program.cs`

```
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("ApiKey", new OpenApi-
        SecurityScheme
    {
        Name = "X-Api-Key",
        Type = SecuritySchemeType.ApiKey,
        In = ParameterLocation.Header,
        Description = "API key required to call Cloud-
        CiApi. Paste the key generated for your envir-
        onment."
    });

    options.AddSecurityRequirement(new OpenApiSecuri-
        tyRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "ApiKey"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

CONCEPT DEEP DIVE

`AddSecurityDefinition` declares a scheme — its name, where it lives (header, query, cookie), and a human-readable description. That alone makes Swagger UI render the **Authorize** button. `AddSecurityRequirement` says **which operations enforce it**. At the global level it applies to every endpoint; that matches our middleware, which is also global. Once the user authorizes, Swagger UI keeps the key in browser memory and attaches `X-Api-Key: <value>` to every request — purely a UI convenience, the same security model is in force regardless of caller.

⚠ **Common Mistakes**

- Calling `AddSecurityDefinition` without `AddSecurityRequirement`. The button appears, the dialog accepts the key, but no operation actually requires the scheme — Swagger never sends the header. Symptom: every call gets `401` even after authorizing.
- Using `SecuritySchemeType.Http` with `Scheme = "Bearer"`. That's JWT from the next exercise, not API key.

✓ **Quick check:** `dotnet build` succeeds. `Program.cs` references both `AddSecurityDefinition` and `AddSecurityRequirement` exactly once each.

Step 9: *Push and watch the pipeline deploy*

The middleware, configuration binding, and Swagger wiring are still only local. The CI/CD pipeline rebuilds and rolls out a new revision on every push to `main`; the secret + env var from Steps 6 and 7 are already in place, so the new revision picks them up automatically.

1. **Update the smoke test first.** The workflow you wrote in the deployment chapter probes `/api/quotes` to validate the deploy. With the API-key middleware in place, that

probe will get back `401` on the very next push and fail the job — even though the deploy itself succeeded. Re-scope the smoke test to a path that's always anonymous, like `/swagger/index.html`:

> `.github/workflows/ci.yml`

```
      - name: Smoke test
        run: |
          FQDN=$(az containerapp show \
            -n ca-api-week6 -g rg-api-week6 \
            --query properties.configuration.ingress.fqdn -o tsv)
          for i in {1..20}; do
            if curl -fsS "https://$FQDN/swagger/index.html" >/dev/null; then
              echo "Smoke test passed."
              exit 0
            fi
            sleep 3
          done
          echo "Smoke test failed."
          exit 1
```

Replace just the `curl -fsS` URL — the rest of the step stays the same.

2. **Commit and push:**

```
git add Middleware/ApiKeyMiddleware.cs \
        Program.cs \
        appsettings.Development.json \
        .github/workflows/ci.yml
git commit -m "Add API-key middleware and Swagger Authorize wiring"
git push
gh run watch
```

Expected: green workflow within a couple of minutes.

3. **Capture** the FQDN:

```
FQDN=$(az containerapp show \
  -g rg-api-week6 -n ca-api-week6 \
  --query properties.configuration.ingress.fqdn -o tsv)

echo "$FQDN"
```

> **QUICK CHECK:**
> Workflow green. A new revision is `Active` with 100% traffic.

Step 10: *Probe the deployed API from the terminal*

The same three calls as Step 4, now against the deployed FQDN. The placeholder `local-dev-key` is **not** in production — only the key you generated in Step 5 will pass.

1. **Anonymous** call. Expect `401`:

```
curl -i "https://$FQDN/api/quotes"
```

   Status `401`, header `WWW-Authenticate: ApiKey`, body `API key is missing.`.

2. **Wrong key** (try the dev placeholder — it's wrong in production):

```
curl -i -H "X-Api-Key: local-dev-key" "https://$FQDN/
        api/quotes"
```

   Status `403`, body `API key is invalid.`.

3. **Right key**. Expect `200` and the JSON array:

```
curl -i -H "X-Api-Key: $API_KEY" "https://$FQDN/api/
        quotes"
```

Status `200`, body the JSON array of quotes.

4. **Swagger UI in the browser.** Visit `https://$FQDN/swagger`. Click the **Authorize** button at the top right; paste `$API_KEY` into the dialog; click **Authorize** then **Close**. Click any operation, then **Try it out**, then **Execute**. The "Curl" preview at the bottom of the operation should now include `-H "X-Api-Key: <your-key>"`, and the response body should contain the JSON array.

> **QUICK CHECK:**
> All three curl calls return the expected status codes. Swagger's Authorize flow accepts the key and subsequent in-browser calls succeed.

Step 11: *Test Your Implementation*

Walk every signal end to end so you finish knowing the gate works in production and locally and the secret is delivered the right way.

1. **Anonymous → 401 + `WWW-Authenticate`:**

```
curl -is "https://$FQDN/api/quotes" | head -n 5
```

Expected: `HTTP/2 401` and `www-authenticate: ApiKey` in the first few lines.

2. **Wrong key → 403:**

```
curl -s -o /dev/null -w "%{http_code}\n" \
  -H "X-Api-Key: not-the-key" "https://$FQDN/api/quotes"
```

Expected: `403`.

3. **Right key → 200 + JSON:**

```
curl -s -H "X-Api-Key: $API_KEY" "https://$FQDN/api/
        quotes" | head -c 80
echo
```

Expected: the start of a JSON array, e.g. `[{"id":1,"author":"...`.

4. **Secret wiring — env var is `secretRef`, not a literal:**

```
az containerapp show -g rg-api-week6 -n ca-api-week6 \
  --query 'properties.template.containers[0].env[?
        name==`ApiKey__Value`]' \
  -o json
```

Expected: an object with `secretRef: "api-key"` and **no** `value` field.

5. **Swagger Authorize flow** — open `https://$FQDN/swagger`, click **Authorize**, paste `$API_KEY`, run any operation. Confirm by hand that the JSON array comes back.

√ ***Final verification checklist:***

- [ ] `Middleware/ApiKeyMiddleware.cs` *exists and implements the 401/403/200 logic*
- [ ] `Program.cs` *calls* `app.UseMiddleware<ApiKeyMiddleware>()` *between* `UseRouting()` *and* `MapControllers()`
- [ ] `appsettings.Development.json` *has* `ApiKey:Value = "local-dev-key"`*;* `appsettings.json` *does not*
- [ ] *Container Apps secret* `api-key` *is set on* `ca-api-week6`
- [ ] *Env var* `ApiKey__Value` *references the secret via* `secretRef:`*, not a literal value*
- [ ] *Anonymous, wrong-key, and right-key probes against the deployed FQDN return 401, 403, 200*
- [ ] *Swagger UI's Authorize button accepts the key and subsequent in-browser calls succeed*
- [ ] *The pipeline still deploys cleanly on push to* `main`

8.4.4 Common Issues

> ***If you encounter problems:***
> ***Every call returns 401, even with the right key.*** *The middleware can't read* `ApiKey:Value`*. Check the env var name is* `ApiKey__Value` *(double underscore) —* `ApiKey_Value` *and* `ApiKey:Value` *both fail silently. Confirm with* `az containerapp show ... --query 'properties.template.containers[0].env'`*.*
> ***Every call returns 403.*** *The middleware reads a value, but not yours. Either the secret stores a different key than* `$API_KEY` *(typo, stray newline), or the new revision hasn't rolled out — check* `az containerapp revision list -g rg-api-week6 -n ca-api-week6 -o table`*.*
> ***Swagger Authorize button accepts the key but every Try-it-out still 401s.*** *You called* `AddSecurityDefinition` *but forgot* `AddSecurityRequirement`*.*
> ***Anonymous calls return 200, not 401.*** *The middleware isn't running. Either you forgot* `app.UseMiddleware<ApiKeyMiddleware>()`*, or you placed it after* `app.MapControllers()` *(terminal for matched endpoints).*
> **`az containerapp update`** ***fails with "secret not found."*** *The* `secretref:` *value must match an existing secret. Run* `az containerapp secret list` *first.*
> ***Still stuck?*** *Verify three things in order: the middleware reads* `ApiKey:Value` *(Step 1),* `Program.cs` *registers it between* `UseRouting` *and* `MapControllers` *(Step 2), and the env var*

> `ApiKey__Value` *references the secret via* `secretRef:` *(Step 7).*

8.4.5 Summary

You added a custom ASP.NET Core middleware that demands an `X-Api-Key` header on every request, returning `401` (with `WWW-Authenticate: ApiKey`) when the header is missing and `403` when the value is wrong. The production key is delivered through the same Container Apps secret + `secretref:` env-var pattern you used for the App Insights connection string. The dev placeholder lives in `appsettings.Development.json`. Swagger UI's Authorize button keeps the in-browser explorer working.

- √ Custom middleware with the standard shape — constructor + `InvokeAsync(HttpContext)`
- √ Pipeline placement after `UseRouting()` and before `MapControllers()` — the slot real auth will occupy
- √ The 401-vs-403 split per RFC 7235 — `WWW-Authenticate: ApiKey` on the 401, none on the 403
- √ Production key generated locally with `openssl rand -base64 48`, delivered as a Container Apps secret + `secretref:` env var
- √ Env var name `ApiKey__Value` — double underscore mapping to the configuration key `ApiKey:Value`
- √ Swagger UI's Authorize button wired via `AddSecurityDefinition` + `AddSecurityRequirement`

> **KEY TAKEAWAY:**
> An API key is the cheapest possible gate, and the secret-store pattern that delivers it is the same regardless of what's behind the gate. What changes between credentials is the *threat model*; the mechanism — Container Apps secret + `secretref:` env var, never a plain env var — is the same every time.

8.4.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add an* `[AllowAnonymous]` *equivalent. Inspect* `context.GetEndpoint()?.Metadata` *in the middleware and skip the check when the endpoint declares itself anonymous — useful for* `/health` *or* `/swagger`*.*
> - *Compare your middleware to the built-in* `UseAuthentication` / `UseAuthorization` *pair. The framework version separates "who are you?" from "what can you do?", so the 401-vs-403 split falls out naturally; your middleware merges both responsibilities.*
> - *Constant-time comparison.* `string.Equals` *short-circuits on the first mismatched byte, leaking timing information. For HTTPS-with-network-jitter the leak is negligible, but the production-grade way is* `CryptographicOperations.FixedTimeEquals` *over byte arrays.*
> - *Read about* ***mutual TLS*** *(mTLS) — the alternative for machine-to-machine auth. The client presents a certificate; the private key never crosses the wire. Container Apps supports mTLS for ingress.*

8.4.7 Done!

The API has a gate. Anonymous requests bounce with 401, wrong keys bounce with 403, the right key gets through, and Swagger UI still works. The production key lives where a

`Reader` on the Container App can't enumerate it; the dev placeholder lives where it doesn't matter that anyone can.

The gate proves only one thing: the caller knows the shared key. It says nothing about *which* caller — every client with the key looks identical to the server, and rotating the key invalidates every client at once. For machine-to-machine traffic that is enough; for per-user attribution it is not. The next exercise replaces the API key with **JWT bearer authentication**: each user gets a signed token carrying their identity, the server validates the signature, and `[Authorize]` does the enforcement.

8.5 JWT Bearer Authentication and Cleanup

8.6 JWT Bearer Authentication and Cleanup

8.6.1 Goal

In the previous exercise you put an API key in front of `CloudCiApi` and gated the quotes endpoints with a small middleware that compared a header against a Container Apps secret. That solved one problem — anonymous callers off the public internet can no longer hit the API — but it left a real gap: the server has no idea *who* is calling. Every request that presents the right key looks identical to every other. There is no per-user identity, no role information, no way to write code that says "this user can read but not write."

In this exercise you'll replace the API key with **JWT bearer authentication**. You'll add a `TokensController` with a `POST /api/tokens/login` action that validates a username and password against an in-memory user list and returns a signed JSON Web Token. The `QuotesController` will be marked with `[Authorize]` so that the framework's authen-

tication middleware enforces the new identity model. Swagger UI's **Authorize** button will be rewired from the API-key scheme to a Bearer scheme so you can paste tokens directly into the UI and try the endpoints from the browser.

This is the last exercise of the chapter. Once the JWT flow works end to end on the deployed Container App, the final step tears down the lab — `rg-api-week6`, the Entra app registration created for OIDC federation, and (optionally) the stale GitHub repo secrets — so that nothing you provisioned this week is left running.

WHAT YOU'LL LEARN:

- What a JWT actually is — three base64url segments, the signature as the integrity check, the payload as readable-but-tamper-evident claims
- How to configure `JwtBearerDefaults.AuthenticationScheme` in ASP.NET Core with `TokenValidationParameters` for issuer, audience, lifetime, and signing-key validation
- How to mint a signed token with `JwtSecurityTokenHandler` using `SymmetricSecurityKey` and `HmacSha256`, with claims for `sub`, `name`, and an optional `role`
- When symmetric signing (HS256) is acceptable and when you need asymmetric signing (RS256/ES256) instead
- How to deliver the signing key to Container Apps as a secret and reference it via `secretref:` — the same pattern as the App Insights connection string in the previous chapter
- How to rewire Swagger UI's Authorize button from the API-key scheme to the Bearer scheme
- How to tear down both the resource group and the tenant-level Entra app registration so the lab leaves no orphans behind

8.6.2 Prerequisites

> **BEFORE STARTING, ENSURE YOU HAVE:**
>
> - ✓ The previous exercise complete: `CloudCiApi` deployed to the Container App `ca-api-week6` in `rg-api-week6`, with `QuotesController` returning JSON DTOs and the API-key middleware rejecting requests that don't carry the `X-Api-Key` header
> - ✓ The CI/CD pipeline from earlier in the course still wired up — `git push` to `main` deploys to the Container App via OIDC federation
> - ✓ The Azure CLI signed in (`az account show` returns your subscription)
> - ✓ A local clone of the `CloudCiApi` ASP.NET Core Web API project where you can edit code and push to `main`
> - ✓ `openssl` available on your machine (default on macOS and Linux; on Windows use Git Bash or WSL)

8.6.3 Exercise Steps

Overview

1. **Add the JWT bearer NuGet package**
2. **Configure JWT validation in `Program.cs`**
3. **Add development-mode JWT settings to `appsettings.Development.json`**
4. **Remove the API-key middleware**
5. **Add the in-memory user store**
6. **Implement `TokensController` with the login action**
7. **Mark `QuotesController` with `[Authorize]`**
8. **Generate the production signing key locally**
9. **Inject the signing key as a Container Apps secret**
10. **Rewire Swagger UI's Authorize button for Bearer**

11. **Push and verify the deployed API**
12. **Test Your Implementation**
13. **Tear down the cloud resources**

Step 1: *Add the JWT bearer NuGet package*

The `Microsoft.AspNetCore.Authentication.JwtBearer` package is the official ASP.NET Core handler for incoming JWTs. It plugs into the same authentication middleware you'd use for cookies or any other scheme — once registered, marking a controller with `[Authorize]` causes the framework to look for an `Authorization: Bearer <token>` header, validate the token against the parameters you configure, and materialize a `ClaimsPrincipal` from the claims inside.

1. **Open** a terminal in the project root (the directory containing `CloudCiApi.csproj`).
2. **Add** the package:

```
dotnet add package
        Microsoft.AspNetCore.Authentication.JwtBearer
```

3. **Inspect** the resulting `.csproj`. The `ItemGroup` for package references should now contain the new entry alongside whatever was there before:

> `CloudCiApi.csproj`

```
<ItemGroup>
  <PackageReference
        Include="Microsoft.AspNetCore.Authentication.JwtBearer"
        Version="10.0.0" />

  <PackageReference Include="Swashbuckle.AspNetCore"
        Version="6.6.2" />
</ItemGroup>
```

The exact version pinned by `dotnet add` will be the latest at the time you run the command.

> **CONCEPT DEEP DIVE**
>
> This single package is everything you need to *validate* incoming JWTs. To *issue* JWTs from the `TokensController` you don't need an extra package — `System.IdentityModel.Tokens.Jwt` and `Microsoft.IdentityModel.Tokens` are pulled in transitively by the bearer package, and they expose `JwtSecurityTokenHandler`, `SymmetricSecurityKey`, and `SigningCredentials`. You'll see those types used in Step 6.
>
> ✓ **Quick check:** `dotnet build` completes with no errors and no new warnings.

Step 2: *Configure JWT validation in* `Program.cs`

Registering the JWT bearer handler is two API calls: one to add the authentication services, one to configure the validation parameters that incoming tokens must satisfy. The validation parameters are the load-bearing piece — they tell the middleware exactly what to check on every request, and *not* checking any of them is a security bug. You want all four checks (`ValidateIssuer`, `ValidateAudience`, `ValidateLifetime`, `ValidateIssuerSigningKey`) on for any production-shaped configuration.

1. **Open** `Program.cs` in the project root.

2. **Add** the `using` statements at the top of the file:

   > `Program.cs`

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
```

3. **Register** the bearer scheme right after `AddControllers()` (or wherever your existing service registrations live). The block below is what the rest of this exercise assumes — copy it as-is:

   > `Program.cs`

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.Authentication-
        Scheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new Token-
        ValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer =
        builder.Configuration["Jwt:Issuer"],
            ValidAudience =
        builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configur-
        ation["Jwt:SigningKey"]!))
        };
    });

builder.Services.AddAuthorization();
```

4. **Add** the authentication and authorization middleware after `app.UseRouting()` (if present) and before `app.MapControllers()`:

    > `Program.cs`

```
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
```

CONCEPT DEEP DIVE

The four validation flags answer four different questions. `ValidateIssuer` asks "did this token come from the issuer I trust?" — it compares the `iss` claim in the payload against `ValidIssuer`. `ValidateAudience` asks "is this token meant for me?" — it compares `aud` against `ValidAudience`. `ValidateLifetime` asks "is this token still in its `nbf` / `exp` window?" `ValidateIssuerSigningKey` asks "was this token signed by a key I trust?" — it recomputes the signature with `IssuerSigningKey` and compares.

Turning any one of those off opens a hole. `ValidateAudience = false` lets a token issued for a different service be replayed against yours. `ValidateLifetime = false` makes expired tokens valid forever. The defaults in the SDK have most flags on, but listing them explicitly is the right habit — it documents the security posture in the same file where the app is configured.

⚠ **Common Mistakes**

- Putting `app.UseAuthentication()` *after* `app.UseAuthorization()` — every authenticated request returns 401 with no obvious diagnostic, because authorization runs against an anonymous principal.
- Forgetting `app.UseAuthentication()` entirely — same symptom: every request looks anonymous to `[Authorize]`.
- Hard-coding the signing key as a string literal in `Program.cs`. It works locally and bakes the key into the image. The whole next chapter on secret management exists because this is wrong; do not start it wrong here.

√ **Quick check:** `dotnet build` succeeds. The app won't start yet without the configuration values from Step 3.

Step 3: *Add development-mode JWT settings to* **`appsettings.Development.json`**

The validation parameters in Step 2 read three configuration keys: `Jwt:Issuer`, `Jwt:Audience`, and `Jwt:SigningKey`. For local development you want these in `appsettings.Development.json` so a fresh clone of the repo runs without any extra environment-variable juggling. The dev signing key is *not* the production key — it's a long, clearly-marked dev string, regenerated whenever it leaks into a screenshot.

1. **Open** `appsettings.Development.json` (create it next to `appsettings.json` if it doesn't exist).
2. **Add** the `Jwt` section at the top level of the JSON, alongside `Logging` and any other settings already present:

   > `appsettings.Development.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "Jwt": {
    "Issuer": "cloudci-api-dev",
    "Audience": "cloudci-api-clients-dev",
    "SigningKey": "DEV-ONLY-NOT-FOR-
        PROD-3f8a2c7e9b1d4e6f0a5b8c2d7e1f4a9c8e3b2a7d5c4f6e8a1b9d0c2e3f5a7b

  }
}
```

3. **Confirm** the file is gitignored only in the way *production* secrets would be. `appsettings.Development.json` is intentionally *committed* — every developer who clones the

repo gets the same dev-mode key. That's safe because it's clearly marked as dev-only and grants nothing in production.

> **CONCEPT DEEP DIVE**
>
> The signing key has to be at least 256 bits long for HS256 to accept it. The handler measures the byte length of the UTF-8 representation: the string above is 87 ASCII characters, so 87 bytes — 696 bits, comfortably above the 256-bit minimum. For the production key in Step 8 you'll use `openssl rand -base64 48`, which produces 64 base64 characters — 64 bytes — 512 bits. ASCII-printable padding strings like `"super-secret-key"` are short, fail the length check at first validation, and produce the famously cryptic `IDX10720` error.
>
> ⚠ **Common Mistakes**
>
> - Using the same signing key in dev and production. The whole point of separating them is that one leaking does not compromise the other.
> - Leaving the dev key short and discovering at first request that HS256 throws `IDX10720`. Use a 64-character (or longer) string.
>
> √ **Quick check:** `dotnet run` starts the app without the `IDX10720` error. The startup logs say `Now listening on: http://localhost:<port>`.

Step 4: *Remove the API-key middleware*

The cleanest story for this exercise is *replace*, not *layer*. The API key answered "is this a known client?" The JWT answers "which user is this, and what can they do?" Layering both adds operational overhead — the same secret has to be rotated twice, the failure modes interact in surprising ways — without adding security, because any caller already authen-

ticated by JWT didn't need the API key. Pull the middleware out cleanly.

1. **Open** `Program.cs`.
2. **Find** the line that registers the API-key middleware from the previous exercise. It looks like:

   > `Program.cs`

   ```
   app.UseMiddleware<ApiKeyMiddleware>();
   ```

3. **Delete** that line. The pipeline now relies on `app.UseAuthentication()` and `app.UseAuthorization()` from Step 2 to gate the controllers.
4. **Delete** the middleware class file itself — typically `Middleware/ApiKeyMiddleware.cs`. If you leave the file around, it compiles into the assembly as dead code; nothing references it, but it's noise.
5. **Remove** the `ApiKey` configuration key from `appsettings.Development.json` and from any `Jwt:`-adjacent settings — it's no longer read.
6. **Plan** to remove the production-side leftovers. The Container Apps secret `api-key` and the env var that referenced it are still on the running app. You'll clean those up implicitly when you delete the resource group in Step 13. If you want to remove them sooner, run:

```
az containerapp update \
  -g rg-api-week6 -n ca-api-week6 \
  --remove-env-vars ApiKey__Value

az containerapp secret remove \
  -g rg-api-week6 -n ca-api-week6 \
  --secret-names api-key
```

CONCEPT DEEP DIVE

Why replace rather than layer. The textbook reason to layer multiple authentication factors is when each factor proves something the others can't — a password proves knowledge, a hardware token proves possession, a fingerprint proves identity. JWT and API key both answer the same kind of question (does this caller hold the right shared secret?), so stacking them just multiplies the operational surface. Once you have per-user JWTs you have strictly more information than the API key gave you, and the API key becomes redundant.

⚠ **Common Mistakes**

- Forgetting the `app.UseMiddleware<ApiKeyMiddleware>()` line. The middleware still runs at startup, the API still expects the `X-Api-Key` header, and your JWT-authenticated requests get rejected for missing the key.
- Deleting the middleware *file* but leaving the registration in `Program.cs`. The build fails immediately with "type or namespace name 'ApiKeyMiddleware' not found." The build break tells you to delete both halves.

✓ **Quick check:** The project builds. `app.UseMiddleware<ApiKeyMiddleware>()` no longer appears in `Program.cs` and the file `Middleware/ApiKeyMiddleware.cs` is gone.

Step 5: *Add the in-memory user store*

Real applications store users in a database with hashed passwords and proper audit trails. For this lab the focus is the JWT mechanics, so a hard-coded list of two or three users keeps the moving parts small. The store is an interface (`IUserStore`) so the controller depends on an abstraction; the implementation (`InMemoryUserStore`) holds the list and exposes a `Validate(username, password)` method that returns the user when credentials match and `null` otherwise.

1. **Open** the existing `Models/` folder created in the first exercise.
2. **Add** a new file:

   > `Models/User.cs`

   ```
   namespace CloudCiApi.Models;

   public record User(string Username, string Password,
           string? Role = null);
   ```

3. **Open** the existing `Services/` folder created in the first exercise.
4. **Add** the interface:

   > `Services/IUserStore.cs`

```
using CloudCiApi.Models;

namespace CloudCiApi.Services;

public interface IUserStore
{
    // Returns the user when credentials match, null
        otherwise.
    User? Validate(string username, string password);
}
```

5. **Add** the implementation:

> `Services/InMemoryUserStore.cs`

```
using CloudCiApi.Models;

namespace CloudCiApi.Services;

public class InMemoryUserStore : IUserStore
{
    // Hard-coded for this lab. Real apps store hashed
        passwords.
    private static readonly List<User> Users =
    [
        new("alice", "alice123", "admin"),
        new("bob",   "bob456",   "reader"),
    ];

    public User? Validate(string username, string pass-
        word) =>
        Users.FirstOrDefault(u =>
            string.Equals(u.Username, username, String-
        Comparison.Ordinal) &&
            string.Equals(u.Password, password, String-
        Comparison.Ordinal));
}
```

6. **Register** the store as a singleton in `Program.cs`, alongside the other service registrations from Step 2:

> `Program.cs`

```
builder.Services.AddSingleton<IUserStore, InMemoryUser-
        Store>();
```

CONCEPT DEEP DIVE

Singleton lifetime fits because the user list is read-only and shared across all requests. If the store talked to a database we'd need scoped, because the underlying `DbContext` is scoped. The lifetime question is always "does the underlying state need to be per-request?" — for a static list, no.

⚠ **Common Mistakes**

- Hashing passwords with a homegrown function. If you ever need real password handling, use ASP.NET Core Identity or `PasswordHasher<T>`. Do not roll your own.
- Comparing passwords with `==` and depending on it being constant-time. `string.Equals` is also not constant-time. For a real app use a constant-time comparison; for this lab, the credentials are public anyway, so the timing-attack surface doesn't matter.

√ **Quick check:** The project builds. `Validate("alice", "alice123")` returns the alice record; `Validate("alice", "wrong")` returns null.

Step 6: *Implement* `TokensController` *with the login action*

This is where the API issues the JWT. The action takes a username and password, asks the user store to validate them, and on success builds a `JwtSecurityToken` with claims, an issuer, an audience, an expiry, and a signature. `JwtSecurityTokenHandler.WriteToken(...)` serializes that object into

the three-segment compact form that the client will paste into the `Authorization: Bearer <...>` header.

1. **Create** a new file:

> `Controllers/TokensController.cs`

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using CloudCiApi.Services;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;

namespace CloudCiApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class TokensController : ControllerBase
{
    private readonly IUserStore _users;
    private readonly IConfiguration _config;

    public TokensController(IUserStore users, IConfigur-
        ation config)
    {
        _users = users;
        _config = config;
    }

    public record LoginRequest(string Username, string
        Password);

    [HttpPost("login")]
    public IActionResult Login([FromBody] LoginRequest
        request)
    {
        var user = _users.Validate(request.Username,
        request.Password);
        if (user is null)
        {
            return Unauthorized();
        }

        var key = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(_config["Jwt:Sign-
        ingKey"]!));
        var creds = new SigningCredentials(key, Secur-
        ityAlgorithms.HmacSha256);
```

```
        var claims = new List<Claim>
        {
            new(JwtRegisteredClaimNames.Sub, user.User-
        name),
            new(ClaimTypes.Name, user.Username),
        };
        if (!string.IsNullOrEmpty(user.Role))
        {
            claims.Add(new Claim(ClaimTypes.Role,
        user.Role));
        }

        var expires = DateTime.UtcNow.AddHours(1);

        var token = new JwtSecurityToken(
            issuer: _config["Jwt:Issuer"],
            audience: _config["Jwt:Audience"],
            claims: claims,
            expires: expires,
            signingCredentials: creds);

        var tokenString = new
        JwtSecurityTokenHandler().WriteToken(token);

        return Ok(new { token = tokenString, expiresAt
        = expires });
    }
}
```

2. **Confirm** the route is `POST /api/tokens/login`. The `[ApiController]` attribute gives you automatic model-state validation and binds `LoginRequest` from the JSON body without further ceremony.

CONCEPT DEEP DIVE: WHAT A JWT ACTUALLY IS

A JWT is three base64url-encoded segments separated by dots: `header.payload.signature`.

The **header** is a tiny JSON object — `{"alg":"HS256","typ":"JWT"}` — base64url-encoded. It says how the signature was produced.

The **payload** is another JSON object — `{"sub":"alice","name":"alice","role":"admin","iss":"cloudci-api-dev","aud":"cloudci-api-clients-dev","exp":1714312345,"nbf":1714308745,"iat":1714308745}` — base64url-encoded. The `iss` (issuer), `aud` (audience), `exp` (expiry as Unix seconds), `nbf` (not-before), and `iat` (issued-at) claims are the standard ones the validator checks; `sub`, `name`, and `role` are the per-user identity. Anyone can decode the payload — paste a token into https://jwt.io and read it. There is no secrecy in a JWT.

The **signature** is `HMAC-SHA256(base64url(header) + "." + base64url(payload), key)` — the entire signed string is the first two segments concatenated with the dot, hashed with the shared key. Anyone holding the key can produce the signature; anyone holding the key can also forge tokens. The signature is the *integrity* check, not a secrecy mechanism. The token is tamper-evident: change one byte of the payload and the signature stops matching.

⚠ **Common Mistakes**

- Putting sensitive data in the payload "because the signature protects it." It doesn't. The payload is plaintext to anyone who sees the token. Put identifiers there, not secrets.
- Returning the raw token in plain log lines at issuance time. The token grants access for its full lifetime; logs are often shipped to systems with broader read access than the API itself.
- Forgetting to set `expires` and shipping a token that is valid forever. The validation parameters with `Valid-`

> `ateLifetime = true` will reject it because it has no `exp` — but the symptom is "every login returns a 401-on-next-request" and is hard to debug.
>
> ✓ **Quick check:** `dotnet run`, then `curl -X POST http://localhost:<port>/api/tokens/login -H 'Content-Type: application/json' -d '{"username":"alice","password":"alice123"}'` returns `{"token":"eyJ...","expiresAt":"2026-..."}`.

Step 7: *Mark* `QuotesController` *with* `[Authorize]`

`[Authorize]` is the framework-level switch that says "this action requires a successfully-authenticated principal." With the bearer scheme registered as the default, the middleware looks for `Authorization: Bearer <token>` on every request, validates the token, and either materializes a `ClaimsPrincipal` from its claims or short-circuits the pipeline with a 401.

1. **Open** `Controllers/QuotesController.cs`.
2. **Add** the using:

> `Controllers/QuotesController.cs`

```
using Microsoft.AspNetCore.Authorization;
```

3. **Add** the attribute at class level:

> `Controllers/QuotesController.cs`

```
[ApiController]
[Route("api/[controller]")]
[Authorize]
public class QuotesController : ControllerBase
{
    // ...existing code from the previous exercise...
}
```

4. **Leave** `TokensController` without `[Authorize]`. The login endpoint must be reachable anonymously; a caller can't authenticate before they have a token. There's no `[AllowAnonymous]` needed here because the parent class isn't `[Authorize]` — only `QuotesController` is.

> **CONCEPT DEEP DIVE: WHY SYMMETRIC SIGNING IS ACCEPTABLE HERE, AND WHEN IT ISN'T**
>
> HS256 — HMAC-SHA256 — uses one secret key for both signing and validating. The same process that issues the token also validates it. That works in this exercise because there is exactly one application: `CloudCiApi` mints the token, `CloudCiApi` validates the token, and the key never has to leave the process boundary.
>
> The moment a separate identity provider issues tokens consumed by multiple validators — a corporate IdP issuing tokens for fifteen microservices, say — symmetric signing falls apart. To validate the token, every microservice would need the signing key, and any one of them could then forge a token impersonating any user. You move to **asymmetric signing** (RS256 or ES256), where the IdP holds a private key and validators only get the public key. Public keys verify signatures but cannot produce them, so a compromised validator cannot mint forged tokens.
>
> Practical heuristic: same process issues *and* validates → HS256 is fine. Multiple validators of tokens from a separate issuer → RS256 with the public key fetched from the IdP's JWKS endpoint.
>
> ✓ **Quick check:** Build succeeds. Hitting `GET http://localhost:<port>/api/quotes` without an `Authorization` header returns `401 Unauthorized`.

Step 8: *Generate the production signing key locally*

The dev signing key in `appsettings.Development.json` is committed and known to anyone with read access to the repo. The production key has to be different, generated freshly, and never written to a file the source control system tracks. `openssl rand -base64 48` is the canonical one-liner — 48

random bytes encoded as 64 base64 characters, comfortably above the 256-bit minimum for HS256.

1. **Generate** the key locally:

   ```
   openssl rand -base64 48
   ```

   The output looks like `Yk0tH3gB...QqQ==` — 64 characters. Copy this string into a temporary shell variable, do **not** paste it into a file the repository tracks:

   ```
   PROD_KEY=$(openssl rand -base64 48)
   echo "$PROD_KEY" | wc -c   # 65 (64 chars + newline)
   ```

2. **Treat** the value as sensitive. Anyone holding it can mint tokens that your production API will accept as authentic. Do not paste it into chat, screenshots, or commit messages.

CONCEPT DEEP DIVE: TOKEN LIFETIME TRADE-OFFS

The token issued in Step 6 lives for one hour. That choice is a compromise. Short lifetimes (minutes) are more secure: a stolen token expires before an attacker can chain many calls. The price is load — clients re-authenticate often, the tokens endpoint takes more traffic, every request flow has to handle the "token expired, re-login" branch.

Long lifetimes (hours, days) are convenient and cheap on the issuer side, but they extend the compromise window. A token leaked in a screenshot or a log file is valid for as long as its `exp` says it is, and there is no equivalent of "log out all sessions" — the validator has no idea which tokens it has issued, only which ones it would currently accept.

The production pattern that combines both is **short access tokens (minutes) plus longer refresh tokens (days, single-use, server-tracked)**. The access token gates API calls; the refresh token is exchanged at the IdP for a fresh access token. A leaked access token is short-lived; a leaked refresh token is invalidated server-side the moment its single use is consumed. For this lab, one hour is a reasonable middle ground that doesn't introduce the operational complexity of refresh tokens while still showing the lifetime concept.

⚠ **Common Mistakes**

- Generating the key with a non-cryptographic source (`echo "myapp$(date)"`). HS256 keys must be unpredictable. `openssl rand` reads from the OS RNG.
- Generating the key shorter than 256 bits. `openssl rand -base64 24` (24 bytes = 192 bits) fails HS256's minimum-key-size check; `IDX10720` is the symptom.

√ **Quick check:** `echo -n "$PROD_KEY" | wc -c` reports 64 (or close — base64 padding can vary). The value is in your shell, not in any file the repo tracks.

Step 9: *Inject the signing key as a Container Apps secret*

The signing key has to reach the running container at startup. The wrong way to deliver it is as a plain environment variable on the Container App — that bakes the key into the revision history, where anyone with `Reader` role can read it via `az containerapp show`. The right way is the same pattern as the App Insights connection string in the previous chapter: store the value as a Container Apps **secret**, then reference it from an environment variable using the `secretref:` prefix. The runtime substitutes the actual value when the container starts.

The issuer and audience are not sensitive — they're values the token-validation logic compares incoming claims against, and they don't grant anything by themselves. You can set them as plain env vars in the same `update` call.

1. **Set** the signing key as a secret on the Container App:

```
az containerapp secret set \
  -g rg-api-week6 -n ca-api-week6 \
  --secrets jwt-signing-key="$PROD_KEY"
```

2. **Wire** the secret to the env var the SDK reads, and set the issuer and audience env vars in the same call. ASP.NET Core maps double underscores in env vars to colons in the configuration tree, so `Jwt__SigningKey` becomes `Jwt:SigningKey`:

```
az containerapp update \
  -g rg-api-week6 -n ca-api-week6 \
  --set-env-vars \
    Jwt__SigningKey=secretref:jwt-signing-key \
    Jwt__Issuer=cloudci-api-prod \
    Jwt__Audience=cloudci-api-clients-prod
```

3. **Confirm** the env vars are wired correctly:

```
az containerapp show \
  -g rg-api-week6 -n ca-api-week6 \
  --query 'properties.template.containers[0].env' -o
        json
```

You should see three entries — `Jwt__SigningKey` with a `secretRef`, `Jwt__Issuer` with a literal `value`, `Jwt__Audience` with a literal `value`. The actual signing key string does **not** appear anywhere in this output. That is the whole point.

COMMON MISTAKES

- Storing the signing key as a literal env var (`--set-env-vars Jwt__SigningKey="$PROD_KEY"`). It works at runtime and bakes the key into the Container App's revision history, exactly the same anti-pattern as a literal App Insights connection string. Use `secretref:`.
- Mismatched `Jwt__Issuer` and `Jwt__Audience` between issuance (the controller reads from configuration) and validation (the bearer handler reads from configuration). They must be the same string in both — the controller and the validator both read from `IConfiguration`, so as long as the env vars are set, this happens automatically. The mistake is setting different values in dev and prod and then wondering why dev tokens fail in prod.
- Forgetting that `__` (double underscore) is the env-var separator that ASP.NET Core maps to `:`. Setting `Jwt:SigningKey` literally in an env var name is invalid; setting `Jwt-SigningKey` is also invalid; only `Jwt__SigningKey` works.

√ **Quick check:** The JSON output shows `secretRef: "jwt-signing-key"` for `Jwt__SigningKey`, with no `value` field. `Jwt__Issuer` and `Jwt__Audience` show plain values.

Step 10: *Rewire Swagger UI's Authorize button for Bearer*

In the previous exercise you wired Swagger UI's **Authorize** button to the API-key scheme — when clicked, it asked for a single string and sent it as `X-Api-Key`. The button needs to be rewired for Bearer tokens: when clicked, it should ask for the JWT, and Swagger UI should send it as `Authorization: Bearer <token>` on subsequent requests.

The mechanics are the same `OpenApiSecurityScheme` and `AddSecurityRequirement` calls; only the values change.

1. **Open** `Program.cs` and find the existing `AddSwaggerGen(...)` block from the previous exercise.
2. **Replace** the API-key security scheme with the Bearer scheme. The full block should look like this — copy it whole, replacing whatever was there before:

> `Program.cs`

```
using Microsoft.OpenApi.Models;

builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title =
        "CloudCi API", Version = "v1" });

    // Bearer scheme: clicking "Authorize" in Swagger UI
        prompts for the JWT,
    // and Swagger sends it as `Authorization: Bearer
        <token>` on every request.
    c.AddSecurityDefinition("Bearer", new OpenApiSecur-
        ityScheme
    {
        Name = "Authorization",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        Description = "Paste the JWT returned by /api/
        tokens/login. Do not include the word
        'Bearer'."
    });

    c.AddSecurityRequirement(new OpenApiSecurityRequire-
        ment
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

3. **Verify** locally before pushing. `dotnet run`, open `http://localhost:<port>/swagger`, click **Authorize**, and confirm the dialog asks for a token (not an API key). Get a token from `POST /api/tokens/login`, paste it into the Authorize dialog, then call `GET /api/quotes` from inside Swagger UI — it should return 200 with the JSON array.

CONCEPT DEEP DIVE

`Type = SecuritySchemeType.Http` together with `Scheme = "bearer"` is the OpenAPI 3.0 way of saying "use the HTTP Authorization header with the Bearer scheme." Swagger UI knows how to render this — it shows a single text box, prepends `Bearer` automatically, and applies the header to every subsequent request as long as the dialog stays "authorized."

The `BearerFormat = "JWT"` field is descriptive only — it's a hint to the UI and to consumers reading the OpenAPI document, with no runtime effect on validation.

⚠ **Common Mistakes**

- Pasting the token *with* the literal `Bearer` prefix into the Authorize dialog. Swagger UI prepends it for you. The result is the request header `Authorization: Bearer Bearer eyJ...`, which the bearer handler rejects.
- Leaving the API-key `AddSecurityDefinition` block in place alongside the Bearer one. Swagger UI then shows two schemes in the Authorize dialog and tries to apply both. Either delete the old block or, if you really want to show both options, add `[AllowAnonymous]` overrides on the tokens endpoint and configure the `AddSecurityRequirement` to require only Bearer.

√ **Quick check:** The Swagger UI Authorize dialog shows a Bearer scheme. Pasting the token (without the word `Bearer`) and calling `GET /api/quotes` returns 200.

Step 11: *Push and verify the deployed API*

The code changes from Steps 1–10 are still local. The pipeline from earlier in the course rebuilds and updates the Container App on every push to `main`, so commit and push to roll the new bearer flow into the running container.

1. **Commit and push:**

```
git add CloudCiApi.csproj Program.cs Controllers/
        Models/ Services/ appsettings.Development.json
git rm Middleware/ApiKeyMiddleware.cs   # if you kept
        it; safe to skip if already deleted
git commit -m "Replace API-key middleware with JWT
        bearer authentication"
git push
gh run watch
```

   The workflow should turn green in a couple of minutes.

2. **Capture** the FQDN:

```
FQDN=$(az containerapp show \
  -g rg-api-week6 -n ca-api-week6 \
  --query properties.configuration.ingress.fqdn -o tsv)

echo "$FQDN"
```

3. **Verify** the three-way behaviour with `curl`. Anonymous → 401, login → 200 + token, with token → 200:

```
# Anonymous request — expected 401
curl -i "https://$FQDN/api/quotes"

# Login — expected 200 with a JWT in the body
TOKEN=$(curl -s -X POST "https://$FQDN/api/tokens/
        login" \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"alice123"}' \
  | jq -r .token)

echo "$TOKEN" | head -c 40 ; echo "..."

# Authorized request — expected 200 with the JSON array
curl -i -H "Authorization: Bearer $TOKEN" "https://
        $FQDN/api/quotes"
```

COMMON MISTAKES

- Forgetting the literal word `Bearer` (and the space) in the curl `-H` argument. The header value must be exactly `Bearer <token>`. `Authorization: <token>` returns 401.
- Using bad credentials in the login request and getting 401, then pasting the empty `$TOKEN` into the next curl — the second request also returns 401, but for a different reason. Always check that `$TOKEN` is non-empty before using it.
- Hitting the API before the new revision is active. `az containerapp revision list -g rg-api-week6 -n ca-api-week6 -o table` shows which revision serves traffic.

√ **Quick check:** Three responses, in order: 401, 200 with token, 200 with JSON array. The Container Apps revision history shows a new revision active with 100% traffic.

Step 12: *Test Your Implementation*

Walk through the flow end to end one more time, both locally and against the deployed API.

1. **Run the app locally:**

```
dotnet run
```

2. **Anonymous request returns 401:**

```
curl -i http://localhost:<port>/api/quotes
```

Expected: `HTTP/1.1 401 Unauthorized` with the response header `WWW-Authenticate: Bearer`.

3. **Login returns 200 with a JWT:**

```
curl -s -X POST http://localhost:<port>/api/tokens/
        login \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"alice123"}'
```

Expected: `{"token":"eyJ...","expiresAt":"2026-..."}`. Decode the `eyJ...` portion at https://jwt.io and confirm the payload contains `sub: "alice"`, `name: "alice"`, `role: "admin"`, `iss: "cloudci-api-dev"`, `aud: "cloudci-api-clients-dev"`, and an `exp` ~1 hour in the future.

4. **Bad credentials return 401:**

```
curl -i -X POST http://localhost:<port>/api/tokens/
        login \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","password":"wrong"}'
```

Expected: `HTTP/1.1 401 Unauthorized` with no body.

5. **Authorized request returns 200:** Use the token from step 3 and call the protected endpoint:

```
curl -i -H "Authorization: Bearer $TOKEN" http://local-
        host:<port>/api/quotes
```

   Expected: `HTTP/1.1 200 OK` with the JSON array of quote DTOs.

6. **Tampered token returns 401:** Change one character in the middle of the token and retry:

```
BAD_TOKEN="${TOKEN:0:50}X${TOKEN:51}"
curl -i -H "Authorization: Bearer $BAD_TOKEN" http://
        localhost:<port>/api/quotes
```

   Expected: 401. The signature check fails because the payload no longer matches.

7. **Repeat steps 2–5 against the deployed API** at `https://$FQDN`.
8. **Swagger UI works:** Open `https://$FQDN/swagger`, click **Authorize**, paste a fresh token (without the word `Bearer`), then exercise `GET /api/quotes` from the UI. Confirm 200 with the JSON array.

√ ***Success indicators:***

- *Anonymous requests to* `/api/quotes` *return 401 both locally and on the Container App*
- `POST /api/tokens/login` *with valid credentials returns 200 with a JWT and an expiry*
- *Bad credentials return 401 with no token*
- *Requests with the Bearer header return 200 with the JSON array*
- *Tampered tokens return 401*
- *Swagger UI's Authorize button accepts the JWT and persists it across requests*

√ ***Final verification checklist:***

- [ ] `Microsoft.AspNetCore.Authentication.JwtBearer` *listed in* `CloudCiApi.csproj`
- [ ] *JWT validation parameters configured in* `Program.cs` *with all four* `Validate*` *flags on*
- [ ] `appsettings.Development.json` *has a 256-bit-or-larger dev signing key*
- [ ] *The* `ApiKeyMiddleware` *registration and class file are gone*
- [ ] `IUserStore` *and* `InMemoryUserStore` *registered as a singleton*
- [ ]

> `TokensController` *issues signed JWTs with* `iss`, `aud`, `exp`, `sub`, `name`, *and an optional* `role`
>
> ☐ `QuotesController` *is* `[Authorize]`*-marked*
>
> ☐ *Container Apps secret* `jwt-signing-key` *set; env var* `Jwt__SigningKey` *references it via* `secretref:`
>
> ☐ `Jwt__Issuer` *and* `Jwt__Audience` *set as plain env vars on the Container App*
>
> ☐ *Swagger UI Authorize button uses the Bearer scheme*

Step 13: *Tear down the cloud resources*

This is the last exercise of the chapter and the last exercise that uses the Week 6 cloud resources. Nothing later in the course needs `rg-api-week6`, the Container App, the ACR, or the Entra app registration created for OIDC federation. Tear all of it down so you finish with no resources running and no orphaned tenant-level identities.

The work splits into two homes — the Azure subscription holds the running resources, and Microsoft Entra ID (a tenant-level service) holds the identity used by the pipeline. Deleting one does **not** delete the other, exactly as you saw at the end of the previous chapter.

1. **Capture** the Entra app's `appId` before you delete it (you'll need it for the second command, and the simplest way to look it up later is by display name):

```
APP_ID=$(az ad app list \
  --display-name github-cloudci-api-oidc \
  --query "[0].appId" -o tsv)

echo "$APP_ID"
```

If `$APP_ID` is empty, the app already doesn't exist — skip the `az ad app delete` step.

2. **Delete** the resource group. This removes the Container App, the ACR with all its images, the Container Apps environment, every Container Apps secret (including `jwt-signing-key`), and every role assignment scoped under the group:

```
# 1. Delete the resource group (Container App, ACR, all
        of it).
az group delete -n rg-api-week6 --yes --no-wait
```

`--no-wait` returns immediately and lets the deletion run in the background. The full teardown takes a few minutes.

3. **Delete** the Entra app registration that was created for OIDC federation. The app registration lives in the tenant, not in any subscription resource group, so the RG delete above does **not** remove it. Delete it explicitly:

```
# 2. Delete the Entra app registration that was created
        for OIDC federation.
# The app registration lives in the tenant, not in any
        subscription resource group,
# so the RG delete above does NOT remove it. Delete it
        explicitly.
az ad app delete --id "$APP_ID"
```

4. **Verify** the resource group is gone (give it a few minutes if `--no-wait` is still working in the background):

```
az group exists -n rg-api-week6
```

Expected: `false`.

5. **Verify** the Entra app registration is gone:

```
az ad app list --display-name github-cloudci-api-oidc
        -o tsv
```

Expected: empty output (no rows).

6. **Optionally** delete the GitHub repo secrets that pointed at the now-deleted Entra app and Azure subscription. They're inert without working resources behind them, but cleaning them up keeps the repo tidy:

```
gh secret delete AZURE_CLIENT_ID --repo <your-username>/
        cloudci-api
gh secret delete AZURE_TENANT_ID --repo <your-username>/
        cloudci-api
gh secret delete AZURE_SUBSCRIPTION_ID --repo <your-
        username>/cloudci-api
gh secret delete ACR_NAME --repo <your-username>/
        cloudci-api
```

Replace `<your-username>/cloudci-api` with the actual owner and name of your repository.

CONCEPT DEEP DIVE

Why the two-step cleanup matters. The resource group is a subscription-level container — `az group delete` cascades to every Azure resource scoped under it, including the Container App, the ACR, the Container Apps environment, every Container Apps secret stored on the app, and every role assignment scoped to those resources.

The Entra app registration, however, is a *tenant-level* identity object. It lives in Microsoft Entra ID, not in your Azure subscription, and survives every subscription operation. Without the second `az ad app delete`, the federated identity sticks around as orphaned tenant clutter — visible in the Entra portal with broken role assignments pointing at deleted resources, contributing nothing and confusing future you.

Note that Entra ID *soft-deletes* app registrations for 30 days; they're recoverable from **App registrations → Deleted applications** in the Entra portal and garbage-collected automatically afterwards. That's a useful safety net if you delete the wrong app — you have a month to restore it. It also means `az ad app list` won't show the deleted app, but a soft-deleted entry still exists for the next 30 days; that's expected and harmless.

⚠ **Common Mistakes**

- Stopping after `az group delete`. The Entra app stays alive and accumulates over multiple cohorts as orphaned `github-cloudci-api-oidc` entries.
- Forgetting that the GitHub repo secrets `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, and `ACR_NAME` in your repository are now stale. The next workflow run will fail at **Sign in to Azure** because the client ID points at a deleted app. Either delete the secrets via `gh secret delete <name>`, or leave them alone — they're inert without working resources behind them.

- Confusing Entra soft-delete with hard-delete. `az ad app delete` does a soft-delete; the app is recoverable for 30 days. To purge immediately, use `az rest --method DELETE --url "https://graph.microsoft.com/v1.0/directory/deletedItems/$APP_ID"`. For this lab, soft-delete is fine — it'll be auto-purged before the next cohort starts.
- Orphaned role assignments at *subscription* scope. If the previous chapter assigned a role to the Entra service principal at subscription scope (rather than at the resource-group scope), `az group delete` does not clean it up. List subscription-scoped assignments with `az role assignment list --assignee "$APP_ID" --scope /subscriptions/<sub-id>` and `az role assignment delete` any that turn up.

√ **Quick check:** `az group exists -n rg-api-week6` returns `false`, `az ad app list --display-name github-cloudci-api-oidc -o tsv` returns nothing, and your subscription's billing is no longer accruing for any of the Week 6 resources.

8.6.4 Common Issues

> ***If you encounter problems:***
> ***Every request returns 401 even with a valid token:*** *Either* `app.UseAuthentication()` *is missing from the pipeline, or it comes after* `app.UseAuthorization()`*. The order is* `UseAuthentication` *first,* `UseAuthorization` *second, then the endpoint mapping.*
> ***Startup throws*** **`IDX10720: Unable to create KeyedHashAlgorithm for algorithm 'HS256'`***: The signing key is shorter than 256 bits. Use* `openssl rand -base64 48` *(or longer) and re-set the secret.*
> ***Login returns 200 with a token, but the next request returns 401:*** *Almost always a mismatch between the issuer/audience used at issuance (*`Jwt:Issuer`*/*`Jwt:Audience` *from configuration in* `TokensController`*) and the values configured for validation (*`ValidIssuer`*/*`ValidAudience` *in* `Program.cs`*). Both are read from the same configuration tree, so the actual mistake is usually different env-var values in dev vs prod. Compare the values directly.*
> ***Container App can't read the signing key — startup logs show a null configuration value:*** *The env var name is wrong. ASP.NET Core maps* `__` *(double underscore) in env vars to* `:` *in the config tree. The env var must be* `Jwt__SigningKey`*, not* `Jwt:SigningKey` *or* `Jwt-SigningKey`*.*
> ***Swagger UI's Authorize button still asks for an API key:*** *The old* `AddSecurityDefinition` *from the previous exercise is still in*

> `Program.cs`. *Delete the API-key block; only the Bearer block should remain.*
>
> **`curl` returns 401 with `Bearer <token>` but Swagger UI works:** *You included the literal word `Bearer` inside the Swagger Authorize dialog. Swagger prepends it for you, so the resulting header is `Authorization: Bearer Bearer <token>`. Paste only the token.*
>
> **`az group delete` returns immediately but resources still appear in the Portal:** *That's `--no-wait` working as intended. Re-check with `az group exists -n rg-api-week6`.*
>
> ***Still stuck?*** *Verify three things in order: the JWT bearer package in `CloudCiApi.csproj`, the `AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(...)` block in `Program.cs`, and the Container App's env var list. All three must be correct for the bearer flow to work.*

8.6.5 Summary

You replaced the API-key middleware with JWT bearer authentication. A `TokensController` issues signed JWTs against an in-memory user list; `QuotesController` is `[Authorize]`-marked and the framework's authentication middleware materializes a `ClaimsPrincipal` from the token on every request. The signing key is delivered to Container Apps as a secret referenced via `secretref:` — the same pattern you used for the App Insights connection string in the previous chapter — so it never appears in `az containerapp show`. Swagger UI's **Authorize** button is rewired for Bearer,

which makes the API explorable from the browser with one paste of a token.

- ✓ JWT bearer authentication configured with all four `TokenValidationParameters` checks on
- ✓ A login endpoint that validates credentials against an in-memory user store and signs a JWT with claims for `sub`, `name`, and an optional `role`
- ✓ The API-key middleware removed cleanly — replacement, not layering
- ✓ Production signing key delivered as a Container Apps secret, never as a literal env var
- ✓ Swagger UI's Authorize button rewired from API-key to Bearer
- ✓ Resource group, Entra app registration, and (optionally) GitHub secrets all torn down — no orphaned cloud or tenant resources

> **KEY TAKEAWAY:**
> A JWT is three base64url segments separated by dots. The signature is the integrity check; the payload is plaintext, readable by anyone who sees the token. ASP.NET Core's bearer handler validates the four standard claims (`iss`, `aud`, `exp`, `nbf`) plus the signature on every request — turning any one of those checks off is a security bug. Per-user identity gives you strictly more than an API key did, which is why the right move was *replace* rather than *layer*.

8.6.6 Going Deeper (Optional)

Want to explore more?

- *Add a* `[Authorize(Roles = "admin")]` *attribute on a write endpoint (a* `POST /api/quotes` *that adds a quote, say) and confirm that alice's token works but bob's returns 403. The role claim from Step 6 is what makes this work.*
- *Replace HS256 with RS256. Generate an RSA keypair with* `openssl genrsa -out signing.key 2048` *and* `openssl rsa -in signing.key -pubout -out signing.pub`*. The issuer signs with the private key; the validator gets the public key only. This is the model real IdPs use.*
- *Add refresh tokens. The access token stays at 1 hour; a separate* `POST /api/tokens/refresh` *endpoint exchanges a long-lived refresh token (single-use, server-tracked) for a fresh access token. This is the production pattern alluded to in the lifetime trade-off discussion above.*
- *Read the JWT spec — RFC 7519 is short and worth one careful read. Pair it with RFC 7515 (JWS, the signature container that JWTs use) and RFC 7518 (JWA, the algorithms registered for use with JWS).*
- *Investigate* ***OAuth 2.1*** *and* ***OpenID Connect****. JWT is a token format; OAuth/OIDC are protocols for getting tokens. The next time you wire authentication against an external IdP —*

> *Microsoft Entra, Google, Auth0 — those protocols are how the token reaches your app.*

8.6.7 Done!

This exercise ends the REST API and DTOs chapter, and the lab the chapter ran on. The resource group is gone, the Entra app registration is gone, the GitHub secrets in your repo are stale and inert. There is nothing left running and nothing left billing.

You started the chapter with a controller returning JSON. You finish it with the same controller behind per-user authentication, a token-issuing endpoint that signs claims with a secret delivered the right way, and a Swagger UI that lets you exercise the protected surface from the browser.

The next thing the course tackles is what happens when one service has to call another — when your API is no longer the leaf of the request graph but a node in the middle, calling downstream APIs and propagating identity across the hops. The patterns you've seen here (a JWT carrying user identity, a validator checking signature and claims, a configuration-driven secret) are the same patterns that scale up to a multi-service architecture; the difference is that the issuer becomes a separate identity provider and the signature scheme moves from HS256 to RS256. That's the road the rest of the program will spend time on.

PART V

Cloud Databases

Learn to provision and use Azure cloud data services through progressive exercises, from visual portal interface to infrastructure as code and application integration.

8.7 Learning Path

This track guides you through managed data service provisioning and usage using five complementary approaches:

1. ***Portal Interface*** *- Graphical approach for understanding CosmosDB resources and configuration visually*

2. ***Command Line Interface*** *- Scriptable approach for database provisioning automation and repeatability*
3. ***ARM and Bicep*** *- Infrastructure as Code approach for declarative, version-controlled database deployments*
4. ***Connecting from Code*** *- Application integration with a .NET proof-of-concept*
5. ***Azure Blob Storage*** *- Provisioning cloud storage for serving static assets like images*

EXERCISE 9

Portal Interface

9.1 1. Provisioning CosmosDB via Portal

9.2 Provisioning CosmosDB via Portal

9.2.1 Goal

Create an Azure Cosmos DB account with the MongoDB API through the Azure Portal, set up a database and collection, explore the Data Explorer, and retrieve the connection string for application use.

WHAT YOU'LL LEARN:

- How to create a Cosmos DB account with MongoDB API compatibility
- How to choose serverless capacity mode to minimize costs
- How to create databases and collections in Cosmos DB
- How to navigate and use Data Explorer to manage data
- How to locate and copy the connection string for application use
- Key differences between Cosmos DB and native MongoDB

9.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Active Azure subscription with resource creation permissions
- ✓ Familiarity with the Azure Portal (or completed Section 1: Server Foundation)
- ✓ Basic understanding of databases and JSON document structure

9.2.3 Exercise Steps

Overview

1. **Create a Cosmos DB Account**
2. **Create a Database**
3. **Create a Collection**
4. **Insert a Test Document**
5. **Retrieve the Connection String**
6. **Explore Data Explorer Features**

Step 1: *Create a Cosmos DB Account*

Set up an Azure Cosmos DB account configured with the MongoDB API. Cosmos DB is Microsoft's globally distributed, multi-model database service. By selecting the MongoDB API, you get a cloud-hosted database that speaks the MongoDB wire protocol — meaning existing MongoDB drivers and tools work with minimal changes. Choosing serverless capacity mode ensures you only pay for what you use, which is ideal for learning and development.

1. **Navigate to** the Azure Portal at https://portal.azure.com
2. **Search for** "Azure Cosmos DB" using the search bar at the top
3. **Select** Azure Cosmos DB from the search results
4. **Click** the **+ Create** button
5. **Select** "Azure Cosmos DB for MongoDB" from the API options
6. **Choose** the "Request unit (RU) database account" option
7. **Configure** the Basics tab with the following settings:
   - **Subscription**: Select your subscription
   - **Resource Group**: Create new or use existing (e.g., `CloudDatabasesRG`)
   - **Account Name**: Enter a globally unique name (e.g., `cosmosdb-yourname-bcd`, lowercase letters, numbers, and hyphens only)
   - **Region**: Select `North Europe` (or a region close to you)
   - **Capacity mode**: Select `Serverless`
8. **Click** Review + Create, then **click** Create
9. **Wait** for the deployment to complete (this may take 2–5 minutes)

CONCEPT DEEP DIVE

Cosmos DB is a fully managed NoSQL database service that supports multiple data models through different APIs: MongoDB, Cassandra, Gremlin (graph), Table, and the native NoSQL API. The MongoDB API compatibility layer allows applications to use standard MongoDB drivers and the MongoDB query language against Cosmos DB — meaning existing MongoDB application code works with minimal or no changes.

Serverless capacity mode charges only for consumed Request Units (RU/s) and storage. There is no minimum charge when idle, unlike provisioned throughput which reserves and charges for RU/s continuously. This makes serverless ideal for development, learning, and workloads with intermittent traffic. A Request Unit (RU) is a normalized measure of database operation cost — a simple point-read of a 1 KB document costs approximately 1 RU.

⚠ **Common Mistakes**

- The account name must be globally unique across all of Azure — if your name is taken, add a random suffix
- Choosing "Provisioned throughput" instead of "Serverless" will incur continuous charges even when idle
- Selecting a region far from your location increases latency for development and testing
- Forgetting to note the resource group name makes it harder to manage or clean up resources later

√ **Quick check:** Navigate to the Cosmos DB account overview page and verify it shows "API: MongoDB" and "Capacity mode: Serverless"

Step 2: *Create a Database*

Create a database within your Cosmos DB account to serve as a logical container for your collections. A database in Cosmos DB is analogous to a database in MongoDB — it

groups related collections together and provides a namespace for organizing your data.

1. **Navigate to** your newly created Cosmos DB account in the Azure Portal
2. **Click** "Data Explorer" in the left menu
3. **Click** the "New Database" button in the toolbar (or use the dropdown arrow next to "New Collection")
4. **Enter** the Database id: `bookmarks_db`
5. **Click** OK to create the database

> **CONCEPT DEEP DIVE**
>
> In Cosmos DB with the MongoDB API, the data hierarchy is: Account → Database → Collection → Document. This mirrors the MongoDB structure where a server hosts databases, databases contain collections, and collections hold JSON documents. The database level is primarily an organizational unit — provisioning and scaling happen at the collection or account level depending on your capacity mode.
>
> ✓ **Quick check:** The `bookmarks_db` database appears in the Data Explorer tree view on the left side

Step 3: *Create a Collection*

Create a collection within your database to store documents. A collection is where your actual data lives — equivalent to a table in relational databases but schema-free. Each collection requires a shard key (partition key) that determines how Cosmos DB distributes data across physical partitions for scalability.

1. **Expand** the `bookmarks_db` database in Data Explorer
2. **Click** the "..." menu next to `bookmarks_db`
3. **Select** "New Collection"

4. **Configure** the collection settings:
   - **Collection id**: `bookmarks`
   - **Shard key**: `/category`
5. **Click** OK to create the collection

> **CONCEPT DEEP DIVE**
>
> The shard key (partition key) is one of the most important design decisions when creating a collection. It determines how Cosmos DB distributes data across physical partitions. A good shard key has high cardinality (many distinct values) and distributes both storage and request volume evenly. For the bookmarks collection, `/category` provides reasonable distribution across values like "cloud", "development", "tools", etc.
>
> In production, the partition key cannot be changed after creation — you would need to create a new collection and migrate your data. Choose carefully by considering your most frequent query patterns: queries that include the partition key in the filter are efficient single-partition queries, while queries without it require expensive cross-partition fan-out.
>
> ⚠ **Common Mistakes**
>
> - Choosing a low-cardinality shard key (like `/status` with only "active"/"inactive") creates hot partitions
> - Using a field that doesn't exist in most documents results in all those documents landing in the same partition
> - The shard key path is case-sensitive — `/Category` and `/category` are different
>
> ✓ **Quick check:** The `bookmarks` collection appears under `bookmarks_db` in the Data Explorer tree, showing the shard key as `/category`

Step 4: *Insert a Test Document*

Insert a document into your collection to verify the setup and explore how Cosmos DB stores data. This step demonstrates the schema-free nature of document databases — you can insert any valid JSON without predefined schemas or migrations.

1. **Expand** `bookmarks_db` → `bookmarks` in Data Explorer
2. **Click** on "Documents" (or "Items")
3. **Click** the "New Document" button (or "New Item")
4. **Replace** the default JSON with the following document:

```
{
    "title": "Azure Cosmos DB Documentation",
    "url": "https://learn.microsoft.com/en-us/azure/
        cosmos-db/",
    "category": "cloud",
    "tags": ["azure", "database", "nosql"],
    "createdAt": "2026-02-22T10:00:00Z"
}
```

5. **Click** Save
6. **Observe** the additional properties that Cosmos DB automatically adds to the document: `_id`, `_rid`, `_self`, `_etag`, and `_ts`
7. **Insert** a second document with a different category to see multiple partitions in use:

```
{
    "title": "MongoDB Driver for .NET",
    "url": "https://www.mongodb.com/docs/drivers/csharp/
        current/",
    "category": "development",
    "tags": ["mongodb", "dotnet", "driver"],
    "createdAt": "2026-02-22T10:05:00Z"
}
```

CONCEPT DEEP DIVE

Cosmos DB adds several system properties to each document. The `_id` field is the unique document identifier (auto-generated if not provided, just like MongoDB). The `_rid` is the resource identifier used internally by Cosmos DB. The `_etag` enables optimistic concurrency — if two users try to update the same document simultaneously, the etag helps detect conflicts. The `_ts` is a Unix timestamp recording the last modification time.

Notice that you did not need to define a schema before inserting data. Each document in the collection can have a different structure. However, the shard key field (`category` in this case) should be present in every document for proper partition distribution.

⚠ **Common Mistakes**

- Forgetting to include the shard key field (`category`) in your document — the document will still be stored but will land in a null-key partition
- Manually setting `_id` to a value that already exists will cause a duplicate key error
- Inserting very large documents (exceeding 2 MB) will be rejected by Cosmos DB

√ **Quick check:** Both documents appear in the Documents view, each with system properties (`_id`, `_rid`, `_etag`, `_ts`) automatically added

Step 5: *Retrieve the Connection String*

Locate and copy the connection string that applications will use to connect to your Cosmos DB account. This connection string uses the standard MongoDB connection format, which is what enables existing MongoDB drivers to connect to Cosmos DB transparently.

1. **Navigate to** the Cosmos DB account overview (click the account name in the breadcrumb)
2. **Click** "Connection strings" under the Settings section in the left menu
3. **Locate** the "PRIMARY CONNECTION STRING" field
4. **Click** the copy button next to the connection string to copy it to your clipboard
5. **Note** the connection string format:

```
mongodb://
<account>:<key>@<account>.mongo.cosmos.azure.com:10255/?
ssl=true&retrywrites=false&...
```

6. **Save** the connection string in a safe, temporary location (you will need it in later exercises)

CONCEPT DEEP DIVE

The connection string uses the standard MongoDB connection URI format (`mongodb://...`), which is what enables existing MongoDB drivers and tools to connect to Cosmos DB without code changes. Key differences from a typical MongoDB connection string:

- **Port 10255**: Cosmos DB uses this port for the MongoDB API endpoint, rather than MongoDB's default port 27017
- **`ssl=true`** : All connections to Cosmos DB are encrypted with TLS/SSL — this is mandatory, not optional
- **`retrywrites=false`** : Cosmos DB does not support MongoDB's retryable writes feature in the same way native MongoDB does, so this must be explicitly disabled

The connection string contains your account access key, which grants full read/write access to all databases in the account. Treat it like a password — never commit it to source code, share it in chat, or include it in screenshots. In production and in later exercises, you will use environment variables and Azure App Service configuration to manage this securely.

⚠ **Common Mistakes**

- Copying the connection string with trailing whitespace can cause connection failures
- Accidentally using the secondary connection string works fine but makes key rotation confusing later
- Sharing the connection string publicly (e.g., in a Git commit) exposes your entire database — Azure will sometimes detect this and send security alerts

√ **Quick check:** The connection string starts with `mongodb://` and contains `.mongo.cosmos.azure.com:10255`

Step 6: *Explore Data Explorer Features*

Familiarize yourself with Data Explorer's query and management capabilities. Data Explorer is your primary tool for interacting with Cosmos DB data directly from the Azure Portal — useful for debugging, ad hoc queries, and verifying that your application is storing data correctly.

1. **Navigate to** Data Explorer in the left menu
2. **Expand** `bookmarks_db` → `bookmarks` → `Documents`
3. **Try a filter query** by entering the following in the filter bar:

```
{ "category": "cloud" }
```

4. **Click** "Apply Filter" to see only documents matching the filter
5. **Try a more complex query** — filter by a nested array value:

```
{ "tags": "azure" }
```

6. **Insert** one or two more documents with different categories (e.g., `"tools"`, `"learning"`) to observe how documents distribute across partition key values
7. **Explore** the Mongo Shell (if available) at the bottom of Data Explorer — you can run MongoDB shell commands directly. First, select the database, then query the collection:

```
use bookmarks_db
db.bookmarks.find({ "category": "cloud" })
```

CONCEPT DEEP DIVE

Data Explorer supports both the visual document browser and a MongoDB-compatible shell. The filter bar accepts standard MongoDB query syntax, making it familiar if you have MongoDB experience. Queries that filter on the shard key (`category`) are efficient single-partition queries. Queries without the shard key require scanning all partitions, which consumes more Request Units and is slower at scale.

The Mongo Shell in Data Explorer supports a subset of MongoDB shell commands. This is useful for quick operations but for more complex scripting, you would use the `mongosh` CLI tool or connect from application code using the MongoDB driver.

√ **Quick check:** Filter queries return the expected subset of documents, and you can see documents from different categories in the collection

9.2.4 Common Issues

> ***If you encounter problems:***
> ***"Account name already exists":*** *Cosmos DB account names must be globally unique. Add a random number or your initials to the name.*
> ***Deployment takes longer than 10 minutes:*** *This is unusual. Check the deployment status under "Deployments" in your resource group. If it fails, delete and try again with a different region.*
> ***Cannot see Data Explorer:*** *Make sure you are navigated to the Cosmos DB account resource, not the resource group. Data Explorer is in the left menu of the Cosmos DB account blade.*
> ***Filter query returns no results:*** *Ensure the field names and values match exactly (they are case-sensitive). Check that you are using double quotes for JSON strings.*
> ***Still stuck?*** *Verify you selected "MongoDB" as the API type when creating the account. If you chose a different API, you will need to create a new account — the API type cannot be changed after creation.*

9.2.5 Summary

You've successfully provisioned an Azure Cosmos DB account with the MongoDB API which:

- √ Provides a fully managed, globally distributable NoSQL database
- √ Uses serverless capacity to minimize costs for development
- √ Supports the MongoDB wire protocol for driver compatibility

- ✓ Stores schema-free JSON documents with automatic indexing

> **KEY TAKEAWAY:**
>
> Cosmos DB with the MongoDB API gives you the managed infrastructure benefits of a cloud database service (automatic scaling, global distribution, built-in high availability) while maintaining compatibility with the MongoDB ecosystem. The connection string format is standard MongoDB, meaning existing MongoDB drivers and tools work without code changes — only the connection string needs to change.

9.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Explore the "Metrics" blade to understand Request Unit consumption for different operations*
> - *Try the "Replicate data globally" option to see how Cosmos DB enables multi-region deployments*
> - *Read about Cosmos DB consistency levels and how they differ from MongoDB's read/write concerns*
> - *Investigate the "Firewall and virtual networks" settings to understand network security options*

9.2.7 Done! 🎉

Great job! You've provisioned your first Azure Cosmos DB account with the MongoDB API, created a database and collection, inserted and queried documents, and retrieved the connection string. This foundation prepares you for automat-

ing the same provisioning with the Azure CLI and Bicep in the next exercises.

EXERCISE 10

Command Line Interface

10.1 2. Provisioning CosmosDB via AZ CLI

10.2 Provisioning CosmosDB via AZ CLI

10.2.1 Goal

Provision an Azure Cosmos DB account with the MongoDB API, create a database and collection, and retrieve the connection string — all using the Azure CLI. Build a reusable shell script that automates the entire provisioning process.

WHAT YOU'LL LEARN:

- How to provision a Cosmos DB account with MongoDB API using `az cosmosdb` commands
- How to create databases and collections from the command line
- How to query connection strings and keys programmatically
- How to build a reusable provisioning script for repeatable deployments
- Key advantages of CLI-based provisioning over portal-based approaches

10.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Active Azure subscription with resource creation permissions
- ✓ Azure CLI installed and authenticated (`az login`)
- ✓ A terminal (bash, zsh, or Azure Cloud Shell)
- ✓ Familiarity with basic shell scripting concepts

10.2.3 Exercise Steps

Overview

1. **Set Up Variables and Resource Group**
2. **Create the Cosmos DB Account**
3. **Create the Database and Collection**
4. **Retrieve the Connection String**
5. **Build the Complete Provisioning Script**
6. **Test and Verify**

Step 1: *Set Up Variables and Resource Group*

Define reusable shell variables and create a resource group to hold your Cosmos DB resources. Using variables at the top of your workflow makes commands reusable and reduces errors from typos. The resource group acts as a logical container for related Azure resources, making it easy to manage lifecycle, access control, and cost tracking together.

1. **Open** your terminal and **ensure** you are logged in to Azure:

```
az login
```

2. **Define** the following shell variables (replace `yourname` with your actual name):

```
RESOURCE_GROUP="CloudDatabasesRG"
LOCATION="northeurope"
ACCOUNT_NAME="cosmosdb-yourname-bcd"
DATABASE_NAME="bookmarks_db"
COLLECTION_NAME="bookmarks"
```

3. **Create** the resource group:

```
az group create \
  --name $RESOURCE_GROUP \
  --location $LOCATION
```

4. **Verify** the resource group was created:

```
az group show --name $RESOURCE_GROUP --query
        "{name:name, location:location}" --output table
```

CONCEPT DEEP DIVE

Shell variables keep configuration values in one place, making scripts easier to maintain and reuse across environments. The `--location` parameter determines the Azure region where the resource group metadata is stored — choose a region close to you for lower latency during development. The resource group itself is free; you only pay for the resources inside it.

⚠ **Common Mistakes**

- Using uppercase letters or spaces in `ACCOUNT_NAME` — Cosmos DB account names must be lowercase letters, numbers, and hyphens only (3–44 characters)
- Forgetting to run `az login` first results in authentication errors on every subsequent command
- Variable names are case-sensitive in bash — `$RESOURCE_GROUP` and `$resource_group` are different

√ **Quick check:** The `az group show` command displays your resource group name and location in a table

Step 2: *Create the Cosmos DB Account*

Create the Cosmos DB account with MongoDB API compatibility and serverless capacity mode. This is the foundational resource — all databases and collections live within this account. The account creation takes several minutes because Azure is provisioning the underlying infrastructure across its data centers.

1. **Run** the following command to create the Cosmos DB account:

```
az cosmosdb create \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --kind MongoDB \
  --server-version "4.2" \
  --capabilities EnableServerless \
  --locations regionName=$LOCATION failoverPriority=0
        isZoneRedundant=false
```

2. **Wait** for the command to complete (this typically takes 3–8 minutes)
3. **Verify** the account was created successfully:

```
az cosmosdb show \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --query "{name:name, kind:kind, location:location}" \
  --output table
```

CONCEPT DEEP DIVE

Each flag in the create command serves a specific purpose:

- `--kind MongoDB` selects the MongoDB API, enabling MongoDB wire protocol compatibility
- `--capabilities EnableServerless` sets the capacity mode to serverless, which charges only for consumed Request Units (RUs) — ideal for development and intermittent workloads with no minimum charge when idle
- `--server-version "4.2"` sets the MongoDB wire protocol version that the account will support
- `--locations` configures the primary region — with serverless capacity, you get a single-region deployment

The `--query` and `--output table` flags use JMESPath expressions to extract specific fields from the JSON response and display them in a readable table format. This technique is useful for quickly verifying resource properties without scrolling through verbose JSON output.

⚠ **Common Mistakes**

- Forgetting `--kind MongoDB` creates a NoSQL (Core) API account instead — the API type cannot be changed after creation
- Omitting `--capabilities EnableServerless` defaults to provisioned throughput, which incurs continuous charges even when idle
- If the account name is already taken globally, you will get an error — try adding a random number suffix

✓ **Quick check:** The `az cosmosdb show` command displays a table with `kind` showing `MongoDB`

Step 3: *Create the Database and Collection*

Create the database and collection within your Cosmos DB account. These commands mirror the data model hierarchy:

account → database → collection → documents. The database is a logical grouping, while the collection is where your actual data will be stored.

1. **Create** the MongoDB database:

```
az cosmosdb mongodb database create \
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --name $DATABASE_NAME
```

2. **Create** the collection with a shard key:

```
az cosmosdb mongodb collection create \
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --database-name $DATABASE_NAME \
  --name $COLLECTION_NAME \
  --shard "category"
```

3. **Verify** the database was created:

```
az cosmosdb mongodb database list \
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --output table
```

4. **Verify** the collection was created:

```
az cosmosdb mongodb collection list \
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --database-name $DATABASE_NAME \
  --output table
```

CONCEPT DEEP DIVE

The `az cosmosdb mongodb` subcommand group provides MongoDB-specific operations. The command hierarchy mirrors the data model: `az cosmosdb mongodb database` for database operations and `az cosmosdb mongodb collection` for collection operations. The `--shard` parameter sets the partition key path — this determines how Cosmos DB distributes documents across physical partitions.

Unlike the Portal experience where you click through forms, CLI provisioning can be scripted, version-controlled in Git, and integrated into CI/CD pipelines. This is a key advantage for infrastructure automation — you can reproduce the exact same infrastructure in development, staging, and production environments.

⚠ **Common Mistakes**

- The `--shard` key path does not include a leading `/` in the CLI command (unlike the Portal which shows `/category`)
- Running the collection create command before the database create command fails — the database must exist first
- Misspelling `--account-name` as `--name` will cause the command to fail with a confusing error

√ **Quick check:** Both `list` commands display your database and collection in table format

Step 4: *Retrieve the Connection String*

Retrieve the connection string that applications will use to connect to your Cosmos DB account. The connection string uses the standard MongoDB URI format, which means existing MongoDB drivers can connect to Cosmos DB without code changes — only the connection string needs to change.

1. **Retrieve** the primary connection string:

```
az cosmosdb keys list \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --type connection-strings \
  --query "connectionStrings[0].connectionString" \
  --output tsv
```

2. **Store** the connection string in an environment variable for use in applications:

```
export COSMOS_CONNECTION_STRING=$(az cosmosdb keys list
        \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --type connection-strings \
  --query "connectionStrings[0].connectionString" \
  --output tsv)
```

3. **Verify** the variable was set:

```
echo "Connection string starts with: ${COSMOS_CONNEC-
        TION_STRING:0:30}..."
```

CONCEPT DEEP DIVE

The `--type connection-strings` flag retrieves the full connection URI instead of raw access keys. The `--query` parameter uses a JMESPath expression to extract just the first connection string from the response array. Using `--output tsv` produces raw text without JSON quotes, which is essential when storing the value in a variable — quoted strings would break application connection attempts.

Storing the connection string in an environment variable keeps it out of shell history files and makes it available to applications that read configuration from environment variables (a twelve-factor app best practice). The variable is only available in the current shell session — for persistence, use a `.env` file (excluded from Git), Azure Key Vault, or your IDE's run configuration.

⚠ **Common Mistakes**

- Using `--type keys` instead of `--type connection-strings` returns raw access keys, not the complete MongoDB connection URI
- Forgetting `--output tsv` wraps the value in JSON quotes, which breaks connection attempts
- The environment variable only persists in the current terminal session — opening a new terminal requires setting it again
- Never echo the full connection string in shared environments or logs — it contains the account access key

√ **Quick check:** The echoed connection string starts with `mongodb://` and contains `.mongo.cosmos.azure.com:10255`

Step 5: *Build the Complete Provisioning Script*

Combine all the individual commands into a single reusable script. Scripting the full provisioning process means you can reproduce your entire database infrastructure with a

single command — essential for setting up new environments, disaster recovery, or onboarding new team members.

1. **Create** a new file named `provision-cosmosdb.sh`
2. **Add** the following script content:

> `provision-cosmosdb.sh`

```bash
#!/bin/bash
set -euo pipefail

# ============================================
# CosmosDB Provisioning Script
# Provisions a CosmosDB account with MongoDB API
# ============================================

# Configuration — change these for your environment
RESOURCE_GROUP="CloudDatabasesRG"
LOCATION="northeurope"
ACCOUNT_NAME="cosmosdb-yourname-bcd"
DATABASE_NAME="bookmarks_db"
COLLECTION_NAME="bookmarks"

echo "=== CosmosDB Provisioning Script ==="
echo ""

# Step 1: Resource Group
echo "Creating resource group '$RESOURCE_GROUP' in
        '$LOCATION'..."
az group create \
  --name $RESOURCE_GROUP \
  --location $LOCATION \
  --output none

# Step 2: CosmosDB Account
echo "Creating CosmosDB account '$ACCOUNT_NAME' (this
        may take several minutes)..."
az cosmosdb create \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --kind MongoDB \
  --server-version "4.2" \
  --capabilities EnableServerless \
  --locations regionName=$LOCATION failoverPriority=0
        isZoneRedundant=false \
  --output none

# Step 3: Database
echo "Creating database '$DATABASE_NAME'..."
az cosmosdb mongodb database create \
```

```
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --name $DATABASE_NAME \
  --output none

# Step 4: Collection
echo "Creating collection '$COLLECTION_NAME' with shard
        key 'category'..."
az cosmosdb mongodb collection create \
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --database-name $DATABASE_NAME \
  --name $COLLECTION_NAME \
  --shard "category" \
  --output none

# Step 5: Retrieve Connection String
echo "Retrieving connection string..."
CONNECTION_STRING=$(az cosmosdb keys list \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --type connection-strings \
  --query "connectionStrings[0].connectionString" \
  --output tsv)

echo ""
echo "=== Provisioning Complete ===="
echo "Account:    $ACCOUNT_NAME"
echo "Database:   $DATABASE_NAME"
echo "Collection: $COLLECTION_NAME"
echo ""
echo "Connection String:"
echo "$CONNECTION_STRING"
```

3. **Make** the script executable:

```
chmod +x provision-cosmosdb.sh
```

4. **Run** the script:

```
./provision-cosmosdb.sh
```

CONCEPT DEEP DIVE

The `set -euo pipefail` line at the top enables strict error handling, which is critical for infrastructure scripts:

- `-e` exits immediately if any command returns a non-zero exit code
- `-u` treats references to unset variables as errors (catches typos in variable names)
- `-o pipefail` ensures that errors in piped commands are not silently swallowed

The `--output none` flag suppresses the verbose JSON output that Azure CLI returns by default. This keeps the script output clean with only the progress messages you defined. Errors will still be printed to stderr, so failures are not hidden.

This script is idempotent for the resource group (`az group create` succeeds even if it already exists) but the `az cosmosdb create` command will fail if an account with the same name already exists. For a fully idempotent script, you would add existence checks or use Bicep templates (covered in the next section).

⚠ **Common Mistakes**

- Forgetting `chmod +x` results in "Permission denied" when trying to run the script
- Running the script without being logged in (`az login`) causes all commands to fail
- The script prints the connection string to stdout — be careful not to share terminal output or logs that contain it

✓ **Quick check:** The script runs without errors and prints the account name, database, collection, and connection string at the end

Step 6: *Test and Verify*

Confirm that all resources were provisioned correctly by checking them both from the CLI and the Azure Portal. This verification step ensures your automation produced the expected results.

1. **Verify** the Cosmos DB account exists and has the correct configuration:

```
az cosmosdb show \
  --name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --query "{name:name, kind:kind, capabilit-
        ies:capabilities[0].name}" \
  --output table
```

2. **Verify** the database and collection exist:

```
az cosmosdb mongodb database list \
  --account-name $ACCOUNT_NAME \
  --resource-group $RESOURCE_GROUP \
  --query "[].name" \
  --output tsv
```

3. **Open** the Azure Portal at https://portal.azure.com and **navigate to** your Cosmos DB account to visually confirm:
   - The account shows "API: MongoDB" and "Capacity mode: Serverless"
   - The `bookmarks_db` database appears in Data Explorer
   - The `bookmarks` collection exists with shard key `/category`
4. **Clean up** resources when you are finished experimenting (optional):

```
az group delete --name $RESOURCE_GROUP --yes --no-wait
```

CONCEPT DEEP DIVE

The `--no-wait` flag on the delete command starts the deletion process and returns immediately without waiting for it to complete. Resource group deletion cascades to all resources inside it — this is both powerful and dangerous, which is why the `--yes` flag is required to skip the confirmation prompt.

Verifying CLI-provisioned resources in the Portal is good practice — it confirms that the automation produced the same result you would get through manual provisioning. In production workflows, this verification step would be replaced by automated integration tests or infrastructure validation scripts.

⚠ **Common Mistakes**

- Running `az group delete` without `--yes` causes the command to hang waiting for interactive confirmation
- Deleting the resource group removes everything inside it — make sure you don't have other resources in the same group that you want to keep
- The delete operation may take several minutes even with `--no-wait` — the flag only makes the CLI return immediately, not the actual deletion

√ **Quick check:** The Portal shows your Cosmos DB account with MongoDB API and Serverless capacity, and the database and collection are visible in Data Explorer

10.2.4 Common Issues

> ***If you encounter problems:***
> ***"The subscription is not registered to use namespace 'Microsoft.DocumentDB'":*** *Run* `az provider register --namespace Microsoft.DocumentDB` *and wait a few minutes before retrying.*
> ***"Account name already exists":*** *Cosmos DB account names are globally unique. Change* `ACCOUNT_NAME` *to include a random suffix or your initials.*
> ***"AuthorizationFailed":*** *Your Azure account may not have the required permissions. Ensure you have at least Contributor role on the subscription or resource group.*
> ***Command hangs or takes very long:*** *The* `az cosmosdb create` *command can take up to 10 minutes. If it exceeds this, check the deployment status in the Azure Portal under your resource group's "Deployments" section.*
> ***Still stuck?*** *Verify you are using the correct Azure subscription with* `az account show` *and switch if needed with* `az account set --subscription "Your Subscription Name"`*.*

10.2.5 Summary

You've successfully provisioned Azure Cosmos DB infrastructure entirely from the command line which:

- ✓ Creates a Cosmos DB account with MongoDB API and serverless capacity
- ✓ Sets up a database and collection with a partition key
- ✓ Retrieves the connection string programmatically

- ✓ Packages everything into a reusable provisioning script

> **KEY TAKEAWAY:**
>
> CLI-based provisioning transforms manual portal clicks into repeatable, scriptable commands. The provisioning script you built can be version-controlled in Git, shared with team members, and integrated into CI/CD pipelines. This is a fundamental shift from "click-ops" to infrastructure automation — the same approach scales from a single development database to provisioning hundreds of environments.

10.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add command-line arguments to the script (using `$1`, `$2`, or `getopts`) to make the account name and region configurable at runtime*
> - *Explore `az cosmosdb mongodb user` commands for creating database users with specific roles*
> - *Add an existence check (`az cosmosdb show`) before creating to make the script fully idempotent*
> - *Investigate `az cosmosdb keys regenerate` for rotating access keys as a security practice*

10.2.7 Done! 🎉

Great job! You've automated the provisioning of Azure Cosmos DB using the Azure CLI. You can now create and tear down database infrastructure with a single script, laying the

foundation for infrastructure as code practices you will build on with Bicep templates.

EXERCISE 11

ARM and Bicep

11.1 3. Provisioning CosmosDB with Bicep

11.2 Provisioning CosmosDB with Bicep

11.2.1 Goal

Define and deploy Azure Cosmos DB infrastructure declaratively using a Bicep template. Create a complete template that provisions a Cosmos DB account with the MongoDB API, a database, and a collection — then deploy it using `az deployment group create` and verify the resources in the Azure Portal.

WHAT YOU'LL LEARN:

- How to write a Bicep template for Cosmos DB with the MongoDB API
- How to use the `Microsoft.DocumentDB/databaseAccounts` resource type
- How to configure serverless capability declaratively
- How to define child resources for databases and collections
- How to deploy Bicep templates and retrieve outputs
- Key advantages of Infrastructure as Code over imperative CLI scripts

11.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Active Azure subscription with resource creation permissions
- ✓ Azure CLI installed and authenticated (`az login`)
- ✓ Bicep CLI installed (bundled with Azure CLI 2.20+, verify with `az bicep version`)
- ✓ A text editor (VS Code with the Bicep extension recommended)
- ✓ Familiarity with Azure CLI basics

11.2.3 Exercise Steps

Overview

1. **Create the Bicep Template with Parameters**
2. **Define the Cosmos DB Account Resource**
3. **Define the Database and Collection**
4. **Add Outputs**

5. **Deploy the Bicep Template**
6. **Verify in the Azure Portal**

Step 1: *Create the Bicep Template with Parameters*

Create a new Bicep file and define the parameters that make the template configurable and reusable. Parameters allow the same template to be deployed across multiple environments — development, staging, and production — by changing only the parameter values.

1. **Create** a new file named `cosmosdb.bicep` in your working directory
2. **Add** the following parameter definitions:

> `cosmosdb.bicep`

```
@description('The name of the Cosmos DB account. Must be
globally unique.')
param accountName string

@description('The Azure region for the Cosmos DB
account.')
param location string = resourceGroup().location

@description('The name of the MongoDB database.')
param databaseName string = 'bookmarks_db'

@description('The name of the MongoDB collection.')
param collectionName string = 'bookmarks'

@description('The shard key for the collection.')
param shardKey string = 'category'
```

CONCEPT DEEP DIVE

Bicep parameters with default values allow flexible reuse — you only need to provide the account name (which must be globally unique), while other values have sensible defaults. The `@description` decorator provides documentation that appears in editor tooltips and ARM template exports. Using `resourceGroup().location` as the default location ensures the Cosmos DB account is created in the same region as its resource group, reducing latency and simplifying deployment.

Bicep is a domain-specific language (DSL) that compiles to ARM JSON templates. It provides a cleaner, more readable syntax while producing the same deployment artifacts. Every Bicep file compiles 1:1 to an equivalent ARM template, so there is no loss of capability.

⚠ **Common Mistakes**

- Forgetting that `accountName` has no default — it must be provided at deployment time
- Using a `.json` extension instead of `.bicep` — the Azure CLI expects Bicep files for Bicep deployments
- Parameter names are case-sensitive in Bicep — `accountName` and `AccountName` are different parameters

✓ **Quick check:** The file is saved with a `.bicep` extension, and VS Code shows no syntax errors if the Bicep extension is installed

Step 2: *Define the Cosmos DB Account Resource*

Add the Cosmos DB account resource definition to the template. This resource represents the top-level account that hosts all databases and collections. The resource type, API version, and property structure follow the Azure Resource

Manager schema for `Microsoft.DocumentDB/databaseAccounts`.

1. **Add** the following resource definition below the parameters in `cosmosdb.bicep`:

> `cosmosdb.bicep`

```
resource cosmosAccount 'Microsoft.DocumentDB/databaseAc-
counts@2024-05-15' = {
  name: accountName
  location: location
  kind: 'MongoDB'
  properties: {
    databaseAccountOfferType: 'Standard'
    capabilities: [
      {
        name: 'EnableServerless'
      }
      {
        name: 'EnableMongo'
      }
    ]
    locations: [
      {
        locationName: location
        failoverPriority: 0
        isZoneRedundant: false
      }
    ]
    apiProperties: {
      serverVersion: '4.2'
    }
  }
}
```

CONCEPT DEEP DIVE

The resource declaration follows the pattern: `resource <symbolic-name> '<type>@<api-version>' = { ... }`. The symbolic name (`cosmosAccount`) is used only within the Bicep file to reference this resource — it does not appear in Azure.

Key properties explained:

- `kind: 'MongoDB'` — Selects the MongoDB API for this account
- `databaseAccountOfferType: 'Standard'` — Required property with only one valid value
- `capabilities` — An array enabling `EnableServerless` (pay-per-use pricing) and `EnableMongo` (MongoDB wire protocol support)
- `locations` — Configures the primary region with `failoverPriority: 0` (single-region for serverless)
- `apiProperties.serverVersion` — Sets the MongoDB wire protocol version

Unlike CLI commands that execute immediately, Bicep describes the desired end state. Azure Resource Manager compares the template against the current state and makes only the necessary changes — this is declarative infrastructure management.

⚠ **Common Mistakes**

- Forgetting `EnableMongo` in the capabilities array alongside `EnableServerless` can cause deployment failures
- The `kind` value is case-sensitive — use `'MongoDB'`, not `'MongoDb'` or `'mongo'`
- Using an API version older than `2021-10-15` may not support serverless capabilities
- Omitting `databaseAccountOfferType` causes a validation error even though `'Standard'` is the only option

√ **Quick check:** No red squiggly lines in VS Code, and the resource block is properly nested within the file

Step 3: *Define the Database and Collection*

Add the database and collection as child resources of the Cosmos DB account. Bicep's `parent` property establishes the resource hierarchy, which is cleaner and more readable than the slash-separated naming convention used in raw ARM JSON templates.

1. **Add** the database resource below the account resource in `cosmosdb.bicep`:

   > `cosmosdb.bicep`

```
resource database 'Microsoft.DocumentDB/databaseAc-
counts/mongodbDatabases@2024-05-15' = {
  parent: cosmosAccount
  name: databaseName
  properties: {
    resource: {
      id: databaseName
    }
  }
}
```

2. **Add** the collection resource below the database resource:

   > `cosmosdb.bicep`

```
resource collection 'Microsoft.DocumentDB/databaseAc-
counts/mongodbDatabases/collections@2024-05-15' = {
  parent: database
  name: collectionName
  properties: {
    resource: {
      id: collectionName
      shardKey: {
        '${shardKey}': 'Hash'
      }
    }
  }
}
```

CONCEPT DEEP DIVE

The `parent` property in Bicep establishes the resource hierarchy: account → database → collection. This tells Azure Resource Manager about the dependency chain — the database cannot be created before the account, and the collection cannot be created before the database. Bicep handles this ordering automatically.

The `resource.id` property inside `properties` is required by the ARM API and must match the resource name. This duplication is an ARM API design pattern — the outer `name` is the resource identifier for Azure Resource Manager, while the inner `resource.id` is the identifier for the Cosmos DB service itself.

The `shardKey` object maps the partition key path to its partitioning strategy. The syntax `'${shardKey}': 'Hash'` uses Bicep string interpolation to set the parameter value as the object key, with `'Hash'` specifying hash-based partitioning (the standard strategy for most workloads).

⚠ **Common Mistakes**

- Forgetting the `parent` property and instead using slash-separated names results in implicit dependencies that can cause race conditions
- The `resource.id` must exactly match the `name` — mismatches cause confusing deployment errors
- Missing the `resource` wrapper inside `properties` causes validation failures — ARM requires this nested structure

✓ **Quick check:** The Bicep file now contains three resources: `cosmosAccount`, `database`, and `collection`, each referencing its parent

Step 4: *Add Outputs*

Add output declarations to make key values easily retrievable after deployment. Outputs are displayed in the terminal

when the deployment completes and can be programmatically consumed by scripts or CI/CD pipelines.

1. **Add** the following output declarations at the bottom of `cosmosdb.bicep`:

> `cosmosdb.bicep`

```
@description('The name of the deployed Cosmos DB
account.')
output accountNameOutput string = cosmosAccount.name

@description('The primary connection string for the
Cosmos DB account.')
output connectionString string = cosmosAc-
count.listConnection-
Strings().connectionStrings[0].connectionString
```

CONCEPT DEEP DIVE

The `listConnectionStrings()` function is a Bicep runtime function that calls the Azure Resource Manager API at deployment time to retrieve the connection string from the deployed account. This is different from a property reference — it makes an API call to get a value that is only available after the resource is created.

Outputs are stored in the deployment metadata and can be retrieved later using `az deployment group show`. In production environments, you would typically avoid outputting sensitive values like connection strings and instead store them directly in Azure Key Vault using a Bicep module. For learning purposes, the output makes it easy to verify the deployment.

⚠ **Common Mistakes**

- The `listConnectionStrings()` function is only available at deployment time — it cannot be used in parameter defaults or variables
- Output values containing secrets will be visible in deployment logs and the Azure Portal's deployment history
- Referencing `cosmosAccount.properties.connectionStrings` does not work — you must use the `listConnectionStrings()` function

√ **Quick check:** The complete file should now have five sections: parameters, account resource, database resource, collection resource, and outputs

Step 5: *Deploy the Bicep Template*

Deploy the template to Azure using the Azure CLI. The `az deployment group create` command sends your Bicep template to Azure Resource Manager, which compiles it to ARM

JSON and orchestrates the resource creation in the correct dependency order.

1. **Ensure** the resource group exists:

```
az group create --name CloudDatabasesRG --location
        northeurope
```

2. **Deploy** the Bicep template (replace `yourname` with your actual name):

```
az deployment group create \
  --resource-group CloudDatabasesRG \
  --template-file cosmosdb.bicep \
  --parameters accountName="cosmosdb-yourname-bcd"
```

3. **Wait** for the deployment to complete (typically 3–8 minutes)
4. **Retrieve** the deployment outputs after completion:

```
az deployment group show \
  --resource-group CloudDatabasesRG \
  --name cosmosdb \
  --query "properties.outputs" \
  --output json
```

5. **Extract** just the connection string for use in applications:

```
az deployment group show \
  --resource-group CloudDatabasesRG \
  --name cosmosdb \
  --query "properties.outputs.connectionString.value" \
  --output tsv
```

CONCEPT DEEP DIVE

Bicep deployments are idempotent — running the same template multiple times with the same parameters produces the same result without errors. Azure Resource Manager compares the template's desired state against the current state and makes only the necessary changes. If all resources already exist with the correct configuration, the deployment completes quickly with no modifications. This is a significant advantage over imperative CLI scripts, which would fail on the second run because the resources already exist.

The deployment name defaults to the template filename without the extension (`cosmosdb` from `cosmosdb.bicep`). You can override this with the `--name` parameter if you want a custom deployment name.

⚠ **Common Mistakes**

- Forgetting to create the resource group first results in a "ResourceGroupNotFound" error
- Providing an account name that already exists globally (even in other subscriptions) causes a conflict error
- Running the command from a directory that does not contain `cosmosdb.bicep` causes a "file not found" error
- The `--parameters` flag uses `key=value` syntax (no spaces around `=`)

✓ **Quick check:** The deployment completes with `"provisioningState": "Succeeded"` and the outputs section shows the account name and connection string

Step 6: *Verify in the Azure Portal*

Confirm that the Bicep deployment created all resources correctly by inspecting them in the Azure Portal. Verifying infrastructure as code deployments visually helps build con-

fidence in the automation and catches any configuration mismatches.

1. **Navigate to** the Azure Portal at https://portal.azure.com
2. **Open** the Cosmos DB account by searching for the account name or navigating through the `CloudDatabasesRG` resource group
3. **Confirm** the following on the account overview page:
   - API shows "MongoDB"
   - Capacity mode shows "Serverless"
   - Region matches your selected location
4. **Click** "Data Explorer" in the left menu
5. **Verify** that `bookmarks_db` database and `bookmarks` collection appear in the tree view
6. **Insert** a test document to verify the collection is functional:

```
{
    "title": "Bicep Documentation",
    "url": "https://learn.microsoft.com/en-us/azure/
        azure-resource-manager/bicep/",
    "category": "cloud",
    "tags": ["azure", "iac", "bicep"],
    "createdAt": "2026-02-22T10:00:00Z"
}
```

7. **Test idempotency** by re-running the deployment command from Step 5 — it should succeed without errors or changes

CONCEPT DEEP DIVE

Idempotency is a core property of declarative infrastructure management. When you re-run the same Bicep template, Azure Resource Manager detects that the resources already exist with the correct configuration and reports "no changes needed." This makes Bicep templates safe to run in CI/CD pipelines on every commit — they converge toward the desired state without causing errors or duplicating resources.

Compare this to the CLI script from the previous approach: running `az cosmosdb create` twice with the same account name would either fail or require additional existence checks. Bicep eliminates this entire category of problems by design.

√ **Quick check:** The Portal shows all resources matching the template configuration, the test document saves successfully, and re-running the deployment produces no errors

11.2.4 Common Issues

> ***If you encounter problems:***
> ***"InvalidTemplateDeployment" error:*** *Check the Bicep file for syntax errors. Run* `az bicep build --file cosmosdb.bicep` *to compile locally and see detailed error messages.*
> ***"AccountNameAlreadyExists":*** *Cosmos DB account names must be globally unique. Change the* `accountName` *parameter to include a random suffix.*
> ***"AuthorizationFailed":*** *Your Azure account may lack the required permissions. Ensure you have at least Contributor role on the resource group.*
> ***Deployment takes longer than 15 minutes:*** *Check the deployment status in the Azure Portal under your resource group → Deployments. Long deployments may indicate a region capacity issue — try a different region.*
> ***"BicepNotInstalled" or version errors:*** *Run* `az bicep install` *or* `az bicep upgrade` *to get the latest Bicep CLI version.*
> ***Still stuck?*** *Run* `az bicep build --file cosmosdb.bicep` *to validate the template locally before deploying. This catches most syntax and schema errors without waiting for a deployment round-trip.*

11.2.5 Summary

You've successfully defined and deployed Cosmos DB infrastructure using Bicep which:

- √ Declaratively defines the complete database infrastructure in a single file

- ✓ Provisions a Cosmos DB account, database, and collection with one command
- ✓ Outputs the connection string for application consumption
- ✓ Supports idempotent deployments — safe to run repeatedly

> **KEY TAKEAWAY:**
> Infrastructure as Code with Bicep transforms infrastructure from manual configuration into version-controlled, reviewable, and repeatable artifacts. The same template can provision identical environments for development, staging, and production. Bicep's declarative approach means you describe what you want, not how to create it — Azure Resource Manager handles the orchestration, dependency ordering, and idempotency automatically.

11.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add a Key Vault resource to the template and store the connection string as a secret instead of outputting it*
> - *Create a parameters file (`cosmosdb.parameters.json`) for different environments (dev, staging, prod)*
> - *Add diagnostic settings to send Cosmos DB metrics to a Log Analytics workspace*
> - *Extract the Cosmos DB resources into a Bicep module for reuse across projects*

11.2.7 Done! 🎉

Great job! You've defined your entire Cosmos DB infrastructure as code using Bicep. This template can be version-controlled in Git, reviewed in pull requests, and deployed automatically through CI/CD pipelines — bringing software engineering practices to infrastructure management.

EXERCISE 12

Connecting from Code

12.1 4. CosmosDB PoC with .NET Script

12.2 CosmosDB PoC with .NET Script

12.2.1 Goal

Build a proof-of-concept application that connects to Azure Cosmos DB using .NET 10 file-based scripting. Use the MongoDB driver to perform all CRUD operations (Create, Read, Update, Delete) against your cloud-hosted database, reading the connection string from an environment variable.

WHAT YOU'LL LEARN:

- How to use .NET 10 file-based app execution (`dotnet run file.cs`)
- How to add NuGet packages with the `#:package` directive
- How to connect to Cosmos DB using the MongoDB.Driver NuGet package
- How to perform CRUD operations against a MongoDB-compatible database
- How to manage connection strings securely via environment variables
- How to verify application data in the Azure Portal Data Explorer

12.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ .NET 10 SDK installed (verify with `dotnet --version`)
- ✓ A provisioned Cosmos DB account with MongoDB API (from a previous exercise in this section)
- ✓ The Cosmos DB connection string available
- ✓ A terminal and a text editor

12.2.3 Exercise Steps

Overview

1. **Set Up the Connection String**
2. **Create the .NET Script File**
3. **Implement Insert Operations**
4. **Implement Query and List Operations**

5. **Implement Update and Delete Operations**
6. **Test the Complete CRUD Workflow**

Step 1: *Set Up the Connection String*

Configure the Cosmos DB connection string as an environment variable. Storing connection strings in environment variables keeps secrets out of source code and follows the twelve-factor app methodology for configuration management.

1. **Retrieve** your Cosmos DB connection string from the Azure Portal (Settings → Connection strings) or from your CLI output
2. **Set** the environment variable in your terminal:

```
export COSMOS_CONNECTION_STRING="mongodb://your-
        account:your-key@your-
        account.mongo.cosmos.azure.com:10255/?
        ssl=true&retry-
        writes=false&maxIdleTimeMS=120000"
```

3. **Verify** the variable is set:

```
echo "${COSMOS_CONNECTION_STRING:0:30}..."
```

CONCEPT DEEP DIVE

Environment variables are the standard way to inject configuration into applications without hardcoding values. This approach has several benefits: the same code works against different databases (development, staging, production) by changing only the environment variable, secrets never appear in source code or version control, and the operating system provides built-in isolation between processes.

The connection string format `mongodb://...` is the standard MongoDB connection URI. Because Cosmos DB supports the MongoDB wire protocol, the standard MongoDB.Driver NuGet package connects to it without any Cosmos-DB-specific code.

⚠ **Common Mistakes**

- Forgetting to wrap the connection string in quotes when it contains special characters like `&` or `=`
- The environment variable only persists in the current terminal session — opening a new terminal requires setting it again
- Accidentally committing a `.env` file or a hardcoded connection string to Git exposes your database credentials

✓ **Quick check:** The echo command shows the first 30 characters of your connection string starting with `mongodb://`

Step 2: *Create the .NET Script File*

Create a .NET 10 file-based application that connects to Cosmos DB using the MongoDB driver. .NET 10 supports running single `.cs` files directly with `dotnet run`, which is ideal for proof-of-concept work — no project file or solution structure needed.

1. **Create** a new file named `explore-cosmos.cs`

2. **Add** the package directive and initial connection setup:

> `explore-cosmos.cs`

```
#:package MongoDB.Driver@3.*

using MongoDB.Driver;
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

// Read the connection string from environment variable
var connectionString = Environment.GetEnvironmentVari-
        able("COSMOS_CONNECTION_STRING")
    ?? throw new InvalidOperationException(
        "COSMOS_CONNECTION_STRING environment variable
        is not set. " +
        "Set it with: export COSMOS_CONNEC-
        TION_STRING=\"your-connection-string\"");

// Connect to Cosmos DB
var client = new MongoClient(connectionString);
var database = client.GetDatabase("bookmarks_db");
var collection = database.GetCollection<Bookmark>("book-
        marks");

Console.WriteLine("Connected to Cosmos DB success-
        fully!");
Console.WriteLine($"Database: bookmarks_db");
Console.WriteLine($"Collection: bookmarks");
Console.WriteLine();
```

3. **Add** the `Bookmark` record class at the bottom of the file:

> `explore-cosmos.cs`

```
// Document model
record Bookmark
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; init; }

    [BsonElement("title")]
    public required string Title { get; init; }

    [BsonElement("url")]
    public required string Url { get; init; }

    [BsonElement("category")]
    public required string Category { get; init; }

    [BsonElement("tags")]
    public string[] Tags { get; init; } = [];

    [BsonElement("createdAt")]
    public DateTime CreatedAt { get; init; } = Date-
        Time.UtcNow;
}
```

CONCEPT DEEP DIVE

The `#:package MongoDB.Driver@3.*` directive is a .NET 10 feature that allows single-file applications to declare NuGet package dependencies inline. The `@3.*` version range syntax specifies any version in the 3.x range. When you run the file with `dotnet run explore-cosmos.cs`, the runtime automatically restores the package before execution.

The `Bookmark` record uses MongoDB.Driver attributes to control serialization: `[BsonId]` marks the primary key field, `[BsonRepresentation(BsonType.ObjectId)]` tells the driver to store it as a MongoDB ObjectId, and `[BsonElement("title")]` maps the C# property to the JSON field name. Using `record` instead of `class` provides value-based equality and immutability by default.

⚠ **Common Mistakes**

- Forgetting the `#:package` directive causes a compilation error — the MongoDB.Driver namespace will not be found
- Using `#r` instead of `#:package` — the `#r` directive is for .NET Interactive notebooks, not file-based apps
- Hardcoding the connection string instead of reading from an environment variable makes the code insecure and non-portable
- The `required` keyword on properties ensures they must be set during object initialization

√ **Quick check:** Run `dotnet run explore-cosmos.cs` — it should print "Connected to Cosmos DB successfully!" without errors

Step 3: *Implement Insert Operations*

Add code to insert bookmark documents into the collection. This demonstrates the Create operation in CRUD, using both single-document and batch insert methods.

1. **Add** the following insert code after the connection setup (before the `Bookmark` record definition):

> `explore-cosmos.cs`

```
// === CREATE: Insert bookmarks ===
Console.WriteLine("--- Inserting Bookmarks ---");

var bookmarks = new List<Bookmark>
{
    new()
    {
        Title = "Microsoft Learn",
        Url = "https://learn.microsoft.com",
        Category = "learning",
        Tags = ["microsoft", "documentation", "tutori-
         als"]
    },
    new()
    {
        Title = "GitHub",
        Url = "https://github.com",
        Category = "development",
        Tags = ["git", "code", "collaboration"]
    },
    new()
    {
        Title = "Azure Portal",
        Url = "https://portal.azure.com",
        Category = "cloud",
        Tags = ["azure", "management", "portal"]
    },
    new()
    {
        Title = "Stack Overflow",
        Url = "https://stackoverflow.com",
        Category = "development",
        Tags = ["questions", "community", "programming"]
    }
};

await collection.InsertManyAsync(bookmarks);
Console.WriteLine($"Inserted {bookmarks.Count} book-
         marks.");
Console.WriteLine();
```

CONCEPT DEEP DIVE

The `InsertManyAsync` method sends all documents to the database in a single network round-trip, which is more efficient than inserting them one at a time with `InsertOneAsync`. The MongoDB driver automatically generates an `_id` (ObjectId) for each document if one is not provided. After insertion, the `Id` property on each `Bookmark` object is populated with the generated value.

Because the `Bookmark` record uses `[BsonElement]` attributes, the C# property names (PascalCase) are mapped to lowercase JSON field names in the database. This follows MongoDB naming conventions while keeping C# code idiomatic.

⚠ **Common Mistakes**

- Inserting documents with duplicate `_id` values causes a `MongoBulkWriteException` — each document must have a unique identifier
- Running the script multiple times inserts duplicate data because `InsertManyAsync` always creates new documents
- Forgetting `await` on async methods causes the program to exit before the operation completes

✓ **Quick check:** The console output shows "Inserted 4 bookmarks." and no exceptions are thrown

Step 4: *Implement Query and List Operations*

Add code to retrieve and filter documents from the collection. This demonstrates the Read operations in CRUD, including listing all documents, filtering by field values, and querying by array contents.

1. **Add** the following query code after the insert section:

> `explore-cosmos.cs`

```
// === READ: Query bookmarks ===
Console.WriteLine("--- All Bookmarks ---");

var allBookmarks = await collection.Find(_ =>
        true).ToListAsync();
foreach (var b in allBookmarks)
{
    Console.WriteLine($"  [{b.Category}] {b.Title} -
        {b.Url}");
}
Console.WriteLine($"Total: {allBookmarks.Count} book-
        marks");
Console.WriteLine();

// Filter by category
Console.WriteLine("--- Development Bookmarks ---");

var devFilter = Builders<Bookmark>.Filter.Eq(b => b.Cat-
        egory, "development");
var devBookmarks = await collec-
        tion.Find(devFilter).ToListAsync();
foreach (var b in devBookmarks)
{
    Console.WriteLine($"  {b.Title} ({string.Join(", ",
        b.Tags)})");
}
Console.WriteLine($"Found: {devBookmarks.Count} develop-
        ment bookmarks");
Console.WriteLine();

// Filter by tag
Console.WriteLine("--- Bookmarks tagged 'azure' ---");

var tagFilter = Builders<Bookmark>.Filter.AnyEq(b =>
        b.Tags, "azure");
var azureBookmarks = await collec-
        tion.Find(tagFilter).ToListAsync();
foreach (var b in azureBookmarks)
{
    Console.WriteLine($"  {b.Title} - {b.Url}");
}
Console.WriteLine();
```

CONCEPT DEEP DIVE

The MongoDB.Driver provides a strongly-typed `Builders<T>.Filter` API for constructing queries. `Filter.Eq` creates an equality filter, while `Filter.AnyEq` matches documents where any element in an array field equals the specified value. These filter builders generate MongoDB query documents (like `{ "category": "development" }`) but with compile-time type safety — renaming a property in the `Bookmark` class will cause a build error in the filter, catching bugs early.

The `Find(_ => true)` expression is a shorthand for "match all documents" — the `_` parameter is a discarded lambda that always returns true. For production code with large collections, you would add pagination using `.Skip()` and `.Limit()` to avoid loading all documents into memory.

Queries that filter on the shard key (`category`) are efficient single-partition queries in Cosmos DB. The `tagFilter` query on `tags` requires scanning across partitions because `tags` is not the shard key — this consumes more Request Units at scale.

⚠ **Common Mistakes**

- Using string-based field names (e.g., `Filter.Eq("category", "development")`) bypasses type safety and is prone to typos
- Forgetting `await` on `ToListAsync()` returns a `Task` object instead of the actual results
- Loading all documents with `Find(_ => true)` on a large collection can consume excessive memory and Request Units

√ **Quick check:** The output lists all 4 bookmarks, then filters to show only development bookmarks (2) and azure-tagged bookmarks

Step 5: *Implement Update and Delete Operations*

Add code to update existing documents and delete documents from the collection. This completes the CRUD operations, demonstrating both targeted single-document updates and filtered deletions.

1. **Add** the following update and delete code after the query section:

> `explore-cosmos.cs`

```
// === UPDATE: Modify a bookmark ===
Console.WriteLine("--- Updating a Bookmark ---");

var updateFilter = Builders<Bookmark>.Filter.Eq(b =>
        b.Title, "GitHub");
var update = Builders<Bookmark>.Update
    .Set(b => b.Title, "GitHub - Where the world builds
        software")
    .Set(b => b.Tags, new[] { "git", "code", "collabora-
        tion", "open-source" });

var updateResult = await collec-
        tion.UpdateOneAsync(updateFilter, update);
Console.WriteLine($"Matched: {upda-
        teResult.MatchedCount}, Modified: {upda-
        teResult.ModifiedCount}");

// Verify the update
var updatedBookmark = await collec-
        tion.Find(updateFilter).FirstOrDefaultAsync();
if (updatedBookmark == null)
{
    // The title changed, so search by URL instead
    var urlFilter = Builders<Bookmark>.Filter.Eq(b =>
        b.Url, "https://github.com");
    updatedBookmark = await collec-
        tion.Find(urlFilter).FirstOrDefaultAsync();
}
Console.WriteLine($"Updated title: {updatedBook-
        mark?.Title}");
Console.WriteLine($"Updated tags: {string.Join(", ",
        updatedBookmark?.Tags ?? [])}");
Console.WriteLine();

// === DELETE: Remove a bookmark ===
Console.WriteLine("--- Deleting a Bookmark ---");

var deleteFilter = Builders<Bookmark>.Filter.Eq(b =>
        b.Title, "Stack Overflow");
var deleteResult = await collec-
        tion.DeleteOneAsync(deleteFilter);
Console.WriteLine($"Deleted: {deleteResult.DeletedCount}
        bookmark(s)");
Console.WriteLine();

// Final count
```

```
var finalCount = await collection.CountDocumentsAsync(_
        => true);
Console.WriteLine($"--- Final bookmark count: {final-
        Count} ---");
```

CONCEPT DEEP DIVE

The `Builders<T>.Update` API provides a fluent interface for constructing update operations. The `.Set()` method updates specific fields without replacing the entire document — this is an atomic, partial update that is both efficient and safe for concurrent access. `UpdateOneAsync` modifies only the first matching document, while `UpdateManyAsync` would modify all matches.

`DeleteOneAsync` removes a single document matching the filter. The `DeletedCount` property confirms how many documents were actually removed — this is useful for verifying that the filter matched the intended document. If no document matches, `DeletedCount` is 0 and no error is thrown.

Both update and delete operations consume Request Units in Cosmos DB. Operations that include the shard key in the filter are more efficient because they target a single partition.

⚠ **Common Mistakes**

- Using `ReplaceOneAsync` instead of `UpdateOneAsync` replaces the entire document, which can unintentionally remove fields
- After updating the `Title` field, any subsequent filter based on the old title value will not find the document
- `DeleteOneAsync` with a filter that matches multiple documents only deletes the first match — use `DeleteManyAsync` to remove all matches

✓ **Quick check:** The update shows "Matched: 1, Modified: 1", the delete shows "Deleted: 1", and the final count is 3

Step 6: *Test the Complete CRUD Workflow*

Run the complete script, verify the output in the terminal, and cross-reference the results in the Azure Portal Data Explorer. This end-to-end verification confirms that your code interacts correctly with the cloud database.

1. **Run** the script:

```
dotnet run explore-cosmos.cs
```

2. **Verify** the console output shows:
   - Successful connection message
   - 4 bookmarks inserted
   - All bookmarks listed with categories
   - Filtered results for "development" and "azure" tags
   - Update confirmation (Matched: 1, Modified: 1)
   - Delete confirmation (Deleted: 1)
   - Final count of 3
3. **Open** the Azure Portal at https://portal.azure.com
4. **Navigate to** your Cosmos DB account → Data Explorer
5. **Expand** `bookmarks_db` → `bookmarks` → Documents
6. **Verify** that:
   - Three documents remain in the collection
   - The GitHub bookmark has the updated title and tags
   - The Stack Overflow bookmark has been removed
7. **Clean up** the test data (optional) — add this at the end of your script or run it separately:

```
// Optional: Clean up all inserted documents
// await collection.DeleteManyAsync(_ => true);
// Console.WriteLine("All bookmarks deleted.");
```

> √ ***Success indicators:***
>
> - *Script runs without exceptions from start to finish*
> - *All CRUD operations produce the expected output counts*
> - *Data Explorer in the Azure Portal reflects the final state (3 documents)*
> - *The updated GitHub bookmark shows the new title and tags array*
> - *The Stack Overflow bookmark no longer exists in the collection*
>
> √ ***Final verification checklist:***
>
> ☐ *Connection string read from environment variable (not hardcoded)*
>
> ☐ *All four CRUD operations execute successfully*
>
> ☐ *Console output matches expected results*
>
> ☐ *Azure Portal Data Explorer confirms the data state*
>
> ☐ *The* `Bookmark` *record maps correctly to MongoDB documents*

12.2.4 Common Issues

> ***If you encounter problems:***
> ***"COSMOS_CONNECTION_STRING environment variable is not set":*** *Set the environment variable with* `export COSMOS_CONNECTION_STRING="your-connection-string"` *in the same terminal where you run the script.*
> ***"Authentication failed":*** *Verify the connection string is correct and complete. Copy it again from the Azure Portal (Settings → Connection strings). Ensure the string is wrapped in quotes when setting the environment variable.*
> ***"MongoCommandException: Command insert failed":*** *This often indicates a shard key issue. Ensure the* `category` *field is present in all inserted documents, as it matches the collection's partition key.*
> ***"The type or namespace name 'MongoDB' could not be found":*** *Ensure the* `#:package MongoDB.Driver@3.*` *directive is the very first line of the file (before any* `using` *statements).*
> ***"dotnet run: command not found" or version error:*** *Verify .NET 10 is installed with* `dotnet --version`*. File-based execution requires .NET 10 or later.*
> ***Still stuck?*** *Try connecting with a minimal script that only creates the* `MongoClient` *and lists database names to isolate whether the issue is connectivity or code logic.*

12.2.5 Summary

You've successfully built a proof-of-concept that connects to Cosmos DB from .NET code which:

- √ Uses .NET 10 file-based execution for rapid prototyping
- √ Connects to Cosmos DB using the standard MongoDB.Driver package
- √ Performs all CRUD operations with strongly-typed models
- √ Manages connection strings securely via environment variables

> **KEY TAKEAWAY:**
>
> Because Cosmos DB supports the MongoDB wire protocol, you can use the standard MongoDB.Driver NuGet package — the same driver you would use with a native MongoDB instance. The only difference is the connection string. This means migrating between MongoDB and Cosmos DB is primarily a configuration change, not a code change. You have validated this concept hands-on with a working proof-of-concept.

12.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Add a* `FindOneAndUpdateAsync` *operation to atomically update and return a document in a single round-trip*
> - *Implement pagination using* `.Skip()` *and* `.Limit()` *to handle large collections efficiently*
> - *Add an index on the* `tags` *field using* `collection.Indexes.CreateOneAsync()` *and observe the query performance difference*
> - *Explore the MongoDB.Driver LINQ provider to write queries using C# LINQ syntax instead of filter builders*

12.2.7 Done! 🎉

Great job! You've connected to Azure Cosmos DB from application code and performed all CRUD operations using the MongoDB driver. This proof-of-concept validates that existing MongoDB driver code works seamlessly with Cosmos DB — a key insight that you will apply when integrating Cosmos DB into a full web application.

EXERCISE 13

Azure Blob Storage

13.1 5. Provisioning Azure Blob Storage

13.2 Provisioning Azure Blob Storage

13.2.1 Goal

Create an Azure Storage Account, configure a Blob container for public image hosting, upload a hero image, and verify that the blob is publicly accessible.

WHAT YOU'LL LEARN:

- How to create an Azure Storage Account through the Azure Portal
- How to enable anonymous access at the account level
- How to create a Blob container with anonymous read access for public assets
- How to upload files to Azure Blob Storage
- How to verify blob access from a browser
- Key concepts of Azure Blob Storage: accounts, containers, and blobs

13.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Active Azure subscription with resource creation permissions
- ✓ Familiarity with the Azure Portal
- ✓ A hero image file (any `.jpg` or `.png` image, ideally 1920×1080 or similar for a hero section)

13.2.3 Exercise Steps

Overview

1. **Create a Storage Account**
2. **Create a Blob Container**
3. **Upload a Hero Image**
4. **Verify Public Access**

Step 1: *Create a Storage Account*

Set up an Azure Storage Account to host your application's static assets. A Storage Account is the top-level resource for all Azure Storage services — it provides a unique namespace

in Azure for your blobs, files, queues, and tables. For this exercise, you will use the Blob service to host images that your web application serves to users.

1. **Navigate to** the Azure Portal at https://portal.azure.com
2. **Search for** "Storage accounts" using the search bar at the top
3. **Select** Storage accounts from the search results
4. **Click** the **+ Create** button
5. **Configure** the Basics tab with the following settings:
   - **Subscription**: Select your subscription
   - **Resource Group**: Use your existing resource group (e.g., `CloudDatabasesRG`) or create a new one
   - **Storage account name**: Enter a globally unique name (e.g., `cloudsoftyourname`, lowercase letters and numbers only, 3–24 characters)
   - **Region**: Select `North Europe` (or a region close to you)
   - **Performance**: Select `Standard`
   - **Redundancy**: Select `Locally-redundant storage (LRS)`
6. **Click** the **Advanced** tab and configure the following:
   - Under **Security**, **check** "Allow enabling anonymous access on individual containers"
   - Leave all other Advanced settings at their defaults

> *This setting is required for Step 2, where you will configure public read access on the Blob container. If this is not enabled at the account level, the anonymous access dropdown will be greyed out when creating the container.*

7. **Click** Review + Create, then **click** Create

8. **Wait** for the deployment to complete (this typically takes 30–60 seconds)

CONCEPT DEEP DIVE

Azure Storage Accounts provide a unified namespace for multiple storage services. The Blob service (Binary Large Object) is optimized for storing unstructured data such as images, videos, documents, and backups. Each storage account supports up to 5 PiB of data and is accessible via HTTP/HTTPS from anywhere in the world.

Performance tiers: Standard uses magnetic drives (HDD) and is cost-effective for most workloads. Premium uses solid-state drives (SSD) and is designed for low-latency scenarios like disk storage for virtual machines. For serving static images, Standard is the appropriate choice.

Redundancy options: LRS (Locally-redundant storage) maintains three copies of your data within a single data center. This is the most affordable option and provides sufficient durability for development and non-critical production workloads. For business-critical applications, you would choose GRS (Geo-redundant storage) which replicates data to a secondary region.

⚠ **Common Mistakes**

- The storage account name must be globally unique across all of Azure, contain only lowercase letters and numbers, and be 3–24 characters long
- Choosing Premium performance for image hosting adds unnecessary cost — Standard is sufficient for serving static content
- Selecting a region far from your users increases latency for image loading

✓ **Quick check:** Navigate to the Storage Account overview page and verify it shows the correct region and “Standard” performance tier

Step 2: *Create a Blob Container*

Create a container within your Storage Account to organize and store your image files. A container is similar to a directory — it groups related blobs together and provides access control at the container level. For serving images publicly through your web application, the container needs anonymous read access.

1. **Navigate to** your newly created Storage Account in the Azure Portal
2. **Click** "Containers" in the left menu under the "Data storage" section
3. **Click** the **+ Container** button
4. **Enter** the container name: `images`
5. **Set** the "Anonymous access level" to **Blob (anonymous read access for blobs only)**
6. **Click** Create

CONCEPT DEEP DIVE

Azure Blob Storage has a three-level hierarchy: Storage Account → Container → Blob. Containers serve as the organizational unit — they are like top-level folders that hold blobs (files). Each container has its own access policy that controls whether unauthenticated users can read the blobs inside it.

Anonymous access levels:

- **Private (no anonymous access)** — Default. All requests must be authenticated with an account key, SAS token, or Azure AD. Use this for sensitive data.
- **Blob (anonymous read access for blobs only)** — Anyone with the URL can read individual blobs, but cannot list the blobs in the container. This is ideal for serving public assets like images — users can access `hero.png` directly but cannot browse the container to discover other files.
- **Container (anonymous read access for containers and blobs)** — Anyone can read blobs and list all blobs in the container. Avoid this unless you intentionally want the container contents to be discoverable.

If the "Anonymous access level" dropdown is greyed out or missing, it means that anonymous access is disabled at the storage account level. To enable it: go to Settings → Configuration in your Storage Account, find "Allow Blob anonymous access", set it to **Enabled**, and save. Then return to create the container.

⚠ **Common Mistakes**

- Leaving the access level as "Private" means your application cannot serve images without authentication tokens — the browser will get a 403 Forbidden error
- Setting the access level to "Container" instead of "Blob" allows anyone to list all files in the container, which may expose assets you did not intend to make discoverable

- Container names must be lowercase, 3–63 characters, and can only contain letters, numbers, and hyphens

√ **Quick check:** The `images` container appears in the container list with "Blob" shown as the access level

Step 3: *Upload a Hero Image*

Upload your hero image to the Blob container. Once uploaded, the image will be accessible via a public URL that follows a predictable pattern based on your storage account name, container name, and blob name.

1. **Click** on the `images` container to open it
2. **Click** the **Upload** button in the toolbar
3. **Click** "Browse for files" and **select** your hero image file
4. **Rename** the file to `hero.png` in the upload dialog (or ensure your file is already named `hero.png`)
5. **Click** Upload
6. **Verify** the upload by checking that `hero.png` appears in the container's blob list
7. **Click** on `hero.png` to open its properties
8. **Copy** the blob URL — it will follow this pattern:

```
https://{accountname}.blob.core.windows.net/images/
hero.png
```

CONCEPT DEEP DIVE

Azure Blob Storage URLs follow a deterministic pattern: `https://{account}.blob.core.windows.net/{container}/{blob}`. This predictable URL structure means your application can construct image URLs by simply knowing the storage account name, container name, and file name — no API calls needed to resolve URLs at runtime.

When you upload a blob, Azure stores it with metadata including the content type (MIME type). The Portal typically auto-detects the content type based on the file extension (e.g., `image/png` for `.png`). Correct content types are important because browsers use them to determine how to handle the response — an incorrect content type could cause the browser to download the image instead of displaying it.

Blob names are case-sensitive in Azure Storage. `hero.png` and `Hero.jpg` are treated as two different blobs. Establish a naming convention (lowercase is recommended) and use it consistently across your application code and storage.

⚠ **Common Mistakes**

- Uploading with a different filename than what the application expects — the CloudSoft application expects `hero.png`
- Very large images (10+ MB) will slow page loading. For a hero section, optimize the image to 500 KB–2 MB for a good balance between quality and performance
- Forgetting to note the full blob URL — you will need the container base URL (without the filename) for the application configuration

✓ **Quick check:** The blob URL displayed in the properties panel matches the pattern `https://{account-name}.blob.core.windows.net/images/hero.png`

Step 4: *Verify Public Access*

Confirm that the uploaded image is publicly accessible by opening the blob URL directly in a browser. This verification step ensures that the container access level is configured correctly before integrating with your application.

1. **Copy** the full blob URL from the previous step (e.g., `https://cloudsoftyourname.blob.core.windows.net/images/hero.png`)
2. **Open** a new browser tab (or an incognito/private window for a clean test)
3. **Paste** the URL into the browser address bar and **press** Enter
4. **Verify** that the image loads and displays correctly in the browser

CONCEPT DEEP DIVE

Testing in an incognito or private browser window is a best practice when verifying public access. Your regular browser session may have cached Azure Portal authentication tokens that could mask access issues — an incognito window has no stored credentials, simulating how an unauthenticated user (or your application's end users) will access the image.

If the image loads successfully, it confirms two things: the container's anonymous access level is set to "Blob" (or "Container"), and the blob was uploaded with the correct content type. If you see an XML error response instead of the image, the access configuration needs to be corrected.

⚠ **Common Mistakes**

- Getting a `ResourceNotFound` error means the URL is incorrect — double-check the storage account name, container name, and blob name (all case-sensitive)
- Getting a `PublicAccessNotPermitted` error means anonymous access is disabled at the storage account level — enable it in Settings → Configuration → "Allow Blob anonymous access"
- Getting an `AuthorizationPermissionMismatch` error means the container access level is set to "Private" — change it to "Blob" in the container's access level settings

√ **Quick check:** The hero image displays correctly in an incognito browser window without any authentication

13.2.4 Common Issues

> ***If you encounter problems:***
> ***Image not loading (403 Forbidden):*** *The container access level is set to "Private". Navigate to the container in the Azure Portal, click "Change access level", and select "Blob (anonymous read access for blobs only)".*
> ***Image not loading (404 Not Found):*** *The blob name in the URL does not match the uploaded file name. Blob names are case-sensitive — verify that the file was uploaded as* `hero.png` *(lowercase).*
> ***"Allow Blob anonymous access" is disabled:*** *Navigate to your Storage Account → Settings → Configuration. Set "Allow Blob anonymous access" to* ***Enabled*** *and save. Then update the container access level.*
> ***Still stuck?*** *Test the blob URL directly in an incognito browser window first. If the image does not load in the browser, the issue is in the Azure Storage configuration.*

13.2.5 Summary

You've successfully provisioned Azure Blob Storage infrastructure for serving public images which:

- √ Provides a globally accessible, high-availability storage service for static assets
- √ Uses anonymous read access for public image serving without authentication overhead
- √ Follows the deterministic URL pattern `https://{account}.blob.core.windows.net/{container}/{blob}`

> **KEY TAKEAWAY:**
>
> Azure Blob Storage provides a scalable, cost-effective solution for serving static assets from the cloud. The deterministic URL pattern means your application can construct image URLs from configuration alone — no SDK or API calls needed at runtime. Your container URL (`https://{accountname}.blob.core.windows.net/images`) is what you will use when configuring your application to serve images from Azure Blob Storage.

13.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Create the same Storage Account and container using the Azure CLI (`az storage account create` and `az storage container create`) for scriptable provisioning*
> - *Configure a custom domain for your Blob Storage endpoint so images are served from your own domain (e.g., `images.cloudsoft.com`)*
> - *Enable Azure CDN (Content Delivery Network) in front of Blob Storage to cache images at edge locations worldwide for faster loading*
> - *Explore SAS tokens (Shared Access Signatures) to provide time-limited access to private blobs without making the container public*

13.2.7 Done! 🎉

Great job! You've provisioned Azure Blob Storage, created a public container, and verified that your hero image is accessible via a public URL. This infrastructure is ready for your

application to use — you will configure the application to point to your container URL in the next exercise.

PART VI

Storage and Resilience

EXERCISE 14

File Uploads and Deep Health Probes

14.1 MVC Uploads, PDF Validation, and First Deploy

14.2 MVC Uploads, PDF Validation, and First Deploy

14.2.1 Goal

Stand up an anonymous recruitment portal called `CloudCi-Careers.Web` and ship it to Azure Container Apps. The portal lists a handful of hard-coded job postings, lets applicants attach a PDF CV, validates each upload with a four-byte signature check, and exposes a recruiter view that lists every

application with status updates, notes, deletion, and CV download. Storage in this exercise is intentionally fragile — an in-memory store and a folder on disk inside the container — because the whole point of the next exercise is to swap that out for managed state. You arrive with the previous chapter's REST and DTO discipline, the deployment chapter's OIDC-federated pipeline, and the logging chapter's `secretref:` pattern for Application Insights — none of those are re-taught here.

WHAT YOU'LL LEARN:

- How to wire an MVC recruitment flow with a multipart upload form end to end
- Why a four-byte magic-bytes check is the load-bearing piece of file-type validation, and what it deliberately doesn't tell you
- How to stream `IFormFile` content directly to a blob abstraction without buffering it again
- How antiforgery interacts with `enctype="multipart/form-data"`
- How to apply the OIDC-federated pipeline pattern to a brand-new repo, with a smoke target that actually exists today
- The difference between an in-memory store that survives one revision and managed state that survives the next one — and why this exercise stops short of fixing it

14.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ .NET 10 SDK installed (`dotnet --version` reports a `10.*` version)
- ✓ Docker Desktop running, and `docker --version` succeeds
- ✓ The `az` CLI signed in (`az login`) and the `gh` CLI signed in (`gh auth status`)
- ✓ Completed the deployment chapter — you've previously built a GitHub Actions pipeline that uses OIDC federation to push to ACR and update a Container App
- ✓ Completed the REST API and DTOs chapter — you're fluent with Container Apps, ACR, federated subjects, and the `secretref:` pattern for App Insights
- ✓ Permission to create Entra app registrations in your tenant and Owner (or User Access Administrator) on the subscription
- ✓ A real PDF file on disk (any small one will do) — you'll upload it during local testing

14.2.3 Exercise Steps

Overview

1. **Scaffold the `CloudCiCareers.Web` MVC project**
2. **Define the `Job`, `Application`, and `ApplyForm` types**
3. **Seed the job catalog**
4. **Add the `IApplicationStore` and the in-memory implementation**
5. **Add the `IBlobService` abstraction and a local-disk implementation**
6. **Add the PDF magic-bytes validator**
7. **Wire dependency injection in `Program.cs`**

8. **Build the `JobsController` and the apply form**
9. **Build the `ApplicationsController` and the recruiter views**
10. **Run locally and exercise the full applicant-then-recruiter flow**
11. **Containerize, set up the OIDC-federated pipeline, and deploy**
12. **Add Application Insights via the `secretref:` pattern**
13. **Test Your Implementation**

Step 1: *Scaffold the* `CloudCiCareers.Web` *MVC project*

Start from the standard MVC template. The default `dotnet new mvc` scaffold gives you Razor views, controllers, antiforgery wired in by default, and the static-files pipeline you'll need for CSS — exactly the right shape for a portal that's going to render HTML.

1. **Create** the project and verify it builds:

```
dotnet new mvc -o CloudCiCareers.Web --framework net10.0
cd CloudCiCareers.Web
dotnet build
dotnet run --launch-profile http
```

   The terminal should report `Now listening on: http://localhost:<port>`. Stop the server with `Ctrl+C` once you've seen it start.

2. **Delete** the demo `Privacy` action and view, plus the `Models/ErrorViewModel.cs` reference if you don't plan to use it. You'll replace `Index` with a job listing in a later step, so leave the controller file in place but plan to gut its body.

```
rm Views/Home/Privacy.cshtml
```

In `Controllers/HomeController.cs`, remove the `Privacy()` action method. Leave the `Index()` action — you'll repoint it at the jobs page in Step 8.

3. **Add** an `uploads/` directory to `.gitignore`. The local-file blob service writes here during development, and the contents are throwaway:

> `.gitignore`

```
uploads/
```

COMMON MISTAKES

- Forgetting `--framework net10.0` and ending up on whatever the default is on your box. The Dockerfile pulls `mcr.microsoft.com/dotnet/sdk:10.0`; if your `.csproj` says `net8.0`, the publish step fails.

√ **Quick check:** `CloudCiCareers.Web.csproj` exists, `dotnet build` returns no errors, and `dotnet run` serves the default home page.

Step 2: *Define the* `Job`*,* `Application`*, and* `ApplyForm` *types*

Three small types capture the entire data model. `Job` is the read-only domain entity for a posting (you never create or edit jobs at runtime — they're hard-coded). `Application` is the entity an applicant creates by submitting the form, with status that the recruiter then mutates. `ApplyForm` is the wire-bound form type — the shape of the multipart request body. Same entity-vs-wire discipline as the previous chapter, just with form binding instead of JSON.

1. **Create** the `Models/Job.cs` file:

> `Models/Job.cs`

```
namespace CloudCiCareers.Web.Models;

public record Job(
    int Id,
    string Title,
    string Department,
    string Description,
    DateTimeOffset Posted);
```

2. **Create** `Models/Application.cs` with the entity and the status enum side by side:

> `Models/Application.cs`

```
namespace CloudCiCareers.Web.Models;

public class Application
{
    public string Id { get; set; } = string.Empty;
    public int JobId { get; set; }
    public string ApplicantName { get; set; } =
        string.Empty;
    public string ApplicantEmail { get; set; } =
        string.Empty;
    public string CvBlobName { get; set; } =
        string.Empty;
    public DateTimeOffset SubmittedAt { get; set; }
    public ApplicationStatus Status { get; set; }
    public string? Notes { get; set; }
}

public enum ApplicationStatus
{
    Submitted,
    UnderReview,
    Rejected,
    Hired,
}
```

3. **Create** `Models/ApplyForm.cs` — the form-binding type with validation attributes:

> `Models/ApplyForm.cs`

```
using System.ComponentModel.DataAnnotations;

namespace CloudCiCareers.Web.Models;

public class ApplyForm
{
    [Required]
    [StringLength(100, MinimumLength = 1)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    [StringLength(200)]
    public string Email { get; set; } = string.Empty;
}
```

> **CONCEPT DEEP DIVE: FORM-BINDING TYPES ARE WIRE TYPES**
>
> `ApplyForm` is to multipart forms what a DTO is to JSON: the exact shape the request binder produces. The entity carries `Id`, `SubmittedAt`, `Status`, and `CvBlobName` — none of which a client should be able to set. Keeping them on the entity but off the form-binding type is what makes the compiler enforce the boundary. The uploaded `IFormFile` is bound separately as an action parameter so the validation attributes don't fight with the upload-stream lifetime.
>
> ✓ **Quick check:** `dotnet build` succeeds.

Step 3: *Seed the job catalog*

The portal has no admin interface — jobs are baked in. A tiny interface plus a static implementation is enough, and it sets up the swap to a backing store later (a different course; not this one) without changing the controller.

1. **Create** the interface:

> `Services/IJobCatalog.cs`

```
using CloudCiCareers.Web.Models;

namespace CloudCiCareers.Web.Services;

public interface IJobCatalog
{
    IReadOnlyList<Job> GetAll();
    Job? GetById(int id);
}
```

2. **Create** the static implementation:

> `Services/StaticJobCatalog.cs`

```
using CloudCiCareers.Web.Models;

namespace CloudCiCareers.Web.Services;

public class StaticJobCatalog : IJobCatalog
{
    private static readonly IReadOnlyList<Job> _jobs =
        new List<Job>
    {
        new(1, "Cloud Engineer", "Platform",
            "Design, deploy, and operate Azure infra-
        structure for the product platform.",
            DateTimeOffset.UtcNow.AddDays(-7)),
        new(2, "Backend Developer", "Engineering",
            "Build and own ASP.NET Core services that
        back the customer-facing apps.",
            DateTimeOffset.UtcNow.AddDays(-5)),
        new(3, "DevOps Specialist", "Platform",
            "Build and maintain CI/CD, observability,
        and incident-response tooling.",
            DateTimeOffset.UtcNow.AddDays(-3)),
        new(4, "Site Reliability Engineer", "Platform",
            "Define SLOs, run game days, and keep
        production boring.",
            DateTimeOffset.UtcNow.AddDays(-1)),
    };

    public IReadOnlyList<Job> GetAll() => _jobs;

    public Job? GetById(int id) =>
        _jobs.FirstOrDefault(j => j.Id == id);
}
```

QUICK CHECK:

Both files compile. `StaticJobCatalog` returns four jobs from `GetAll()`.

Step 4: *Add the* `IApplicationStore` *and the in-memory implementation*

The recruiter side needs a place to read applications back, mutate their status, and delete them. A thread-safe `ConcurrentDictionary` does the job for one replica — same singleton-store pattern as the previous chapter's `IQuoteStore`.

1. **Create** the interface:

> `Services/IApplicationStore.cs`

```
using CloudCiCareers.Web.Models;

namespace CloudCiCareers.Web.Services;

public interface IApplicationStore
{
    IEnumerable<Application> GetAll();
    Application? GetById(string id);
    Application Create(Application application);
    bool UpdateStatus(string id, ApplicationStatus newS-
        tatus, string? notes);
    bool Delete(string id);
}
```

2. **Create** the in-memory implementation:

> `Services/InMemoryApplicationStore.cs`

```
using System.Collections.Concurrent;
using CloudCiCareers.Web.Models;

namespace CloudCiCareers.Web.Services;

public class InMemoryApplicationStore : IApplication-
        Store
{
    private readonly ConcurrentDictionary<string,
        Application> _applications = new();

    public IEnumerable<Application> GetAll() =>
        _applications.Values.OrderByDescending(a =>
        a.SubmittedAt);

    public Application? GetById(string id) =>
        _applications.TryGetValue(id, out var applica-
        tion) ? application : null;

    public Application Create(Application application)
    {
        if (string.IsNullOrEmpty(application.Id))
        {
            application.Id =
        Guid.NewGuid().ToString("n");
        }
        if (application.SubmittedAt == default)
        {
            application.SubmittedAt = DateTimeOff-
        set.UtcNow;
        }
        _applications[application.Id] = application;
        return application;
    }

    public bool UpdateStatus(string id, Application-
        Status newStatus, string? notes)
    {
        if (!_applications.TryGetValue(id, out var
        application))
        {
            return false;
        }
        application.Status = newStatus;
```

```
        application.Notes = notes;
        return true;
    }

    public bool Delete(string id) => _applica-
        tions.TryRemove(id, out _);
}
```

CONCEPT DEEP DIVE

The store survives every request because it's registered as `Singleton` (Step 7); it does **not** survive a Container App revision rollover, because every revision is a fresh process. That's the motivation for the next exercise.

⚠ **Common Mistakes**

- Registering this as `Scoped`. Every request gets a fresh empty dictionary; the recruiter view always shows zero applications.

✓ **Quick check:** Both files compile.

Step 5: *Add the* `IBlobService` *abstraction and a local-disk implementation*

The CV bytes go somewhere — for now, a folder on disk. Putting an interface in front of `File.WriteAllBytesAsync` is what lets the next exercise replace the implementation with `BlobContainerClient` calls without touching the controllers. One line in `Program.cs` swaps the registration; upstream keeps compiling.

1. **Create** the interface:

> `Services/IBlobService.cs`

```
namespace CloudCiCareers.Web.Services;

public interface IBlobService
{
    Task UploadAsync(string name, Stream content, Can-
        cellationToken ct);
    Task<Stream> OpenReadAsync(string name, Cancella-
        tionToken ct);
    Task DeleteAsync(string name, CancellationToken ct);
}
```

2. **Create** the local-disk implementation:

> `Services/LocalFileBlobService.cs`

```
namespace CloudCiCareers.Web.Services;

public class LocalFileBlobService : IBlobService
{
    private readonly string _root;

    public LocalFileBlobService(IWebHostEnvironment env)
    {
        _root = Path.Combine(env.ContentRootPath,
        "uploads");
        Directory.CreateDirectory(_root);
    }

    public async Task UploadAsync(string name, Stream
        content, CancellationToken ct)
    {
        var path = Path.Combine(_root, name);
        await using var file = File.Create(path);
        await content.CopyToAsync(file, ct);
    }

    public Task<Stream> OpenReadAsync(string name, Can-
        cellationToken ct)
    {
        var path = Path.Combine(_root, name);
        Stream stream = File.OpenRead(path);
        return Task.FromResult(stream);
    }

    public Task DeleteAsync(string name, Cancellation-
        Token ct)
    {
        var path = Path.Combine(_root, name);
        if (File.Exists(path))
        {
            File.Delete(path);
        }
        return Task.CompletedTask;
    }
}
```

> **CONCEPT DEEP DIVE**
>
> The interface speaks only in `Stream`s and string names — no filesystem, no Azure Blob, no specific SDK. That's the abstraction boundary. The next exercise registers a managed-identity-authenticated Azure Blob implementation against the same three methods, and the controllers don't notice.
>
> ✓ **Quick check:** Both files compile.

Step 6: *Add the PDF magic-bytes validator*

Every PDF on Earth starts with the same four bytes: `%PDF` (`0x25 0x50 0x44 0x46`). Reading those four bytes and comparing tells you whether the file is structurally a PDF, regardless of what the filename says. This is the core of trustworthy file-type validation: don't believe the extension, don't believe the `Content-Type` header — read the bytes the file actually starts with.

1. **Create** the validator:

> `Services/PdfValidation.cs`

```
namespace CloudCiCareers.Web.Services;

public static class PdfValidation
{
    // PDF files always start with the four bytes "%PDF"
        (0x25 0x50 0x44 0x46).
    private static readonly byte[] PdfSignature = {
        0x25, 0x50, 0x44, 0x46 };

    public static bool IsPdf(Stream s)
    {
        if (s is null || !s.CanRead)
        {
            return false;
        }

        Span<byte> header = stackalloc byte[4];
        var read = s.Read(header);

        // CRITICAL: rewind so the caller's upload
        pipeline reads from byte zero.
        if (s.CanSeek)
        {
            s.Position = 0;
        }

        if (read < 4)
        {
            return false;
        }

        return header.SequenceEqual(PdfSignature);
    }
}
```

> **CONCEPT DEEP DIVE: MAGIC BYTES, ISO 32000, AND WHAT THIS CHECK DOES NOT DO**
>
> The four-byte `%PDF` header is part of the PDF file format specification (ISO 32000); RFC 8118 registers `application/pdf` as the canonical media type. Every conforming PDF starts with these bytes — anything that doesn't is either truncated, corrupt, or pretending.
>
> What a magic-bytes check **does** buy you: confirmation that the bytes you received are structurally a PDF, not a renamed `.exe` or a text file given a `.pdf` extension. What it does **not** buy you: any guarantee that the PDF is *safe*. PDFs can carry JavaScript, embedded files, malformed cross-references that crash parsers, viewer-specific exploits. Defending against those is layered separately — antivirus scanning at rest (Microsoft Defender for Storage), sandboxed parsing, content stripping with PdfPig, file-size limits. Magic bytes is layer one; the other layers are real and outside this chapter's scope.
>
> ⚠ **Common Mistakes**
>
> - Forgetting `s.Position = 0` after the read. The four-byte header is consumed, the upload pipeline writes from byte four onwards, and the saved file fails to open in any PDF viewer.
> - Treating `Content-Type` from the browser as a security check. Browsers send what the OS guessed from the extension; both are attacker-controlled. Magic bytes is what makes the check trustworthy.
>
> √ **Quick check:** `dotnet build` succeeds.

Step 7: *Wire dependency injection in* `Program.cs`

Three singletons plus the standard MVC pipeline. Same shape as the BCD webapp series.

1. **Replace** the contents of `Program.cs`:

> `Program.cs`

```
using CloudCiCareers.Web.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

// Singletons — one shared instance per process (per
        Container App replica).
builder.Services.AddSingleton<IJobCatalog, StaticJob-
        Catalog>();
builder.Services.AddSingleton<IApplicationStore,
        InMemoryApplicationStore>();
builder.Services.AddSingleton<IBlobService, Local-
        FileBlobService>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Jobs}/{action=Index}/{id?}");

app.Run();
```

CONCEPT DEEP DIVE

The default route now points at `JobsController.Index` — the home page IS the jobs page, no extra hop. Singleton is correct for all three services since none depend on a per-request type like `DbContext`.

√ **Quick check:** `dotnet build` succeeds.

Step 8: *Build the* `JobsController` *and the apply form*

Two GETs and one POST: the home page lists jobs, `/Jobs/Apply/3` renders the form for a specific job, and submitting that form does the validation, the upload, and the redirect. The POST is where every interesting decision lives — antiforgery, model state, magic bytes, blob streaming.

1. **Create** the controller:

> `Controllers/JobsController.cs`

```
using CloudCiCareers.Web.Models;
using CloudCiCareers.Web.Services;
using Microsoft.AspNetCore.Mvc;

namespace CloudCiCareers.Web.Controllers;

public class JobsController : Controller
{
    private readonly IJobCatalog _catalog;
    private readonly IApplicationStore _store;
    private readonly IBlobService _blobs;

    public JobsController(
        IJobCatalog catalog,
        IApplicationStore store,
        IBlobService blobs)
    {
        _catalog = catalog;
        _store = store;
        _blobs = blobs;
    }

    public IActionResult Index()
    {
        return View(_catalog.GetAll());
    }

    public IActionResult Apply(int id)
    {
        var job = _catalog.GetById(id);
        if (job is null)
        {
            return NotFound();
        }
        ViewData["Job"] = job;
        return View(new ApplyForm());
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Apply(int id,
        ApplyForm form, IFormFile? cv,
```

```
        CancellationToken ct)
    {
        var job = _catalog.GetById(id);
        if (job is null)
        {
            return NotFound();
        }
        ViewData["Job"] = job;

        if (cv is null || cv.Length == 0)
        {
            ModelState.AddModelError(nameof(cv),
                "Please attach your CV as a PDF file.");
        }
        else if (!
        PdfValidation.IsPdf(cv.OpenReadStream()))
        {
            ModelState.AddModelError(nameof(cv),
                "The uploaded file is not a valid PDF
        document.");
        }

        if (!ModelState.IsValid)
        {
            return View(form);
        }

        var blobName = $"{Guid.NewGuid():n}.pdf";

        // Stream straight from the request body to the
        blob — don't re-buffer.
        await using (var upload = cv!.OpenReadStream())
        {
            await _blobs.UploadAsync(blobName, upload,
        ct);
        }

        var application = new Application
        {
            JobId = job.Id,
            ApplicantName = form.Name,
            ApplicantEmail = form.Email,
            CvBlobName = blobName,
```

```
            Status = ApplicationStatus.Submitted,
        };

        _store.Create(application);
        TempData["Thanks"] =
        "Thanks for applying! We'll be in touch.";

        return RedirectToAction("Details", "Applica-
        tions",
            new { id = application.Id });
    }
}
```

2. **Create** the home-page view at `Views/Jobs/Index.cshtml`:

    > `Views/Jobs/Index.cshtml`

```
@model IEnumerable<CloudCiCareers.Web.Models.Job>
@{ ViewData["Title"] = "Open positions"; }

<h1>Open positions</h1>
<p class="lead">We're hiring across Platform and Engin-
eering.</p>

<div class="row">
@foreach (var job in Model)
{
    <div class="col-md-6 mb-3">
        <div class="card h-100"><div class="card-body">
            <h5 class="card-title">@job.Title</h5>
            <h6 class="card-subtitle text-muted
mb-2">@job.Department</h6>
            <p class="card-text">@job.Description</p>
            <a class="btn btn-primary" asp-
action="Apply"
               asp-route-id="@job.Id">Apply</a>
        </div></div>
    </div>
}
</div>
```

3. **Create** the apply-form view at `Views/Jobs/Apply.cshtml`:

    > `Views/Jobs/Apply.cshtml`

```
@model CloudCiCareers.Web.Models.ApplyForm
@{
    var job = (CloudCiCa-
reers.Web.Models.Job)ViewData["Job"]!;
    ViewData["Title"] = $"Apply for {job.Title}";
}

<h1>Apply for @job.Title</h1>
<p class="text-muted">@job.Department</p>

<form asp-action="Apply" asp-route-id="@job.Id"
      method="post" enctype="multipart/form-data">
    @Html.AntiForgeryToken()
    <div asp-validation-summary="All" class="text-
danger"></div>

    <div class="mb-3">
        <label asp-for="Name" class="form-label">Full
name</label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-
danger"></span>
    </div>
    <div class="mb-3">
        <label asp-for="Email" class="form-
label">Email</label>
        <input asp-for="Email" class="form-control"
type="email" />
        <span asp-validation-for="Email" class="text-
danger"></span>
    </div>
    <div class="mb-3">
        <label for="cv" class="form-label">CV (PDF)</
label>
        <input id="cv" name="cv" type="file"
accept="application/pdf"
               class="form-control" />
    </div>

    <button type="submit" class="btn btn-primary">Submit
```

```
application</button>
</form>
```

CONCEPT DEEP DIVE: ANTIFORGERY ON MULTIPART FORMS

Antiforgery protects every state-changing request — without it, a malicious page on another origin could submit your form using the user's logged-in cookie. The mechanism: a server-stamped hidden form field plus a matching cookie, both validated by `[ValidateAntiForgeryToken]`. A cross-origin form can't read the cookie, so the replay fails. `@Html.AntiForgeryToken()` writes the field; the attribute validates it.

The multipart caveat: `enctype="multipart/form-data"` does **not** break antiforgery — the token field travels in the multipart envelope just like any other form field. What it *does* mean is that you can't use `[FromBody]` (that's for JSON), so each field is bound by name from the multipart parts.

⚠ **Common Mistakes: stream the upload, don't buffer it**

- Reading the upload into a `MemoryStream` first, then passing that to `IBlobService.UploadAsync`. Don't. `IFormFile.OpenReadStream()` is already buffered to disk by the request body reader once the body crosses 64 KB; re-buffering through a `MemoryStream` wastes process memory for a copy you don't need. Pipe the stream straight in.
- Forgetting `enctype="multipart/form-data"` on the `<form>`. The browser then sends `application/x-www-form-urlencoded`, the file is omitted, and `cv` arrives as `null`.
- Forgetting `[ValidateAntiForgeryToken]`. The submit appears to work in dev; the moment another origin can reach the form, it's exploitable.

√ **Quick check:** `dotnet build` succeeds.

Step 9: *Build the* `ApplicationsController` *and the recruiter views*

A list, a detail page with status + delete forms, and a CV-streaming action that proxies bytes from the blob service. `Content-Disposition: inline` makes the browser render the PDF in-tab — easier to verify visually that the upload round-tripped.

1. **Create** the controller:

> `Controllers/ApplicationsController.cs`

```
using CloudCiCareers.Web.Models;
using CloudCiCareers.Web.Services;
using Microsoft.AspNetCore.Mvc;

namespace CloudCiCareers.Web.Controllers;

public class ApplicationsController : Controller
{
    private readonly IApplicationStore _store;
    private readonly IJobCatalog _catalog;
    private readonly IBlobService _blobs;

    public ApplicationsController(
        IApplicationStore store,
        IJobCatalog catalog,
        IBlobService blobs)
    {
        _store = store;
        _catalog = catalog;
        _blobs = blobs;
    }

    public IActionResult Index()
    {
        ViewData["Catalog"] = _catalog;
        return View(_store.GetAll().ToList());
    }

    public IActionResult Details(string id)
    {
        var application = _store.GetById(id);
        if (application is null)
        {
            return NotFound();
        }
        ViewData["Job"] = _cata-
         log.GetById(application.JobId);
        return View(application);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
```

```
    public IActionResult UpdateStatus(string id,
        ApplicationStatus newStatus, string? notes)
    {
        if (!_store.UpdateStatus(id, newStatus, notes))
        {
            return NotFound();
        }
        return RedirectToAction(nameof(Details), new {
         id });
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Delete(string id)
    {
        if (!_store.Delete(id))
        {
            return NotFound();
        }
        return RedirectToAction(nameof(Index));
    }

    public async Task<IActionResult> Cv(string id, Can-
        cellationToken ct)
    {
        var application = _store.GetById(id);
        if (application is null)
        {
            return NotFound();
        }
        var stream = await _blobs.OpenReadAsync(applica-
        tion.CvBlobName, ct);
        Response.Headers["Content-Disposition"] =
        "inline; filename=\"cv.pdf\"";
        return File(stream, "application/pdf");
    }
}
```

2. **Create** the recruiter list view at `Views/Applications/Index.cshtml`:

> `Views/Applications/Index.cshtml`

```
@model List<CloudCiCareers.Web.Models.Application>
@{
    ViewData["Title"] = "Applications";
    var catalog = (CloudCiCa-
reers.Web.Services.IJobCatalog)ViewData["Catalog"]!;
}

<h1>Applications</h1>

@if (!Model.Any())
{
    <p class="text-muted">No applications yet.</p>
}
else
{
    <table class="table">
        <thead><tr>
            <th>Submitted</th><th>Job</th><th>Applic-
ant</th>
            <th>Status</th><th></th>
        </tr></thead>
        <tbody>
        @foreach (var a in Model)
        {
            var job = catalog.GetById(a.JobId);
            <tr>
                <td>@a.SubmittedAt.LocalDateTime</td>
                <td>@(job?.Title ?? "(unknown)")</td>
                <td>@a.ApplicantName<br/><small
class="text-muted">@a.ApplicantEmail</small></td>
                <td><span class="badge bg-second-
ary">@a.Status</span></td>
                <td><a class="btn btn-sm btn-outline-
primary"
                       asp-action="Details" asp-route-
id="@a.Id">Open</a></td>
            </tr>
        }
        </tbody>
    </table>
}
```

3. **Create** the detail view at `Views/Applications/Details.cshtml`:

    > `Views/Applications/Details.cshtml`

```
@model CloudCiCareers.Web.Models.Application
@using CloudCiCareers.Web.Models
@{
    var job = (Job?)ViewData["Job"];
    ViewData["Title"] = $"Application — {Model.Applic-
antName}";
}

@if (TempData["Thanks"] is string thanks)
{
    <div class="alert alert-success">@thanks</div>
}

<h1>@Model.ApplicantName</h1>
<p class="text-muted">
    Applied for <strong>@(job?.Title ?? "(unknown)")</
strong>
    on @Model.SubmittedAt.LocalDateTime
</p>
<p><a asp-action="Cv" asp-route-id="@Model.Id"
target="_blank">View CV (PDF)</a></p>

<h2>Update status</h2>
<form asp-action="UpdateStatus" asp-route-id="@Model.Id"
method="post">
    @Html.AntiForgeryToken()
    <div class="mb-3">
        <label class="form-label">Status</label>
        <select name="newStatus" class="form-select">
        @foreach (var value in Enum.GetVal-
ues<ApplicationStatus>())
        {
            <option value="@value" selected="@(value ==
Model.Status)">@value</option>
        }
        </select>
    </div>
    <div class="mb-3">
        <label class="form-label">Notes</label>
        <textarea name="notes" class="form-control"
rows="3">@Model.Notes</textarea>
```

```
    </div>
    <button type="submit" class="btn btn-primary">Save</
button>
</form>

<hr/>

<form asp-action="Delete" asp-route-id="@Model.Id"
method="post"
      onsubmit="return confirm('Delete this applica-
tion?');">
    @Html.AntiForgeryToken()
    <button type="submit" class="btn btn-outline-
danger">Delete</button>
</form>
```

> **QUICK CHECK:**
> `dotnet build` succeeds with no warnings.

Step 10: *Run locally and exercise the full applicant-then-recruiter flow*

A real end-to-end pass before you reach for Docker. The point is to see all the pieces — the validator, the upload, the in-memory store, the recruiter views — connected and working in a single process.

1. **Start** the app:

   ```
   dotnet run --launch-profile http
   ```

   Note the port; call it `<port>` below.

2. **Open** `http://localhost:<port>/` in a browser. You should see four job cards.
3. **Click** *Apply* on any job, fill in name and email, attach a real PDF (any small one), and submit. You should land on `/`

`Applications/Details/<some-guid>` with a green "Thanks for applying!" banner.

4. **Try** uploading a non-PDF: rename a `notes.txt` file to `cv.pdf` and submit. The form should re-render with a validation error — *"The uploaded file is not a valid PDF document."*
5. **Visit** `http://localhost:<port>/Applications`. The successful application appears in the table.
6. **Open** the detail page, change the status to `UnderReview`, save, and confirm the badge updates back on the listing.
7. **Click** the *View CV* link — the PDF should render inline in a new tab.
8. **Stop** the server with `Ctrl+C`. The `uploads/` directory now contains your test CV — confirm the file size and that opening it in a real PDF viewer works. Delete the directory if you want a clean slate.

> **QUICK CHECK:**
> Four jobs on the home page, valid PDF accepted, invalid file rejected with a clear error, status updates persist within the running process, CV renders inline.

Step 11: *Containerize, set up the OIDC-federated pipeline, and deploy*

Same OIDC pattern as the previous chapter — fresh Azure resources, a fresh Entra app registration with a federated credential, a workflow that signs in via OIDC, builds, pushes, updates the Container App, and smoke-tests the deployment. Container ingress on `8080`. The smoke target for this exercise is `/` (the anonymous home page) — the deeper `/health/live` endpoint that future-you will smoke against doesn't exist yet, and trying to use it here would fail every

deploy until the third exercise lands. See the deployment chapter for why `id-token: write` is load-bearing and how the federated subject claim is matched.

1. **Add** the Dockerfile at the project root:

   > `Dockerfile`

```
# Build stage — full SDK, restores and publishes a
        Release build.
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . ./
RUN dotnet publish -c Release -o /app /p:Use-
        AppHost=false

# Runtime stage — slim ASP.NET image, runs as the non-
        root 'app' user.
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app ./

# LocalFileBlobService writes uploads to ContentRoot/
        uploads at runtime.
# /app is owned by root, so create the directory and
        chown it to the non-root
# 'app' user before dropping privileges. (Replaced by
        Azure Blob in the next
# exercise; harmless to keep either way.)
RUN mkdir -p /app/uploads && chown app:app /app/uploads

EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
USER app

ENTRYPOINT ["dotnet", "CloudCiCareers.Web.dll"]
```

2. **Add** a `.dockerignore`:

   > `.dockerignore`

```
bin/
obj/
.vs/
.vscode/
.idea/
*.user
.git/
.github/
uploads/
Dockerfile
.dockerignore
README.md
```

3. **Push** the project to a fresh GitHub repo (substitute your username for `<your-gh-user>`; the example uses `larsappel`):

```
git init
git add .
git commit -m "Scaffold CloudCiCareers.Web with PDF
        uploads"
gh repo create <your-gh-user>/cloudci-careers --public
        --source=. --push
```

4. **Provision** the Azure resources. Pick a 5–6 character random alphanumeric suffix for the ACR name (it must be globally unique); the example uses `7g8h2j` but you should generate your own:

```
az group create -n rg-careers-week7 -l northeurope

az acr create -n acrcareers<rand> -g rg-careers-week7 \
  --sku Basic --admin-enabled false

az containerapp env create -n cae-careers-week7 \
  -g rg-careers-week7 -l northeurope

az containerapp create \
  -n ca-careers-week7 \
  -g rg-careers-week7 \
  --environment cae-careers-week7 \
  --image mcr.microsoft.com/k8se/quickstart:latest \
  --target-port 8080 \
  --ingress external \
  --min-replicas 1 \
  --max-replicas 1
```

5. **Create** the Entra app, the service principal, the role assignments, and the federated credential. Substitute your GitHub username for `<your-gh-user>` in the federated subject — leaving the angle brackets in literally is the single most common authentication failure:

```
az ad app create --display-name "github-cloudci-careers-
        oidc"
export APP_ID="<paste appId from previous output>"
az ad sp create --id "$APP_ID"

ACR_ID=$(az acr show -n acrcareers<rand> --query id -o
        tsv)
CA_ID=$(az containerapp show -n ca-careers-week7 \
  -g rg-careers-week7 --query id -o tsv)

az role assignment create --assignee "$APP_ID" \
  --role AcrPush --scope "$ACR_ID"
az role assignment create --assignee "$APP_ID" \
  --role "Container Apps Contributor" --scope "$CA_ID"

az ad app federated-credential create \
  --id "$APP_ID" \
  --parameters '{
    "name": "main-branch",
    "issuer": "https://
        token.actions.githubusercontent.com",
    "subject": "repo:<your-gh-user>/cloudci-
        careers:ref:refs/heads/main",
    "audiences": ["api://AzureADTokenExchange"],
    "description": "GitHub Actions, main branch only"
  }'
```

6. **Set** the four GitHub secrets via stdin (avoids leaving values in shell history):

```
printf '%s' "$APP_ID" | gh secret set AZURE_CLIENT_ID \
  --repo <your-gh-user>/cloudci-careers
printf '%s' "$(az account show --query tenantId -o
        tsv)" | \
  gh secret set AZURE_TENANT_ID --repo <your-gh-user>/
        cloudci-careers
printf '%s' "$(az account show --query id -o tsv)" | \
  gh secret set AZURE_SUBSCRIPTION_ID --repo <your-gh-
        user>/cloudci-careers
printf '%s' "acrcareers<rand>" | gh secret set ACR_NAME
        \
  --repo <your-gh-user>/cloudci-careers
```

7. **Grant** the Container App's managed identity `AcrPull`, then point the app at the registry over that identity:

```
az containerapp identity assign \
  -n ca-careers-week7 -g rg-careers-week7 --system-
        assigned

IDENTITY_PRINCIPAL_ID=$(az containerapp show \
  -n ca-careers-week7 -g rg-careers-week7 \
  --query identity.principalId -o tsv)

az role assignment create \
  --assignee "$IDENTITY_PRINCIPAL_ID" \
  --role AcrPull --scope "$ACR_ID"

az containerapp registry set \
  -n ca-careers-week7 -g rg-careers-week7 \
  --server acrcareers<rand>.azurecr.io \
  --identity system
```

8. **Add** the workflow at `.github/workflows/ci.yml`:

> `.github/workflows/ci.yml`

```
name: build-and-deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  id-token: write
  contents: read

env:
  RESOURCE_GROUP: rg-careers-week7
  CONTAINER_APP: ca-careers-week7

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Sign in to Azure
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: $
        {{ secrets.AZURE_SUBSCRIPTION_ID }}

      - name: Resolve ACR login server
        run: |
          echo "ACR_LOGIN_SERVER=$
        {{ secrets.ACR_NAME }}.azurecr.io" \
            >> "$GITHUB_ENV"

      - name: Log in to ACR
        run: az acr login --name ${{ secrets.ACR_NAME }}

      - name: Build and push image
        uses: docker/build-push-action@v6
        with:
          context: .
```

```
          push: true
          tags: |
            ${{ env.ACR_LOGIN_SERVER }}/cloudci-careers:
        ${{ github.sha }}
            ${{ env.ACR_LOGIN_SERVER }}/cloudci-
        careers:latest

      - name: Update Container App
        run: |
          az containerapp update \
            -n "$CONTAINER_APP" -g "$RESOURCE_GROUP" \
            --image "${{ env.ACR_LOGIN_SERVER }}/
        cloudci-careers:${{ github.sha }}"

      - name: Smoke test
        run: |
          FQDN=$(az containerapp show \
            -n "$CONTAINER_APP" -g "$RESOURCE_GROUP" \
            --query proper-
        ties.configuration.ingress.fqdn -o tsv)
          for i in {1..20}; do
            if curl -fsS "https://$FQDN/" >/dev/null;
        then
              echo "Smoke test passed."
              exit 0
            fi
            sleep 3
          done
          echo "Smoke test failed."
          exit 1
```

9. **Push** to trigger the first real deployment:

```
git add Dockerfile .dockerignore .github/workflows/
        ci.yml
git commit -m "Add OIDC-federated build-and-deploy work-
        flow"
git push
gh run watch
```

10. **Verify** the deployed FQDN serves the home page:

```
FQDN=$(az containerapp show \
  -n ca-careers-week7 -g rg-careers-week7 \
  --query properties.configuration.ingress.fqdn -o tsv)
echo "https://$FQDN/"
curl -I "https://$FQDN/"
```

Expected: `HTTP/2 200`. Open the URL — the four job cards render.

COMMON MISTAKES

- Pointing the smoke test at `/health/live`. That endpoint doesn't exist yet — it lands in the third exercise of this chapter. Pointing the smoke step at it now causes every deploy to fail.
- Federated subject typo: `refs/head/main` instead of `refs/heads/main`. The token is rejected with `AADSTS70021: No matching federated identity record found`.
- `--target-port` not `8080`. Container Apps' health probe fails because Kestrel listens on `:8080` (set by the Dockerfile `ENV`) but the platform asks port-something-else.

√ **Quick check:** The workflow run is green, `https://$FQDN/` returns `200`, and the home page lists four jobs.

Step 12: *Add Application Insights via the* `secretref:` *pattern*

Same pattern as the logging chapter — workspace-based component, SDK pinned to 2.22.0, connection string injected as a Container Apps secret and referenced by env var. The pin is **mandatory** here too: the 3.x line throws `System.InvalidOperationException: A connection string was not found...` at host startup when the env var is empty (i.e. every

`dotnet run` on your laptop), while 2.22.0 silently drops telemetry when missing.

1. **Provision** the workspace-based component against the auto-managed Log Analytics workspace inside `rg-careers-week7`:

```
WS_ID=$(az monitor log-analytics workspace list \
  -g rg-careers-week7 --query '[0].id' -o tsv)

az monitor app-insights component create \
  --app cloudci-careers-insights \
  -g rg-careers-week7 --location northeurope \
  --workspace "$WS_ID"

CONN=$(az monitor app-insights component show \
  --app cloudci-careers-insights -g rg-careers-week7 \
  --query connectionString -o tsv)
```

2. **Add** the SDK and register it:

```
dotnet add package
        Microsoft.ApplicationInsights.AspNetCore --ver-
        sion 2.22.0
```

In `Program.cs`, alongside the other service registrations:

> `Program.cs`

```
builder.Services.AddApplicationInsightsTelemetry();
```

3. **Inject** the connection string and bind the env var:

```
az containerapp secret set \
  -g rg-careers-week7 -n ca-careers-week7 \
  --secrets appinsights-connstr="$CONN"

az containerapp update \
  -g rg-careers-week7 -n ca-careers-week7 \
  --set-env-vars APPLICATIONIN-
        SIGHTS_CONNECTION_STRING=secretref:appinsights-
        connstr
```

4. **Commit and push** to roll the SDK out:

```
git add CloudCiCareers.Web.csproj Program.cs
git commit -m "Add Application Insights SDK"
git push
gh run watch
```

> **QUICK CHECK:**
> `az containerapp show -g rg-careers-week7 -n ca-careers-week7 --query 'properties.template.containers[0].env'` shows `APPLICATIONINSIGHTS_CONNECTION_STRING` with a `secretRef`, not a literal value.

Step 13: *Test Your Implementation*

End-to-end check against the live deployment.

1. **Capture** the FQDN:

```
FQDN=$(az containerapp show \
  -n ca-careers-week7 -g rg-careers-week7 \
  --query properties.configuration.ingress.fqdn -o tsv)
echo "https://$FQDN/"
```

2. **Verify** the home page lists 3–4 jobs:

```
curl -I "https://$FQDN/"
```

Expected: `HTTP/2 200`.

3. **Apply** for a job in the browser with a valid PDF. Confirm:
   - The form posts and you land on `/Applications/Details/<guid>`.
   - The success banner shows.
   - The application appears in `https://$FQDN/Applications`.
4. **Re-submit** the form with a non-PDF (rename `notes.txt` to `cv.pdf`). The form re-renders with the validation error, no application is created.
5. **Edit** the status on the detail page → the badge updates on the listing.
6. **Click** *View CV* → the PDF renders inline in the browser tab.
7. **Generate** traffic and check Application Insights:

```
for i in {1..30}; do curl -s "https://$FQDN/" >/dev/
        null; sleep 0.3; done
```

In the Azure Portal, navigate to Application Insights `cloudci-careers-insights` → **Logs**, and run:

```
requests
| where timestamp > ago(10m)
| summarize count() by name, resultCode
```

You should see incoming requests within one to three minutes of the curl loop.

√ ***Success indicators:***

- `https://$FQDN/` *returns* `200` *and lists four jobs*
- *Valid PDF upload creates an application visible at* `/Applications`
- *Invalid file (renamed* `notes.txt`*) is rejected with a clear validation message*
- *Status edits persist between page loads*
- *The CV link renders the original PDF inline*
- *App Insights* `requests` *table records traffic against the deployed FQDN*

√ ***Final verification checklist:***

- [ ] `CloudCiCareers.Web` *project scaffolded with* `dotnet new mvc --framework net10.0`
- [ ] `Job`, `Application` + `ApplicationStatus`, *and* `ApplyForm` *types under* `Models/`
- [ ] `IJobCatalog` + `StaticJobCatalog`, `IApplicationStore` + `InMemoryApplicationStore`, `IBlobService` + `LocalFileBlobService`, *plus* `PdfValidation` *under* `Services/`
- [ ] `JobsController` + `Apply.cshtml` + `Index.cshtml`, `ApplicationsController` + `Index.cshtml` + `Details.cshtml`
- [ ]

> *Dockerfile produces a working image listening on* `:8080` *running as non-root*
>
> ☐ *GitHub repo* `<your-gh-user>/cloudci-careers` *pushed and federated against an Entra app* `github-cloudci-careers-oidc`
>
> ☐ *Workflow runs green on push to* `main`*, deploys to Container App* `ca-careers-week7` *in* `rg-careers-week7`*, smoke-tests* `/`
>
> ☐ *App Insights* `cloudci-careers-insights` *workspace-based, connection string delivered via* `secretref:`

Persistence in this exercise is fragile by design — the in-memory store and the `./uploads/` directory are wiped on every Container App revision rollover, every restart, every scale event. Apply for a job, redeploy, and watch the application disappear. This is **not** a bug to fix here; it's the motivating problem for what comes next.

14.2.4 Common Issues

> ***If you encounter problems:***
>
> **`The required antiforgery cookie ... is not present`** *or* **`Antiforgery token mismatch`***: Either* `@Html.AntiForgeryToken()` *is missing from the form or* `[ValidateAntiForgeryToken]` *is missing from the action. MVC's default scaffolding adds neither — you have to.*
>
> **`System.InvalidOperationException: A connection string was not found for the Application Insights connection.`***: The Application Insights SDK package on disk is 3.x. Check* `dotnet list package` *— confirm* `Microsoft.ApplicationInsights.AspNetCore 2.22.0`*. If it's drifted,* `dotnet remove package` *then re-add with* `--version 2.22.0`*.*
>
> ***Smoke test fails on every deploy with*** **`curl: (22) The requested URL returned error: 404`***: The workflow is hitting* `/health/live` *or some other endpoint that doesn't exist yet. The smoke target during this exercise must be* `/`*. The deeper health probes arrive later in this chapter.*
>
> **`AADSTS70021: No matching federated identity record found for presented assertion`***: The* `subject` *claim from the GitHub OIDC token doesn't match the federated credential. Exact form:* `repo:<your-gh-user>/cloudci-careers:ref:refs/heads/main`*. Watch for* `head` *vs* `heads`*, the wrong username, the wrong branch, or angle brackets left in literally.*
>
> ***Workflow fails at the push step with*** **`unauthorized: authentication required`** ***or***

> ***401:*** *The federated subject is correct but the Entra app doesn't have* `AcrPush` *on the registry yet, or role-assignment propagation hasn't finished. Wait one to two minutes and re-run.*
> ***Apply form rejects valid PDFs:*** `PdfValidation.IsPdf` *is reading the four bytes but not rewinding. Confirm the* `s.Position = 0` *line. Symptom: the saved file in* `uploads/` *is exactly four bytes shorter than the original and starts with whatever followed* `%PDF`*.*
> ***Container App revision goes*** **`Failed`** ***with*** **`Failed to pull image`***: Either the system-assigned managed identity wasn't granted* `AcrPull`*, or* `az containerapp registry set --identity system` *was skipped. Both are required.*
> **`Live Metrics: Not available`***: The connection string env var didn't make it into the container. Check* `az containerapp show ... --query 'properties.template.containers[0].env'` *for the* `secretRef` *and* `az containerapp secret list ...` *for the underlying value.*
> ***Still stuck?*** *Re-read the deployment chapter for the OIDC pipeline mechanics and the logging chapter for the* `secretref:` *pattern. The names changed; the patterns didn't.*

14.2.5 Summary

You have a working anonymous recruitment portal — applicants browse four hard-coded jobs, attach a PDF, and submit, while recruiters see every application and walk it through a

status workflow. The PDF validation actually checks the file's bytes rather than trusting the extension. The whole thing builds into a non-root multi-stage Docker image, ships through an OIDC-federated GitHub Actions pipeline to a Container App in `rg-careers-week7`, and emits telemetry to Application Insights via the `secretref:` pattern.

- √ MVC project shape — controllers + Razor views + antiforgery on every form — fits a portal with shared layout and multiple actions per resource
- √ Form-binding types (`ApplyForm`) are wire types; entities (`Application`) are domain types — same discipline as DTOs in the previous chapter
- √ The four-byte `%PDF` magic-bytes check is the load-bearing piece of file-type validation; magic bytes confirm the file *is* a PDF, not that the PDF is *safe*
- √ `IFormFile.OpenReadStream()` pipes straight to `IBlobService.UploadAsync` — don't re-buffer through `MemoryStream`
- √ The `IBlobService` abstraction is what makes the next exercise's swap to Azure Blob a one-line registration change
- √ The OIDC-federated pipeline pattern from the deployment chapter ports cleanly to a brand-new repo, with a smoke target (`/`) that exists today

> **KEY TAKEAWAY:**
>
> Storage in this exercise is intentionally throwaway — the in-memory dictionary and the local `uploads/` folder are wiped on every revision rollover. The motivation for managed state is much sharper after you've felt the fragility of process-local state in production than after reading about it. Stop here, deploy a few applications, redeploy, watch them vanish — that lived experience is what makes the next change feel like a fix instead of a chore.

14.2.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Antivirus scanning at rest with **Microsoft Defender for Storage** — once CVs land in Azure Blob, Defender can scan each new blob and quarantine matches.*
> - *Larger-than-30 MB uploads via **direct-to-blob with SAS** — the browser PUTs straight to a pre-signed blob URL, skipping the application server entirely. The default end-to-end limit (Container Apps ingress + Kestrel + ASP.NET Core multipart, all 30 MB) only matters for the through-the-server path.*
> - *Content stripping with **PdfPig** — parse the uploaded PDF and re-emit only text and known-safe constructs, dropping JavaScript and embedded files. Layer two on top of magic bytes.*
> - *The **OWASP File Upload Cheat Sheet** for the broader threat model: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html.*

14.2.7 Done!

The portal runs in the cloud. Anyone with the FQDN can browse jobs, submit applications, and walk them through the recruiter workflow. It works — until the next deploy, when the in-memory store and the uploaded PDFs both vanish. In the next exercise we replace the in-memory store with managed state in CosmosDB and the local files with Azure Blob, both authenticated via the Container App's managed identity. The controller code does not change; only the registrations in `Program.cs` do.

14.3 CosmosDB and Azure Blob via Managed Identity

14.4 CosmosDB and Azure Blob via Managed Identity

14.4.1 Goal

The previous exercise left you with `CloudCiCareers.Web` deployed at `https://ca-careers-week7.<env>.northeurope.azurecontainerapps.io/`, an OIDC-federated pipeline pushing to ACR on every commit, and App Insights ingesting telemetry. It also left you with two pieces of state that survive only as long as the current container does: `IApplicationStore` is registered as `InMemoryApplicationStore` (a `ConcurrentDictionary<string, Application>` in process memory), and `IBlobService` writes uploaded CVs to `./uploads/` on the local filesystem. Every revision rollover wipes both — that fragility was deliberate, and it exists to motivate this exercise.

This exercise replaces both with managed Azure services. Application records move into **CosmosDB serverless**; CV

PDFs move into **Azure Blob Storage**. Crucially, neither uses a connection string or an account key — both authenticate via the Container App's **system-assigned managed identity**. There is no `secretref:` for storage credentials here because there are no storage credentials to deliver. This is a deliberate contrast with the prior chapter's API key and JWT signing key, which lived behind `secretref:`. Those were shared secrets; managed identity is an attested compute identity. The trust-boundary shift is the load-bearing lesson.

WHAT YOU'LL LEARN:

- How to provision CosmosDB serverless and Azure Blob Storage from the CLI and grant data-plane access without account keys
- The difference between **Cosmos DB control-plane RBAC** (Azure RBAC roles like Cosmos DB Account Reader) and **Cosmos DB data-plane RBAC** (the GUID role definition `00000000-0000-0000-0000-000000000002`)
- How `DefaultAzureCredential` from `Azure.Identity` picks up the managed identity in Container Apps and your `az login` token on a developer laptop, with no code changes
- Why managed identity beats connection strings on every axis that matters — rotation, attribution, blast radius
- When `secretref:` is still the right answer (Application Insights ingestion) and when it stops being (Cosmos and Blob)
- How to register cloud implementations conditionally so `dotnet run` from localhost still works without any Azure resources

14.4.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ The previous exercise complete: `ca-careers-week7` deployed in `rg-careers-week7`, system-assigned managed identity already on, App Insights wired via `secretref:appinsights-connstr`
- ✓ The CI/CD workflow still wired up — `git push` on `main` deploys a new revision via OIDC federation
- ✓ Azure CLI signed in (`az account show` returns your subscription) with permission to create resources and assign roles in `rg-careers-week7`
- ✓ A local clone of `CloudCiCareers.Web` and a working .NET 10 SDK (`dotnet --version` reports a `10.*` version)
- ✓ The system-assigned managed identity's `principalId` accessible via `az containerapp show -n ca-careers-week7 -g rg-careers-week7 --query identity.principalId -o tsv`

14.4.3 Exercise Steps

Overview

1. **Provision the Storage Account and the `cvs` blob container**
2. **Provision a CosmosDB serverless account, database, and container**
3. **Grant managed identity Storage Blob Data Contributor on the Storage Account**
4. **Grant managed identity Cosmos DB data-plane access**
5. **Add the SDK packages**
6. **Refactor `IApplicationStore` to async and implement `CosmosApplicationStore`**

7. **Implement `AzureBlobService`**
8. **Wire DI in `Program.cs` with a dev/prod registration switch**
9. **Update Container App env vars (no `secretref:` — managed identity is the auth)**
10. **Push, deploy a new revision, verify persistence survives the rollover**
11. **Test Your Implementation**

Step 1: *Provision the Storage Account and the* `cvs` *blob container*

Storage comes first because the role assignment in Step 3 needs the account to already exist. Pick a 5–6 character random suffix for the account name — globally unique, lowercase, alphanumeric only. The example below uses `7g8h2j`; substitute your own and **keep using the same suffix in every command**. The `--allow-blob-public-access false` flag is non-negotiable: this account holds CVs containing personal data, and a public-access default would mean a single misconfigured container could leak everything.

1. **Create** the Storage Account in `rg-careers-week7`:

```
az storage account create -n stcareers7g8h2j -g rg-
        careers-week7 \
  -l northeurope --sku Standard_LRS --allow-blob-public-
        access false
```

2. **Create** the `cvs` container, authenticating as your signed-in user (not via account key):

```
az storage container create --account-name stcareer-
        s7g8h2j --name cvs --auth-mode login
```

CONCEPT DEEP DIVE: `--AUTH-MODE LOGIN`

By default `az storage` reaches for an account key — a string that grants full plane-wide access to every container, blob, queue, and table in the account. That key is exactly what this chapter is moving away from. `--auth-mode login` tells the CLI to authenticate as your signed-in Azure AD user instead, using the role assignments your subscription account already has. No account key is read, none is cached. The flag works on every `az storage` subcommand and is the right default for any chapter taking the no-shared-secrets posture seriously.

⚠ **Common Mistakes**

- Omitting `--auth-mode login` and silently falling back to account-key lookup — works locally but trains you for the wrong pattern.
- Picking a suffix with uppercase letters or hyphens. Storage account names allow only `a-z0-9` and must be 3–24 characters.

✓ **Quick check:** `az storage container list --account-name stcareers7g8h2j --auth-mode login -o table` lists exactly one container named `cvs`.

Step 2: *Provision a CosmosDB serverless account, database, and container*

CosmosDB has two billing modes: **provisioned throughput** (pre-purchase RU/s, pay whether you use them or not) and **serverless** (per-request billing, no minimum). Provisioned's 400 RU/s floor runs roughly 24 USD per month even idle; serverless runs cents per chapter. Use serverless. It also sidesteps the free-tier-already-claimed question — every subscription gets one free-tier Cosmos account, and if you've

used it elsewhere, provisioned stops being free. Serverless doesn't depend on the free-tier slot.

1. **Create** the CosmosDB account in serverless mode (same suffix as Step 1):

```
az cosmosdb create -n cosmos-careers-7g8h2j -g rg-
        careers-week7 \
  --capabilities EnableServerless --default-consistency-
        level Session
```

   This takes 4–6 minutes. Cosmos accounts are slow to provision because the control plane spins up regional infrastructure even for a single region.

2. **Create** the `careers` database:

```
az cosmosdb sql database create -a cosmos-
        careers-7g8h2j -g rg-careers-week7 -n careers
```

3. **Create** the `applications` container, partitioned on `/id`:

```
az cosmosdb sql container create -a cosmos-
        careers-7g8h2j -g rg-careers-week7 \
  -d careers -n applications --partition-key-path /id
```

CONCEPT DEEP DIVE: WHY SERVERLESS

Serverless charges per request unit consumed, not per provisioned RU/s. For an app that handles a handful of reads and writes per minute the total runs in cents per chapter, predictable and bounded. No RU/s tuning step, no autoscale curve, no idle cost. When the chapter ends and you delete `rg-careers-week7`, the resource group teardown deletes the account too — no "did I forget to scale this back down?" trap. Serverless trades off advanced features (no multi-region writes, no analytical store) that don't matter for a single-tenant teaching app.

⚠ **Common Mistakes**

- Forgetting `--capabilities EnableServerless`. The default is provisioned throughput, which silently costs 24 USD per month per database.
- Using a different suffix than the storage account. The rest of the chapter assumes both names share one suffix.

√ **Quick check:** `az cosmosdb sql container show -a cosmos-careers-7g8h2j -g rg-careers-week7 -d careers -n applications --query 'resource.partitionKey.paths' -o tsv` prints `/id`.

Step 3: *Grant managed identity Storage Blob Data Contributor on the Storage Account*

Storage uses **regular Azure RBAC**. The role is `Storage Blob Data Contributor` — data-plane only (read, write, delete blobs), not control-plane (no key rotation, no encryption-setting changes). Scope it to the Storage Account, not the subscription — the Container App's identity should not have access to *every* storage account in your subscription.

1. **Capture** the Container App's managed identity `principalId` and the Storage Account resource ID:

```
PRINCIPAL_ID=$(az containerapp show -n ca-careers-week7
        -g rg-careers-week7 --query iden-
        tity.principalId -o tsv)
STORAGE_ID=$(az storage account show -n stcareers7g8h2j
        -g rg-careers-week7 --query id -o tsv)
echo "PRINCIPAL_ID=$PRINCIPAL_ID  STOR-
        AGE_ID=$STORAGE_ID"
```

Both should be non-empty. `PRINCIPAL_ID` is a GUID; `STORAGE_ID` is an ARM resource path.

2. **Assign** the role at the Storage Account scope:

```
az role assignment create \
  --assignee "$PRINCIPAL_ID" \
  --role "Storage Blob Data Contributor" \
  --scope "$STORAGE_ID"
```

CONCEPT DEEP DIVE

"Data Contributor" vs "Account Contributor" is the load-bearing distinction. **Account Contributor** is control-plane: rename, rotate keys, change firewall rules. **Data Contributor** is data-plane: actually read and write blobs. The Container App needs only data-plane access; granting Account Contributor would mean a compromised app could rotate keys or delete the account. Always grant the narrowest data-plane role that does the job.

⚠ **Common Mistakes**

- Granting at subscription scope. Works, but the identity now has blob access to every storage account in your subscription — far more blast radius than needed.
- Granting `Storage Account Contributor` because the name sounds adjacent. That's control-plane and won't grant blob read/write at all.

✓ **Quick check:** `az role assignment list --assignee "$PRINCIPAL_ID" --scope "$STORAGE_ID" -o table` lists exactly one row with role `Storage Blob Data Contributor`.

Step 4: *Grant managed identity Cosmos DB data-plane access*

Cosmos DB has its own RBAC system parallel to Azure RBAC, and the day you discover this is usually the day you've already lost two hours to 403s. The role definition ID `00000000-0000-0000-0000-000000000002` is the built-in **Cosmos DB Built-in Data Contributor** — read, write, delete documents within the account. It is *not* discoverable via `az role definition list`; it lives in a separate registry queried via `az cosmosdb sql role definition list`.

1. **Assign** the data-plane role at the account scope:

```
az cosmosdb sql role assignment create \
  --account-name cosmos-careers-7g8h2j -g rg-careers-
        week7 \
  --scope "/" --principal-id "$PRINCIPAL_ID" \
  --role-definition-id
        00000000-0000-0000-0000-000000000002
```

2. **List** the data-plane role definitions for orientation:

```
az cosmosdb sql role definition list --account-name
        cosmos-careers-7g8h2j -g rg-careers-week7 -o
        table
```

Expected: two built-in entries — `Cosmos DB Built-in Data Reader` (`...0001`) and `Cosmos DB Built-in Data Contributor` (`...0002`).

CONCEPT DEEP DIVE: COSMOS DATA-PLANE RBAC VS CONTROL-PLANE RBAC

Why two RBAC systems? Because the operations split cleanly. **Control-plane operations** go through regular Azure RBAC: rename the database, increase RU/s, change consistency level, enable multi-region writes. Roles like `Cosmos DB Account Reader Role` and `DocumentDB Account Contributor` cover those. **Data-plane operations** — read, write, query, delete documents — go through the Cosmos-specific registry with GUID role definition IDs you'll never recognise on first sight. The split exists because data-plane permissions are enforced by the data-plane endpoints, not by ARM, and at a per-document granularity ARM doesn't model. The scope `/` here means "everything in this account"; tighter scopes are possible but rarely worth the complexity for a single-tenant app. The role-definition ID is mechanically the same shape as an Azure RBAC ID, but **lives in a different registry** — that's why `az role assignment list` won't show it. Use `az cosmosdb sql role assignment list` to see Cosmos data-plane assignments.

⚠ **Common Mistakes**

- Granting `Cosmos DB Account Reader Role` (control-plane) and expecting reads to work. They won't — that role lets you `az cosmosdb show`, not `ReadItemAsync`. Symptom: 403 on every data-plane SDK call.
- Forgetting `--scope "/"`. The scope is required, and `/` is the right value for "the whole account".
- Looking for the role in `az role definition list` and concluding it doesn't exist. Wrong registry.

√ **Quick check:** `az cosmosdb sql role assignment list --account-name cosmos-careers-7g8h2j -g rg-careers-week7 -o table` lists one row with `RoleDefinitionId` ending in `...0002`.

Step 5: *Add the data-plane SDK packages*

Four NuGet packages: the Cosmos SDK, the Blob SDK, `Azure.Identity` for `DefaultAzureCredential`, and an explicit `Newtonsoft.Json` reference. The first three are obvious; the fourth needs a one-paragraph explanation. `Microsoft.Azure.Cosmos` 3.x uses **Newtonsoft.Json internally** for document (de)serialisation — not `System.Text.Json`. The Cosmos targets file fails the build with `The Newtonsoft.Json package must be explicitly referenced with version >= 10.0.2` if you don't add it yourself, even though it's already a transitive dependency. Pinning it removes that surprise. `Azure.Identity` is the load-bearing one — it provides the credential type that picks up the managed identity in Container Apps and your `az login` token on a laptop, with no code changes.

1. **From the project root** of `CloudCiCareers.Web`:

```
dotnet add package Microsoft.Azure.Cosmos --version
        3.49.0
dotnet add package Azure.Storage.Blobs --version 12.24.0
dotnet add package Azure.Identity --version 1.13.1
dotnet add package Newtonsoft.Json --version 13.0.3
```

2. **Verify** with `dotnet list package` — all four should be present at the versions above.

CONCEPT DEEP DIVE: `DEFAULTAZURECREDENTIAL`

A chain-of-responsibility credential that tries auth sources in order and returns the first that works. The chain is roughly: env vars → workload identity → managed identity → Visual Studio → Azure CLI → interactive browser. In Container Apps the managed identity step succeeds; on a laptop with `az login` the CLI step succeeds. **The same code works in both environments without changes** — that's the value proposition.

⚠ **Common Mistakes**

- Using `DefaultAzureCredential` from a laptop without `az login`. The chain falls through to interactive browser auth.
- Mixing `Azure.Storage.Blobs` (data-plane SDK) with `Microsoft.Azure.Storage` (legacy SDK). Use only `Azure.Storage.Blobs`.

✓ **Quick check:** `dotnet build` succeeds with no missing-package errors.

Step 6: *Refactor* `IApplicationStore` *to async and implement* `CosmosApplicationStore`

The existing `IApplicationStore` has synchronous methods. The Cosmos SDK is async-only, so the cleanest path is to refactor the interface to async, update `InMemoryApplicationStore` to wrap each body in `Task.FromResult`, and update controllers to `await`. This is more code change than blocking with `.GetAwaiter().GetResult()` would be, but it's the right ergonomic fit — async is what real Cosmos code looks like. The `Application` domain itself is unchanged — same fields, same `ApplicationStatus` enum, same `string` Id (the previous exercise generated it as `Guid.NewGuid().ToString()`). All you add is two `Newtonsoft.Json` annotations — Cosmos 3.x uses Newtonsoft

internally — so Cosmos can find the partition-key value and so the status is stored as a readable string instead of an integer.

1. **Open** `Models/Application.cs` and add two attributes — `[JsonProperty("id")]` on `Id` (Cosmos requires the partition-key property to serialise to lowercase `id`), and `[JsonConverter(typeof(StringEnumConverter))]` on `Status` so the enum lands in Cosmos as `"Submitted"` instead of `0`. Keep every other field exactly as it was:

> `Models/Application.cs`

```
using Newtonsoft.Json;
using Newtonsoft.Json.Converters;

namespace CloudCiCareers.Web.Models;

public class Application
{
    [JsonProperty("id")]
    public string Id { get; set; } = string.Empty;
    public int JobId { get; set; }
    public string ApplicantName { get; set; } =
        string.Empty;
    public string ApplicantEmail { get; set; } =
        string.Empty;
    public string CvBlobName { get; set; } =
        string.Empty;
    public DateTimeOffset SubmittedAt { get; set; }
    [JsonConverter(typeof(StringEnumConverter))]
    public ApplicationStatus Status { get; set; }
    public string? Notes { get; set; }
}

public enum ApplicationStatus
{
    Submitted, UnderReview, Rejected, Hired
}
```

2. **Refactor** `Services/IApplicationStore.cs` to async signatures. Same five methods, same parameters, same `string` Id, same `ApplicationStatus` enum, same nullable `Notes` — only the return shapes change to `Task<…>`:

> `Services/IApplicationStore.cs`

```
using CloudCiCareers.Web.Models;

namespace CloudCiCareers.Web.Services;

public interface IApplicationStore
{
    Task<IEnumerable<Application>> GetAllAsync(Cancella-
        tionToken ct = default);
    Task<Application?> GetByIdAsync(string id, Cancella-
        tionToken ct = default);
    Task<Application> CreateAsync(Application applica-
        tion, CancellationToken ct = default);
    Task<bool> UpdateStatusAsync(string id, Application-
        Status newStatus, string? notes, Cancellation-
        Token ct = default);
    Task<bool> DeleteAsync(string id, CancellationToken
        ct = default);
}
```

3. **Update** `Services/InMemoryApplicationStore.cs` so each method wraps the existing logic in `Task.FromResult` — kept as the dev fallback:

> `Services/InMemoryApplication-Store.cs`

```
using CloudCiCareers.Web.Models;
using System.Collections.Concurrent;

namespace CloudCiCareers.Web.Services;

public class InMemoryApplicationStore : IApplication-
        Store
{
    private readonly ConcurrentDictionary<string,
        Application> _store = new();

    public Task<IEnumerable<Application>>
        GetAllAsync(CancellationToken ct = default)
        =>
        Task.FromResult<IEnumerable<Application>>(_store.Values.ToLis

    public Task<Application?> GetByIdAsync(string id,
        CancellationToken ct = default)
        => Task.FromResult(_store.TryGetValue(id, out
        var a) ? a : null);

    public Task<Application> CreateAsync(Application
        application, CancellationToken ct = default)
    {
        _store[application.Id] = application;
        return Task.FromResult(application);
    }

    public Task<bool> UpdateStatusAsync(string id,
        ApplicationStatus newStatus, string? notes,
        CancellationToken ct = default)
    {
        if (!_store.TryGetValue(id, out var existing))
        return Task.FromResult(false);
        existing.Status = newStatus;
        existing.Notes = notes;
        return Task.FromResult(true);
    }

    public Task<bool> DeleteAsync(string id, Cancella-
        tionToken ct = default)
        => Task.FromResult(_store.TryRemove(id, out _));
}
```

4. **Create** the production implementation. The `Id` property is a `string` (matching the partition-key path `/id`), so the Cosmos calls take the string directly — no `.ToString()` on a Guid:

> `Services/CosmosApplicationStore.cs`

```
using CloudCiCareers.Web.Models;
using Microsoft.Azure.Cosmos;
using System.Net;

namespace CloudCiCareers.Web.Services;

public class CosmosApplicationStore : IApplicationStore
{
    private readonly Container _container;

    public CosmosApplicationStore(CosmosClient client,
        IConfiguration config)
    {
        var db = config["Cosmos:Database"] ?? throw new
        InvalidOperationException("Cosmos:Database");
        var name = config["Cosmos:Container"] ?? throw
        new InvalidOperationException("Cosmos:Con-
        tainer");
        _container = client.GetContainer(db, name);
    }

    public async Task<IEnumerable<Application>> GetAllA-
        sync(CancellationToken ct = default)
    {
        var query = _con-
        tainer.GetItemQueryIterator<Application>(new
        QueryDefinition("SELECT * FROM c"));
        var results = new List<Application>();
        while (query.HasMoreResults)
            foreach (var item in await query.ReadNextA-
        sync(ct)) results.Add(item);
        return results;
    }

    public async Task<Application?> GetByIdAsync(string
        id, CancellationToken ct = default)
    {
        try
        {
            var r = await _con-
        tainer.ReadItemAsync<Application>(
                id, new PartitionKey(id), cancellation-
        Token: ct);
            return r.Resource;
        }
```

```
        catch (CosmosException ex) when (ex.StatusCode
        == HttpStatusCode.NotFound) { return null; }
    }

    public async Task<Application> CreateAsync(Applica-
        tion application, CancellationToken ct =
        default)
    {
        var r = await _container.CreateItemAsync(applic-
        ation, new PartitionKey(application.Id), can-
        cellationToken: ct);
        return r.Resource;
    }

    public async Task<bool> UpdateStatusAsync(string
        id, ApplicationStatus newStatus, string?
        notes, CancellationToken ct = default)
    {
        try
        {
            await _con-
        tainer.PatchItemAsync<Application>(
                id, new PartitionKey(id),
                patchOperations: new[]
                {
                    PatchOperation.Replace("/Status",
        newStatus.ToString()),
                    PatchOperation.Replace("/Notes",
        notes)
                },
                cancellationToken: ct);
            return true;
        }
        catch (CosmosException ex) when (ex.StatusCode
        == HttpStatusCode.NotFound) { return false; }
    }

    public async Task<bool> DeleteAsync(string id, Can-
        cellationToken ct = default)
    {
        try
        {
            await _con-
        tainer.DeleteItemAsync<Application>(id, new
        PartitionKey(id), cancellationToken: ct);
            return true;
```

```
        }
        catch (CosmosException ex) when (ex.StatusCode
        == HttpStatusCode.NotFound) { return false; }
    }
}
```

5. **Update** the two controllers so every call site of the old synchronous store methods becomes `await` of the async equivalent. The action methods that touch the store change from `IActionResult` to `async Task<IActionResult>`, and each call gets renamed and `await`'d (`_store.GetAll()` → `await _store.GetAllAsync()`, `_store.GetById(id)` → `await _store.GetByIdAsync(id)`, `_store.Create(application)` → `await _store.CreateAsync(application)`, `_store.UpdateStatus(id, newStatus, notes)` → `await _store.UpdateStatusAsync(id, newStatus, notes)`, `_store.Delete(id)` → `await _store.DeleteAsync(id)`). The compiler guides you — every old call site produces a build error pointing at exactly the line that needs updating. Touch `Controllers/JobsController.cs` and `Controllers/ApplicationsController.cs`.

CONCEPT DEEP DIVE: `PATCHITEMASYNC` OVER `REPLACEITEMASYNC`

`ReplaceItemAsync` requires reading the document, mutating the field, and writing the whole thing back — extra latency and a potential lost-update on races. `PatchItemAsync` is a single round trip and changes only the named properties. For a status + notes update that's the obvious win. Trade-off: patch operations are limited to a fixed set (replace, add, remove, increment) and can't do conditional updates without ETags.

The patch paths above use **PascalCase** (`/Status`, `/Notes`) because the model has no `[JsonProperty]` on those properties — Newtonsoft.Json defaults to PascalCase serialisation, so the document keys are `Status` and `Notes`. Only `Id` is renamed to lowercase `id` (because Cosmos requires that for the partition key). Patch paths are case-sensitive; mismatching them silently no-ops.

⚠ **Common Mistakes**

- Forgetting `[JsonProperty("id")]` on `Id`. Cosmos rejects writes with `Required property 'id' is missing`.
- Forgetting `[JsonConverter(typeof(StringEnumConverter))]` on `Status`. The status lands as `0`/`1`/`2`/`3` in Cosmos, the patch path then mismatches the type, and `az cosmosdb sql query` results become unreadable.
- Reaching for `System.Text.Json`'s `[JsonPropertyName]` instead of Newtonsoft's `[JsonProperty]`. Cosmos 3.x ignores STJ attributes — they apply to ASP.NET Core's HTTP serialisation but not to what Cosmos writes to disk.
- Using `new PartitionKey(applicationId)` with the wrong casing on the path. The partition-key path is `/id` (lowercase, set when you created the container in Step 2); the JSON property is also `id` because of

> `JsonPropertyName`. All consistent — keep them that way.
>
> √ **Quick check:** `dotnet build` succeeds. Every controller action that uses `_store` is `async Task<IActionResult>` and every store call has `await` in front of it.

Step 7: *Implement* **`AzureBlobService`**

The blob side mirrors the Cosmos side. The `IBlobService` interface keeps its existing async signatures. The new implementation takes a `BlobServiceClient` from DI, reads the container name from config, and forwards calls to `BlobContainerClient`. Same `DefaultAzureCredential` story — every call rides on the managed identity.

1. **Create** the production implementation:

> `Services/AzureBlobService.cs`

```
using Azure.Storage.Blobs;
using Azure.Storage.Blobs.Models;

namespace CloudCiCareers.Web.Services;

public class AzureBlobService : IBlobService
{
    private readonly BlobContainerClient _container;

    public AzureBlobService(BlobServiceClient client,
        IConfiguration config)
    {
        var name = config["Storage:Container"]
            ?? throw new
        InvalidOperationException("Storage:Container
        not configured");
        _container =
        client.GetBlobContainerClient(name);
    }

    public Task UploadAsync(string name, Stream con-
        tent, CancellationToken ct = default)
        => _container.GetBlobClient(name).UploadAsync(
            content,
            new BlobUploadOptions { HttpHeaders = new
        BlobHttpHeaders { ContentType = "application/
        pdf" } },
            cancellationToken: ct);

    public async Task<Stream> OpenReadAsync(string
        name, CancellationToken ct = default)
        => await _container.GetBlobClient(name).Open-
        ReadAsync(cancellationToken: ct);

    public Task DeleteAsync(string name, Cancellation-
        Token ct = default)
        => _container.GetBlobClient(name).DeleteIfExist-
        sAsync(cancellationToken: ct);
}
```

> **CONCEPT DEEP DIVE: SETTING `CONTENTTYPE` ON UPLOAD**
>
> Without `ContentType = "application/pdf"` the blob defaults to `application/octet-stream`, and browsers force a download instead of rendering the PDF inline. Setting it at upload time means the metadata is correct from the start; no rewrite later.
>
> ⚠ **Common Mistakes**
>
> - Calling `UploadAsync(stream, overwrite: true)`. GUID-based naming makes collisions improbable but catastrophic if they happen — keep the default `overwrite: false`.
>
> ✓ **Quick check:** `dotnet build` succeeds.

Step 8: *Wire DI in* `Program.cs` *with a dev/prod registration switch*

You now have two implementations of each interface. The registration logic picks between them based on whether `Cosmos:Endpoint` is configured: present → cloud implementations; absent → in-memory and local-file fallbacks. This keeps `dotnet run` working locally without any Azure resources, while the deployed Container App always picks the cloud path because its env vars are always set.

1. **Open** `Program.cs`.
2. **Add** the usings at the top:

> `Program.cs`

```
using Azure.Identity;
using Azure.Storage.Blobs;
using CloudCiCareers.Web.Services;
using Microsoft.Azure.Cosmos;
```

3. **Replace** the existing single-line registrations of `IApplicationStore` and `IBlobService` with the conditional block:

> `Program.cs`

```
var cosmosEndpoint = builder.Configuration["Cosmos:End-
        point"];
var blobEndpoint = builder.Configuration["Stor-
        age:BlobEndpoint"];

if (!string.IsNullOrWhiteSpace(cosmosEndpoint) && !
        string.IsNullOrWhiteSpace(blobEndpoint))
{
    // Production / cloud path: managed identity
        authenticates both clients.
    builder.Services.AddSingleton(_ =>
        new CosmosClient(cosmosEndpoint, new
        DefaultAzureCredential()));
    builder.Services.AddSingleton(_ =>
        new BlobServiceClient(new Uri(blobEndpoint),
        new DefaultAzureCredential()));

    builder.Services.AddSingleton<IApplicationStore,
        CosmosApplicationStore>();
    builder.Services.AddSingleton<IBlobService,
        AzureBlobService>();
}
else
{
    // Dev fallback: no Azure resources required for
        `dotnet run`.
    builder.Services.AddSingleton<IApplicationStore,
        InMemoryApplicationStore>();
    builder.Services.AddSingleton<IBlobService, Local-
        FileBlobService>();
}
```

> **CONCEPT DEEP DIVE: CONVENTION, NOT FEATURE FLAGS**
>
> This is deliberately the simplest possible "if cloud config is present, use cloud impls" switch — five lines of `if/else`, no library, no abstraction. Production-grade systems would use `IOptions<CloudStorageOptions>` with `[Required]` validation, fail-fast at startup if half the cloud config is provided, and centralise the decision in a typed extension method. The rough shape is here so the lesson is the registration logic, not a feature-flag library like `Microsoft.FeatureManagement`.
>
> ⚠ **Common Mistakes**
>
> - Using `IsDevelopment()` as the switch instead of "is the cloud config present?". A locally-run app pointed at real Azure resources would then refuse to use them. Configure based on configuration, not on environment name.
> - Registering both pairs unconditionally. You'd construct a `CosmosClient` that opens TCP connections at startup and fails if the endpoint is missing.
>
> ✓ **Quick check:** `dotnet build` succeeds. `dotnet run` from localhost (with no `Cosmos:Endpoint` set) starts and serves `/Applications` from the in-memory store.

Step 9: *Update Container App env vars*

Five env vars carry the cloud configuration, **none** of them a secret — managed identity replaces every connection string in this design. The Cosmos endpoint URL, the blob endpoint, the database, container, and account names are all public information. Compare with the previous chapter's API key and JWT signing key, both of which lived behind `secretref:` because they were shared secrets.

1. **Set** all five env vars in one `az containerapp update` call:

```
az containerapp update -n ca-careers-week7 -g rg-
        careers-week7 \
  --set-env-vars \
    Cosmos__Endpoint=https://cosmos-
        careers-7g8h2j.documents.azure.com:443/ \
    Cosmos__Database=careers Cosmos__Con-
        tainer=applications \
    Storage__BlobEndpoint=https://stcareer-
        s7g8h2j.blob.core.windows.net \
    Storage__Container=cvs
```

2. **Verify** the env vars are set with `value` (not `secretRef`):

```
az containerapp show -n ca-careers-week7 -g rg-careers-
        week7 \
  --query 'properties.template.containers[0].env' -o
        json
```

Expected: five new entries with `name` + `value` keys (the cloud config), plus the existing `secretRef: appinsights-connstr` entry.

> **CONCEPT DEEP DIVE: WHY MANAGED IDENTITY BEATS CONNECTION STRINGS**
>
> Three axes, all in managed identity's favour. **Rotation**: a connection string is "fresh" only when you rotate it; a managed identity token is reissued every hour automatically by the platform. **Attribution**: a connection string in a log line tells you "someone with this string did this"; a managed identity entry in Azure Activity Log tells you exactly which Container App revision did it. **Blast radius**: a connection string in `.env` or a Container Apps secret is a string that, if exfiltrated, lets an attacker call the storage account from anywhere; a managed identity token can only be obtained by the compute identity it's bound to.
>
> The exception is App Insights — the app-side telemetry SDK doesn't yet support managed identity on the ingestion path, so the connection string stays as a `secretref:`. Pick the right tool per resource: managed identity wherever the SDK supports it, `secretref:` for the cases that still need a shared secret.
>
> ⚠ **Common Mistakes**
>
> - Falling back to `Storage__ConnectionString=secretref:storage-connstr` "because it's easier". It is — for two minutes. Then you have a key to rotate, audit, and worry about leaking.
> - Putting `Cosmos__AccountKey` anywhere. There is no account key in this design.
>
> √ **Quick check:** None of the five new env vars use `secretRef`. The only `secretRef` in the env list is `APPLICATIONINSIGHTS_CONNECTION_STRING`.

Step 10: *Push, deploy, and verify persistence survives a revision rollover*

Push the code; the CI/CD workflow rebuilds, pushes to ACR, and updates the Container App. The new revision picks up the env vars from Step 9 and the role assignments from Steps

3 and 4. Once `Active`, submit a fresh application; then deliberately roll over and confirm it's still there.

1. **Commit and push:**

```
git add Models/Application.cs Services/ Controllers/
        Program.cs CloudCiCareers.Web.csproj
git commit -m "Move application store to Cosmos and CV
        blobs to Azure Blob Storage"
git push
gh run watch
```

   Expected: green workflow within a couple of minutes.

2. **Capture** the FQDN:

```
FQDN=$(az containerapp show -n ca-careers-week7 -g rg-
        careers-week7 \
  --query properties.configuration.ingress.fqdn -o tsv)
echo "$FQDN"
```

3. **Submit** a fresh application in the browser. Open `https://$FQDN/`, click any of the four jobs, fill in name + email, attach a small PDF, submit. After the redirect, you land on `https://$FQDN/Applications/Details/<id>` — copy that `<id>` for the next step.
4. **Force** a new revision rollover. Bumping a harmless env var is enough — Container Apps treats any template change as a new revision:

```
az containerapp update -n ca-careers-week7 -g rg-
        careers-week7 --set-env-vars BUMP=$(date +%s)
az containerapp revision list -n ca-careers-week7 -g rg-
        careers-week7 -o table
```

5. **Reload** `https://$FQDN/Applications/<id>` from Step 3. The application is still there — the Cosmos document and

the blob both survived the rollover. This is the moment the chapter's investment pays off.

6. **Verify** the data directly in Azure:

```
az cosmosdb sql query --account-name cosmos-
        careers-7g8h2j \
  --database-name careers --container-name applications
        --query-text "SELECT * FROM c"

az storage blob list --account-name stcareers7g8h2j --
        container-name cvs --auth-mode login -o table
```

> **QUICK CHECK:**
> The application submitted before the rollover survives the rollover. The Cosmos query and blob list both show the record.

Step 11: *Test Your Implementation*

Walk every signal end to end so you finish knowing both stores work, the managed identity is the auth path, and App Insights sees the dependencies.

1. **Submit** a fresh application from `https://$FQDN/` (pick any job, fill the apply form) with a recognisable name like `Test Sigrid Larsson`.
2. **Force** a revision rollover (`az containerapp update -n ca-careers-week7 -g rg-careers-week7 --set-env-vars BUMP=$(date +%s)`) and confirm the application is still listed at `https://$FQDN/Applications`.
3. **Verify** Cosmos has the document:

```
az cosmosdb sql query \
  --account-name cosmos-careers-7g8h2j \
  --database-name careers --container-name applications
        \
  --query-text "SELECT * FROM c WHERE c.candidateName =
        'Test Sigrid Larsson'"
```

4. **Verify** the blob exists:

```
az storage blob list --account-name stcareers7g8h2j --
        container-name cvs --auth-mode login -o table
```

   Expected: a `<guid>.pdf` whose name matches `cvBlobName` from Step 3.

5. **Open** the detail page `https://$FQDN/Applications/<id>` and click the CV link. The PDF renders inline (because `ContentType` was set at upload). The link is proxied through `IBlobService.OpenReadAsync`, not a direct blob URL — that's how the managed-identity path stays in force.

6. **Check** App Insights sees the dependency calls:

```
az monitor app-insights query \
  --app cloudci-careers-insights -g rg-careers-week7 \
  --analytics-query "dependencies | where timestamp >
        ago(15m) | project timestamp, type, name,
        success | take 20"
```

   Expected: rows with `type = "Azure DocumentDB"` (Cosmos) and `type = "Azure blob"` (Storage), all `success = true`.

√ ***Success indicators:***

- *The browser flow (submit → list → detail → CV inline) works against the deployed FQDN*
- *A revision rollover does not lose the data*
- *Cosmos and Blob both contain the records you submitted*
- *App Insights shows successful* `Azure DocumentDB` *and* `Azure blob` *dependency entries*
- *Nothing in* `az containerapp show ... --query 'properties.template.containers[0].env'` *carries a* `secretRef` *for storage or Cosmos*

√ ***Final verification checklist:***

- [ ] *Storage Account* `stcareers<rand>` *and container* `cvs` *exist*
- [ ] *CosmosDB serverless account,* `careers` *database,* `applications` *container with* `/id` *partition key all exist*
- [ ] `Storage Blob Data Contributor` *role is assigned to the Container App's principal at the Storage Account scope*
- [ ] `Cosmos DB Built-in Data Contributor` *(*`...0002`*) is assigned to the same principal at account scope*
- [ ]

> `Models/Application.cs` *carries* `[JsonPropertyName("id")]` *on* `Id`
>
> ☐ `IApplicationStore` *is async;* `InMemoryApplicationStore` *and* `CosmosApplicationStore` *both implement it*
>
> ☐ `Program.cs` *switches on* `Cosmos:Endpoint` *to pick cloud vs in-memory implementations*
>
> ☐ *Five env vars on the Container App carry the cloud config; none uses* `secretRef`
>
> ☐ *A new revision rollover does not wipe applications or CVs*
>
> ☐ *The CV link on the detail page renders the PDF inline*

14.4.4 Common Issues

> ***If you encounter problems:***
> ***403 on the very first request after role assignment.*** *Azure RBAC and Cosmos data-plane RBAC both have propagation lag of 5–30 seconds; retry. If 403s persist beyond a minute, re-run* `az role assignment list` *and* `az cosmosdb sql role assignment list`*.*
> `CosmosException: Unauthorized` ***mentioning "Request blocked by Auth".*** *Wrong Cosmos data-plane role definition ID. The right value is* `00000000-0000-0000-0000-000000000002`*. Regular Azure RBAC roles like* `DocumentDB Account Contributor` *don't apply to data-plane calls.*
> ***Reads return zero results but writes succeed.*** *Partition key path is* `/Id` *(capital) but the JSON property is* `id` *(lowercase), or vice versa. Confirm* `[JsonPropertyName("id")]` *and* `--partition-key-path /id`*.*
> `Required property 'id' is missing` ***on writes.*** `[JsonPropertyName("id")]` *is missing from* `Models/Application.cs`*.*
> ***Local*** `dotnet run` ***hangs at startup.*** `DefaultAzureCredential` *is trying every source and one is timing out. Confirm* `az login` *works and* `az account show` *returns the right subscription. If on multiple subscriptions,* `az account set --subscription <id>` *to disambiguate.*
> ***The CV link downloads instead of rendering inline.*** *Blob* `ContentType` *wasn't set. Re-upload with* `BlobHttpHeaders { ContentType = "application/pdf" }`*.*

> ***Still stuck?*** *Check in order: are the role assignments listed (Steps 3 and 4)? Is* `[JsonPropertyName("id")]` *on the* `Id` *property? Are the env vars set with* `value`*, not* `secretRef`*?*

14.4.5 Summary

You replaced the in-memory application store with **CosmosDB serverless** and the local-file blob store with **Azure Blob Storage**. Both authenticate via the Container App's system-assigned managed identity, with **regular Azure RBAC** for Storage (`Storage Blob Data Contributor`) and **Cosmos data-plane RBAC** (`Cosmos DB Built-in Data Contributor`, role definition `...0002`) for Cosmos. No connection strings, no `secretref:`, no shared keys. The DI registration switches on whether the cloud config is present, so `dotnet run` still works on a developer laptop without any Azure resources.

- √ CosmosDB serverless + database + container with `/id` partition key
- √ Azure Blob Storage with `--allow-blob-public-access false` and a `cvs` container
- √ Two role assignments: regular Azure RBAC for Storage, Cosmos-specific RBAC for Cosmos
- √ `DefaultAzureCredential` from `Azure.Identity` — picks managed identity in Azure, `az login` token locally, no code changes
- √ Async `IApplicationStore` interface with both `InMemoryApplicationStore` (dev) and `CosmosApplicationStore` (prod)
- √ Conditional DI registration in `Program.cs` based on `Cosmos:Endpoint` presence

- ✓ Five non-secret env vars carrying the cloud configuration; App Insights `secretref:` stays where it is

> **KEY TAKEAWAY:**
>
> Managed identity is the right answer wherever the Azure service supports it. The trust boundary moves from "anyone holding this string" to "this specific compute identity, attested by Azure", and rotation, attribution, and blast radius all improve. The remaining `secretref:` for App Insights is the exception that proves the rule — use the platform-attested identity wherever you can; reserve shared secrets for the resources whose SDKs don't yet support the better path.

14.4.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - ***Cross-region writes**: enable Cosmos multi-region writes for a globally-distributed setup. Serverless doesn't support it; switch to provisioned throughput first.*
> - ***Private endpoints**: put the Storage Account and Cosmos account behind private endpoints in a VNet so the public endpoint goes away entirely. Container Apps environments can be deployed into a VNet to reach them.*
> - ***Customer-managed keys (CMK)**: replace the platform-managed encryption keys on the Storage Account with keys you hold in Key Vault, with the Container App's identity granted `wrap`/`unwrap` permissions on the key.*
> - ***Azure Cosmos DB analytical store + Synapse Link**: enable the column-store sibling of the transactional store and query application records from a Synapse workspace without affecting the OLTP path. Note: not available in serverless mode.*

14.4.7 Done!

The application's state survives the box it runs on. CosmosDB holds the records, Azure Blob Storage holds the CVs, and both are reachable only via the Container App's managed identity — no connection strings to leak, no keys to rotate manually, no `.env` file to forget. Revision rollovers, scale-to-zero recoveries, full redeploys: all lose nothing.

In the next exercise we add deep health probes that distinguish liveness (am I running?) from readiness (am I able to

serve real traffic?), wire them as Container Apps probes so the platform stops routing to revisions that have lost their dependencies, and end the chapter with a cleanup substep that tears down the resource group cleanly.

14.5 Deep Health Probes and Cleanup

14.6 Deep Health Probes and Cleanup

14.6.1 Goal

In the previous exercise you finished wiring `CloudCiCareers.Web` to Cosmos and Blob via the Container App's system-assigned managed identity. `CosmosApplicationStore` and `AzureBlobService` are the registered production implementations, the App Insights connection is still in place, the pipeline still rolls a fresh image on every push to `main`, and persistence survives revision rollovers. What's missing is a way for the platform — and for you — to tell whether the running replica is actually healthy. There are no health-check endpoints in the app, and Container Apps' liveness and readiness probes are at their defaults, which means the platform infers liveness from the TCP listener alone. A replica whose Cosmos data-plane role assignment got revoked still listens on port 8080 and still receives traffic; it just returns 500 on every request that touches the database.

In this exercise you'll add three health endpoints with three distinct semantics, wire two of them as Container Apps probes, and end the chapter with a cleanup substep that tears down `rg-careers-week7` and the `github-cloudci-careers-oidc` app registration in Entra. The three endpoints are `/health/live` (liveness — answers "is this process alive"), `/health/ready` (readiness — answers

"is this replica ready to serve traffic, given its dependencies"), and `/health` (diagnostic — JSON breakdown for humans, dashboards, and on-call investigation). The first two are consumed by the platform; the third is consumed by people.

This is the last exercise of the chapter. Once the cleanup substep finishes, the cloud lab is fully torn down and you can re-run the chapter from a clean slate any time.

WHAT YOU'LL LEARN:

- How to register the ASP.NET Core health-checks pipeline and tag individual checks so different endpoints expose different subsets
- Why liveness must answer "is *this process* alive" and must NOT depend on external services
- Why readiness gates on dependencies, and how to wire it into Container Apps so the platform drains traffic from a degraded replica
- How to write a tiny custom `IHealthCheck` that probes Cosmos and Blob with explicit timeouts
- How to hand-roll a JSON `ResponseWriter` for the diagnostic endpoint, and which library replaces it in production
- How to tear down both subscription resources (resource group) and tenant-level identity objects (Entra app registration) so the lab leaves no orphans

14.6.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- √ The previous exercise complete: `CloudCiCareers.Web` deployed to `https://ca-careers-week7.<env>.northeurope.azurecontainerapps.io/`, with `CosmosApplicationStore` and `AzureBlobService` registered as the production implementations, both authenticated via the Container App's system-assigned managed identity
- √ Cosmos and Storage reachable from the running container; persistence verified to survive revision rollovers
- √ App Insights still wired and the CI/CD pipeline still rolling a fresh image on every push to `main`
- √ The Azure CLI signed in (`az account show` returns your subscription)
- √ A local clone of `CloudCiCareers.Web` where you can edit code and push to `main`
- √ `curl` and `jq` available on your machine

14.6.3 Exercise Steps

Overview

1. **Add the health-checks package and registration scaffold**
2. **Implement `CosmosHealthCheck`**
3. **Implement `BlobHealthCheck`**
4. **Map the three endpoints with distinct predicates**
5. **Hand-roll the JSON `ResponseWriter`**
6. **Push, deploy, and verify the endpoints respond**
7. **Update the smoke test in the workflow to target `/health/live`**
8. **Wire Container Apps liveness and readiness probes**

9. **Tear down the cloud resources**
10. **Test Your Implementation**

Step 1: *Register the health-checks pipeline scaffold*

The ASP.NET Core health-checks pipeline lives in `Microsoft.Extensions.Diagnostics.HealthChecks` — a package that's already on the implicit `Microsoft.AspNetCore.App` framework reference, so no `dotnet add package` is required. Register the services once with `AddHealthChecks()`, then attach individual checks via `.AddCheck(...)`. The first check is the always-true `self` probe — it reports `Healthy` unconditionally and is the load-bearing check behind `/health/live`. Liveness asks one question: "is this process running and able to execute code?" If the always-true delegate returns, the answer is yes.

1. **Open** `Program.cs`.
2. **Register** the health-checks pipeline alongside the other service registrations:

> `Program.cs`

```
builder.Services.AddHealthChecks()
    .AddCheck("self", () => HealthCheckRes-
        ult.Healthy());
```

3. **Add** the `using` statement at the top of the file:

> `Program.cs`

```
using Microsoft.Extensions.Diagnostics.HealthChecks;
```

> **CONCEPT DEEP DIVE**
>
> The `self` check seems trivial — and it is — but it's the right primitive for liveness. The whole point of liveness is to ask a question that has nothing to do with dependencies: if the delegate runs, the process is alive. The endpoint that maps to this check returns 200 the moment ASP.NET Core can execute middleware, and 503 only if the runtime is so wedged that even an always-true callback can't return. That's exactly what Container Apps wants to consume to decide whether to restart the replica.
>
> ✓ **Quick check:** `dotnet build` succeeds with no new warnings.

Step 2: *Implement* `CosmosHealthCheck`

`/health/ready` needs to know whether Cosmos is reachable from this replica. A custom `IHealthCheck` is the right shape — it has access to the DI container (so it can resolve `CosmosClient` and `IConfiguration`), it returns a `HealthCheckResult` that the pipeline aggregates with the other checks, and it's where you put the explicit timeout so a hung Cosmos call doesn't freeze the readiness endpoint.

1. **Navigate to** the `Services` directory.
2. **Create a new file** named `CosmosHealthCheck.cs`.
3. **Add the following code:**

> `Services/CosmosHealthCheck.cs`

```
using Microsoft.Azure.Cosmos;
using Microsoft.Extensions.Diagnostics.HealthChecks;

namespace CloudCiCareers.Web.Services;

public class CosmosHealthCheck : IHealthCheck
{
    private readonly CosmosClient _client;
    private readonly string _databaseName;
    private readonly string _containerName;

    public CosmosHealthCheck(CosmosClient client, ICon-
        figuration config)
    {
        _client = client;
        _databaseName = config["Cosmos:Database"]!;
        _containerName = config["Cosmos:Container"]!;
    }

    public async Task<HealthCheckResult> Check-
        HealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        using var cts = CancellationTokenSource.Cre-
        ateLinkedTokenSource(cancellationToken);
        cts.CancelAfter(TimeSpan.FromSeconds(5));

        try
        {
            var container = _client.GetContainer(_data-
        baseName, _containerName);
            await container.ReadContainerAsync(cancella-
        tionToken: cts.Token);
            return HealthCheckResult.Healthy("Cosmos
        reachable");
        }
        catch (Exception ex)
        {
            return HealthCheckResult.Unhealthy("Cosmos
        unreachable", ex);
        }
```

```
    }
}
```

4. **Open** `Program.cs` and extend the existing health-checks registration:

> `Program.cs`

```
builder.Services.AddHealthChecks()
    .AddCheck("self", () => HealthCheckResult.Healthy())
    .AddCheck<CosmosHealthCheck>("cosmos", tags: new[]
        { "ready" });
```

CONCEPT DEEP DIVE: HEALTH-CHECK TIMEOUTS

Without an explicit timeout, a hung Cosmos call freezes the readiness endpoint, which makes Container Apps think the app is failing. The 5-second `CancellationTokenSource` here is the maximum time the check is willing to wait before declaring the dependency unreachable. Tune the value based on the dependency's SLA — Cosmos in the same region typically responds in tens of milliseconds, so 5 seconds is generous. For a slower or noisier dependency, bump it; for a fast in-VNet service, lower it. The point is that the timeout exists at all.

⚠ **Common Mistakes**

- Letting the check inherit the ambient request cancellation token without adding its own deadline. The pipeline's outer timeout is much longer than what you want for a single-dependency probe.
- Returning `Healthy` on a swallowed exception. If `ReadContainerAsync` throws, the dependency is *not* healthy from this replica's point of view; surface that.

√ **Quick check:** `dotnet build` succeeds. The new check is registered with the tag `"ready"`.

Step 3: *Implement* `BlobHealthCheck`

The same shape as the Cosmos check, except the probe is `BlobContainerClient.ExistsAsync(...)`. That call is cheap — it's a HEAD on the container — and it covers the two failure modes that matter: the storage account is unreachable, or the managed identity can't authenticate against it.

1. **Navigate to** the `Services` directory.
2. **Create a new file** named `BlobHealthCheck.cs`.
3. **Add the following code:**

> `Services/BlobHealthCheck.cs`

```
using Azure.Storage.Blobs;
using Microsoft.Extensions.Diagnostics.HealthChecks;

namespace CloudCiCareers.Web.Services;

public class BlobHealthCheck : IHealthCheck
{
    private readonly BlobServiceClient _client;
    private readonly string _containerName;

    public BlobHealthCheck(BlobServiceClient client,
        IConfiguration config)
    {
        _client = client;
        _containerName = config["Storage:Container"]!;
    }

    public async Task<HealthCheckResult> Check-
        HealthAsync(
        HealthCheckContext context,
        CancellationToken cancellationToken = default)
    {
        using var cts = CancellationTokenSource.Cre-
        ateLinkedTokenSource(cancellationToken);
        cts.CancelAfter(TimeSpan.FromSeconds(5));

        try
        {
            var containerClient = _client.GetBlobCon-
        tainerClient(_containerName);
            var response = await containerClient.Exist-
        sAsync(cancellationToken: cts.Token);
            return response.Value
                ? HealthCheckResult.Healthy("Blob con-
        tainer reachable")
                : HealthCheckResult.Unhealthy("Blob con-
        tainer missing");
        }
        catch (Exception ex)
        {
            return HealthCheckResult.Unhealthy("Blob
        unreachable", ex);
        }
```

```
    }
}
```

4. **Extend** the registration in `Program.cs`:

> `Program.cs`

```
builder.Services.AddHealthChecks()
    .AddCheck("self", () => HealthCheckResult.Healthy())
    .AddCheck<CosmosHealthCheck>("cosmos", tags: new[]
        { "ready" })
    .AddCheck<BlobHealthCheck>("blob", tags: new[] {
        "ready" });
```

COMMON MISTAKES

- Using `GetPropertiesAsync` instead of `ExistsAsync`. `GetPropertiesAsync` requires `Storage Blob Data Reader` at minimum and pulls more data; `ExistsAsync` is cheaper and works with the same RBAC role you already granted the managed identity in the previous exercise.
- Forgetting the `tags: new[] { "ready" }` argument. Without the tag, the check still runs on `/health` (the diagnostic endpoint) but is invisible to `/health/ready` because the predicate filters by tag.

√ **Quick check:** Both custom checks build, both are registered, and both carry the `"ready"` tag.

Step 4: *Map the three endpoints with distinct predicates*

This is the load-bearing piece of the exercise: three endpoints, three audiences, three predicates. `/health/live` runs no checks at all — its predicate is `_ => false`, which means "include nothing." If the pipeline can run the predicate, the process is alive; that's the entire assertion. `/health/ready` runs only checks tagged `"ready"`, so it gates on

Cosmos and Blob. `/health` runs every registered check and uses a custom JSON writer to render a per-check breakdown.

1. **Open** `Program.cs`.
2. **Locate** the existing endpoint mappings (after `var app = builder.Build();`).
3. **Add** the three health-check mappings before `app.MapControllers()` (or wherever your other endpoint mappings live):

> `Program.cs`

```
app.MapHealthChecks("/health/live", new HealthCheckOp-
        tions
{
    Predicate = _ => false   // No checks. Process is
        alive — that's all this endpoint asserts.
});

app.MapHealthChecks("/health/ready", new HealthCheckOp-
        tions
{
    Predicate = c => c.Tags.Contains("ready")
});

app.MapHealthChecks("/health", new HealthCheckOptions
{
    ResponseWriter = WriteJsonResponse
});
```

4. **Add** the `using` for `HealthCheckOptions`:

> `Program.cs`

```
using Microsoft.AspNetCore.Diagnostics.HealthChecks;
```

5. **Leave** `WriteJsonResponse` undefined for now — Step 5 adds it as a static local method in the same file.

> **CONCEPT DEEP DIVE: THREE ENDPOINTS, THREE AUDIENCES**
>
> The temptation is to ship a single `/health` endpoint and wire every consumer to it. Resist. Each of the three endpoints answers a different question for a different consumer.
>
> `/health/live` is for the platform's liveness probe. The platform reads it and decides whether to restart the replica. The right answer is "yes, the process is alive" or "no, restart it." Anything more nuanced — "Cosmos is degraded but the process is fine" — confuses the platform into restarting replicas that don't need restarting.
>
> `/health/ready` is for the platform's readiness probe. The platform reads it and decides whether to send the replica traffic. If Cosmos is unreachable from this replica, traffic should drain to other replicas (which may have working connections) until the dependency recovers.
>
> `/health` is for *humans* — dashboards, alerting, on-call investigation. It returns a per-check JSON breakdown. It is **not** wired as a probe. Container Apps shouldn't react to "blob check is degraded but cosmos is fine"; that's a human signal that needs human judgement.
>
> Three endpoints, three audiences. That distinction is what makes the difference between a service that recovers gracefully when its dependencies wobble and one that flaps on every transient hiccup.
>
> ✓ **Quick check:** All three mappings compile. The endpoints aren't reachable yet because `WriteJsonResponse` doesn't exist; Step 5 fixes that.

Step 5: *Hand-roll the JSON* `ResponseWriter`

The default response writer for `MapHealthChecks` writes the plain-text status (`Healthy`, `Degraded`, `Unhealthy`) and nothing else. That's fine for `/health/live` and `/health/ready` — the platform only cares about the HTTP status code,

not the body — but `/health` is for humans, and humans want a per-check breakdown. Hand-roll a tiny `System.Text.Json` writer; ~10 lines, no extra NuGet dependency.

1. **Open** `Program.cs`.
2. **Add** the `using`:

> `Program.cs`

```
using System.Text.Json;
```

3. **Add** the static helper at the bottom of `Program.cs` (after `app.Run()` is fine — top-level files allow trailing static locals via a regular method declaration):

> `Program.cs`

```
static Task WriteJsonResponse(HttpContext ctx, Heal-
        thReport report)
{
    ctx.Response.ContentType = "application/json";
    var payload = JsonSerializer.Serialize(new
    {
        status = report.Status.ToString(),
        checks = report.Entries.Select(e => new
        {
            name = e.Key,
            status = e.Value.Status.ToString(),
            duration_ms = (int)e.Value.Duration.Total-
        Milliseconds
        })
    });
    return ctx.Response.WriteAsync(payload);
}
```

4. **Verify** the shape of the JSON. Locally, `dotnet run` then `curl -s http://localhost:<port>/health | jq .` should return:

```
{
  "status": "Healthy",
  "checks": [
    {"name": "self", "status": "Healthy",
        "duration_ms": 0},
    {"name": "cosmos", "status": "Healthy", "dura-
        tion_ms": 12},
    {"name": "blob", "status": "Healthy",
        "duration_ms": 8}
  ]
}
```

CONCEPT DEEP DIVE

Real services usually take a NuGet dependency on `AspNetCore.HealthChecks.UI.Client` and use `UIResponseWriter.WriteHealthCheckUIResponse` instead of a hand-rolled writer. That writer produces a richer schema (durations, descriptions, tags, exceptions in non-production environments) and is the canonical shape parsed by the `AspNetCore.HealthChecks.UI` dashboard. Hand-rolling here keeps the dependency surface small and shows what the writer is actually doing under the hood; once you've seen the moving parts, switching to the library writer is a one-line change.

√ **Quick check:** Locally, `curl -s http://localhost:<port>/health | jq .` returns the JSON shown above with `"status": "Healthy"` and three entries.

Step 6: *Push, deploy, and verify the endpoints respond*

The endpoint code is ready. Push to `main`, let the pipeline roll a new revision, and verify all three endpoints respond correctly against the deployed Container App.

1. **Commit and push:**

```
git add CloudCiCareers.Web.csproj Program.cs Services/
        CosmosHealthCheck.cs Services/Blob-
        HealthCheck.cs
git commit -m "Add deep health probes for liveness,
        readiness, and diagnostics"
git push
gh run watch
```

The workflow should turn green in a couple of minutes.

2. **Capture** the FQDN:

```
FQDN=$(az containerapp show \
  -g rg-careers-week7 -n ca-careers-week7 \
  --query properties.configuration.ingress.fqdn -o tsv)

echo "$FQDN"
```

3. **Verify** all three endpoints with `curl`:

```
curl -is "https://$FQDN/health/live"   # 200, body
        "Healthy"
curl -is "https://$FQDN/health/ready"  # 200, body
        "Healthy"
curl -s  "https://$FQDN/health" | jq . # full JSON
```

COMMON MISTAKES

- Hitting the API before the new revision is active. `az containerapp revision list -g rg-careers-week7 -n ca-careers-week7 -o table` shows which revision serves traffic.
- `/health/ready` returning `503` from a fresh deploy because role-assignment propagation is still settling. Wait a minute and retry; the smoke-test loop in Step 7 handles this automatically.

√ **Quick check:** Three responses: `/health/live` and `/health/ready` return 200 with body `Healthy`; `/health` returns a JSON document with three entries, all `Healthy`.

Step 7: *Update the smoke test in the workflow to target* `/health/live`

The earlier chapters smoke-tested `/` (the home page). That worked, but `/health/live` is the better probe target now that it exists — it doesn't depend on Razor view rendering, doesn't depend on session state, and doesn't need any database round-trip. It's the cleanest "is the process up" assertion the app exposes.

The order matters: push the code change first (Step 6) so `/health/live` is reachable on the deployed revision *before* the workflow change goes in. Otherwise the very first deploy of the new workflow smoke-tests an endpoint that doesn't exist yet on the running revision.

1. **Open** the workflow file at `.github/workflows/ci.yml`.
2. **Locate** the smoke-test step that hits `/`.
3. **Replace** the smoke-test loop body with one that targets `/health/live`:

> `.github/workflows/ci.yml`

```yaml
- name: Smoke test
  run: |
    for i in {1..20}; do
      curl -fsS https://$FQDN/health/live && break ||
        sleep 6
    done
```

4. **Commit and push:**

```bash
git add .github/workflows/ci.yml
git commit -m "Smoke-test /health/live instead of /"
git push
gh run watch
```

COMMON MISTAKES

- Pushing the workflow change *before* the code change. The deploy succeeds, the smoke test runs against `/health/live`, the endpoint doesn't exist yet on the live revision, and the workflow turns red. Push code first, watch deploy succeed, then update the smoke target.
- Using `/health` (the diagnostic endpoint) as the smoke target. The 200/503 mapping for that endpoint is "did at least one check fail?" — which is the wrong signal for "is the deploy healthy enough to call green?" `/health/live` is the right target.

√ **Quick check:** The smoke-test step in the workflow file targets `/health/live`. The next pipeline run is green, with the smoke-test step passing on the first or second iteration.

Step 8: *Wire Container Apps liveness and readiness probes*

The endpoints exist; the platform doesn't know to consume them yet. `az containerapp update --probe-type Liveness` adds a liveness probe to the container template; the same command with `--probe-type Readiness` adds a

readiness probe. Both go on port 8080 (the container's listening port) and use the paths from Step 4.

1. **Wire** the liveness probe:

```
az containerapp update -n ca-careers-week7 -g rg-
        careers-week7 \
  --probe-type Liveness \
  --probe-path /health/live \
  --probe-port 8080 \
  --probe-period-seconds 10 \
  --probe-timeout-seconds 3 \
  --probe-failure-threshold 3
```

2. **Wire** the readiness probe:

```
az containerapp update -n ca-careers-week7 -g rg-
        careers-week7 \
  --probe-type Readiness \
  --probe-path /health/ready \
  --probe-port 8080 \
  --probe-period-seconds 10 \
  --probe-timeout-seconds 3 \
  --probe-failure-threshold 3
```

3. **Verify** the probes are wired:

```
az containerapp show \
  -g rg-careers-week7 -n ca-careers-week7 \
  --query 'properties.template.containers[0].probes' -o
        json
```

The output should show two probes: one with `type: Liveness` and `httpGet.path: /health/live`, one with `type: Readiness` and `httpGet.path: /health/ready`. Both on port 8080.

CONCEPT DEEP DIVE: WHY LIVENESS MUST NOT DEPEND ON EXTERNAL SERVICES

The temptation is to point liveness at the same endpoint as readiness — "if anything is wrong, restart the replica." That is the wrong instinct, and it's the source of the most expensive production outages.

Imagine Cosmos is down. If liveness checks Cosmos, every replica fails its liveness probe, every replica gets killed, and every replacement replica fails its first liveness probe and gets killed before it can serve a single request. The platform is now in a restart loop that does nothing to bring Cosmos back AND prevents the few read-only paths that didn't need Cosmos from serving traffic at all. The blast radius is amplified — a Cosmos outage becomes an application outage, plus a thundering-herd recovery once Cosmos returns.

Liveness asks "is *this process* alive?" — restart trigger. Readiness asks "should I send this replica traffic right now?" — traffic gate. Two different questions, two different probes, two different endpoints.

Optional operational simulation — if you have a few extra minutes, try this. Remove the Cosmos data-plane role assignment to deliberately break readiness:

```
# Find the role assignment
SP=$(az containerapp identity show -g rg-careers-
        week7 -n ca-careers-week7 \
     --query principalId -o tsv)
COSMOS=$(az cosmosdb list -g rg-careers-week7 --
        query '[0].name' -o tsv)
az cosmosdb sql role assignment list \
  --account-name "$COSMOS" -g rg-careers-week7 -o
        table
# Delete the matching assignment, then watch
        readiness flip:
while true; do curl -is "https://$FQDN/health/ready"
        | head -1; sleep 5; done
        # Ctrl+C to stop. macOS users without
        `watch` can keep this loop running in a side
        terminal.
```

Within a probe period, `/health/ready` returns 503 and the platform stops sending the replica traffic. Liveness keeps returning 200 the whole time — the process is fine; it just can't serve. Restore the role assignment with `az cosmosdb sql role assignment create ...` and watch recovery.

⚠ **Common Mistakes**

- Forgetting `--probe-port 8080`. The default port for the probe is not the container's listening port; if you omit it, every probe lands somewhere the container isn't listening, every probe fails, and you've engineered a restart loop.
- Wiring `/health` (the diagnostic endpoint) as a probe. The 200/503 mapping is "did at least one check fail?" — which is wrong for liveness (it shouldn't depend on Cosmos) and slightly wrong for readiness (it includes the trivial `self` check, which doesn't matter). Use the dedicated paths.
- Setting `--probe-failure-threshold 1`. One failed probe should not be enough to kill a replica; transient blips happen. Three is a reasonable default.

> √ **Quick check:** `az containerapp show ... --query 'properties.template.containers[0].probes'` shows both a Liveness and a Readiness probe wired to the right paths and port 8080.

Step 9: *Tear down the cloud resources*

This is the last exercise of the chapter and the last exercise that uses the Week 7 cloud resources. Nothing later in the course needs `rg-careers-week7`, the Container App, the storage account, the Cosmos account, the ACR, or the Entra app registration created for OIDC federation. Tear all of it down so you finish with no resources running and no orphaned tenant-level identities.

The work splits into two homes — the Azure subscription holds the running resources, and Microsoft Entra ID (a tenant-level service) holds the identity the pipeline used. Deleting the resource group does **not** delete the Entra app registration. They live in different planes; cleanup needs two commands.

1. **Capture** the Entra app's `appId` before you delete it:

```
APP_ID=$(az ad app list \
  --display-name github-cloudci-careers-oidc \
  --query "[0].appId" -o tsv)

echo "$APP_ID"
```

   If `$APP_ID` is empty, the app already doesn't exist — skip the `az ad app delete` step.

2. **Delete** the resource group. This removes the Container App, the storage account, the Cosmos account, the ACR with all its images, the Container Apps environment, every

Container Apps secret, and every role assignment scoped under the group:

```
az group delete -n rg-careers-week7 --yes --no-wait
```

`--no-wait` returns immediately and lets the deletion run in the background. The full teardown takes 3–5 minutes.

3. **Delete** the Entra app registration. The app registration lives in the tenant, not in any subscription resource group, so the RG delete above does **not** remove it. Delete it explicitly:

```
az ad app delete --id "$APP_ID"
```

4. **Verify** the resource group is gone (give it a few minutes if `--no-wait` is still working in the background):

```
az group exists -n rg-careers-week7
```

Expected: `false`.

5. **Verify** the Entra app registration is gone:

```
az ad app list --display-name github-cloudci-careers-
        oidc -o tsv
```

Expected: empty output (no rows).

6. **Optionally** delete the GitHub repo secrets that pointed at the now-deleted Entra app and Azure subscription. They're inert without working resources behind them — the Entra app they reference no longer exists — but cleaning them up is good hygiene so a future viewer doesn't think the pipeline is still wired:

```
gh secret delete ACR_NAME --repo <your-gh-user>/cloudci-
        careers
gh secret delete AZURE_CLIENT_ID --repo <your-gh-user>/
        cloudci-careers
gh secret delete AZURE_TENANT_ID --repo <your-gh-user>/
        cloudci-careers
gh secret delete AZURE_SUBSCRIPTION_ID --repo <your-gh-
        user>/cloudci-careers
```

Replace `<your-gh-user>/cloudci-careers` with the actual owner and name of your repository.

CONCEPT DEEP DIVE: TENANT-VS-SUBSCRIPTION SPLIT FOR CLEANUP

Azure resources and Entra identities live on two different planes. The resource group is a subscription-level container — `az group delete` cascades to every Azure resource scoped under it (Container App, storage account, Cosmos account, ACR, Container Apps environment, all secrets, all role assignments scoped to those resources).

The Entra app registration is a *tenant-level* identity object. It lives in Microsoft Entra ID, not in your Azure subscription, and survives every subscription operation. Without the second `az ad app delete`, the federated identity sticks around as orphaned tenant clutter — visible in the Entra portal with broken role assignments pointing at deleted resources, contributing nothing and confusing future you.

Note that Entra ID *soft-deletes* app registrations for 30 days; they're recoverable from **App registrations → Deleted applications** in the Entra portal and garbage-collected automatically afterwards. That's a useful safety net if you delete the wrong app — you have a month to restore it.

⚠ **Common Mistakes**

- Stopping after `az group delete`. The Entra app stays alive and accumulates over multiple cohorts as orphaned `github-cloudci-careers-oidc` entries.
- Passing the display name to `az ad app delete --id`. The flag wants the `appId` GUID, not the display name. The `az ad app list --query "[0].appId"` extraction in step 1 is what makes this work.
- Orphaned role assignments at *subscription* scope. If anything assigned a role to the Entra service principal at subscription scope (rather than at the resource-group scope), `az group delete` does not clean it up. List subscription-scoped assignments with `az role assignment list --assignee "$APP_ID" --`

> `scope /subscriptions/<sub-id>` and `az role assignment delete` any that turn up.
>
> √ **Quick check:** `az group exists -n rg-careers-week7` returns `false`, `az ad app list --display-name github-cloudci-careers-oidc -o tsv` returns nothing.

Step 10: *Test Your Implementation*

Walk through the assertions one more time, in order. Run the first four against the deployed Container App *before* the cleanup substep; run the last two *after*.

1. **Liveness returns 200:**

   ```
   curl -i https://$FQDN/health/live
   ```

   Expected: `HTTP/2 200`, body `Healthy`.

2. **Readiness returns 200:**

   ```
   curl -i https://$FQDN/health/ready
   ```

   Expected: `HTTP/2 200`, body `Healthy` (Cosmos and Blob both healthy).

3. **Diagnostic endpoint returns the JSON breakdown:**

   ```
   curl -s https://$FQDN/health | jq .
   ```

   Expected: a JSON object with `"status": "Healthy"` and three entries — `self`, `cosmos`, `blob` — all `Healthy`.

4. **Probes are wired on the Container App:**

```
az containerapp show -g rg-careers-week7 -n ca-careers-
        week7 \
  --query 'properties.template.containers[0].probes' -o
        json
```

Expected: a Liveness probe at `/health/live` and a Readiness probe at `/health/ready`, both on port 8080.

5. **After cleanup, the resource group is gone:**

```
az group exists -n rg-careers-week7
```

Expected: `false`.

6. **After cleanup, the Entra app is gone:**

```
az ad app list --display-name github-cloudci-careers-
        oidc -o tsv
```

Expected: empty output.

√ ***Success indicators:***

- `/health/live` *returns 200 with body* `Healthy`
- `/health/ready` *returns 200 with body* `Healthy` *when Cosmos and Blob are reachable*
- `/health` *returns a JSON breakdown with three healthy entries*
- `az containerapp show` *confirms both probes are wired to the right paths and port*
- *After cleanup:* `az group exists` *returns* `false`, `az ad app list` *returns nothing*

√ ***Final verification checklist:***

- [ ] `Microsoft.Extensions.Diagnostics.HealthChecks` *package added*
- [ ] `self`, `cosmos`, `blob` *checks all registered;* `cosmos` *and* `blob` *tagged* `"ready"`
- [ ] *Three endpoints mapped:* `/health/live` *(no checks),* `/health/ready` *(tag predicate),* `/health` *(custom JSON writer)*
- [ ] *Smoke test in the workflow targets* `/health/live`
- [ ] *Container Apps Liveness and Readiness probes wired to* `/health/live` *and* `/health/ready` *on port 8080*
- [ ]

> - [ ] *Resource group* `rg-careers-week7` *deleted*
> - [ ] *Entra app* `github-cloudci-careers-oidc` *deleted*
> - [ ] *(Optional) GitHub repo secrets cleaned up*

14.6.4 Common Issues

> ***If you encounter problems:***
>
> **`/health/ready` returns 503 in CI from a fresh deploy:** *Role-assignment propagation is still settling. The smoke-test loop in Step 7 retries up to 20 times with 6-second backoff, which usually covers it. If it doesn't, the role assignment is missing or scoped wrong — re-run the role-assignment commands from the previous exercise.*
>
> **First deploy after the workflow change fails the smoke test:** *You pushed the workflow change before the code change.* `/health/live` *doesn't exist on the live revision yet. Push the code first, watch the deploy go green, then update the smoke target.*
>
> **`az containerapp update --probe-type` fails with "unrecognized arguments":** *Some* `az` *versions are positional about the order of flags. Try moving* `--probe-type Liveness` *immediately after* `-n`/`-g`*, or upgrade* `az` *with* `az upgrade`*.*
>
> **Every probe fails and the replica is in a restart loop:** *Almost always missing* `--probe-port 8080`*. The default port for the probe is not the container's listening port; specify it explicitly.*
>
> **`/health` returns 503 but `/health/live` and `/health/ready` return 200:** *A check that's not tagged* `"ready"` *is failing. The diagnostic endpoint runs every check, including* `self` *— if* `self` *is somehow unhealthy, the aggregated status flips. Inspect the JSON body to see which entry is failing.*
>
> **`az ad app delete` says "Resource not found":** *You passed the display name instead of*

> *the* `appId`*. The* `az ad app list --query "[0].appId"` *extraction in Step 9 returns the GUID; pass that to* `--id`*.*
> `az group delete` ***returns immediately but resources still appear in the Portal:*** *That's* `--no-wait` *working as intended. Re-check with* `az group exists -n rg-careers-week7` *after a few minutes.*
> ***Still stuck?*** *Verify three things in order: the three checks are registered with the right tags (*`AddCheck<CosmosHealthCheck>("cosmos", tags: new[] { "ready" })`*), the three endpoint mappings have the right predicates (*`_ => false` *for live,* `c => c.Tags.Contains("ready")` *for ready), and the Container Apps probes are wired to the matching paths. All three must agree for the platform-level signals to make sense.*

14.6.5 Summary

You added three health-check endpoints with three distinct semantics, wired two of them as Container Apps probes, and finished the chapter with a clean teardown. `/health/live` is the always-true liveness probe — it answers "is this process alive" and is consumed by the platform's Liveness probe to decide whether to restart the replica. `/health/ready` runs the `cosmos` and `blob` checks via the `"ready"` tag predicate and is consumed by the Readiness probe to decide whether to send the replica traffic. `/health` is the diagnostic endpoint — a per-check JSON breakdown for humans, dashboards, and on-call investigation; it is **not** wired as a probe, because the platform shouldn't react to "blob check is degraded but cosmos is fine." Three endpoints, three audiences. The

cleanup substep tore down the resource group and the Entra app registration in two separate commands, because they live on two different planes.

- ✓ ASP.NET Core health-checks pipeline registered with `self`, `cosmos`, and `blob` checks
- ✓ `CosmosHealthCheck` and `BlobHealthCheck` use 5-second timeouts so a hung dependency doesn't freeze the readiness endpoint
- ✓ Three endpoints mapped with distinct predicates: `_ => false` for liveness, `Tags.Contains("ready")` for readiness, custom JSON writer for diagnostics
- ✓ Container Apps Liveness and Readiness probes wired to `/health/live` and `/health/ready` on port 8080
- ✓ Smoke test in the workflow updated to target `/health/live`
- ✓ Resource group, Entra app registration, and (optionally) GitHub secrets all torn down — no orphaned cloud or tenant resources

> **KEY TAKEAWAY:**
> Liveness asks "is *this process* alive?" and must NOT depend on external services. Readiness asks "should I send this replica traffic right now?" and gates on dependencies. Two questions; two endpoints; two probes. A diagnostic JSON endpoint exists alongside, for humans — never wire it as a probe. And cleanup needs two commands, because Azure resources and Entra identities live on different planes.

14.6.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Replace the hand-rolled writer with* `AspNetCore.HealthChecks.UI.Client.UIResponseWriter.WriteHea` *It's the canonical JSON writer that ships with the AspNetCore.HealthChecks NuGet ecosystem, and it produces a richer schema parsed by the* `AspNetCore.HealthChecks.UI` *dashboard.*
> - *Tune* `--probe-failure-threshold` *for noisier dependencies. The default of 3 is fine for in-region Cosmos and Blob; for cross-region or third-party dependencies, bump to 5 or 7 so transient blips don't flap the replica.*
> - *Use* `--probe-initial-delay-seconds` *to give slow-warming apps a grace period before the first probe runs. Useful for apps that do significant startup work — JIT warmup, connection-pool priming, schema migration on boot.*
> - *Read the Microsoft docs for Container Apps health probes: https://learn.microsoft.com/en-us/azure/container-apps/health-probes. The page covers the full set of probe knobs (*`--probe-success-threshold`*, TCP vs HTTPS schemes, custom HTTP headers) that aren't covered here.*

14.6.7 Done!

That's the chapter. Storage and resilience — durable Cosmos and Blob through managed identity, file uploads with explicit lifecycle, deep health probes that the platform consumes to

make restart and traffic decisions. The cloud lab is fully torn down; you can re-run the chapter from a clean slate any time.

The next chapter shifts gears entirely — from durable persistence to async messaging and queue-driven workflows. Many of the patterns you've leaned on so far (synchronous request/response, an HTTP API as the leaf of every interaction, durable state read inline with the request) start to creak when work needs to happen out-of-band. The next chapter introduces the primitives — queues, message envelopes, idempotent handlers — that let you take long-running work off the critical path without losing reliability.

PART VII

Webapp Development

Learn to build well-structured ASP.NET Core MVC web applications using the three-layer architecture pattern: Presentation, Service, and Data layers.

EXERCISE 15

Presentation Layer

15.1 1. Hello World

15.2 Hello World MVC Application

15.2.1 Goal

Create a basic ASP.NET Core MVC application, run it locally, and prepare it for version control with Git.

15.2.2 Learning Objectives

By the end of this exercise, you will:

- Create a new ASP.NET Core MVC project
- Understand the basic project structure of an MVC application
- Run a web application locally

- Initialize Git for version control
- Publish your code to GitHub

15.2.3 Step-by-Step Instructions

Step 1: Create a New ASP.NET Core MVC Application

1. Open a terminal or command prompt
2. Navigate to your desired project location
3. Run the following command to create a new MVC application:

```
dotnet new mvc -n CloudSoft
```

> ***Information***
>
> - *The `dotnet new mvc` command creates a new ASP.NET Core MVC project*
> - *The `-n CloudSoft` parameter sets the project name to "CloudSoft"*
> - *This creates a folder structure with Controllers, Models, and Views folders*

1. Navigate to the newly created project folder:

```
cd CloudSoft
```

Step 2: Explore the Project Structure

Take a moment to explore the generated project structure:

- **Controllers/** - Contains controller classes that handle HTTP requests
- **Models/** - Contains data model classes
- **Views/** - Contains Razor view files (.cshtml) for rendering HTML

- **wwwroot/** - Contains static files (CSS, JavaScript, images)
- **Program.cs** - The application entry point
- **CloudSoft.csproj** - The project file with dependencies and configuration

> ***Information***
> *The MVC (Model-View-Controller) pattern separates an application into three main components: - **Models**: Data and business logic - **Views**: User interface (UI) - **Controllers**: Handle user input and responses*

Step 3: Run the Application Locally

1. Start the application:

```
dotnet run
```

1. Open a web browser and navigate to:

```
https://localhost:7240
```

or

```
http://localhost:5000
```

(Note: The exact port numbers may vary)

> ***Verification Step:***
> *You should see the default ASP.NET Core welcome page with links to resources and documentation.*

Step 4: Initialize Git for Version Control

1. Create a .gitignore file to exclude unnecessary files from version control:

```
dotnet new gitignore
```

1. Initialize a new Git repository:

```
git init
```

1. Add all files to the Git staging area:

```
git add .
```

1. Create your first commit:

```
git commit -m "Initial commit: Create ASP.NET Core MVC
        application"
```

> ***Information***
>
> - *Git tracks changes to your codebase*
> - *The .gitignore file prevents unnecessary files (like build artifacts) from being tracked*
> - *Each commit represents a snapshot of your code at a specific point in time*

Step 5: Publish to GitHub

1. Create a new repository on GitHub:
   - Go to GitHub
   - Click the "+" icon in the top right and select "New repository"
   - Name your repository (e.g., "CloudSoft")
   - Do not initialize with a README, .gitignore, or license
   - Click "Create repository"
2. Link your local repository to GitHub and push your code:

```
git remote add origin https://github.com/YourUsername/
        CloudSoft.git
git branch -M main
git push -u origin main
```

> ***Information***
>
> - *Replace "YourUsername" with your actual GitHub username*
> - *This establishes a connection between your local repository and GitHub*
> - *The `-u` flag sets up tracking, which simplifies future pushes*

15.2.4 Final Tests

Test 1: Verify Your Web Application

1. Ensure your application is running:

```
dotnet run
```

1. Open a browser and navigate to the application URL
2. Verify the homepage loads correctly

Expected Result: - The application runs without errors - The welcome page appears in your browser - You can navigate to the Privacy page using the link in the header

Test 2: Verify Your GitHub Repository

1. Go to your GitHub account
2. Navigate to your new repository
3. Verify that your code has been pushed successfully

Expected Result: - All project files are visible in your GitHub repository - You can see your commit message - The repository structure matches your local project

15.2.5 Exercise Complete!

Congratulations!

15.3 2. Create A Form With Basic HTML

15.4 Create A Form With Basic HTML

15.4.1 Goal

Create a **basic HTML form** that collects **name** and **email** and submits it via `POST`. This step introduces fundamental form handling in an ASP.NET Core MVC web application without using advanced features like model binding or validation.

15.4.2 Learning Objectives

By the end of this exercise, you will:

- Understand how to create an HTML form with basic input fields.
- Learn how to submit form data using an HTTP `POST` request.
- Implement a controller to handle form display and submission.
- Display submitted data in the application console.

15.4.3 Step-by-Step Instructions

Step 1: Create a Controller

1. Navigate to the `Controllers` folder and create a new file named `NewsletterController.cs`.

2. Add action methods to **display the form** and **handle form submission**.

> ***File:*** `Controllers/NewsletterController.cs`

```
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Controllers;

public class NewsletterController : Controller
{

    // GET: /Newsletter/Subscribe
    public IActionResult Subscribe()
    {
        return View();
    }

    // POST: /Newsletter/Subscribe
    [HttpPost]
    public IActionResult Subscribe(string name, string
        email)
    {
        // Add subscription logic here
        // ...

        // Write to the console
        Con-
        sole.WriteLine($"New subscription - Name: {name}
        Email: {email}");

        // Send a message to the user
        return Content($"Thank you {name} for subscribing
        to our newsletter!");
    }
}
```

> ***Information***
>
> - *The "Get"* `Subscribe()` *method* ***renders the page with the form.***
> - *The "Post"* `Subscribe()` *method* ***handles the form submission*** *and returns a simple message.*

Step 2: Create the Form View

1. Navigate to the **`Views/Newsletter/`** folder.
2. Create a new file named **`Subscribe.cshtml`**.
3. Add the following HTML code for the form:

> ***File:*** `Views/Newsletter/Subscribe.cshtml`

```
@{
    ViewData["Title"] = "Sign up for our newsletter";
}

<h2>Signup up for our newsletter</h2>

<form action="/Newsletter/Subscribe" method="post">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name">

    <label for="email">Email:</label>
    <input type="email" id="email" name="email">

    <button type="submit">Sign Up</button>
</form>
```

> ***Information***
>
> - *This form submits data to* `/Newsletter/Signup` *via* ***HTTP POST****.*
> - *When the form is submitted, the following data is sent back to the server via* `POST`*:*
> - `name`*: The user's entered name.*
> - `email`*: The user's entered email address.*
> - *It is the name attribute (name="email") in the form input field that is sent back to the server. The id attribute is used within the browser page for linking labels to input fields.*

15.4.4 Final Tests

Step 1: Run the Application

Start the application:

```
dotnet run
```

Then open the browser and navigate to:

```
http://localhost:<PORT>/Newsletter/Subscribe
```

> *Change the port to what you have*

Expected Result

- You should see your form page

Step 2: Fill Out and Submit the Form

- Enter a **name** and an incorrect **email** (without @)
- Click **Sign Up**.
- Enter a **name** and a correct **email**

- Click **Sign Up**

Expected Result

- The incorrect email should show an information message
- Upon correct submission, the browser should display:

```
Thank you [entered name] for subscribing to our newslet-
ter!
```

- In the application console, you should see:

```
New subscription - Name: [Your Name], Email: [Your
Email]
```

15.5 You have a form!

15.6 3. Helper Tags and Model Binding

15.7 Helper Tags and Model Binding

15.7.1 Goal

Enhance the existing form by introducing **ASP.NET Core helper tags**, **model binding (without annotations)**, and **ViewBag for feedback messages**.

15.7.2 Learning Objectives

By the end of this exercise, you will:

- Use **ASP.NET Core Tag Helpers** (`asp-for`, `asp-action`)
- Implement **model binding** without validation attributes
- Utilize **ViewBag** to display feedback messages after form submission

15.7.3 Step-by-Step Instructions

Step 1: Create a Model for Form Data

1. Navigate to the `Models` folder and create a new file named `Subscriber.cs`.
2. Define a simple class with `Name` and `Email` properties.

> `Models/Subscriber.cs`

```
namespace CloudSoft.Models;

public class Subscriber
{
    public string? Name { get; set; }
    public string? Email { get; set; }
}
```

> ***Information***
> *This model will store the form input values and be used for **model binding** in the controller.*

Step 2: Update the Controller

1. Open `NewsletterController.cs` in the `Controllers` folder.
2. Modify the `Subscribe` action to accept a `Subscriber` model and store feedback in `ViewBag`.

> `Controllers/NewsletterController.cs`

```
using CloudSoft.Models;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Controllers;

public class NewsletterController : Controller
{

    [HttpGet]
    public IActionResult Subscribe()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Subscribe(Subscriber subscriber)
    {
        // Add subscription logic here
        // ...

        // Write to the console
        Console.WriteLine($"New subscription - Name: {sub-
        scriber.Name} Email: {subscriber.Email}");

        // Send a message to the user
        ViewBag.Message = $"Thank you for subscribing,
        {subscriber.Name}!";

        // Return the view
        return View();
    }
}
```

> ***Information***
>
> - *The* `Subscribe` *method now* ***accepts a*** **`Subscriber`** ***model****, allowing model binding to populate properties automatically.*
> - `ViewBag.Message` *stores feedback that can be displayed in the view.*

Step 3: Update the View to Use Tag Helpers and Display Feedback

1. Open `Views/Newsletter/Subscribe.cshtml`.
2. Modify the form to use **Tag Helpers** and display the feedback message.

> `Views/Newsletter/Subscribe.cshtml`

```
@model CloudSoft.Models.Subscriber

@{
    ViewData["Title"] = "Newsletter Signup";
}

<h2>Newsletter Signup</h2>

@if (ViewBag.Message != null)
{
    <p style="color: green;">@ViewBag.Message</p>
}

<form asp-action="Subscribe" method="post">
    <label asp-for="Name"></label>
    <input asp-for="Name" class="form-control" />

    <label asp-for="Email"></label>
    <input asp-for="Email" class="form-control" />

    <button type="submit">Sign Up</button>
</form>
```

> ***Information***
>
> - `@model CloudSoft.Models.Subscriber` *tells the view what model to use.*
> - `ViewBag.Message` *is displayed after a successful submission.*
> - *The* `asp-action` *helper automatically sets the form action to* `Subscribe`*.*
> - *The* `asp-for` ***Tag Helpers*** *bind input fields directly to the* `Subscriber` *model.*

Step 4: Update the NavBar

The layout defines the shared structure of your application.

1. Open the _Layout.cshtml file in the Views/Shared folder.
2. Add a navigation link to the Newsletter Subscription feature:

> `Views/Shared/_Layout.cshtml`

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-control-
        ler="Newsletter" asp-
        action="Subscribe">Subscribe</a>
</li>
```

15.7.4 Final Tests

Run the Application and Validate Your Work

1. Start the application:

```
dotnet run
```

2. Open a browser and navigate to:

```
http://localhost:5000/Newsletter/Subscribe
```

3. Fill out the form and submit it.
4. Ensure the page reloads and displays a confirmation message.

15.7.5 Shout Out!

Great job implementing **Tag Helpers and Model Binding!** Next, we'll enhance validation to improve user experience.

15.8 4. Data Annotations and Client-Side Validation

15.9 Data Annotations and Client-Side Validation

15.9.1 Goal

Enhance the existing form by introducing **data annotations** for **client-side validation** to improve user experience and enforce input constraints.

15.9.2 Learning Objectives

By the end of this exercise, you will:

- Use **data annotations** (`[Required]`, `[EmailAddress]`, `[StringLength]`)
- Enable **client-side validation** using ASP.NET Core built-in validation
- Improve form feedback with validation messages

15.9.3 Step-by-Step Instructions

Step 1: Update the Model with Data Annotations

1. Open `Subscriber.cs` in the `Models` folder.
2. Add **validation attributes** to enforce required fields and format constraints.

> `Models/Subscriber.cs`

```
using System.ComponentModel.DataAnnotations;

namespace CloudSoft.Models;

public class Subscriber
{
    [Required]
    [StringLength(20, ErrorMessage =
        "Name cannot exceed 20 characters")]
    public string? Name { get; set; }

    [Required]
    [EmailAddress]
    public string? Email { get; set; }
}
```

> ***Information***
>
> - `[Required]` *ensures the field* ***must be filled****.*
> - `[StringLength]` *limits the length of the* `Name` *field.*
> - `[EmailAddress]` *ensures a* ***valid email format****.*

Step 2: Update the View to Display Validation Messages

1. Open `Views/Newsletter/Subscribe.cshtml`.
2. Add validation messages for form fields.

> `Views/Newsletter/Subscribe.cshtml`

```
@model CloudSoft.Models.Subscriber

@{
    ViewData["Title"] = "Newsletter Signup";
}

<h2>Newsletter Signup</h2>

@if (ViewBag.Message != null)
{
    <p style="color: green;">@ViewBag.Message</p>
}

<form asp-action="Subscribe" method="post">
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-
        danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-
        danger"></span>
    </div>
    <button type="submit">Sign Up</button>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

> ***Information***
>
> - *The* `asp-validation-for` ***displays validation errors*** *next to fields.*
> - `_ValidationScriptsPartial` ***enables client-side validation*** *using jQuery.*

15.9.4 Final Tests

Run the Application and Validate Your Work

1. Start the application:

```
dotnet run
```

2. Open a browser and navigate to:

```
http://localhost:5000/Newsletter/Subscribe
```

3. Try submitting the form **with missing or invalid data**.
4. Ensure **error messages** appear and **prevent submission**.
5. Fill in valid data and confirm successful submission.

15.9.5 Shout Out!

Fantastic job! You've successfully added **data validation and client-side validation**

15.10 5. Server-Side Validation and Improved Feedback

15.11 Server-Side Validation and Improved Feedback

15.11.1 Goal

Enhance the form by introducing **server-side validation**, managing a **list of subscribers**, and improving feedback handling using **TempData** and **model-level errors**.

15.11.2 Learning Objectives

By the end of this exercise, you will:

- Implement **server-side validation** for better security.
- Store **subscribers in a list** within the controller.
- Use **regular expressions** to validate email format.
- Prevent **duplicate email signups**.
- Replace **ViewBag with TempData** for better feedback persistence.
- Use **ModelState.AddModelError** to improve error handling.

15.11.3 Step-by-Step Instructions

Step 1: Update the Model with Server-Side Validation

1. Open `Subscriber.cs` in the `Models` folder.
2. Improve **client-side email validation** using **regular expressions**.

> `Models/Subscriber.cs`

```
using System.ComponentModel.DataAnnotations;

namespace CloudSoft.Models;

public class Subscriber
{
    [Required]
    [StringLength(20, ErrorMessage =
        "Name cannot exceed 20 characters")]
    public string? Name { get; set; }

    [Required]
    [EmailAddress]
    [RegularExpression("^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\
        \.[a-zA-Z]{2,}$", ErrorMessage = "Missing top
        level domain")]
    public string? Email { get; set; }
}
```

Information

- The `RegularExpression` **attribute** enforces stricter email format validation.
- This prevents malformed or **suspicious email input**.

Step 2: Update the Controller to Store Subscribers and Prevent Duplicates

1. Open `NewsletterController.cs`.
2. Modify the `Subscribe` method to check for **duplicate emails** and store data in memory.

> `Controllers/NewsletterController.cs`

```
using System.Text.RegularExpressions;
using CloudSoft.Models;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Controllers;

public class NewsletterController : Controller
{
    // Create a "database" of subscribers for demonstration
        purposes
    private static List<Subscriber> _subscribers = [];

    [HttpGet]
    public IActionResult Subscribe()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Subscribe(Subscriber subscriber)
    {
        // Validate the model
        if (!ModelState.IsValid)
        {
            return View(subscriber);
        }

        // Check if the email is already subscribed and
        return a general model level error
        if (_subscribers.Any(s => s.Email == sub-
        scriber.Email))
        {
            ModelState.AddModelError("Email",
        "The email is already subscribed. Please use a
        different email.");
            return View(subscriber);
        }

        // Add the subscriber to the list
        _subscribers.Add(subscriber);

        // Write to the console
```

```
        Console.WriteLine($"New subscription - Name: {sub-
        scriber.Name} Email: {subscriber.Email}");


        // Send a message to the user
        TempData["SuccessMessage"] = $"Thank you for sub-
        scribing, {subscriber.Name}! You will receive our
        newsletter at {subscriber.Email}";


        // Return the view (using the POST-REDIRECT-GET
        pattern)
        return
        RedirectToAction(nameof(Subscribe));  // use
        nameof() to find the action by name during compile
        time
    }
}
```

> ***Information***
>
> - `_subscribers` ***list*** *stores all registered users* ***in memory****.*
> - ***Checks if the email already exists*** *before allowing submission.*
> - *Uses* ***ModelState.AddModelError*** *for* ***field-specific error messages****.*
> - ***TempData replaces ViewBag*** *for better feedback persistence across redirects.*

Step 3: Update the View to Display Errors and TempData Feedback

1. Open `Views/Newsletter/Subscribe.cshtml`.
2. Modify the form to display **server-side validation errors** and **TempData messages**.

> `Views/Newsletter/Subscribe.cshtml`

```
@model CloudSoft.Models.Subscriber

@{
    ViewData["Title"] = "Newsletter Signup";
}

<h2>Newsletter Signup</h2>

<!-- Display validation summary for model-level errors -->
@if (!ViewData.ModelState.IsValid)
{
    <div class="alert alert-danger alert-dismissible fade
        show" role="alert">
        @Html.ValidationSummary(false, null, new { @class =
        "text-danger" })
        <button type="button" class="btn-close" data-bs-
        dismiss="alert" aria-label="Close"><i class="fas
        fa-times"></i></button>
    </div>
}

<!-- Display message from the TempData sent by the
        controller -->
@if (TempData["SuccessMessage"] != null)
{
    <div class="alert alert-success alert-dismissible fade
        show" role="alert">
        @TempData["SuccessMessage"]
        <button type="button" class="btn-close" data-bs-
        dismiss="alert" aria-label="Close"><i class="fas
        fa-times"></i></button>
    </div>
}

<form asp-action="Subscribe" method="post">
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-
        danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
```

```
        <span asp-validation-for="Email" class="text-
        danger"></span>
    </div>
    <div class="mt-3">
        <button type="submit" class="btn btn-primary">Sign
        Up</button>
    </div>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

Information

- **TempData message** is displayed when **redirecting after a successful submission**.
- `asp-validation-for` **displays field-specific validation errors**.
- `_ValidationScriptsPartial` **enables client-side validation**.

15.11.4 Final Tests

Run the Application and Validate Your Work

1. Start the application:

```
dotnet run
```

2. Open a browser and navigate to:

```
http://localhost:5000/Newsletter/Subscribe
```

3. Try submitting **invalid or empty input** and confirm error messages.

4. Try **registering with the same email twice** and confirm the duplicate warning.
5. Submit valid data and check if feedback persists correctly using **TempData**.

15.11.5 Shout Out!

Awesome work! You've implemented **server-side validation**, handled **duplicate entries**, and improved **feedback management**.

15.12 6. Displaying a List of Subscribers

15.13 Displaying a List of Subscribers

15.13.1 Goal

Enhance the application by adding a **new view to list all subscribers** and updating the **navigation bar** to include a button linking to the new page.

15.13.2 Learning Objectives

By the end of this exercise, you will:

- Create a **new view** to display a list of subscribers.
- Modify the **controller** to pass subscriber data to the view.
- Update the **navigation bar** to provide access to the new page.

15.13.3 Step-by-Step Instructions

Step 1: Update the Controller to Provide Subscriber Data

1. Open `NewsletterController.cs`.
2. Add a new action method to return a list of subscribers to the view.

> `Controllers/NewsletterController.cs`

```
...

    [HttpGet]
    public IActionResult Subscribers()
    {
        return View(_subscribers);
    }

...
```

> ***Information***
>
> - `Subscribers()` *action returns the* ***list of subscribers*** *to the view.*
> - *The list is retrieved from the* ***in-memory static list****.*

Step 2: Create the Subscriber List View

1. Navigate to the `Views/Newsletter/` folder.
2. Create a new file named `Subscribers.cshtml`.
3. Add the following code to display subscribers in a table.

> `Views/Newsletter/Subscribers.cshtml`

```
@model List<CloudSoft.Models.Subscriber>

@{
    ViewData["Title"] = "Subscriber List";
}

<h2>Subscriber List</h2>

@if (Model.Count == 0)
{
    <p>No subscribers yet.</p>
}
else
{
    <table class="table">
        <thead>
            <tr>
                <th>Name</th>
                <th>Email</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var subscriber in Model)
            {
                <tr>
                    <td>@subscriber.Name</td>
                    <td>@subscriber.Email</td>
                </tr>
            }
        </tbody>
    </table>
}
```

> ***Information***
>
> - *Displays **all subscribers** in a table.*
> - *If no subscribers exist, it shows a **fallback message**.*

Step 3: Add a Navigation Button to the Navbar

1. Open `Views/Shared/_Layout.cshtml`.
2. Add a new navigation link to the **Subscriber List** page.

> `Views/Shared/_Layout.cshtml`

```
...

    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-con-
        troller="Newsletter" asp-action="Subscribers">Sub-
        scribers</a>
    </li>

...
```

> ***Information***
>
> - *Adds a **navigation button** to access the **Subscriber List page**.*

15.13.4 Final Tests

Run the Application and Validate Your Work

1. Start the application:

```
dotnet run
```

2. Open a browser and navigate to:

```
http://localhost:5000/Newsletter/Subscribe
```

3. Subscribe with **multiple names and emails**.
4. Click **Subscribers** in the navbar to view the **subscriber list**.
5. Ensure all **submitted data appears correctly** in the table.

15.13.5 Shout Out!

Great job! You've successfully added a **subscriber list page** and a **navigation button** to improve user experience!

15.14 7. Implement Unsubscribe Functionality

15.15 Implement Unsubscribe Functionality

15.15.1 Goal

Add the ability for users to **unsubscribe** from the newsletter and enhance the subscriber list with **action buttons**.

15.15.2 Learning Objectives

By the end of this exercise, you will:

- Add an **unsubscribe action** to the controller
- Implement **confirmation dialogs** for destructive actions
- Add **action buttons** to table rows
- Handle **POST requests** securely with **anti-forgery tokens**

15.15.3 Step-by-Step Instructions

Step 1: Update the Controller with Unsubscribe Action

1. Open `NewsletterController.cs`
2. Add a new action method for handling unsubscribe requests

> `Controllers/NewsletterController.cs`

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Unsubscribe(string email)
{
    var subscriber = _subscribers.FirstOrDefault(s =>
    s.Email == email);
    if (subscriber != null)
    {
        _subscribers.Remove(subscriber);
        TempData["SuccessMessage"] = $"Successfully
    unsubscribed {email} from the newsletter.";
    }
    return RedirectToAction(nameof(Subscribers));
}
```

> ***Information***
>
> - *Uses* `HttpPost` *to prevent accidental unsubscribes via GET requests*
> - *Removes subscriber from list based on email*
> - *Provides feedback via TempData*
> - *Redirects back to subscriber list*
> - *The* `[ValidateAntiForgeryToken]` *attribute ensures that:*
> - *The form submission includes a valid anti-forgery token*
> - *Protects against Cross-Site Request Forgery (CSRF) attacks*
> - *Will return a 400 Bad Request if the token is missing or invalid*
> - *The client-side token we have in the View* `@Html.AntiForgeryToken()` *works together with this server-side validation to provide security against CSRF attacks.*

Step 2: Update the Subscriber List View

1. Open `Views/Newsletter/Subscribers.cshtml`
2. Add an unsubscribe button and confirmation dialog

> `Views/Newsletter/Subscribers.cshtml`

```
@model List<CloudSoft.Models.Subscriber>

@{
    ViewData["Title"] = "Subscriber List";
}

<h2>Subscriber List</h2>

<!-- Display message from the TempData sent by the
        controller -->
@if (TempData["SuccessMessage"] != null)
{
    <div class="alert alert-success alert-dismissible fade
        show" role="alert">
        @TempData["SuccessMessage"]
        <button type="button" class="btn-close" data-bs-
        dismiss="alert" aria-label="Close"><i class="fas
        fa-times"></i></button>
    </div>
}

@if (Model.Count == 0)
{
    <p>No subscribers yet.</p>
}
else
{
    <table class="table">
        <thead>
            <tr>
                <th>Name</th>
                <th>Email</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
            @foreach (var subscriber in Model)
            {
                <tr>
                    <td>@subscriber.Name</td>
                    <td>@subscriber.Email</td>
                    <td>
```

```
                        <form asp-action="Unsubscribe"
        method="post" class="d-inline"
                                onsubmit="return con-
        firm('Are you sure you want to unsubscribe @sub-
        scriber.Email?');">
                            @Html.AntiForgeryToken()
                            <input type="hidden"
        name="email" value="@subscriber.Email" />
                            <button type="submit"
        class="btn btn-outline-danger btn-sm">
                                <i
        class="fas fa-user-minus me-1"></i>Unsubscribe
                            </button>
                        </form>
                    </td>
                </tr>
            }
        </tbody>
    </table>
}
```

> ***Information***
>
> - *Uses* `form` *tag for secure POST requests*
> - *Includes* ***anti-forgery token*** `@Html.AntiForgeryToken()`
> - *Adds confirmation dialog using JavaScript* `onsubmit="return confirm('Are you sure you want to unsubscribe @subscriber.Email?');"`

15.15.4 Final Tests

Functionality Test

1. Subscribe a new user
2. View the subscriber list
3. Click unsubscribe button
4. Verify confirmation dialog appears
5. Confirm unsubscribe

Expected Results

- Confirmation dialog shows correct email
- Success message appears after unsubscribe
- User is removed from list
- List updates immediately

Security Test

1. Temporarily comment out the `[ValidateAntiForgeryToken]`
2. Add subscriber: `test@example.com`
3. Go to the subscriber list page
4. Open a terminal and run this command:

```
curl -X POST http://localhost:5220/Newsletter/Unsubscribe -d "email=test@example.com"
```

5. Refresh the subscriber list page and verify the subscriber `test@example.com` is gone
6. Remove the comment and run step 2. 3. 4. again
7. Refresh the subscriber list page and verify the subscriber `test@example.com` is still there

> `-d "email=test@example.com"` *specifies the data that is being sent in the body of the POST request.*

15.16 Exercise Done!

You've successfully added unsubscribe functionality with proper security measures and user feedback!

15.17 8. Enhancing UI with Bootstrap

15.17.1 Goal

Transform the basic newsletter form and subscriber list into **professional-looking interfaces** using Bootstrap's components, utilities, and responsive design features.

15.17.2 Learning Objectives

By the end of this exercise, you will:

- Use Bootstrap's **card component** for content organization
- Implement **responsive grid system** for layout control
- Style forms using Bootstrap's **form components and utilities**
- Create **responsive tables** with proper styling
- Apply Bootstrap's **utility classes** for spacing and typography

15.17.3 Step-by-Step Instructions

Step 1: Style the Newsletter Signup Form

1. Open `Views/Newsletter/Subscribe.cshtml`
2. Update the form with Bootstrap components and utilities

> ***File:*** `Views/Newsletter/Subscribe.cshtml`

```
@model CloudSoft.Models.Subscriber

@{
    ViewData["Title"] = "Newsletter Signup";
}

<div class="container py-4">
    <div class="row justify-content-center">
        <div class="col-12 col-md-8 col-lg-6">
            <div class="card shadow">
                <div class="card-header bg-primary text-
        white">
                    <h2 class="card-title h4 mb-0">Newslet-
        ter Signup</h2>
                </div>
                <div class="card-body">
                    <!-- Display validation summary for
        model-level errors -->
                    @if (!ViewData.ModelState.IsValid)
                    {
                        <div class="alert alert-danger
        alert-dismissible fade show" role="alert">
                            @Html.ValidationSummary(false,
        null, new { @class = "text-danger" })
                            <button type="button"
        class="btn-close" data-bs-dismiss="alert" aria-
        label="Close"><i class="fas fa-times"></i></
        button>
                        </div>
                    }

                    <!-- Display message from the TempData
        sent by the controller -->
                    @if (TempData["SuccessMessage"] !=
        null)
                    {
                        <div class="alert alert-success
        alert-dismissible fade show" role="alert">
                            @TempData["SuccessMessage"]
                            <button type="button"
        class="btn-close" data-bs-dismiss="alert" aria-
        label="Close"><i class="fas fa-times"></i></
        button>
                        </div>
                    }
```

```
                <form asp-action="Subscribe"
        method="post">
                    <div class="mb-3">
                        <label asp-for="Name"
        class="form-label"></label>
                        <div class="input-group">
                            <span class="input-group-
        text"><i class="fas fa-user"></i></span>
                            <input asp-for="Name"
        class="form-control" placeholder="Enter your
        name" />
                        </div>
                        <span asp-validation-
        for="Name" class="text-danger small"></span>
                    </div>
                    <div class="mb-3">
                        <label asp-for="Email"
        class="form-label"></label>
                        <div class="input-group">
                            <span class="input-group-
        text"><i class="fas fa-envelope"></i></span>
                            <input asp-for="Email"
        class="form-control" placeholder="Enter your
        email" />
                        </div>
                        <span asp-validation-
        for="Email" class="text-danger small"></span>
                    </div>
                    <div class="d-grid">
                        <button type="submit"
        class="btn btn-primary">
                            <i class="fas fa-paper-
        plane me-2"></i>Sign Up
                        </button>
                    </div>
                </form>
            </div>
        </div>
    </div>
</div>

@section Scripts {
```

```
    <partial name="_ValidationScriptsPartial" />
}
```

> ***Information***
>
> - ***Container & Grid**: `container` provides padding and max-width, while `row` and `col-*` create a responsive layout*
> - ***Card Component**: `card` creates a bordered box with header and body sections*
> - ***Input Group**: Combines form controls with icons using `input-group` and `input-group-text`*
> - ***Utilities**:*
> - *`shadow` adds box shadow*
> - *`py-4` adds vertical padding*
> - *`mb-3` adds bottom margin*
> - *`d-grid` makes button full-width*

Step 2: Style the Subscriber List

1. Open `Views/Newsletter/Subscribers.cshtml`
2. Enhance the table with Bootstrap components

> ***File:*** `Views/Newsletter/Subscribers.cshtml`

```
@model List<CloudSoft.Models.Subscriber>

@{
    ViewData["Title"] = "Subscriber List";
}

<div class="container py-4">
    <div class="row justify-content-center">
        <div class="col-12 col-lg-8">
            <div class="card shadow">
                <div class="card-header bg-primary text-
        white d-flex justify-content-between align-items-
        center">
                    <h2 class="card-title h4 mb-0">Sub-
        scriber List</h2>
                    <span class="badge bg-light text-
        primary">Total: @Model.Count</span>
                </div>
                <div class="card-body">
                    @if (TempData["SuccessMessage"] !=
        null)
                    {
                        <div class="alert alert-success
        alert-dismissible fade show" role="alert">
                            @TempData["SuccessMessage"]
                            <button type="button"
        class="btn-close" data-bs-dismiss="alert" aria-
        label="Close"><i class="fas fa-times"></i></
        button>
                        </div>
                    }

                    @if (Model.Count == 0)
                    {
                        <div class="text-center py-4">
                            <i class="fas fa-users fa-3x
        text-muted mb-3"></i>
                            <p class="lead text-muted">No
        subscribers yet.</p>
                        </div>
                    }
                    else
                    {
                        <div class="table-responsive">
```

```
                        <table
class="table table-hover align-middle mb-0">
                            <thead class="table-light">
                                <tr>
                                    <th><i class="fas
fa-user me-2"></i>Name</th>
                                    <th><i class="fas
fa-envelope me-2"></i>Email</th>
                                    <th class="text-
end">Actions</th>
                                </tr>
                            </thead>
                            <tbody>
                                @foreach (var
subscriber in Model)
                                {
                                    <tr>
                                        <td>@sub-
scriber.Name</td>
                                        <td>
                                            <a
href="mailto:@subscriber.Email" class="text-decor-
ation-none">
                                                @sub-
scriber.Email
                                            </a>
                                        </td>
                                        <td
class="text-end">
                                            <form asp-
action="Unsubscribe" method="post" class="d-
inline"

onsubmit="return confirm('Are you sure you want to
unsubscribe @subscriber.Email?');">

@Html.AntiForgeryToken()
                                                <input
type="hidden" name="email" value="@sub-
scriber.Email" />

<button type="submit" class="btn btn-outline-
danger btn-sm">
                                                    <i
class="fas fa-user-minus me-1"></i>Unsubscribe
                                                </
button>
                                            </form>
```

```
                                                    </td>
                                                </tr>
                                            }
                                        </tbody>
                                    </table>
                                </div>
                            }
                        </div>
                    </div>
                </div>
            </div>
</div>
```

> ***Information***
>
> - ***Table Classes:***
> - `table-responsive` *enables horizontal scrolling on small screens*
> - `table-hover` *adds hover effect to rows*
> - `align-middle` *vertically centers content*
> - ***Flex Utilities:***
> - `d-flex` *creates flexible container*
> - `justify-content-between` *spaces items*
> - `align-items-center` *vertically centers items*
> - ***Badge Component:*** `badge` *creates a small count indicator*
> - ***Empty State:*** *Uses* `text-center` *and icon for better visual feedback*

15.17.4 Final Tests

Visual Inspection

1. Check responsive behavior:
    - Resize browser window
    - Test on mobile device or DevTools mobile view
    - Verify form and table adapt correctly

Expected Results

- Form should be:
    - Centered on page
    - Full width on mobile
    - ~50% width on desktop
    - Have consistent spacing
    - Show icons in input fields
- Table should:
    - Scroll horizontally on mobile
    - Show hover effects
    - Display badge with correct count
    - Have centered content

Functionality Test

1. Submit form with errors
2. Check success message
3. View subscriber list

Expected Results

- Error messages properly styled in red
- Success alert in green with close button
- Table rows properly aligned with hover effect
- Clickable email addresses

- Correct icon display

15.18 Exercise Done!

You've transformed basic HTML into a professional-looking interface using Bootstrap's powerful component library!

15.19 9. Landing Page with Hero Section

15.19.1 Goal

Create an engaging **landing page** with a hero section and call-to-action button that directs users to the newsletter subscription page.

15.19.2 Learning Objectives

By the end of this exercise, you will:

- Create a **hero section** using Bootstrap
- Implement **call-to-action buttons**
- Use Bootstrap's **spacing utilities**
- Apply **responsive text classes**
- Create **gradient backgrounds**

15.19.3 Step-by-Step Instructions

Step 1: Create the Landing Page

1. Open `Views/Home/Index.cshtml`
2. Replace the content with a hero section and call-to-action

> `Views/Home/Index.cshtml`

```
@{
    ViewData["Title"] = "Welcome to CloudSoft";
}

<div class="container-fluid px-0">
    <!-- Hero Section -->
    <div class="hero-gradient text-white py-5">
        <div class="container py-5">
            <div class="row align-items-center g-5">
                <div class="col-lg-6 mx-auto text-center">
                    <h1 class="display-3 fw-bold
        mb-3">Stay Connected with CloudSoft</h1>
                    <p class="lead mb-4">
                        Get the latest updates, news, and
        special offers delivered directly to your inbox.
                        Join our newsletter community
        today!
                    </p>
                    <div class="d-grid gap-2 d-md-flex jus-
        tify-content-center">
                        <a asp-controller="Newsletter" asp-
        action="Subscribe"

        class="btn btn-light btn-lg px-4 shadow">
                            <i class="fas fa-envelope
        me-2"></i>Subscribe Now
                        </a>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
```

Information

- ***Container Classes:***
  - `container-fluid px-0` *removes horizontal padding for full-width sections*
  - `container` *centers content with max-width*
- ***Background & Text:***
  - `bg-gradient` *with custom linear gradient creates modern look*
  - `text-white` *ensures text is visible on dark background*
- ***Spacing Utilities:***
  - `py-5` *adds vertical padding*
  - `mb-3/4` *adds margin bottom*
  - `g-5` *adds gutters between columns*
- ***Typography:***
  - `display-3` *makes heading larger*
  - `lead` *styles paragraph as introduction text*
- ***Flex Utilities:***
  - `d-grid d-md-flex` *changes button layout on different screens*
  - `align-items-center` *vertically centers content*
- ***Components:***
  - `btn-lg` *makes buttons larger*
  - `shadow` *adds depth to primary button*

> - `opacity-75` *reduces icon opacity for visual effect*

Step 2: Add Custom CSS Styles

1. Open `wwwroot/css/site.css`
2. Add custom styles for form and gradient backgrounds

> `wwwroot/css/site.css`

```
...

.hero-gradient {
    background: linear-gradient(135deg, var(--bs-primary)
        0%, var(--bs-info) 100%);
}

...
```

> ***Information***
>
> - ***Gradient Background:***
> - *Creates a diagonal gradient using Bootstrap theme colors*
> - *Uses CSS variables to maintain theme consistency*
> - *135-degree angle creates visual interest*

Step 3: Style the About Page

The About page maintains the same card-based layout and styling patterns as the other pages while introducing new elements like images and feature highlights.

1. Add an image to the `wwwroot/images` folder and name it `about-header.jpg`
2. Create the About page with consistent styling

> `Views/Home/About.cshtml`

```
@{
    ViewData["Title"] = "About CloudSoft";
}

<div class="container py-4">
    <div class="row justify-content-center">
        <div class="col-12 col-lg-8">
            <div class="card shadow">
                <img src="~/images/about-header.jpg"
        class="card-img-top" alt="CloudSoft Office">
                <div class="card-body">
                    <h2 class="card-title h3 mb-4">About
        CloudSoft</h2>
                    <div class="row g-4">
                        <div class="col-md-6">
                            <h3 class="h5 mb-3">Our Mis-
        sion</h3>
                            <p>At CloudSoft, we're passion-
        ate about delivering cutting-edge cloud solutions
        that help businesses thrive in the digital age.</
        p>
                        </div>
                        <div class="col-md-6">
                            <h3 class="h5 mb-3">Our
        Vision</h3>
                            <p>We envision a future where
        every business, regardless of size, can harness
        the power of cloud technology to achieve their
        goals.</p>
                        </div>
                        <div class="col-12">
                            <div class="bg-light p-4 roun-
        ded">
                                <h3 class="h5 mb-3">Why
        Choose Us?</h3>
                                <div class="row g-3">
                                    <div class="col-md-4">
                                        <div class="d-flex
        align-items-center">
                                            <i class="fas
        fa-cloud-upload-alt text-primary fa-2x me-3"></i>
                                            <div>
                                                <h4
        class="h6 mb-1">Cloud Expertise</h4>
                                                <p
        class="small mb-0">10+ years experience</p>
```

```
                                        </div>
                                    </div>
                                </div>
                                <div class="col-md-4">
                                    <div class="d-flex
        align-items-center">
                                        <i class="fas
        fa-users text-primary fa-2x me-3"></i>
                                        <div>
                                            <h4
        class="h6 mb-1">Support Team</h4>
                                            <p
        class="small mb-0">24/7 availability</p>
                                        </div>
                                    </div>
                                </div>
                                <div class="col-md-4">
                                    <div class="d-flex
        align-items-center">
                                        <i class="fas
        fa-shield-alt text-primary fa-2x me-3"></i>
                                        <div>
                                            <h4
        class="h6 mb-1">Security First</h4>
                                            <p
        class="small mb-0">Enterprise-grade</p>
                                        </div>
                                    </div>
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
```

> ***Information***
>
> - ***Image Handling***:
> - `card-img-top` *properly sizes the header image*
> - *Images should be placed in* `wwwroot/images`
> - ***Grid Layout***:
> - *Uses nested rows and columns for complex layouts*
> - `g-4` *adds gutters between grid items*
> - ***Content Cards***:
> - `bg-light` *creates subtle section backgrounds*
> - `rounded` *adds consistent border radius*
> - ***Icons & Typography***:
> - *Font Awesome icons with consistent sizing*
> - *Hierarchical heading structure with Bootstrap classes*
> - ***Responsive Behavior***:
> - *Single column on mobile*
> - *Multi-column layout on larger screens*

15.19.4 Final Tests

Visual Inspection

1. Check responsive behavior:
   - Resize browser window

Expected Results

- Hero section should:
  - Fill viewport width
  - Have smooth gradient background

Functionality Test

1. Click “Subscribe Now” button
2. Verify navigation to subscription page

Expected Results

- Buttons navigate to correct pages
- Links work correctly
- Smooth transitions between pages
- Consistent styling across pages

15.20 Exercise Done!

You’ve created an engaging landing page that draws users to your newsletter subscription!

EXERCISE 16

Service Layer

16.1 10. Implement Service Layer

16.2 Implement Service Layer

16.2.1 Goal

Refactor the newsletter functionality by implementing a **service layer** with dependency injection and async operations.

16.2.2 Learning Objectives

By the end of this exercise, you will:

- Create an **interface** and **service implementation**
- Implement **dependency injection** in ASP.NET Core
- Use the **Result pattern** for operation outcomes

- Work with **async/await** patterns
- Understand **service lifetimes** in ASP.NET Core

16.2.3 Step-by-Step Instructions

Step 1: Create the Operation Result Pattern

1. Create a new class `OperationResult.cs` in the Models folder
2. Implement the Result pattern for handling operation outcomes

> `Models/OperationResult.cs`

```
namespace CloudSoft.Models;

public class OperationResult
{
    public bool IsSuccess { get; private set; }
    public string Message { get; private set; }

    private OperationResult(bool success, string message)
    {
        IsSuccess = success;
        Message = message;
    }

    public static OperationResult Success(string message)
        => new(true, message);
    public static OperationResult Failure(string message)
        => new(false, message);
}
```

> ***Information***
> *The Result pattern provides:*
>
> - *A standardized way to handle operation outcomes*
> - *Clear separation between success and failure cases*
> - *Immutable results through private setters*
> - *Factory methods for creating results*
> - *Consistent messaging across the application*

Step 2: Create the Newsletter Service Interface

1. Create a new interface `INewsletterService.cs` in the Services folder
2. Define the contract for newsletter operations

> `Services/INewsletterService.cs`

```
using CloudSoft.Models;

namespace CloudSoft.Services;

public interface INewsletterService
{
    Task<OperationResult> SignUpForNewsletterAsync(Sub-
        scriber subscriber);
    Task<OperationResult> OptOutFromNewsletterAsync(string
        email);
    Task<IEnumerable<Subscriber>>
        GetActiveSubscribersAsync();
}
```

> ***Information***
> *Interfaces provide:*
>
> - *A contract for implementation*
> - *Dependency inversion principle*
> - *Easier testing through mocking*
> - *Clear separation of concerns*
>
> *Using Task enables:*
>
> - *Asynchronous operations*
> - *Non-blocking execution*
> - *Better scalability*
> - *Future-proofing for database operations*

Step 3: Implement the Newsletter Service

1. Create a new class `NewsletterService.cs` in the Services folder
2. Implement the interface with async operations

> `Services/NewsletterService.cs`

```
using CloudSoft.Models;

namespace CloudSoft.Services;

public class NewsletterService : INewsletterService
{
    // Simulate a database for storing subscribers
    private static readonly List<Subscriber> _subscribers
        = [];

    public async Task<OperationResult> SignUpForNewsletter-
        Async(Subscriber subscriber)
    {
        // Simulate a long running operation
        return await Task.Run(() =>
        {
            // Check subscriber is not null and has a valid
        email
            if (subscriber == null || string.IsNullOr-
        WhiteSpace(subscriber.Email))
            {
                return OperationResult.Failure("Invalid
        subscriber information.");
            }

            // Check if the email is already subscribed
            if (IsAlreadySubscribed(subscriber.Email))
            {
                return OperationResult.Failure("You are
        already subscribed to our newsletter.");
            }

            // Add the subscriber to the list
            _subscribers.Add(subscriber);

            // Return a success message
            return OperationRes-
        ult.Success($"Welcome to our newsletter, {sub-
        scriber.Name}! You'll receive updates soon.");
        });
    }

    public async Task<OperationResult> OptOutFromNewslet-
        terAsync(string email)
```

```
    {
        // Simulate a long running operation
        return await Task.Run(() =>
        {
            // Check if the email is valid
            if (string.IsNullOrWhiteSpace(email))
            {
                return OperationResult.Failure("Invalid
    email address.");
            }

            // Find the subscriber by email
            var subscriber = FindSubscriberByEmail(email);

            if (subscriber == null)
            {
                return OperationRes-
    ult.Failure("We couldn't find your subscription in
    our system.");
            }

            // Remove the subscriber from the list
            _subscribers.Remove(subscriber);

            // Return a success message
            return OperationResult.Success("You have been
    successfully removed from our newsletter. We're
    sorry to see you go!");
        });
    }

    public async Task<IEnumerable<Subscriber>> GetActive-
        SubscribersAsync()
    {
        // Simulate a long running operation and return the
        list of subscribers
        return await Task.Run(() => _subscribers.ToList());
    }

    private static bool IsAlreadySubscribed(string email)
    {
        return _subscribers.Any(s =>
        s.Email!.Equals(email, StringComparison.Ordin-
        alIgnoreCase));
    }
```

```
    private static Subscriber?
        FindSubscriberByEmail(string email)
    {
        return _subscribers.FirstOrDefault(s =>
            s.Email!.Equals(email, StringComparison.Ordin-
        alIgnoreCase));
    }
}
```

> ***Information***
> *Task.Run() is used here because:*
>
> - *We're working with in-memory operations*
> - *In real applications, you'd use actual async operations (e.g., database calls)*
> - *It demonstrates the async pattern structure*
> - *Prepares the code for future async implementations*

Step 4: Register the Service

1. Open `Program.cs`
2. Add the service registration in the dependency injection container
   - `builder.Services.AddScoped<INewsletterService, NewsletterService>();`

> `Program.cs`

```
using CloudSoft.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

...

builder.Services.AddScoped<INewsletterService, Newslet-
        terService>();

...

var app = builder.Build();
```

> ***Information***
> *AddScoped means:*
>
> - *A new instance is created for each HTTP request*
> - *All components within the same request share the same instance*
> - *The instance is disposed at the end of the request*
>
> *Other lifetimes include:*
>
> - *Singleton: One instance for the entire application*
> - *Transient: New instance every time requested*
> - *Scoped: New instance per request (what we're using)*

Step 5: Update the Controller

1. Update `NewsletterController.cs` to use dependency injection

2. Modify methods to use async/await

> `Controllers/NewsletterController.cs`

```
using CloudSoft.Models;
using CloudSoft.Services;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Controllers;

public class NewsletterController : Controller
{
    private readonly INewsletterService _newsletterService;

    public NewsletterController(INewsletterService newslet-
        terService)
    {
        // Inject the INewsletterService via the con-
        structor from the DI container
        _newsletterService = newsletterService;
    }

    [HttpGet]
    public IActionResult Subscribe()
    {
        return View();
    }

    [HttpPost]
    public async Task<IActionResult> Subscribe(Subscriber
        subscriber)
    {
        // Validate the model
        if (!ModelState.IsValid)
        {
            return View(subscriber);
        }

        // Check if the email is already subscribed and
        return a general model level error
        var result = await _newsletterService.SignUp-
        ForNewsletterAsync(subscriber);
        if (!result.IsSuccess)
        {
            ModelState.AddModelError("Email", result.Mes-
        sage);
            return View(subscriber);
```

```
        }

        // Write to the console
        Console.WriteLine($"New subscription - Name: {sub-
        scriber.Name} Email: {subscriber.Email}");

        // Send a message to the user
        TempData["SuccessMessage"] = result.Message;

        // Return the view (using the POST-REDIRECT-GET
        pattern)
        return RedirectToAction(nameof(Subscribe));
    }

    [HttpGet]
    public async Task<IActionResult> Subscribers()
    {
        var subscribers = await _newsletterService.GetAct-
        iveSubscribersAsync();
        return View(subscribers);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Unsubscribe(string
        email)
    {
        var result = await _newsletterService.OptOutFrom-
        NewsletterAsync(email);
        if (result.IsSuccess)
        {
            TempData["SuccessMessage"] = result.Message;
        }
        return RedirectToAction(nameof(Subscribers));
    }
}
```

16.2.4 Final Tests

Test Service Registration

1. Run the application
2. Navigate to the Subscribe page

3. Submit a subscription

Expected Result

- Application starts without errors
- Subscribe form works
- Successful subscription shows success message

16.3 Exercise Done!

You've successfully implemented a service layer with modern patterns and practices!

16.4 11. Implement Unit Tests and Organize Solution

16.4.1 Goal

Organize the solution with proper project structure and implement unit tests for the service layer.

16.4.2 Learning Objectives

By the end of this exercise, you will:

- Understand .NET solution and project organization
- Learn how to create and structure test projects
- Write unit tests using xUnit
- Use dependency injection in unit tests
- Work with test assertions and test patterns

16.4.3 Step-by-Step Instructions

Step 1: Move the Web project to /src folder

1. Open a terminal in your project root folder
2. Create a new solution file:

Step 1: Create the Solution Structure

1. Delete the .sln file
2. Move the Web project to /src folder
   - all files except
     - `.gitignore`
     - `ReadMe.md`
3. Open a terminal in your project root folder
4. Create a new solution file:

```
dotnet new sln
dotnet sln add src/CloudSoft.csproj
```

5. Run the web project and check it works:

```
dotnet run --project src/CloudSoft.csproj
```

> ***Information***
> *This creates a clean solution structure with:*
>
> - *A root solution file*
> - *Source code under the src folder*
> - *Clear separation of concerns*

Step 2: Create the Test Project

1. Create a new test project in the `test` folder:

```
mkdir tests
cd tests
dotnet new xunit -n Services.UnitTests
dotnet add Services.UnitTests reference ../src/Cloud-
        Soft.csproj
```

2. Add the test project to the solution. Go back to the solution root folder:

```
dotnet sln add tests/Services.UnitTests/Ser-
        vices.UnitTests.csproj
```

3. Add required package references. Go to the test projects root folder:

```
dotnet add package Microsoft.NET.Test.Sdk
dotnet add package xunit
dotnet add package xunit.runner.visualstudio
dotnet add package coverlet.collector
```

4. Go back to the solution root folder and run the example test

```
dotnet test
```

> ***Information***
> *The test project setup:*
>
> - *Uses xUnit as the testing framework*
> - *Includes necessary testing tools*
> - *Links to the main project for testing*
> - *Follows standard naming conventions*

Step 3: Create the Newsletter Service Tests

1. Create a new test class `NewsletterServiceTests.cs` in the test project:

> `tests/Services.UnitTests/Newslet-terServiceTests.cs`

```
using CloudSoft.Models;
using CloudSoft.Services;

namespace CloudSoft.Services.UnitTests;

public class NewsletterServiceTests
{
    private readonly INewsletterService _sut;

    public NewsletterServiceTests()
    {
        _sut = new NewsletterService();
    }

    [Fact]
    public async Task SignUpForNewsletterA-
        sync_WithValidSubscriber_ReturnsSuccess()
    {
        // Arrange
        var subscriber = new Subscriber { Name = "Test
        User", Email = "user@example.com" };

        // Act
        var result = await
        _sut.SignUpForNewsletterAsync(subscriber);

        // Assert
        Assert.True(result.IsSuccess);
        Assert.Contains("Welcome to our newsletter",
        result.Message);
    }

    [Fact]
    public async Task SignUpForNewsletterA-
        sync_WithDuplicateEmail_ReturnsFailure()
    {
        // Arrange
        var subscriber1 = new Subscriber { Name = "Test
        User 1", Email = "duplicate@example.com" };
        var subscriber2 = new Subscriber { Name = "Test
        User 2", Email = "duplicate@example.com" };
        await _sut.SignUpForNewsletterAsync(subscriber1);

        // Act
```

```
        var result = await
        _sut.SignUpForNewsletterAsync(subscriber2);

        // Assert
        Assert.False(result.IsSuccess);
        Assert.Contains("already subscribed", result.Mes-
        sage);
    }

    [Fact]
    public async Task OptOutFromNewsletterA-
        sync_WithExistingEmail_ReturnsSuccess()
    {
        // Arrange
        var subscriber = new Subscriber { Name = "Test
        User", Email = "optoutuser@example.com" };
        await _sut.SignUpForNewsletterAsync(subscriber);

        // Act
        var result = await
        _sut.OptOutFromNewsletterAsync("optou-
        tuser@example.com");

        // Assert
        Assert.True(result.IsSuccess);
        Assert.Contains("successfully removed", result.Mes-
        sage);
    }

    [Fact]
    public async Task OptOutFromNewsletterA-
        sync_WithNonexistentEmail_ReturnsFailure()
    {
        // Act
        var result = await
        _sut.OptOutFromNewsletterAsync("nonexist-
        ent@example.com");

        // Assert
        Assert.False(result.IsSuccess);
        Assert.Contains("couldn't find your subscription",
        result.Message);
    }

    [Fact]
```

```
    public async Task GetActiveSubscribersA-
        sync_ReturnsAllSubscribers()
    {
        // Arrange
        var subscriber1 = new Subscriber { Name = "Test
        User 1", Email = "test1@example.com" };
        var subscriber2 = new Subscriber { Name = "Test
        User 2", Email = "test2@example.com" };
        await _sut.SignUpForNewsletterAsync(subscriber1);
        await _sut.SignUpForNewsletterAsync(subscriber2);

        // Act
        var subscribers = await _sut.GetActiveSubscribersA-
        sync();

        // Assert
        Assert.True(subscribers.Count() >= 2); // At least
        2 subscribers. Other tests add subscribers.
        Assert.Contains(subscribers, s => s.Email ==
        "test1@example.com");
        Assert.Contains(subscribers, s => s.Email ==
        "test2@example.com");
    }
}
```

> ***Information***
>
> *The unit tests follow the AAA pattern:*
>
> - *Arrange: Set up the test data*
> - *Act: Execute the method being tested*
> - *Assert: Verify the results*
>
> *Each test focuses on a specific scenario and has a clear purpose.*

Step 4: Run the Tests

1. Run all tests from the command line:

```
dotnet test
```

2. Run tests with coverage:

```
dotnet test /p:CollectCoverage=true
```

> ***Information***
> *The test runner will:*
>
> - *Execute all tests in the solution*
> - *Report test results and failures*
> - *Generate code coverage metrics (when enabled)*

16.4.4 Final Tests

Step 1: Verify Solution Structure

1. Open the solution in Visual Studio Code
2. Verify the following structure exists:

```
CloudSoft.sln
src/
  CloudSoft.csproj
  [other project files]
tests/
  Services.UnitTests/
    Services.UnitTests.csproj
    NewsletterServiceTests.cs
```

Expected Result

- Clean, organized project structure
- Solution builds successfully
- Project references are correctly set up

Step 2: Run All Tests

1. Run the tests using the command line
2. Check the test results

Expected Result

- All tests pass successfully

Step 3: Run Tests with Coverage in VS Code

1. Run tests with coverage:
    - Open Test Explorer in VS Code (left side bar, E-kolv)
    - Click the “Run Tests” button with coverage icon (beaker with checkmark)
    - View coverage results in the Test Explorer panel

Expected Result

- Test coverage report shows in VS Code
- Coverage information highlights tested/untested code
- Coverage percentage is displayed for each class

16.5 Exercise Done!

You’ve successfully organized your project and implemented unit tests!

EXERCISE 17

Data Layer

17.1 12. Implement the Repository Pattern

17.2 Implement the Repository Pattern

17.2.1 Goal

Implement a **thread-safe in-memory database** using the repository pattern for the newsletter subscription system.

17.2.2 Learning Objectives

By the end of this exercise, you will:

- Understand the **Repository Pattern** and its benefits
- Implement a **thread-safe in-memory database** using `ConcurrentDictionary`

- Create an **abstraction layer** between data access and business logic
- Register and use **repositories** with dependency injection
- Apply proper **async/await patterns** with repositories

17.2.3 Step-by-Step Instructions

Step 1: Create the Repository Interface

1. Create a new interface `ISubscriberRepository.cs` in the Repositories folder
2. Define all CRUD operations needed for subscribers

> ***File:*** `Repositories/ISubscriberRepository.cs`

```
using CloudSoft.Models;

namespace CloudSoft.Repositories;

public interface ISubscriberRepository
{
    Task<IEnumerable<Subscriber>> GetAllAsync();
    Task<Subscriber?> GetByEmailAsync(string email);
    Task<bool> AddAsync(Subscriber subscriber);
    Task<bool> UpdateAsync(Subscriber subscriber);
    Task<bool> DeleteAsync(string email);
    Task<bool> ExistsAsync(string email);
}
```

> ***Information***
> *The repository interface:*
>
> - *Defines a **contract** for data access operations*
> - *Uses **async/await pattern** for future database compatibility*
> - *Follows **SOLID principles** by depending on abstractions*
> - *Returns `Task<bool>` for operations that indicate success/failure*
> - *Uses nullable reference types for operations that might return null*

Step 2: Implement the Thread-Safe Repository

1. Create a new class `InMemorySubscriberRepository.cs` in the Repositories folder
2. Implement the interface using a thread-safe collection

> ***File:***
> `Repositories/InMemorySubscriberRepos-itory.cs`

```
using CloudSoft.Models;
using System.Collections.Concurrent;

namespace CloudSoft.Repositories;

public class InMemorySubscriberRepository : ISubscriberRe-
        pository
{
    // Using ConcurrentDictionary for thread safety
    private readonly ConcurrentDictionary<string, Sub-
        scriber> _subscribers = new(StringComparer.Ordin-
        alIgnoreCase);

    public Task<IEnumerable<Subscriber>> GetAllAsync()
    {
        return Task.FromResult(_subscribers.Values.AsEnu-
        merable());
    }

    public Task<Subscriber?> GetByEmailAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
            return Task.FromResult<Subscriber?>(null);
        }

        _subscribers.TryGetValue(email, out var sub-
        scriber);
        return Task.FromResult(subscriber);
    }

    public Task<bool> AddAsync(Subscriber subscriber)
    {
        if (subscriber == null || string.IsNullOrEmpty(sub-
        scriber.Email))
        {
            return Task.FromResult(false);
        }

        // TryAdd returns true if the key was added, false
        if it already exists
        return Task.FromResult(_subscribers.TryAdd(sub-
        scriber.Email, subscriber));
    }
```

```
    public Task<bool> UpdateAsync(Subscriber subscriber)
    {
        if (subscriber == null || string.IsNullOrEmpty(sub-
        scriber.Email))
        {
            return Task.FromResult(false);
        }

        // We need to handle the update manually since we
        need to check if key exists first
        if (!_subscribers.ContainsKey(subscriber.Email))
        {
            return Task.FromResult(false);
        }

        // Using AddOrUpdate to ensure thread safety
        _subscribers.AddOrUpdate(
            subscriber.Email,
            subscriber,
            (key, oldValue) => subscriber
        );

        return Task.FromResult(true);
    }

    public Task<bool> DeleteAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
            return Task.FromResult(false);
        }

        // TryRemove returns true if the item was removed
        return
        Task.FromResult(_subscribers.TryRemove(email, out
        _));
    }

    public Task<bool> ExistsAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
```

```
            return Task.FromResult(false);
        }

        return
        Task.FromResult(_subscribers.ContainsKey(email));
    }
}
```

> ***Information***
> *This implementation:*
>
> - *Uses **ConcurrentDictionary** for thread-safe operations*
> - *Uses **Task.FromResult** to simulate async operations (in real apps, these would be actual async database calls)*
> - *Uses **email as the key** for quick lookups*
> - *Performs **null checks** before operations*
> - *Has **case-insensitive key comparison** for email addresses*

Step 3: Update the Newsletter Service

1. Update the `NewsletterService.cs` to use the repository
2. Inject the repository through the constructor

> ***File:*** `Services/NewsletterService.cs`

```
using CloudSoft.Models;
using CloudSoft.Repositories;

namespace CloudSoft.Services;

public class NewsletterService : INewsletterService
{
    private readonly ISubscriberRepository _subscriberRe-
        pository;

    public NewsletterService(ISubscriberRepository subscri-
        berRepository)
    {
        _subscriberRepository = subscriberRepository;
    }

    public async Task<OperationResult> SignUpForNewsletter-
        Async(Subscriber subscriber)
    {
        // Check subscriber is not null and has a valid
        email
        if (subscriber == null ||
        string.IsNullOrWhiteSpace(subscriber.Email))
        {
            return OperationResult.Failure("Invalid
        subscriber information.");
        }

        // Check if the email is already subscribed
        if (await _subscriberRepository.ExistsAsync(sub-
        scriber.Email))
        {
            return OperationRes-
        ult.Failure("You are already subscribed to our
        newsletter.");
        }

        // Add the subscriber to the repository
        var success = await _subscriberReposit-
        ory.AddAsync(subscriber);

        if (!success)
        {
            return OperationResult.Failure("Failed to add
        your subscription. Please try again.");
```

```
        }

        // Return a success message
        return OperationResult.Success($"Welcome to our
        newsletter, {subscriber.Name}! You'll receive
        updates soon.");
    }

    public async Task<OperationResult> OptOutFromNewslet-
        terAsync(string email)
    {
        // Check if the email is valid
        if (string.IsNullOrWhiteSpace(email))
        {
            return OperationResult.Failure("Invalid email
        address.");
        }

        // Find the subscriber by email
        var subscriber = await _subscriberRepository.Get-
        ByEmailAsync(email);

        if (subscriber == null)
        {
            return OperationResult.Failure("We couldn't
        find your subscription in our system.");
        }

        // Remove the subscriber from the repository
        var success = await _subscriberRepository.DeleteA-
        sync(email);

        if (!success)
        {
            return OperationResult.Failure("Failed to
        remove your subscription. Please try again.");
        }

        // Return a success message
        return OperationResult.Success("You have been suc-
        cessfully removed from our newsletter. We're sorry
        to see you go!");
    }
```

```
    public async Task<IEnumerable<Subscriber>> GetActive-
        SubscribersAsync()
    {
        // Get all subscribers from the repository and
        convert to List to match the interface
        var subscribers = await _subscriberRepository.Get-
        AllAsync();
        return subscribers.ToList();
    }
}
```

> ***Information***
> *The updated service:*
>
> - ***Depends on the abstraction*** *(ISubscriberRepository) not the implementation*
> - *Uses* ***constructor injection*** *for better testability*
> - *Handles* ***failure cases*** *from repository operations*
> - *Maintains the same* ***business logic*** *but delegates data access to the repository*

Step 4: Register the Repository in DI Container

1. Open `Program.cs`
2. Register the repository as a singleton service

> ***File:*** `Program.cs`

```
using CloudSoft.Repositories;
using CloudSoft.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

// Register repository
builder.Services.AddSingleton<ISubscriberRepository,
        InMemorySubscriberRepository>();

// Register service (depends on repository)
builder.Services.AddScoped<INewsletterService, Newslet-
        terService>();

// ...rest of the application setup...
```

> ***Information***
>
> - *The repository is registered as a **singleton** since it represents our in-memory database*
> - *This ensures all services **share the same data store***
> - *The service remains **scoped** (one instance per request)*
> - *The repository is registered **before** the service that depends on it*
> - *This layered dependency chain follows the **Dependency Inversion Principle***

17.2.4 Final Tests

Testing the Repository Implementation

1. Run the application
2. Go to the Subscribe page and add a new subscriber

3. Check the Subscribers page to verify the subscriber was added
4. Try to subscribe with the same email to test duplicate prevention
5. Test the unsubscribe functionality

Expected Result

- Subscription form works correctly
- Subscribers are stored in the in-memory database
- The application prevents duplicate subscriptions
- Subscribers can be removed successfully
- The data persists between requests (while the app is running)

Testing the Thread Safety

1. Open the application in multiple browser tabs/windows
2. Try to add multiple subscribers simultaneously
3. Verify that all subscribers are correctly added without data corruption

Expected Result

- All subscribers are correctly added to the repository
- No exceptions or data corruption occurs during concurrent access
- The application handles multiple requests properly

17.3 Exercise Done!

You've successfully implemented the Repository Pattern with a thread-safe in-memory database for newsletter subscriptions!

17.4 13. Implementing MongoDB Repository

17.5 Implementing MongoDB Repository (Development)

17.5.1 Goal

Implement a **MongoDB repository** for subscribers and configure a **local MongoDB environment** using Docker with a **feature flag toggle**.

17.5.2 Learning Objectives

By the end of this exercise, you will:

- Understand how to **integrate MongoDB** with ASP.NET Core
- Use **Docker Compose** to create a development database environment
- Implement the **Repository Pattern** with MongoDB
- Configure **dependency injection** with MongoDB
- Apply the **Options Pattern** for database configuration
- Use **Feature Flags** for toggling implementations

17.5.3 Step-by-Step Instructions

Step 1: Update the Subscriber Model with MongoDB Annotations

1. Add the MongoDB.Driver NuGet package to the project using the terminal command

```
# Add MongoDB.Driver NuGet package to the project
dotnet add package MongoDB.Driver
```

> ***Information***
> *The MongoDB.Driver package:*
>
> - *Provides the official .NET driver for MongoDB*
> - *Supports both synchronous and asynchronous operations*
> - *Includes BSON serialization capabilities*
> - *Enables integration with Azure Cosmos DB's MongoDB API*

2. Update the Subscriber model with MongoDB annotations

> ***File:*** `Models/Subscriber.cs`

```
using System.ComponentModel.DataAnnotations;
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace CloudSoft.Models;

public class Subscriber
{
    [BsonId]
    [BsonRepresentation(BsonType.ObjectId)]
    public string? Id { get; set; }

    [Required]
    [StringLength(20, ErrorMessage =
        "Name cannot exceed 20 characters")]
    [BsonElement("name")]
    public string? Name { get; set; }

    [Required]
    [EmailAddress]
    [RegularExpression("^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]
        +\\.[a-zA-Z]{2,}$", ErrorMessage =
        "Missing top level domain")]
    [BsonElement("email")]
    public string? Email { get; set; }
}
```

> ***Information***
> *The BSON annotations:*
>
> - ***[BsonId]*** *marks the Id property as the document's primary key*
> - ***[BsonRepresentation]*** *specifies how to store the Id as an ObjectId*
> - ***[BsonElement]*** *maps C# property names to MongoDB field names*
> - *Maintaining existing validation attributes ensures the model works for both web and database layers*

Step 2: Create MongoDB Configuration Options

1. Create a new class for MongoDB configuration options in a new directory called `Configurations`
2. Define the MongoDB connection properties

> ***File:*** `Configurations/MongoDbOptions.cs`

```
namespace CloudSoft.Configurations;

public class MongoDbOptions
{
    public const string SectionName = "MongoDb";

    public string ConnectionString { get; set; } =
        string.Empty;
    public string DatabaseName { get; set; } =
        string.Empty;
    public string SubscribersCollectionName { get;
        set; } = string.Empty;
}
```

> ***Information***
> *Using the Options Pattern provides:*
>
> - *Strong typing for configuration values*
> - *Validation of configuration data*
> - *Grouping of related settings*
> - *Isolation of configuration concerns*
> - *A consistent approach throughout your application*

Step 3: Implement MongoDB Repository

1. Create the MongoDB implementation of the ISubscriberRepository interface
2. Implement CRUD operations using MongoDB

> ***File:*** `Repositories/MongoDbSubscriberRepository.cs`

```
using CloudSoft.Models;
using MongoDB.Driver;

namespace CloudSoft.Repositories;

public class MongoDbSubscriberRepository : ISubscri-
        berRepository
{
    private readonly IMongoCollection<Subscriber> _sub-
        scribers;

    public
        MongoDbSubscriberRepository(IMongoCollection<Sub-
        scriber> subscribers)
    {
        _subscribers = subscribers;
    }

    public async Task<IEnumerable<Subscriber>> GetAllA-
        sync()
    {
        return await _subscribers.Find(_ =>
        true).ToListAsync();
    }

    public async Task<Subscriber?>
        GetByEmailAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
            return null;
        }

        return await _subscribers.Find(s => s.Email ==
        email).FirstOrDefaultAsync();
    }

    public async Task<bool> AddAsync(Subscriber sub-
        scriber)
    {
        if (subscriber == null ||
        string.IsNullOrEmpty(subscriber.Email))
        {
            return false;
        }
```

```
        // Check if subscriber with this email already
        exists
        var existingSubscriber = await
        GetByEmailAsync(subscriber.Email);
        if (existingSubscriber != null)
        {
            return false;
        }

        try
        {
            await _sub-
        scribers.InsertOneAsync(subscriber);
            return true;
        }
        catch
        {
            return false;
        }
    }

    public async Task<bool> UpdateAsync(Subscriber sub-
        scriber)
    {
        if (subscriber == null ||
        string.IsNullOrEmpty(subscriber.Email))
        {
            return false;
        }

        try
        {
            var result = await _subscribers.ReplaceOneA-
        sync(
                s => s.Email == subscriber.Email,
                subscriber,
                new ReplaceOptions { IsUpsert =
        false });

            return result.ModifiedCount > 0;
        }
        catch
        {
```

```
            return false;
        }
    }

    public async Task<bool> DeleteAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
            return false;
        }

        try
        {
            var result = await _subscribers.DeleteOneA-
        sync(s => s.Email == email);
            return result.DeletedCount > 0;
        }
        catch
        {
            return false;
        }
    }

    public async Task<bool> ExistsAsync(string email)
    {
        if (string.IsNullOrEmpty(email))
        {
            return false;
        }

        return await _subscribers.CountDocumentsAsync(s
        => s.Email == email) > 0;
    }
}
```

> ***Information***
> *Key MongoDB operations:*
>
> - ***Find()*** *to query documents with filters*
> - ***InsertOneAsync()*** *to add new documents*
> - ***ReplaceOneAsync()*** *to update existing documents*
> - ***DeleteOneAsync()*** *to remove documents*
> - ***CountDocumentsAsync()*** *to check for existence*
> - *Each method includes proper error handling and validation*

Step 4: Configure Docker for MongoDB

1. Create the infra directory for Docker Compose files
2. Create a `docker-compose.yml` file with MongoDB and Mongo Express

> ***File:*** `infra/docker-compose.yml`

```yaml
services:
  mongodb:
    image: mongo:latest
    container_name: mongodb
    restart: always
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USERNAME:-
        root}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD:-
        example}
    volumes:
      - mongodb_data:/data/db
    networks:
      - mongo_network

  mongo-express:
    image: mongo-express:latest
    container_name: mongo-express
    restart: always
    ports:
      - "8081:8081"
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: ${MONGO_USER-
        NAME:-root}
      ME_CONFIG_MONGODB_ADMINPASSWORD: ${MONGO_PASS-
        WORD:-example}
      ME_CONFIG_MONGODB_SERVER: mongodb
    depends_on:
      - mongodb
    networks:
      - mongo_network

volumes:
  mongodb_data:

networks:
  mongo_network:
    driver: bridge
```

3. Start the MongoDB containers using Docker Compose

```
# Navigate to the infra directory
cd infra

# Start MongoDB and Mongo Express containers in detached
        mode
docker-compose up -d
```

4. Stop the containers when you're done

```
# Stop the containers
docker-compose down

# Stop and remove volumes (cleans up data)
docker-compose down -v
```

> ***Information***
> *The Docker Compose configuration:*
>
> - *Creates a **MongoDB container** with volume for data persistence*
> - *Includes **Mongo Express** for easy database management through a web UI*
> - *Uses **environment variables** for credentials to avoid hardcoding*
> - *Creates a dedicated **network** for container communication*
> - *Exposes necessary **ports** for access from the host machine*

Step 5: Configure Application Settings

1. Update appsettings.json with MongoDB configuration and feature flag
2. Set up development-specific settings in appsettings.Development.json

> ***File:*** `appsettings.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "FeatureFlags": {
    "UseMongoDb": false
  },
  "MongoDb": {
    "ConnectionString": "mongodb://{username}:{password}
        @{hostname}:{port}",
    "DatabaseName": "cloudsoft",
    "SubscribersCollectionName": "subscribers"
  }
}
```

> ***File:*** `appsettings.Development.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "FeatureFlags": {
    "UseMongoDb": true
  },
  "MongoDb": {
    "ConnectionString": "mongodb://
        root:example@localhost:27017",
    "DatabaseName": "cloudsoft",
    "SubscribersCollectionName": "subscribers"
  }
}
```

> ***Information***
> *The configuration approach:*
>
> - *Separates **production** and **development** settings*
> - *Uses **placeholders** in production configuration*
> - *Provides **concrete values** for development*
> - *Follows the **convention over configuration** principle*
> - *Keeps sensitive data out of source control*
> - *Includes a **feature flag** to easily toggle between repository implementations*
> - *Defaults to the **in-memory database** if not specified*

3. Create a .env file for environment variables

> ***File:*** `infra/.env`

```
# Make sure this file is not checked into source con-
trol!

# Use the following commands to export the environment
variables to the current shell
# set -a
# source ./.env
# set +a

# You can also use dotnet user-secrets to set the envir-
onment variables
# dotnet user-secrets set "MongoDb__ConnectionString"
"mongodb:..."
# dotnet user-secrets set "FeatureFlags__UseMongoDb"
"false"
# dotnet user-secrets list
# dotnet user-secrets clear

# Note the double underscore (__) to indicate hierarchi
in environment variables

MongoDb__ConnectionString="mongodb://{username}:{pass-
word}@{hostname}:{port}"
FeatureFlags__UseMongoDb=false
```

Step 6: Update Program.cs to Configure MongoDB with Feature Flag

1. Add code to check the feature flag before registering repositories
2. Configure the MongoDB client and collection only when needed

> ***File:*** `Program.cs`

```
using CloudSoft.Models;
using CloudSoft.Configurations;
using MongoDB.Driver;

...


// Add services to the container.
builder.Services.AddControllersWithViews();

// Check if MongoDB should be used (default to false if
        not specified)
bool useMongoDb =
        builder.Configuration.GetValue<bool>("Feature-
        Flags:UseMongoDb");

if (useMongoDb)
{
    // Configure MongoDB options
    builder.Services.Configure<MongoDbOptions>(

        builder.Configuration.GetSection(MongoDbOptions.Sec-
        tionName));

    // Configure MongoDB client
    builder.Services.AddSingleton<IMongoClient>(service-
        Provider => {
        var mongoDbOptions = builder.Configuration.Get-
        Section(MongoDbOptions.SectionName).Get<MongoD-
        bOptions>();
        return new MongoClient(mongoDbOptions?.Connec-
        tionString);
    });

    // Configure MongoDB collection
    builder.Services.AddSingleton<IMongoCollection<Sub-
        scriber>>(serviceProvider => {
        var mongoDbOptions = builder.Configuration.Get-
        Section(MongoDbOptions.SectionName).Get<MongoD-
        bOptions>();
        var mongoClient = serviceProvider.GetRequired-
        Service<IMongoClient>();
        var database = mongoClient.GetDatabase(mongoDb-
        Options?.DatabaseName);
        return database.GetCollection<Subscriber>(mon-
        goDbOptions?.SubscribersCollectionName);
```

```
    });

    // Register MongoDB repository

        builder.Services.AddSingleton<ISubscriberRepos-
        itory, MongoDbSubscriberRepository>();

    Console.WriteLine("Using MongoDB repository");
}
else
{
    // Register in-memory repository as fallback

        builder.Services.AddSingleton<ISubscriberRepos-
        itory, InMemorySubscriberRepository>();

    Console.WriteLine("Using in-memory repository");
}

// Register service (depends on repository)
builder.Services.AddScoped<INewsletterService, Newslet-
        terService>();



...
```

> ***Information***
> *The Program.cs configuration:*
>
> - *Uses a **feature flag** to determine which repository implementation to use*
> - *Registers the **MongoDB services conditionally** to optimize resources*
> - *Provides **console logging** to indicate which repository is being used*
> - *Defaults to the **in-memory repository** for simplicity when MongoDB isn't needed*
> - *Maintains the same **service layer** regardless of the repository implementation*
> - *Follows the **Strategy Pattern** to easily swap implementations*

17.5.4 Final Tests

Starting the MongoDB Environment

1. Navigate to the infra directory in the terminal
2. Start the MongoDB containers using Docker Compose

```
# Navigate to the infra directory
cd infra

# Start containers
docker-compose up -d

# Check container status
docker-compose ps
```

1. Access Mongo Express at http://localhost:8081 to verify the database setup
2. When finished with testing, stop the containers

```
# Stop the containers
docker-compose down
```

Expected Result

- Both MongoDB and Mongo Express containers should be running
- The cloudsoft database will be created automatically on first access

Testing the Repository Implementation

1. Run the application
2. Go to the Subscribe page and add a new subscriber
3. Check the Subscribers page to verify the subscriber was added
4. Check Mongo Express to verify the data was stored in MongoDB

Expected Result

- Subscription form works correctly
- Subscribers are stored in MongoDB
- The application prevents duplicate subscriptions
- You can view the MongoDB data through Mongo Express

Testing Repository Toggle

1. In appsettings.Development.json, set "UseMongoDb" to false
2. Run the application and verify it uses the in-memory repository (check console output)
3. Set "UseMongoDb" back to true
4. Run the application and verify it uses the MongoDB repository

Expected Result

- The application correctly switches between repository implementations
- No code changes are needed to toggle between implementations
- The console output confirms which repository is being used
- The application functions correctly with both implementations

17.6 Exercise Done!

You've successfully implemented a MongoDB repository with Docker for local development, including a feature flag to toggle between implementations!

17.7 14. Migrating to CosmosDB

17.8 Migrating to CosmosDB

17.8.1 Goal

Migrate the Newsletter application's data layer from a local Docker-hosted MongoDB instance to Azure Cosmos DB. Leverage the repository pattern abstraction to make this migration primarily a configuration change — updating the connection string and adjusting compatibility settings — without modifying application code.

WHAT YOU'LL LEARN:

- How repository pattern abstraction enables painless database migration
- How to configure Cosmos DB connection strings in ASP.NET Core
- How to handle Cosmos DB MongoDB API compatibility differences
- How to use feature flags to toggle between database backends
- How to verify data operations against a cloud-hosted database

17.8.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- √ Completed the Newsletter app with repository pattern and MongoDB repository implementation
- √ Completed the Cloud Databases section (a provisioned Cosmos DB account with MongoDB API)
- √ The Cosmos DB connection string available
- √ The Newsletter application running locally with Docker MongoDB

17.8.3 Exercise Steps

Overview

1. **Prepare the Cosmos DB Database**
2. **Update the Connection String Configuration**
3. **Handle Cosmos DB Compatibility Settings**
4. **Configure the Feature Flag**
5. **Test the Migration**

Step 1: *Prepare the Cosmos DB Database*

Ensure your Cosmos DB account has a database and collection ready to receive the Newsletter application's subscriber data. The collection needs a shard key that aligns with the application's query patterns.

1. **Open** the Azure Portal at https://portal.azure.com
2. **Navigate to** your Cosmos DB account
3. **Open** Data Explorer from the left menu
4. **Create** a new database named `cloudsoft` (if it does not already exist)
5. **Create** a new collection with the following settings:
   - **Collection id**: `subscribers`
   - **Shard key**: `/email`
6. **Verify** the empty collection appears in the Data Explorer tree

CONCEPT DEEP DIVE

The shard key `/email` is chosen because email addresses have high cardinality (every subscriber has a unique email) and the application frequently queries by email (for duplicate checking and unsubscribe operations). This ensures efficient single-partition queries for the most common operations.

The database and collection names must match what the application expects in its `MongoDbOptions` configuration. Your `appsettings.json` sets `DatabaseName` to `cloudsoft` and `SubscribersCollectionName` to `subscribers`, so the Cosmos DB resources must use these exact names.

⚠ **Common Mistakes**

- Creating a collection with a shard key that does not match a field in your documents causes all documents to land in a single null-key partition
- The shard key path is case-sensitive — `/email` and `/Email` are different. Check your C# model's `[BsonElement]` attribute to confirm the exact field name

✓ **Quick check:** Data Explorer shows the `cloudsoft` database with an empty `subscribers` collection

Step 2: *Update the Connection String Configuration*

Replace the local MongoDB connection string with the Cosmos DB connection string in the application configuration. ASP.NET Core's configuration system supports environment-specific overrides, so you can maintain separate settings for development (Docker MongoDB) and cloud (Cosmos DB).

1. **Open** the `appsettings.json` file in the project root
2. **Locate** the MongoDB configuration section

3. **Update** the connection string and database name to point to Cosmos DB:

   > `appsettings.json`

```
{
  "MongoDb": {
    "ConnectionString": "mongodb://your-account:your-
        key@your-account.mongo.cosmos.azure.com:10255/?
        ssl=true&retry-
        writes=false&maxIdleTimeMS=120000",
    "DatabaseName": "cloudsoft",
    "SubscribersCollectionName": "subscribers"
  }
}
```

4. **Alternatively**, use User Secrets to keep the connection string out of source code:

```
dotnet user-secrets set "MongoDb:ConnectionString" "mon-
        godb://your-account:your-key@your-
        account.mongo.cosmos.azure.com:10255/?
        ssl=true&retry-
        writes=false&maxIdleTimeMS=120000"
```

CONCEPT DEEP DIVE

The connection string is the only infrastructure-level change needed to switch from MongoDB to Cosmos DB. This is possible because Cosmos DB implements the MongoDB wire protocol — the MongoDB.Driver NuGet package communicates with Cosmos DB using the same protocol it uses with native MongoDB.

Using `dotnet user-secrets` stores the connection string outside the project directory (in `~/.microsoft/usersecrets/`), preventing accidental commits to Git. User Secrets are automatically loaded in the Development environment by the ASP.NET Core configuration system. For production, you would use Azure App Service configuration or Key Vault references.

⚠ **Common Mistakes**

- Forgetting `retrywrites=false` in the connection string causes write operations to fail — Cosmos DB's MongoDB API does not support retryable writes
- Leaving the old Docker MongoDB connection string active while Docker is stopped causes connection timeout errors
- Committing the Cosmos DB connection string (which contains the access key) to Git exposes your database credentials

√ **Quick check:** The `MongoDb:ConnectionString` value starts with `mongodb://` and contains `.mongo.cosmos.azure.com:10255`

Step 3: *Handle Cosmos DB Compatibility Settings*

Verify that the Cosmos DB connection string includes the parameters needed for compatibility with the MongoDB.Driver. In your application, the `MongoClient` is created

in `Program.cs` using the connection string directly — no code changes are needed, only connection string parameters.

1. **Open** `Program.cs` and **locate** the `MongoClient` registration
2. **Verify** that the existing code creates the client directly from the connection string:

   > `Program.cs`

   ```
   builder.Services.AddSingleton<IMongoClient>(servicePro-
           vider => {
       var mongoDbOptions = builder.Configuration.GetSec-
           tion(MongoDbOptions.SectionName).Get<MongoDbOp-
           tions>();
       return new MongoClient(mongoDbOptions?.Connection-
           String);
   });
   ```

3. **Confirm** that the connection string from Step 2 includes `retrywrites=false` and `ssl=true`:

   ```
   mongodb://...?ssl=true&retry-
   writes=false&maxIdleTimeMS=120000
   ```

   Because `new MongoClient(connectionString)` parses these parameters automatically, no code changes are required in `Program.cs` or the repository.

CONCEPT DEEP DIVE

Cosmos DB's MongoDB API supports most MongoDB operations but has some differences in behavior. The most impactful for this migration is retryable writes — MongoDB 4.2+ enables retryable writes by default, but Cosmos DB does not support the server-side session mechanics that retryable writes require. Including `retrywrites=false` in the connection string prevents the driver from attempting this unsupported feature.

The `maxIdleTimeMS=120000` parameter sets the maximum idle time for connections in the connection pool. Cosmos DB may close idle connections after a period, so this setting helps the driver manage the pool proactively.

Since your `Program.cs` already creates the `MongoClient` with `new MongoClient(connectionString)`, these connection string parameters are parsed and applied automatically. No code changes are needed — the migration is purely configuration.

⚠ **Common Mistakes**

- The MongoDB.Driver defaults to `RetryWrites = true` in newer versions — `retrywrites=false` in the connection string overrides this
- Setting `ssl=false` or omitting `ssl=true` causes connection failures — Cosmos DB requires TLS/SSL on all connections
- Forgetting `retrywrites=false` causes write operations to fail with "Command insert failed" errors

√ **Quick check:** The application starts without MongoDB connection errors in the console output

Step 4: *Configure the Feature Flag*

If your application uses feature flags to toggle between data layer implementations, update the configuration to activate the MongoDB (Cosmos DB) repository. Because the `MongoDb-`

`SubscriberRepository` uses the standard MongoDB.Driver, it works with both native MongoDB and Cosmos DB without modification.

1. **Open** `appsettings.json`
2. **Locate** the feature flags section
3. **Ensure** the MongoDB feature flag is enabled:

   > `appsettings.json`

   ```
   {
     "FeatureFlags": {
       "UseMongoDb": true
     }
   }
   ```

4. **Verify** that the service registration in `Program.cs` uses this flag to select the repository implementation:

   > `Program.cs`

```
bool useMongoDb =
        builder.Configuration.GetValue<bool>("Feature-
        Flags:UseMongoDb");

if (useMongoDb)
{
    // Configure MongoDB options
    builder.Services.Configure<MongoDbOptions>(

        builder.Configuration.GetSection(MongoDbOptions.Sec-
        tionName));

    // Configure MongoDB client
    builder.Services.AddSingleton<IMongoClient>(service-
        Provider => {
        var mongoDbOptions = builder.Configuration.Get-
        Section(MongoDbOptions.SectionName).Get<MongoD-
        bOptions>();
        return new MongoClient(mongoDbOptions?.Connec-
        tionString);
    });

    // Configure MongoDB collection
    builder.Services.AddSingleton<IMongoCollection<Sub-
        scriber>>(serviceProvider => {
        var mongoDbOptions = builder.Configuration.Get-
        Section(MongoDbOptions.SectionName).Get<MongoD-
        bOptions>();
        var mongoClient = serviceProvider.GetRequired-
        Service<IMongoClient>();
        var database = mongoClient.GetDatabase(mongoDb-
        Options?.DatabaseName);
        return database.GetCollection<Subscriber>(mon-
        goDbOptions?.SubscribersCollectionName);
    });

    // Register MongoDB repository

        builder.Services.AddSingleton<ISubscriberRepos-
        itory, MongoDbSubscriberRepository>();

    Console.WriteLine("Using MongoDB repository");
}
else
{
    // Register in-memory repository as fallback
```

```
        builder.Services.AddSingleton<ISubscriberRepos-
        itory, InMemorySubscriberRepository>();

    Console.WriteLine("Using in-memory repository");
}
```

The important thing to note is that **this code does not change** — it was already written in the previous exercise. The `MongoClient` receives the connection string from `appsettings.json`, so switching from Docker MongoDB to Cosmos DB is handled entirely by the configuration update in Step 2.

CONCEPT DEEP DIVE

The feature flag pattern allows you to switch between repository implementations without changing code or redeploying. With `UseMongoDb` set to `true`, the dependency injection container registers the full MongoDB service chain: `MongoDbOptions` → `IMongoClient` → `IMongoCollection<Subscriber>` → `MongoDbSubscriberRepository`. The `MongoClient` reads the connection string from configuration, so it now connects to Cosmos DB instead of Docker MongoDB. Setting the flag to `false` falls back to the in-memory implementation for local development without any database dependency.

This is the key benefit of the repository pattern in action: the controller and service layer code has no knowledge of whether subscribers are stored in memory, in Docker MongoDB, or in Azure Cosmos DB. The `ISubscriberRepository` interface abstracts away the storage mechanism entirely.

√ **Quick check:** The application starts with the console output showing “Using MongoDB repository”, and no errors about missing services or unresolvable dependencies

Step 5: *Test the Migration*

Verify that all CRUD operations work correctly against the Cosmos DB backend. Test the same user workflows you would test with the local MongoDB — subscribing, listing, and unsubscribing — and confirm the data appears in the Azure Portal.

1. **Start** the application:

```
dotnet run
```

2. **Navigate to** the application in your browser (typically `http://localhost:5000` or the URL shown in the console)
3. **Test Subscribe** — add a new subscriber:
   - Enter an email address and name in the subscription form
   - Submit the form
   - Verify the subscriber appears in the subscriber list
4. **Test List** — view all subscribers:
   - Navigate to the subscribers page
   - Verify the newly added subscriber is displayed
5. **Test Unsubscribe** — remove a subscriber:
   - Click the unsubscribe button for the test subscriber
   - Verify the subscriber is removed from the list
6. **Verify in the Azure Portal:**
   - **Open** Data Explorer in your Cosmos DB account
   - **Expand** `cloudsoft` → `subscribers`
   - **Click** on Documents to view stored subscriber data

  - **Confirm** the documents contain the expected fields (email, name, etc.)
  - **Confirm** that unsubscribed entries have been removed (or marked as unsubscribed, depending on your implementation)

7. **Test adding multiple subscribers** and verify they appear in both the application and Data Explorer

√ ***Success indicators:***

- *Subscribe operation creates a document visible in both the app and Data Explorer*
- *List operation displays all subscribers stored in Cosmos DB*
- *Unsubscribe operation removes (or marks) the subscriber in Cosmos DB*
- *No errors appear in the application console or browser developer tools*
- *Data Explorer shows documents with the correct structure and field names*

√ ***Final verification checklist:***

☐ *Application connects to Cosmos DB without errors*

☐ *All CRUD operations (subscribe, list, unsubscribe) work correctly*

☐ *Data is visible in Azure Portal Data Explorer*

☐ *No code changes were needed in the repository or controller (only configuration)*

☐ *The feature flag correctly toggles between in-memory and Cosmos DB repositories*

17.8.4 Common Issues

> ***If you encounter problems:***
> ***"Connection refused" or timeout errors:*** *Verify the connection string is correct and your network allows outbound connections on port 10255. Some corporate networks or VPNs block non-standard ports.*
> ***"Command insert failed" or write errors:*** *Ensure* `retrywrites=false` *is included in the connection string in* `appsettings.json`*.*
> ***"Collection doesn't exist" errors:*** *The MongoDB.Driver may auto-create collections in native MongoDB but may not in Cosmos DB depending on the API version. Create the collection manually in Data Explorer first.*
> ***Data appears in Cosmos DB but not in the app (or vice versa):*** *Check that the database name and collection name in* `appsettings.json` *exactly match the names in Cosmos DB (case-sensitive).*
> ***Application still connects to Docker MongoDB:*** *Verify the feature flag is enabled and the connection string points to Cosmos DB. Check if Docker is still running and whether the application might be reading from an environment variable that overrides* `appsettings.json`*.*
> ***Still stuck?*** *Start Docker MongoDB and switch back to the local configuration (change the connection string back) to confirm the issue is Cosmos DB-specific rather than a general application problem.*

17.8.5 Summary

You've successfully migrated the Newsletter application from Docker MongoDB to Azure Cosmos DB which:

- √ Demonstrates the power of the repository pattern — migration was a configuration change, not a code change
- √ Connects to a fully managed cloud database with automatic scaling and high availability
- √ Uses the same MongoDB.Driver code for both local MongoDB and cloud Cosmos DB
- √ Maintains the feature flag toggle for flexible development workflows

> **KEY TAKEAWAY:**
>
> The repository pattern abstraction you built earlier paid off here — switching the data backend from a local Docker MongoDB container to a globally distributed Azure Cosmos DB instance required only a connection string change and a compatibility setting. No controller code, no service code, and no repository code needed modification. This is the concrete benefit of programming against interfaces and following the Dependency Inversion Principle: your application's core logic is decoupled from its infrastructure dependencies.

17.8.6 Going Deeper (Optional)

> ***Want to explore more?***
>
> - *Configure separate* `appsettings.Development.json` *(Docker MongoDB) and* `appsettings.Production.json` *(Cosmos DB) for automatic environment-based switching*
> - *Explore Cosmos DB's Metrics blade to understand Request Unit consumption for each operation*
> - *Add a health check endpoint that verifies database connectivity using* `IHealthCheck`
> - *Investigate Cosmos DB's global distribution by adding a read replica in a second Azure region*

17.8.7 Done! 🎉

Great job! You've migrated your Newsletter application to Azure Cosmos DB with zero code changes. This exercise demonstrates why architectural patterns like the repository pattern and dependency injection matter — they make significant infrastructure changes into routine configuration updates.

17.9 15. Azure Blob Storage for Images

17.10 Azure Blob Storage for Images

17.10.1 Goal

Create a **hero section** in the About page with an image background that uses **local storage** in development and **Azure Blob Storage** in production.

17.10.2 Learning Objectives

By the end of this exercise, you will:

- Implement a **service abstraction** for image serving
- Configure **Azure Blob Storage** for production image hosting
- Create a **feature flag** to toggle between local and cloud storage
- Build a **responsive hero section** with a background image
- Use the **Options Pattern** for Azure Blob Storage configuration

17.10.3 Step-by-Step Instructions

Step 1: Create the Image Service Interface

1. Create a new interface `IImageService.cs` in a new `Storage` folder
2. Define a method to get an image URL based on an image name

> ***File:*** `Storage/IImageService.cs`

```
namespace CloudSoft.Storage;

public interface IImageService
{
    /// <summary>
    /// Gets the URL for an image based on the specified
        image name
    /// </summary>
    /// <param name="imageName">The name of the image
        (e.g. "hero.png")</param>
    /// <returns>The full URL to the image</returns>
    string GetImageUrl(string imageName);
}
```

> ***Information***
> *This interface:*
>
> - *Follows the **Interface Segregation Principle** with a focused responsibility*
> - *Uses **clean documentation** with XML comments*
> - *Provides a **simple contract** that can have multiple implementations*
> - *Follows the **dependency inversion principle** for decoupling*

Step 2: Create Local Image Service Implementation

1. Create a class `LocalImageService.cs` in the `Storage` folder
2. Implement the interface to serve images from local wwwroot/images folder

> ***File:*** `Storage/LocalImageService.cs`

```
using Microsoft.AspNetCore.Hosting;

namespace CloudSoft.Storage;

public class LocalImageService : IImageService
{
    private readonly IWebHostEnvironment _webHostEnvir-
        onment;
    private readonly IHttpContextAccessor _httpContex-
        tAccessor;

    public LocalImageService(
        IWebHostEnvironment webHostEnvironment,
        IHttpContextAccessor httpContextAccessor)
    {
        _webHostEnvironment = webHostEnvironment;
        _httpContextAccessor = httpContextAccessor;
    }

    public string GetImageUrl(string imageName)
    {
        // Build the relative URL to the image in the
        wwwroot/images folder
        var request = _httpContextAc-
        cessor.HttpContext?.Request;
        var baseUrl = $"{request?.Scheme}://
        {request?.Host}";

        return $"{baseUrl}/images/{imageName}";
    }
}
```

> ***Information***
> *This implementation:*
>
> - *Uses **HttpContextAccessor** to get the current request information*
> - *Builds a **fully qualified URL** to the image*
> - *Returns a path to the image in the **wwwroot/images** folder*
> - *Uses proper **dependency injection** for required services*

Step 3: Create Azure Blob Storage Implementation

1. Create a configuration class for Azure Blob Storage options

> ***File:*** `Configurations/AzureBlobOptions.cs`

```
namespace CloudSoft.Configurations;

public class AzureBlobOptions
{
    public const string SectionName = "AzureBlob";

    public string ContainerUrl { get; set; } =
        string.Empty;
}
```

2. Create an Azure Blob Storage service implementation

> ***File:*** `Services/AzureBlobImageService.cs`

```
using CloudSoft.Configurations;
using Microsoft.Extensions.Options;

namespace CloudSoft.Storage;

public class AzureBlobImageService : IImageService
{
    private readonly string _blobContainerUrl;

    public AzureBlobImageService(IOptions<AzureBlobOp-
        tions> options)
    {
        _blobContainerUrl = options.Value.ContainerUrl;
    }

    public string GetImageUrl(string imageName)
    {
        // Azure Blob Storage URLs are case sensitive
        // For production, images will be served
        directly from the Blob Storage CDN
        return $"{_blobContainerUrl}/{imageName}";
    }
}
```

> ***Information***
> *These files provide:*
>
> - ***Options pattern*** *for storing Azure Blob Storage configuration*
> - *A clean way to* ***inject configuration*** *into the service*
> - *A simple implementation that* ***concatenates*** *the container URL with image name*
> - *A* ***production-ready*** *solution for serving images from Azure Blob Storage*

Step 4: Configure Application Settings

1. Update `appsettings.json` to include Azure Blob Storage settings and feature flag

> ***File:*** `appsettings.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "FeatureFlags": {
    "UseMongoDb": false,
    "UseAzureStorage": false
  },
  "MongoDb": {
    "ConnectionString": "mongodb://{username}:{password}
        @{hostname}:{port}",
    "DatabaseName": "cloudsoft",
    "SubscribersCollectionName": "subscribers"
  },
  "AzureBlob": {
    "ContainerUrl": "https://{account-
        name}.blob.core.windows.net/{container}"
  }
}
```

2. Configure development-specific settings in `appsettings.Development.json`

> ***File:*** `appsettings.Development.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "FeatureFlags": {
    "UseMongoDb": true,
    "UseAzureStorage": false
  },
  "MongoDb": {
    "ConnectionString": "mongodb://
        root:example@localhost:27017",
    "DatabaseName": "cloudsoft",
    "SubscribersCollectionName": "subscribers"
  },
  "AzureBlob": {
    "ContainerUrl": "https://localhost:7240/images"
  }
}
```

> ***Information***
> *The configuration:*
>
> - *Uses **feature flags** to toggle between local and Azure Blob Storage*
> - *Keeps **placeholder values** in the production configuration*
> - *Provides **concrete local values** for development*
> - *Follows the **convention over configuration** principle*
> - *Makes it easy to **switch between environments** without code changes*

Step 5: Register Services in Program.cs

1. Update `Program.cs` to register the image service based on the feature flag
2. Configure the Azure Blob Storage options

> ***File:*** `Program.cs`

```
using CloudSoft.Storage;

...

// Add HttpContextAccessor for URL generation
builder.Services.AddHttpContextAccessor();

// Configure Azure Blob options
builder.Services.Configure<AzureBlobOptions>(

        builder.Configuration.GetSection(AzureBlobOp-
        tions.SectionName));

// Check if Azure Storage should be used
bool useAzureStorage =
        builder.Configuration.GetValue<bool>("Feature-
        Flags:UseAzureStorage");

if (useAzureStorage)
{
    // Register Azure Blob Storage image service for
        production
    builder.Services.AddSingleton<IImageService,
        AzureBlobImageService>();
    Console.WriteLine("Using Azure Blob Storage for
        images");
}
else
{
    // Register local image service for development
    builder.Services.AddSingleton<IImageService, Loc-
        alImageService>();
    Console.WriteLine("Using local storage for images");
}

...


app.UseStaticFiles();

...
```

> ***Information***
> *This setup:*
>
> - *Adds **HttpContextAccessor** which is required by the local image service*
> - *Uses the **Options Pattern** to bind configuration to the options class*
> - ***Conditionally registers** the image service based on the feature flag*
> - *Uses **singleton lifetime** for the image service which is appropriate for stateless services*
> - *Adds **console logging** to indicate which service is being used*

Step 6: Update the HomeController

1. Inject the IImageService into the HomeController
2. Provide the hero image URL to the About view

> ***File:*** `Controllers/HomeController.cs`

```
using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using CloudSoft.Models;
using CloudSoft.Storage;

namespace CloudSoft.Controllers;

public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;
    private readonly IImageService _imageService;

    public HomeController(ILogger<HomeController>
        logger, IImageService imageService)
    {
        _logger = logger;
        _imageService = imageService;
    }

    public IActionResult Index()
    {
        return View();
    }

    public IActionResult Privacy()
    {
        return View();
    }

    [ResponseCache(Duration = 0, Location = Respon-
        seCacheLocation.None, NoStore = true)]
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId =
        Activity.Current?.Id ?? HttpContext.TraceIden-
        tifier });
    }

    public IActionResult About()
    {
        // Get hero image URL from the image service
        ViewData["HeroImageUrl"] = _imageService.GetIm-
        ageUrl("hero.png");
```

```
        return View();
    }
}
```

> ***Information***
> *The controller updates:*
>
> - ***Inject the IImageService*** *dependency via constructor*
> - *Use the service to* ***get the hero image URL*** *based on the image name*
> - *Pass the URL to the view via* ***ViewData***
> - *Keep the controller* ***lean and focused*** *on its responsibilities*

Step 7: Create the Hero Section in the About View

1. Update the About view to add a hero section with the background image
2. Create a responsive layout with the hero content

> ***File:*** `Views/Home/About.cshtml`

```
@{
    ViewData["Title"] = "About";
    var heroImageUrl = ViewData["HeroImageUrl"] as
        string;
}

<div class="hero-section" style="background-image:
        url('@heroImageUrl')">
    <div class="hero-content text-white">
        <h1 class="hero-title display-4">Welcome to
        CloudSoft</h1>
        <div class="hero-text">
            <p class="lead">We're passionate about
        crafting innovative cloud solutions that
        empower businesses to thrive.
            Explore our services and discover how we can
        help you reach new heights.</p>
        </div>
    </div>
</div>

<div class="container mt-5">
    <div class="row">
        <div class="col-md-6">
            <h2>Our Mission</h2>
            <p>
                At CloudSoft, we believe in the power of
        cloud computing to transform businesses and
        drive innovation.
                Our mission is to provide reliable,
        scalable, and secure cloud solutions that meet
        the unique needs of our clients.
            </p>
        </div>
        <div class="col-md-6">
            <h2>Our Team</h2>
            <p>
                Our team of cloud experts is dedicated
        to delivering excellence in every project.
                With years of experience in cloud tech-
        nologies, we have the knowledge and skills to
        help you succeed.
            </p>
        </div>
```

```
    </div>
</div>
```

> ***Information***
> *The view implementation:*
>
> - *Uses **inline style** for the background image from the ViewData*
> - *Creates a **hero section** with text overlay on the image*
> - *Adds a **dark overlay** for better text visibility (via CSS)*
> - *Follows **responsive design** best practices*
> - *Uses **Bootstrap** for layout and styling*

Step 8: Add CSS Styles for the Hero Section

1. Add CSS styles to make the hero section visually appealing
2. Handle text contrast and overlays for better readability

> ***File:*** `wwwroot/css/site.css` *(Add to the end of the file)*

```
/* Hero Section Styles */
.hero-section {
  position: relative;
  height: 500px;
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  margin-bottom: 2rem;
  display: flex;
  align-items: center;
  justify-content: center;
}

.hero-section::before {
  content: "";
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5); /* Dark overlay
        for better text visibility */
}

.hero-content {
  position: relative; /* To appear above the overlay */
  text-align: center;
  padding: 2rem;
  max-width: 800px;
  z-index: 1;
}

.hero-title {
  font-weight: 700;
  margin-bottom: 1rem;
  text-shadow: 2px 2px 4px rgba(0, 0, 0, 0.5);
}

.hero-text {
  font-size: 1.2rem;
  max-width: 600px;
  margin: 0 auto;
```

```
  text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.5);
}
```

> ***Information***
> *The CSS provides:*
>
> - *A **pseudo-element overlay** for better text readability over any image*
> - ***Text shadows** for improved contrast*
> - ***Responsive sizing** and positioning*
> - ***Flexbox** for centering content*
> - ***Z-index** management to layer elements correctly*

Step 9: Prepare Local and Cloud Image Assets

1. Create an images folder in wwwroot and add a low-resolution hero image for development
2. Upload a high-resolution version to Azure Blob Storage for production

```
# Create the images directory in wwwroot
mkdir -p wwwroot/images

# Add a placeholder hero image for development (you'll need
        to provide an actual image)
# The image should be named hero.png
```

> ***Information***
> *For local development: - Use a small, **optimized image** (~300-500KB) - Place it in the **wwwroot/images** folder - Name it **hero.png** to match the code*
> *For Azure Blob Storage: - Use a **higher resolution** version (1-2MB) - Upload to your **Azure Blob container** - Use the same name **hero.png** for consistency*

17.10.4 Final Tests

Testing Local Image Serving

1. Make sure `UseAzureStorage` is set to `false` in appsettings.Development.json
2. Place a hero.png image in the wwwroot/images folder
3. Run the application and navigate to the About page

Expected Result

- The hero section appears with the local image as the background
- The image URL should be served from your local site (e.g., https://localhost:7240/images/hero.png)
- Text should be visible and properly formatted on top of the image

Testing Azure Blob Storage (For Production)

1. Create an Azure Storage Account and a blob container for images
2. Upload hero.png to the blob container
3. Update the production configuration with the container URL

4. Set `UseAzureStorage` to `true` in the environment settings
5. Run the application in production mode

Expected Result

- The hero section loads with the image from Azure Blob Storage
- The image URL should point to your Azure Blob Storage (e.g., https://{account}.blob.core.windows.net/{container}/hero.png)
- The application should use the higher resolution image in production

17.11 Exercise Done!

You've successfully implemented a hero section with an image that uses local storage in development and Azure Blob Storage in production!

EXERCISE 18

Authentication and Authorization

18.1 1. Cookie Authentication and the Who Am I Page

18.2 Cookie Authentication and the Who Am I Page

18.2.1 Goal

Add cookie-based authentication to a fresh ASP.NET Core MVC app using hardcoded dummy credentials and build a **Who Am I?** page that shows the current identity state (authenticated or not, the signed-in user's name, and which

authentication scheme issued the identity). This page becomes the laboratory surface for the rest of the chapter.

> **WHAT YOU'LL LEARN:**
>
> - How the authentication and authorization middleware fit into the request pipeline
> - How to hand-build a `ClaimsPrincipal` and sign it into a cookie with `HttpContext.SignInAsync`
> - How `User.Identity.Name`, `IsAuthenticated`, and `AuthenticationType` surface that cookie back to your code
> - Why the order of `UseAuthentication` and `UseAuthorization` matters

18.2.2 Prerequisites

> **BEFORE STARTING, ENSURE YOU HAVE:**
>
> - ✓ .NET 10 SDK installed (`dotnet --version` reports a `10.*` version)
> - ✓ A Git-tracked working directory for the exercise (a fresh folder is fine)
> - ✓ Basic familiarity with controllers, views, and `Program.cs`

18.2.3 Exercise Steps

Overview

1. **Scaffold** a new ASP.NET Core MVC project
2. **Register cookie authentication** in `Program.cs`
3. **Create a hardcoded user list** in `Data/DummyUsers.cs`

4. **Build the `AccountController`** with Login and Logout actions
5. **Build the `WhoAmIController`** and view that renders the identity state
6. **Add the footer link** to the Who Am I page
7. **Test Your Implementation**

Step 1: *Scaffold a new ASP.NET Core MVC project*

Start from the default MVC template. The scaffold gives you a home controller, a shared layout, and Bootstrap pre-wired — everything you'll extend over this chapter without distractions from extra boilerplate.

1. **Create** the project folder and scaffold the app:

```
mkdir -p src
dotnet new mvc -o src/CloudSoft.Auth.Web --framework
        net10.0
```

2. **Verify** it builds and runs:

```
dotnet build src/CloudSoft.Auth.Web
dotnet run --project src/CloudSoft.Auth.Web --launch-
        profile http
```

Opening `http://localhost:5017` should show the default MVC welcome page. Stop the server with `Ctrl+C`.

> **CONCEPT DEEP DIVE**
>
> `dotnet run` with the `http` profile binds only to port `5017` over plain HTTP. The scaffold's `app.UseHttpsRedirection()` call is effectively a no-op in this mode — it only kicks in when an HTTPS endpoint is also configured (the `https` launch profile). Keep the scaffold's HTTPS and HSTS lines as-is; you may want HTTPS locally later when testing Google OIDC, and they're the right defaults for production anyway.
>
> ✓ **Quick check:** The project builds and `http://localhost:5017` loads the default MVC home page.

Step 2: *Register cookie authentication*

Cookie authentication is the simplest auth scheme ASP.NET Core ships with. On sign-in, the framework serializes a `ClaimsPrincipal` into an encrypted cookie; on every subsequent request the middleware deserializes that cookie back into `HttpContext.User`. To use it, you register the scheme with dependency injection and add two middleware to the request pipeline.

1. **Open** `src/CloudSoft.Auth.Web/Program.cs`
2. **Add** the `using` at the top:

   > `src/CloudSoft.Auth.Web/Program.cs`

   ```
   using Microsoft.AspNetCore.Authentication.Cookies;
   ```

3. **Register** the cookie scheme right after `AddControllersWithViews()`:

   > `src/CloudSoft.Auth.Web/Program.cs`

```
builder.Services.AddControllersWithViews();

builder.Services
    .AddAuthentication(CookieAuthenticationDefaults.Authen-
        ticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/Access-
        Denied";
    });

builder.Services.AddAuthorization();
```

4. **Add** the authentication and authorization middleware after `UseRouting()` and before the endpoint mapping. Keep the existing `UseHttpsRedirection()` call untouched — it's inherited from the scaffold.

> `src/CloudSoft.Auth.Web/Program.cs`

```
app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapStaticAssets();
```

> **CONCEPT DEEP DIVE**
>
> `AddAuthentication(scheme)` tells ASP.NET Core which authentication scheme to use by default when a request arrives without one specified. `AddCookie` registers the handler for that scheme. `LoginPath` is where the middleware redirects anonymous users who hit a protected action; `AccessDeniedPath` is where it redirects signed-in users who lack the required role or claim.
>
> ⚠ **Common Mistakes**
>
> - `UseAuthentication()` **must** come before `UseAuthorization()`. Reversed, the authorization middleware sees an anonymous request every time because no `ClaimsPrincipal` has been built yet.
> - Both middleware must come **after** `UseRouting()` so the matched endpoint is known.
>
> √ **Quick check:** `dotnet build` succeeds with no errors.

Step 3: *Create the hardcoded user list*

Real applications store users in a database. For this chapter we want to keep the focus on the cookie mechanics, so we use a static in-memory list instead. The data structure deliberately carries only a username and password for now — we'll extend it with roles and claims in later exercises.

1. **Create** a new folder: `src/CloudSoft.Auth.Web/Data/`
2. **Add** a new file:

> `src/CloudSoft.Auth.Web/Data/DummyUsers.cs`

```
namespace CloudSoft.Auth.Web.Data;

public static class DummyUsers
{
    public record User(string Username, string Pass-
        word);

    public static readonly List<User> All =
    [
        new("admin", "admin"),
        new("candidate", "candidate"),
    ];

    public static User? Find(string username, string
        password) =>
        All.FirstOrDefault(u =>
            string.Equals(u.Username, username, String-
        Comparison.Ordinal) &&
            string.Equals(u.Password, password, String-
        Comparison.Ordinal));
}
```

COMMON MISTAKES

- Storing passwords in source code is **only** acceptable for a lab. Never do this in production — ASP.NET Core Identity (Chapter 5) handles proper password hashing and persistence for you.

Step 4: *Build the AccountController*

The `AccountController` handles two flows: showing the login form and processing submitted credentials. On a successful POST it builds a `ClaimsPrincipal` containing one claim — the user's name — and hands it to `HttpContext.SignInAsync`, which encrypts and sets the auth cookie on the response.

1. **Create** a new file:

> `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`

```
using System.Security.Claims;
using CloudSoft.Auth.Web.Data;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Auth.Web.Controllers;

public class AccountController : Controller
{
    [HttpGet]
    public IActionResult Login(string? returnUrl = null)
    {
        ViewData["ReturnUrl"] = returnUrl;
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Login(string user-
        name, string password, string? returnUrl =
        null)
    {
        var user = DummyUsers.Find(username, password);
        if (user is null)
        {
            ModelState.AddModelError(string.Empty,
        "Invalid username or password.");
            ViewData["ReturnUrl"] = returnUrl;
            return View();
        }

        var claims = new List<Claim>
        {
            new(ClaimTypes.Name, user.Username),
        };

        var identity = new ClaimsIdentity(claims, Cook-
        ieAuthenticationDefaults.AuthenticationScheme);
        var principal = new ClaimsPrincipal(identity);

        await HttpContext.SignInAsync(
```

```
            CookieAuthenticationDefaults.Authentication-
        Scheme,
            principal);

        if (!string.IsNullOrEmpty(returnUrl) &&
        Url.IsLocalUrl(returnUrl))
        {
            return Redirect(returnUrl);
        }

        return RedirectToAction("Index", "WhoAmI");
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Logout()
    {
        await HttpContext.SignOutAsync(CookieAuthentica-
        tionDefaults.AuthenticationScheme);
        return RedirectToAction("Index", "Home");
    }

    [HttpGet]
    public IActionResult AccessDenied() => View();
}
```

2. **Create** the Login view:

> `src/CloudSoft.Auth.Web/Views/Account/Login.cshtml`

```
@{
    ViewData["Title"] = "Log in";
}

<h1>Log in</h1>

@if (!ViewData.ModelState.IsValid)
{
    <div class="alert alert-danger" data-testid="login-
        error">
        @Html.ValidationSummary()
    </div>
}

<form method="post" data-testid="login-form" style="max-
        width: 420px;">
    @Html.AntiForgeryToken()
    <input type="hidden" name="returnUrl"
        value="@ViewData["ReturnUrl"]" />

    <div class="mb-3">
        <label for="username" class="form-label">User-
        name</label>
        <input type="text" id="username"
        name="username" class="form-control"
               data-testid="login-username" autocom-
        plete="username" />
    </div>

    <div class="mb-3">
        <label for="password" class="form-label">Pass-
        word</label>
        <input type="password" id="password" name="pass-
        word" class="form-control"
               data-testid="login-password" autocom-
        plete="current-password" />
    </div>

    <button type="submit" class="btn btn-primary" data-
        testid="login-submit">Log in</button>
</form>

<p class="mt-4 text-muted">
    Dummy credentials for this exercise:
```

```
    <code>admin / admin</code> or <code>candidate / can-
        didate</code>.
</p>
```

3. **Create** the AccessDenied view:

> `src/CloudSoft.Auth.Web/Views/Account/AccessDenied.cshtml`

```
@{
    ViewData["Title"] = "Access denied";
}

<h1 data-testid="access-denied-heading">Access denied</
        h1>
<p class="lead">You are signed in but your account is
        not permitted to view that page.</p>

<a asp-controller="WhoAmI" asp-action="Index"
        class="btn btn-outline-secondary">Who am I?</a>
```

CONCEPT DEEP DIVE

`ClaimsPrincipal` is ASP.NET Core's universal user abstraction. It wraps one or more `ClaimsIdentity` objects, each a bag of `Claim` tuples: (type, value, issuer). After sign-in the middleware deserializes the cookie back into this shape on every request, and your controllers read it via `HttpContext.User`.

⚠ **Common Mistakes**

- Omitting `[ValidateAntiForgeryToken]` on state-changing POSTs opens the app to CSRF. The token pairs with `@Html.AntiForgeryToken()` in the form.
- Returning an absolute URL from `returnUrl` without validating `Url.IsLocalUrl` is an open-redirect vulnerability.

√ **Quick check:** The project builds. The login view renders at `/Account/Login`.

Step 5: *Build the WhoAmI controller and view*

The Who Am I page is the single window into the current authentication state. It reads `HttpContext.User` — the `ClaimsPrincipal` the cookie middleware materialized — and renders what it finds. Later exercises will extend this view; today it shows three fields and a login/logout control.

1. **Create** the controller:

> `src/CloudSoft.Auth.Web/Controllers/WhoAmIController.cs`

```
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Auth.Web.Controllers;

public class WhoAmIController : Controller
{
    public IActionResult Index() => View();
}
```

2. **Create** the view:

> `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`

```
@{
    ViewData["Title"] = "Who Am I?";
}

<h1>Who Am I?</h1>
<p class="lead">What the server currently knows about
        you.</p>

<dl class="row">
    <dt class="col-sm-4">Is authenticated?</dt>
    <dd class="col-sm-8" data-testid="whoami-authentic-
        ated">
        @(User.Identity?.IsAuthenticated ?? false)
    </dd>

    <dt class="col-sm-4">Name</dt>
    <dd class="col-sm-8" data-testid="whoami-name">
        @(User.Identity?.Name ?? "(anonymous)")
    </dd>

    <dt class="col-sm-4">Authentication type</dt>
    <dd class="col-sm-8" data-testid="whoami-authtype">
        @(User.Identity?.AuthenticationType ?? "(none)")
    </dd>
</dl>

@if (User.Identity?.IsAuthenticated == true)
{
    <form asp-controller="Account" asp-action="Logout"
        method="post" data-testid="logout-form">
        @Html.AntiForgeryToken()
        <button type="submit" class="btn btn-outline-
        secondary" data-testid="logout-submit">
            Log out
        </button>
    </form>
}
else
{
    <a asp-controller="Account" asp-action="Login"
        class="btn btn-primary" data-testid="whoami-
        loginlink">Log in</a>
}
```

> **CONCEPT DEEP DIVE**
>
> `User.Identity` returns an `IIdentity` that is null-safe only if the request has no authentication data at all. With cookie authentication registered, the middleware always gives you an identity — possibly unauthenticated. `IsAuthenticated` is the check that tells you the cookie was present and valid.

Step 6: *Link the Who Am I page from the footer*

You want the Who Am I page accessible from everywhere in the app. The simplest place is the shared footer.

1. **Open** `src/CloudSoft.Auth.Web/Views/Shared/_Layout.cshtml`
2. **Replace** the footer block with:

> `src/CloudSoft.Auth.Web/Views/Shared/_Layout.cshtml`

```
<footer class="border-top footer text-muted">
    <div class="container">
        &copy; 2026 - CloudSoft.Auth.Web -
        <a asp-area="" asp-controller="Home" asp-
        action="Privacy">Privacy</a> -
        <a asp-area="" asp-controller="WhoAmI" asp-
        action="Index" data-testid="footer-whoami">Who
        Am I?</a>
    </div>
</footer>
```

Step 7: *Test Your Implementation*

1. **Run the app:**

```
dotnet run --project src/CloudSoft.Auth.Web --launch-
        profile http
```

2. **Browse to** `http://localhost:5017` and click **Who Am I?** in the footer.
3. **Verify anonymous state:**
   - `Is authenticated?` shows `False`
   - `Name` shows `(anonymous)`
   - `Authentication type` shows `(none)`
   - A **Log in** button is present
4. **Verify the login flow:**
   - Click **Log in**, submit `admin` / `admin`
   - You land back on **Who Am I?**
   - `Is authenticated?` now shows `True`
   - `Name` shows `admin`
   - `Authentication type` shows `Cookies`
5. **Verify the logout flow:**
   - Click **Log out**
   - You return to the home page
   - Revisit **Who Am I?** and confirm it reports anonymous again
6. **Verify error handling:**
   - Go to `/Account/Login` and submit bad credentials (for example `admin` / `wrong`)
   - An error message appears and you stay on the login page

> ✓ ***Success indicators:***
>
> - *The Who Am I page correctly reports both authenticated and anonymous states*
> - *Login sets a cookie that survives page navigation*
> - *Logout clears the cookie and the anonymous state returns*
> - *Bad credentials show a validation error*

18.2.4 Common Issues

> ***If you encounter problems:***
> ***"No authentication scheme was specified" at runtime:*** *The middleware is registered but* `AddAuthentication` *wasn't called, or the scheme name doesn't match. Confirm* `CookieAuthenticationDefaults.AuthenticationScheme` *is used consistently.*
> ***Login redirects you straight back to the login page:*** *Either* `UseAuthentication` *is missing from the pipeline, or the order is reversed — authentication must come before authorization.*
> ***The logout button does nothing:*** `Logout` *is a POST, so a GET request (for example typing* `/Account/Logout` *in the address bar) returns 405. The* `<form method="post">` *wrapper is how the view submits it.*
> ***Antiforgery token errors on login:*** `@Html.AntiForgeryToken()` *must be inside the form;* `[ValidateAntiForgeryToken]` *must be on the* `HttpPost` *action. Both halves are required.*

18.2.5 Summary

You've wired up cookie authentication against a hardcoded user list and built the Who Am I page that shows its effect in real time. You've seen the three primitives the rest of the chapter builds on:

- √ `ClaimsPrincipal` — the universal user abstraction
- √ `SignInAsync` / `SignOutAsync` — the APIs that set and clear the cookie
- √ `UseAuthentication` / `UseAuthorization` middleware — the pipeline that turns the cookie back into `HttpContext.User`

> **KEY TAKEAWAY:**
> Authentication in ASP.NET Core is just a `ClaimsPrincipal` wrapped in a cookie. Every method you'll learn in later exercises — roles, claims, policies, external providers — is a different way of building or interpreting that principal.

18.2.6 Done!

The Who Am I page works. Next exercise: add **roles** to the principal and use `[Authorize(Roles = "Admin")]` to gate an admin-only page.

18.3 2. Roles and the [Authorize] Attribute

18.4 Roles and the `[Authorize]` Attribute

18.4.1 Goal

Extend the cookie-based authentication from Exercise 1 so users carry **roles**. Add `Admin` and `Candidate` roles to the dummy user list, emit them as claims during sign-in, and

gate a new `/AdminOnly` page with `[Authorize(Roles = "Admin")]`. The Who Am I page gets a new row showing the current user's roles.

WHAT YOU'LL LEARN:

- How roles are represented as `ClaimTypes.Role` claims on `ClaimsPrincipal`
- How `[Authorize(Roles = "…")]` maps to claim matching
- How the cookie middleware handles a forbidden user (redirects to `AccessDeniedPath`)
- How to conditionally render UI based on `User.IsInRole(...)`

18.4.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed Exercise 1 — cookie authentication and Who Am I page working
- ✓ The project builds and the Who Am I page shows `IsAuthenticated`, `Name`, `AuthenticationType`

18.4.3 Exercise Steps

Overview

1. **Extend `DummyUsers`** with a `Roles` array
2. **Emit role claims** during sign-in in `AccountController`
3. **Create the `AdminOnly` page** guarded by `[Authorize(Roles = "Admin")]`
4. **Show roles on Who Am I** and conditionally link to the admin page

5. **Test Your Implementation**

Step 1: *Extend the dummy user list with roles*

Roles are just strings. A user can belong to zero, one, or many. For this lab `admin` carries `Admin`, and `candidate` carries `Candidate` — two mutually-exclusive examples that make role-based tests easy to reason about.

1. **Replace** the contents of `src/CloudSoft.Auth.Web/Data/DummyUsers.cs`:

> `src/CloudSoft.Auth.Web/Data/DummyUsers.cs`

```
namespace CloudSoft.Auth.Web.Data;

public static class DummyUsers
{
    public record User(string Username, string Pass-
        word, string[] Roles);

    public static readonly List<User> All =
    [
        new("admin", "admin", Roles: ["Admin"]),
        new("candidate", "candidate", Roles: ["Candid-
        ate"]),
    ];

    public static User? Find(string username, string
        password) =>
        All.FirstOrDefault(u =>
            string.Equals(u.Username, username, String-
        Comparison.Ordinal) &&
            string.Equals(u.Password, password, String-
        Comparison.Ordinal));
}
```

> **CONCEPT DEEP DIVE**
>
> ASP.NET Core treats a role as a claim whose type is `ClaimTypes.Role`. There is no separate "role store" in the primitive auth stack — roles and claims are the same data shape, just a convention on the type string. `User.IsInRole("Admin")` is effectively `User.HasClaim(ClaimTypes.Role, "Admin")`.

Step 2: *Emit role claims during sign-in*

The `ClaimsPrincipal` you build in `AccountController` needs a `Role` claim for every role the user belongs to. The cookie middleware serializes all of them, so every request that comes back with the cookie will materialize a principal whose `IsInRole` knows about them.

1. **Open** `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`
2. **In** the `Login` POST action, **add** a `foreach` loop that adds a role claim per role, just after the `ClaimTypes.Name` claim:

> `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`

```
var claims = new List<Claim>
{
    new(ClaimTypes.Name, user.Username),
};

foreach (var role in user.Roles)
{
    claims.Add(new Claim(ClaimTypes.Role, role));
}

var identity = new ClaimsIdentity(claims, CookieAu-
        thenticationDefaults.AuthenticationScheme);
```

COMMON MISTAKES

- Using your own string key (`"role"`, `"Roles"`) instead of `ClaimTypes.Role` — the `[Authorize(Roles = ...)]` attribute only looks at `ClaimTypes.Role`, so your roles will be invisible to it.
- Adding the roles as a single comma-separated claim value — each role is its own claim.

Step 3: *Create the admin-only page*

`[Authorize]` without arguments only requires the user to be authenticated. Passing `Roles = "Admin"` adds a second condition: the principal must carry a `Role` claim with value `Admin`. A logged-in Candidate hitting this page is redirected to the `AccessDeniedPath` configured in Exercise 1.

1. **Create** the controller:

> `src/CloudSoft.Auth.Web/Controllers/AdminOnlyController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Auth.Web.Controllers;

[Authorize(Roles = "Admin")]
public class AdminOnlyController : Controller
{
    public IActionResult Index() => View();
}
```

2. **Create** the view:

> `src/CloudSoft.Auth.Web/Views/AdminOnly/Index.cshtml`

```
@{
    ViewData["Title"] = "Admin only";
}

<h1 data-testid="adminonly-heading">Admin only</h1>
<p class="lead">You only see this page because your
        identity carries the <code>Admin</code> role.</
        p>

<a asp-controller="WhoAmI" asp-action="Index"
        class="btn btn-outline-secondary">Who Am I?</a>
```

CONCEPT DEEP DIVE

`[Authorize(Roles = "Admin,Moderator")]` lets you accept any of several roles (the principal must carry at least one). Stacking two attributes — `[Authorize(Roles = "Admin")] [Authorize(Roles = "Moderator")]` — means the principal must carry **both** roles. Slightly different semantics; the first is usually what you want.

Step 4: *Show roles on the Who Am I page*

The Who Am I page should reveal what the principal currently carries. Add a roles row and — because there is now something gated — a conditional link to the admin page.

1. **Open** `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`
2. **Add** a `Roles` row inside the definition list, and **add** the admin link block just after the `</dl>`:

> `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`

```
<dt class="col-sm-4">Roles</dt>
<dd class="col-sm-8" data-testid="whoami-roles">
    @{
        var roles = User.Find-
        All(System.Security.Claims.ClaimTypes.Role)
                        .Select(c => c.Value).ToArray();
    }
    @(roles.Length == 0 ? "(none)" : string.Join(", ",
        roles))
</dd>
```

After the `</dl>` (and before the `@if (User.Identity?.IsAuthenticated == true)` block):

```
@if (User.IsInRole("Admin"))
{
    <a asp-controller="AdminOnly" asp-action="Index"
        class="btn btn-outline-primary me-2" data-
        testid="whoami-adminlink">
        Admin-only page
    </a>
}
```

Step 5: *Test Your Implementation*

1. **Run** the app:

```
dotnet run --project src/CloudSoft.Auth.Web --launch-
        profile http
```

2. **Log in as admin** (`admin` / `admin`) and go to **Who Am I?**:
   - The `Roles` row shows `Admin`
   - An **Admin-only page** button is visible
   - Clicking it loads `/AdminOnly` successfully
3. **Log out**, **log in as candidate** (`candidate` / `candidate`):
   - The `Roles` row shows `Candidate`
   - The **Admin-only page** button is **not** visible on Who Am I
   - Typing `/AdminOnly` in the address bar redirects you to `/Account/AccessDenied`
4. **Log out** entirely and type `/AdminOnly` again:
   - The cookie middleware redirects you to `/Account/Login` with `?returnUrl=/AdminOnly`
   - After signing in as admin you land on `/AdminOnly`

> √ ***Success indicators:***
>
> - *Role claims flow from* `DummyUsers` *through the cookie and back into* `HttpContext.User`
> - `[Authorize(Roles = "Admin")]` *lets admin through and bounces everyone else*
> - *The admin link on Who Am I is purely visual — even hidden it cannot be bypassed because the server re-checks on the* `/AdminOnly` *request*

18.4.4 Common Issues

> ***If you encounter problems:***
> ***Admin login still can't access `/AdminOnly`:*** *The role claim is probably missing. Log in and view Who Am I — the roles row should list `Admin`. If it says `(none)`, the `foreach` in `AccountController` isn't running. A common cause is rebuilding `DummyUsers.User` without the `Roles` parameter.*
> ***Candidate lands on a blank page instead of AccessDenied:*** *Check that `AccessDeniedPath = "/Account/AccessDenied"` is still configured on `AddCookie`.*
> ***The admin link stays visible for candidates:*** *`User.IsInRole("Admin")` uses exact string matching. If you typed `admin` (lowercase) in `DummyUsers`, the check won't find it.*

18.4.5 Summary

You've added the second primitive on top of Exercise 1's cookie authentication:

- √ Role claims added to the signed-in `ClaimsPrincipal`
- √ `[Authorize(Roles = "Admin")]` gating a controller
- √ Server-side 403 via `AccessDeniedPath`, not a client-side hide-and-hope
- √ Conditional UI using `User.IsInRole(...)`

> **KEY TAKEAWAY:**
> Roles are ordinary claims with a conventional type (`ClaimTypes.Role`). Understanding that unifies the mental model — the next exercise generalizes to **any** claim and to **policies** that combine them.

18.4.6 Done!

Admin and Candidate now see different pages. Next: arbitrary claims and policy-based authorization — plus CSRF protection for state-changing POSTs.

18.5 3. Claims, Policies, and CSRF Protection

18.6 Claims, Policies, and CSRF Protection

18.6.1 Goal

Generalize what you built in Exercise 2 to any claim — not just roles — and use **named policies** to express authorization rules declaratively.

By the end, the Who Am I page renders the full claim list (type, value, issuer) and a new `/Engineering` page is accessible only to users whose identity carries `Department = Engineering`.

CSRF protection is already present on the login and logout forms from Exercise 1 via the `[ValidateAntiForgeryToken]` attribute paired with `@Html.AntiForgeryToken()`. Keep that pairing in mind whenever you add a state-changing POST from here on.

WHAT YOU'LL LEARN:

- How to add arbitrary custom claims to a `ClaimsPrincipal`
- How `AddAuthorization(options => options.AddPolicy(...))` registers a named policy
- How `[Authorize(Policy = "…")]` differs from `[Authorize(Roles = "…")]`
- What antiforgery tokens protect against and how ASP.NET Core enforces them

18.6.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- √ Completed Exercise 2 — roles and `[Authorize(Roles = "…")]` working
- √ The Who Am I page shows roles for each logged-in user

18.6.3 Exercise Steps

Overview

1. **Extend `DummyUsers`** with a `Claims` dictionary
2. **Emit custom claims** during sign-in in `AccountController`
3. **Register a policy** in `Program.cs`
4. **Create the Engineering page** gated by the policy
5. **Extend the Who Am I view** — full claims table and conditional link
6. **Test Your Implementation**

Step 1: *Extend the dummy user list with claims*

Roles are just claims with a well-known type. Other claims can carry anything — organization, department, age, subscription tier. Here we add a single `Department` claim per user to keep the demo focused.

1. **Replace** `src/CloudSoft.Auth.Web/Data/DummyUsers.cs`:

> `src/CloudSoft.Auth.Web/Data/DummyUsers.cs`

```
namespace CloudSoft.Auth.Web.Data;

public static class DummyUsers
{
    public record User(
        string Username,
        string Password,
        string[] Roles,
        Dictionary<string, string> Claims);

    public static readonly List<User> All =
    [
        new("admin", "admin",
            Roles: ["Admin"],
            Claims: new() { ["Department"] = "Engineer-
        ing" }),

        new("candidate", "candidate",
            Roles: ["Candidate"],
            Claims: new() { ["Department"] = "Sales" }),
    ];

    public static User? Find(string username, string
        password) =>
        All.FirstOrDefault(u =>
            string.Equals(u.Username, username, String-
        Comparison.Ordinal) &&
            string.Equals(u.Password, password, String-
        Comparison.Ordinal));
}
```

Step 2: *Emit the custom claims at sign-in*

`AccountController.Login` already loops over roles. Add a second loop for the generic claims dictionary.

1. **Open** `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`
2. **After** the role `foreach`, add:

> `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`

```
foreach (var (type, value) in user.Claims)
{
    claims.Add(new Claim(type, value));
}
```

> **CONCEPT DEEP DIVE**
>
> A claim is a tuple of `(type, value, issuer)`. The issuer defaults to `LOCAL AUTHORITY` when you build it yourself; external providers (Google, Microsoft) set the issuer to their identity URL. In Chapter 5 you'll see Google-issued claims appear in this same table with a different issuer.

Step 3: *Register the policy*

A policy is a named set of requirements. `RequireEngineering` is satisfied when the principal carries a `Department` claim whose value is `Engineering`.

1. **Open** `src/CloudSoft.Auth.Web/Program.cs`
2. **Replace** `builder.Services.AddAuthorization();` with:

> `src/CloudSoft.Auth.Web/Program.cs`

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireEngineering", policy =>
        policy.RequireClaim("Department",
         "Engineering"));
});
```

> **CONCEPT DEEP DIVE**
>
> `policy.RequireClaim(type, value)` is the simplest requirement. You can also combine requirements — `.RequireRole("Admin").RequireClaim("Department", "Engineering")` means the principal must satisfy **both**. For more elaborate logic, implement `IAuthorizationRequirement` and pair it with an `AuthorizationHandler<T>`.

Step 4: *Create the Engineering page*

The Engineering page is visible only when the policy is satisfied. `[Authorize(Policy = "...")]` looks the same as `[Authorize(Roles = "...")]` at the usage level — the difference is that `Roles` is baked in while `Policy` is arbitrary.

1. **Create** the controller:

> `src/CloudSoft.Auth.Web/Controllers/EngineeringController.cs`

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Auth.Web.Controllers;

[Authorize(Policy = "RequireEngineering")]
public class EngineeringController : Controller
{
    public IActionResult Index() => View();
}
```

2. **Create** the view:

> `src/CloudSoft.Auth.Web/Views/Engineering/Index.cshtml`

```
@{
    ViewData["Title"] = "Engineering";
}

<h1 data-testid="engineering-heading">Engineering</h1>
<p class="lead">You only see this page because your
        identity carries <code>Department = Engineer-
        ing</code>.</p>
<p>The gate is the <code>RequireEngineering</code>
        policy, not a role.</p>

<a asp-controller="WhoAmI" asp-action="Index"
        class="btn btn-outline-secondary">Who Am I?</a>
```

Step 5: *Show the full claims table and conditional link*

The Who Am I page can now render every claim on the identity. Each row includes the issuer so you can see which came from your sign-in code versus (in Chapter 5) from Google.

1. **Open** `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`
2. **Add** `@using System.Security.Claims` at the top of the file (removing the inline namespace you used in Exercise 2)
3. **Replace** the inline `ClaimTypes.Role` reference in the roles row with just `ClaimTypes.Role` (thanks to the new `@using`)
4. **Add** the claims table **after** the `</dl>`:

> `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`

```
<h2 class="mt-4">All claims</h2>
<table class="table table-sm table-striped" data-
        testid="whoami-claims-table">
    <thead>
        <tr>
            <th>Type</th>
            <th>Value</th>
            <th>Issuer</th>
        </tr>
    </thead>
    <tbody>
        @if (User.Identity?.IsAuthenticated == true)
        {
            foreach (var claim in User.Claims)
            {
                <tr>
                    <td><code>@claim.Type</code></td>
                    <td>@claim.Value</td>
                    <td>@claim.Issuer</td>
                </tr>
            }
        }
        else
        {
            <tr>
                <td colspan="3" class="text-muted">(no
        identity — log in to see claims)</td>
            </tr>
        }
    </tbody>
</table>
```

5. **Replace** the admin link block with a row that also includes a conditional Engineering link:

> `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`

```
<div class="mb-4">
    @if (User.IsInRole("Admin"))
    {
        <a asp-controller="AdminOnly" asp-
        action="Index" class="btn btn-outline-primary
        me-2" data-testid="whoami-adminlink">
            Admin-only page
        </a>
    }
    @if (User.HasClaim("Department", "Engineering"))
    {
        <a asp-controller="Engineering" asp-
        action="Index" class="btn btn-outline-primary"
        data-testid="whoami-engineeringlink">
            Engineering page
        </a>
    }
</div>
```

Step 6: *Test Your Implementation*

1. **Run** the app and log in as `admin`.
2. On Who Am I, confirm:
   - The `All claims` table includes `http://schemas.microsoft.com/ws/2008/06/identity/claims/role = Admin` and `Department = Engineering`.
   - An **Engineering page** button appears (and navigates successfully).
3. Log out, log in as `candidate`:
   - Claims table now shows `Department = Sales`.
   - The Engineering button is gone. Typing `/Engineering` in the address bar redirects to `/Account/AccessDenied`.

> √ ***Success indicators:***
>
> - *The claims table shows every claim you added plus* `ClaimTypes.Name`
> - `[Authorize(Policy = "RequireEngineering")]` *behaves exactly like* `[Authorize(Roles = ...)]`*: allow or redirect to access denied*

18.6.4 Common Issues

> ***If you encounter problems:***
> ***Policy never satisfied:*** *Both the claim type and value match exactly (string equality, case-sensitive).* `Department` *is not the same as* `department`*.*
> **`InvalidOperationException: No policy named…`***: You referenced* `[Authorize(Policy = "Foo")]` *without calling* `AddPolicy("Foo", ...)` *in* `Program.cs`*.*

18.6.5 Summary

You've generalized the primitive one more step:

- √ Any claim, not just roles, can live on the principal
- √ Named policies let you express authorization rules in one place and re-use them on many controllers
- √ Antiforgery token protection is easy to add with `[ValidateAntiForgeryToken]` + `@Html.AntiForgeryToken()` and catastrophic to forget

> **KEY TAKEAWAY:**
> `[Authorize(Policy = ...)]` is the general form. `[Authorize(Roles = ...)]` is syntactic sugar over a role-claim check. Reach for policies when the rule is anything other than "is the user in this role?".

18.6.6 Done!

You've completed the cookie-auth fundamentals — authentication, role-based gating, custom claims, named policies, and antiforgery protection — all built on top of a hand-assembled `ClaimsPrincipal`.

EXERCISE 19

Identity and User Stores

19.1 1. Introducing ASP.NET Core Identity

19.2 Introducing ASP.NET Core Identity

19.2.1 Goal

Replace the hand-rolled authentication stack from Chapter 4 with **ASP.NET Core Identity**. Identity provides `UserManager`, `RoleManager`, and `SignInManager` — high-level APIs that handle user persistence, password hashing, role assignment, and the cookie round-trip. The Who Am I page will look almost identical; only the scheme name (`Identity.Application`) and the origin of the claims will change.

To avoid introducing a database too early, we use **EF Core InMemory** as the store. Users and roles live in the process only — every restart wipes them, and a seed method recreates them. Exercise 5.2 introduces a feature flag that swaps this for SQLite persistence.

WHAT YOU'LL LEARN:

- How `IdentityUser`, `IdentityRole`, and `IdentityDbContext` fit together
- How to register Identity with DI and tell it which store to use
- How `SignInManager.PasswordSignInAsync` replaces the hand-built `ClaimsPrincipal` + `SignInAsync`
- How role and claim data flow through Identity's user-roles and user-claims tables into the cookie

19.2.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed Chapter 4 Exercises 1–3 — the cookie flow, roles, claims, policies, and CSRF work end-to-end
- ✓ `.NET` 10 SDK installed
- ✓ The project builds and tests pass at the end of Exercise 4.3

19.2.3 Exercise Steps

Overview

1. **Install** the Identity + EF Core packages
2. **Create** `ApplicationUser` and `ApplicationDbContext`

3. **Register** Identity in `Program.cs` with the InMemory provider
4. **Replace** `AccountController`'s hand-rolled logic with `SignInManager`
5. **Seed** the fixed test users, roles, and claims on startup
6. **Delete** the old `DummyUsers.cs`
7. **Test Your Implementation**

Step 1: *Install the Identity and EF Core packages*

```
dotnet add src/CloudSoft.Auth.Web package
        Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet add src/CloudSoft.Auth.Web package
        Microsoft.EntityFrameworkCore.InMemory
```

Step 2: *Create the application user and database context*

Identity is designed to be extended. `IdentityUser` gives you the standard columns (id, username, email, password hash, security stamp); you subclass it to add your own. For this chapter the empty subclass is enough.

1. **Create** `src/CloudSoft.Auth.Web/Data/ApplicationUser.cs`:

   > `src/CloudSoft.Auth.Web/Data/ApplicationUser.cs`

   ```
   using Microsoft.AspNetCore.Identity;

   namespace CloudSoft.Auth.Web.Data;

   public class ApplicationUser : IdentityUser
   {
   }
   ```

2. **Create** `src/CloudSoft.Auth.Web/Data/ApplicationDbContext.cs`:

> `src/CloudSoft.Auth.Web/Data/ApplicationDbContext.cs`

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace CloudSoft.Auth.Web.Data;

public class ApplicationDbContext : IdentityDbCon-
        text<ApplicationUser, IdentityRole, string>
{
    public ApplicationDbContext(DbContextOptions<Applic-
        ationDbContext> options) : base(options)
    {
    }
}
```

> **CONCEPT DEEP DIVE**
>
> `IdentityDbContext<TUser, TRole, TKey>` wires up the standard Identity tables: `AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, and a few more. You don't write any of these — they come from Identity's `OnModelCreating`.

Step 3: *Register Identity with DI*

The registration in `Program.cs` replaces most of your Chapter 4 authentication code. Identity installs its own cookie scheme (`Identity.Application`) with sensible defaults; the `ConfigureApplicationCookie` call just tweaks the paths you care about.

1. **Open** `src/CloudSoft.Auth.Web/Program.cs`

2. **Replace** the old `AddAuthentication(Cookies).AddCookie(...)` block with:

> `src/CloudSoft.Auth.Web/Program.cs`

```
using CloudSoft.Auth.Web.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

// Data: EF Core InMemory store backing ASP.NET Core
        Identity.
builder.Services.AddDbContext<ApplicationDbContext>(options
        =>
    options.UseInMemoryDatabase("CloudSoftAuthDb"));

builder.Services
    .AddIdentity<ApplicationUser, IdentityRole>(options
        =>
    {
        // Relax password rules for the lab: Chapter-4
        credentials still work.
        options.Password.RequireDigit = false;
        options.Password.RequireLowercase = false;
        options.Password.RequireUppercase = false;
        options.Password.RequireNonAlphanumeric = false;
        options.Password.RequiredLength = 3;
        options.User.RequireUniqueEmail = false;
        options.SignIn.RequireConfirmedAccount = false;
    })
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
});
```

COMMON MISTAKES

- Calling `AddAuthentication(scheme).AddCookie()` alongside `AddIdentity` clobbers Identity's cookie configuration. `AddIdentity` already sets up the scheme; you only configure it.
- Forgetting `AddDefaultTokenProviders()` breaks password reset and email confirmation (not used in this lab, but needed in Ex 5.4).

Step 4: *Replace AccountController with SignInManager*

The controller no longer builds a `ClaimsPrincipal` by hand. `SignInManager.PasswordSignInAsync` takes a username and password, looks up the user, verifies the hash, and issues the cookie — all at once.

1. **Replace** `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`:

> `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`

```
using CloudSoft.Auth.Web.Data;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;

namespace CloudSoft.Auth.Web.Controllers;

public class AccountController : Controller
{
    private readonly SignInManager<ApplicationUser>
        _signInManager;

    public AccountController(SignInManager<Applicatio-
        nUser> signInManager)
    {
        _signInManager = signInManager;
    }

    [HttpGet]
    public IActionResult Login(string? returnUrl = null)
    {
        ViewData["ReturnUrl"] = returnUrl;
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Login(string user-
        name, string password, string? returnUrl =
        null)
    {
        var result = await _signInManager.PasswordSign-
        InAsync(
            username, password, isPersistent: false,
        lockoutOnFailure: false);

        if (!result.Succeeded)
        {
            ModelState.AddModelError(string.Empty,
        "Invalid username or password.");
            ViewData["ReturnUrl"] = returnUrl;
            return View();
        }
```

```
        if (!string.IsNullOrEmpty(returnUrl) &&
        Url.IsLocalUrl(returnUrl))
        {
            return Redirect(returnUrl);
        }

        return RedirectToAction("Index", "WhoAmI");
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Logout()
    {
        await _signInManager.SignOutAsync();
        return RedirectToAction("Index", "Home");
    }

    [HttpGet]
    public IActionResult AccessDenied() => View();
}
```

> **CONCEPT DEEP DIVE**
>
> `SignInManager` sits on top of `UserManager`. Internally it calls `UserManager.FindByNameAsync`, verifies the password hash, and issues the `Identity.Application` cookie via `SignInAsync` on `HttpContext`. The claims that end up in the cookie are assembled by a `IUserClaimsPrincipalFactory` which, by default, includes the user's roles and all `AspNetUserClaims` rows.

Step 5: *Seed fixed test users at startup*

InMemory storage means every restart starts with zero users. A small seeder populates admin and candidate on every boot. Exercise 5.3 replaces this with a config-driven

seeder that reads the admin credentials from `user-secrets`; for now, hardcoded is fine.

1. **Create** `src/CloudSoft.Auth.Web/Data/TestUserSeeder.cs`:

> `src/CloudSoft.Auth.Web/Data/TestUserSeeder.cs`

```
using System.Security.Claims;
using Microsoft.AspNetCore.Identity;

namespace CloudSoft.Auth.Web.Data;

public static class TestUserSeeder
{
    private record SeedUser(string Username, string
        Email, string Password, string Role, string
        Department);

    private static readonly SeedUser[] Seeds =
    [
        new("admin",     "admin@example.com",
        "admin",     "Admin",     "Engineering"),
        new("candidate", "candidate@example.com", "can-
        didate", "Candidate", "Sales"),
    ];

    public static async Task SeedAsync(IServiceProvider
        services)
    {
        using var scope = services.CreateScope();
        var userManager = scope.ServiceProvider.GetRe-
        quiredService<UserManager<ApplicationUser>>();
        var roleManager = scope.ServiceProvider.GetRe-
        quiredService<RoleManager<IdentityRole>>();

        foreach (var role in new[] { "Admin", "Candid-
        ate" })
        {
            if (!await roleMan-
        ager.RoleExistsAsync(role))
                await roleManager.CreateAsync(new Iden-
        tityRole(role));
        }

        foreach (var seed in Seeds)
        {
            var user = await userMan-
        ager.FindByNameAsync(seed.Username);
            if (user is null)
            {
                user = new ApplicationUser
                {
```

```
                    UserName = seed.Username,
                    Email = seed.Email,
                    EmailConfirmed = true,
                };
                var create = await userManager.CreateA-
        sync(user, seed.Password);
                if (!create.Succeeded)
                {
                    throw new InvalidOperationException(
                        $"Failed to seed {seed.User-
        name}: {string.Join("; ",
        create.Errors.Select(e => e.Description))}");
                }
                await userManager.AddToRoleAsync(user,
        seed.Role);
                await userManager.AddClaimAsync(user,
        new Claim("Department", seed.Department));
            }
        }
    }
}
```

2. **Call** the seeder from `Program.cs`, just before `app.Run()`:

> `src/CloudSoft.Auth.Web/Program.cs`

```
await TestUserSeeder.SeedAsync(app.Services);

app.Run();
```

CONCEPT DEEP DIVE

The seeder is idempotent because it checks `FindBy-NameAsync` before creating. With InMemory the check always returns null on the first boot and the user is created; with SQLite (Exercise 5.2) the user is created once and every subsequent boot finds them. Same code, different persistence.

Step 6: *Delete the old DummyUsers.cs*

`AccountController` no longer references `DummyUsers`. The class is dead code.

```
rm src/CloudSoft.Auth.Web/Data/DummyUsers.cs
```

Step 7: *Test Your Implementation*

1. **Run** the app:

   ```
   dotnet run --project src/CloudSoft.Auth.Web --launch-
           profile http
   ```

2. Sign in as `admin` / `admin`. The Who Am I page now shows:
   - `Authentication type: Identity.Application` (was `Cookies` in Chapter 4)
   - `Roles: Admin` (from Identity's user-roles join)
   - Claims table includes the role claim and `Department = Engineering`
3. The admin-only page and engineering page still work exactly as before.
4. Restart the app (Ctrl+C, `dotnet run` again). Log in as admin again. This proves the seeder re-runs on every InMemory boot.

> √ ***Success indicators:***
>
> - *Login still works with the Chapter-4 credentials*
> - *The authentication scheme name changed to* `Identity.Application`
> - *Roles and claims surface through Identity rather than being assembled by hand*
> - *The seeder runs cleanly; a second login attempt doesn't create duplicate users*

19.2.4 Common Issues

> ***If you encounter problems:***
> ***PasswordValidator error when seeding:*** *The password doesn't meet the configured policy. Either loosen* `PasswordOptions` *further (for dev only) or use stronger seed passwords.*
> ***"Scheme Cookies does not exist":*** *You removed* `AddCookie(...)` *but something (old code) still references the* `Cookies` *scheme name. Identity's scheme is* `IdentityConstants.ApplicationScheme` *(*`"Identity.Application"`*).*

19.2.5 Summary

You've migrated from a hand-rolled cookie sign-in to Identity-managed authentication:

- √ `ApplicationUser` + `ApplicationDbContext` as the data model
- √ `AddIdentity` + `AddEntityFrameworkStores` for the plumbing

- ✓ `SignInManager.PasswordSignInAsync` replaces manual `ClaimsPrincipal` construction
- ✓ A startup seed populates the same two test users the lab has used all along

> **KEY TAKEAWAY:**
> Identity is a thin abstraction over the primitives you built in Chapter 4. The cookie is the same kind of cookie; the claims are the same kind of claims. The gain is persistence-ready user management for free.

19.2.6 Done!

Users live in memory. **Exercise 5.2** adds a feature flag that lets you swap the store for SQLite without changing any other code.

19.3 2. User Store: InMemory vs SQLite with a Feature Flag

19.4 User Store: InMemory vs SQLite with a Feature Flag

19.4.1 Goal

The Identity integration from Exercise 5.1 uses EF Core InMemory — fast, zero setup, but volatile. Real applications need persistence. Instead of committing to one or the other, **a feature flag in configuration chooses at startup**. The rest of the code doesn't change.

This teaches the broader pattern: when a component has a clean interface (here, `DbContext`), swapping implementations is a DI registration change and nothing else. The same

`UserManager`, the same `SignInManager`, the same views — two different storage layers.

WHAT YOU'LL LEARN:

- How `AddDbContext` is the seam where you swap EF Core providers
- How configuration (flags and connection strings) controls runtime behavior
- When `EnsureCreated()` is appropriate versus EF Core migrations
- How to verify a feature flag is actually in effect at runtime

19.4.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed Exercise 5.1 — Identity with InMemory works
- ✓ The Chapter-4 regression tests still pass

19.4.3 Exercise Steps

Overview

1. **Install** the SQLite EF Core provider
2. **Add** the `IdentityStore:Provider` flag to `appsettings.json`
3. **Swap** `AddDbContext` to a `switch` on the provider name
4. **Call** `EnsureCreated()` for relational providers
5. **Build** a `/Diagnostics/Store` page that reports the active provider

6. **Test Your Implementation** under both stores

Step 1: *Install the SQLite provider*

```
dotnet add src/CloudSoft.Auth.Web package
        Microsoft.EntityFrameworkCore.Sqlite
```

Step 2: *Configure the flag and the connection string*

Configuration settings are the right home for "what provider?" and "where's the file?" — the application code stays free of literals.

1. **Edit** `src/CloudSoft.Auth.Web/appsettings.json`:

> `src/CloudSoft.Auth.Web/appsettings.json`

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "IdentityStore": {
    "Provider": "InMemory"
  },
  "ConnectionStrings": {
    "Identity": "Data Source=cloudsoft-auth.db"
  }
}
```

> **CONCEPT DEEP DIVE**
>
> Configuration keys with colons (`IdentityStore:Provider`) map to nested objects in JSON. The same key can be overridden via environment variable: `IdentityStore__Provider=SQLite` (`__` in env-var syntax for the colon). That's how the SQLite test harness overrides the default without editing files.

Step 3: *Branch AddDbContext on the flag*

This is the entire behavior change. Two lines of configuration control which provider runs.

1. **Open** `src/CloudSoft.Auth.Web/Program.cs`
2. **Replace** the `AddDbContext` call with:

> `src/CloudSoft.Auth.Web/Program.cs`

```
var storeProvider = builder.Configuration["Iden-
        tityStore:Provider"] ?? "InMemory";
builder.Services.AddDbContext<ApplicationDbContext>(options
        =>
{
    switch (storeProvider.ToLowerInvariant())
    {
        case "sqlite":
            var connectionString =
        builder.Configuration.GetConnectionString("Iden-
        tity")
                ?? "Data Source=cloudsoft-auth.db";
            options.UseSqlite(connectionString);
            break;

        case "inmemory":
        default:
            options.UseInMemoryDatabase("CloudSoftAu-
        thDb");
            break;
    }
});
```

COMMON MISTAKES

- Using `AddDbContext<ApplicationDbContext>(options => …)` twice. The second call overrides the first; DI holds exactly one registration per service type.
- Hardcoding `UseInMemoryDatabase` and `UseSqlite` without the switch — the whole point is one source of truth for the flag.

Step 4: *Ensure the schema exists before first use*

InMemory doesn't need schema — EF creates its internal maps on first use. SQLite does: the file has to exist with the right tables. `EnsureCreated()` handles this; it creates the

schema only if the DB has no tables and is a no-op on subsequent runs.

1. **In** `Program.cs`, **just before** the `TestUserSeeder.SeedAsync(...)` call, **add**:

> `src/CloudSoft.Auth.Web/Program.cs`

```
using (var scope = app.Services.CreateScope())
{
    var db =
        scope.ServiceProvider.GetRequiredService<Applic-
        ationDbContext>();
    if (db.Database.IsRelational())
    {
        db.Database.EnsureCreated();
    }
}
```

> **CONCEPT DEEP DIVE**
>
> `EnsureCreated` is fine for demo apps, tests, and early development. For production you want **EF Core migrations** — they track schema versions and upgrade existing databases. The teaching point here is the swap mechanism, not the migration tooling.

Step 5: *Add a diagnostics page*

A tiny endpoint that reveals the active provider makes the flag concrete. Students can flip the flag, reload, and see the change. Tests use it to verify the wiring.

1. **Create** `src/CloudSoft.Auth.Web/Controllers/DiagnosticsController.cs`:

> `src/CloudSoft.Auth.Web/Controllers/DiagnosticsController.cs`

```
using CloudSoft.Auth.Web.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace CloudSoft.Auth.Web.Controllers;

public class DiagnosticsController : Controller
{
    private readonly ApplicationDbContext _db;
    private readonly IConfiguration _config;

    public DiagnosticsController(ApplicationDbContext
        db, IConfiguration config)
    {
        _db = db;
        _config = config;
    }

    [HttpGet]
    public IActionResult Store()
    {
        ViewBag.ConfiguredProvider = _config["Iden-
        tityStore:Provider"] ?? "InMemory";
        ViewBag.ProviderName =
        _db.Database.ProviderName ?? "(unknown)";
        ViewBag.IsRelational =
        _db.Database.IsRelational();
        return View();
    }
}
```

2. **Create** `src/CloudSoft.Auth.Web/Views/Diagnostics/Store.cshtml`:

> `src/CloudSoft.Auth.Web/Views/Diagnostics/Store.cshtml`

```
@{
    ViewData["Title"] = "Store Diagnostics";
}

<h1>Identity store diagnostics</h1>

<dl class="row">
    <dt class="col-sm-4">Configured provider</dt>
    <dd class="col-sm-8" data-testid="diag-configured-
        provider">@ViewBag.ConfiguredProvider</dd>

    <dt class="col-sm-4">EF Core provider in use</dt>
    <dd class="col-sm-8" data-testid="diag-provider-
        name">@ViewBag.ProviderName</dd>

    <dt class="col-sm-4">Is relational?</dt>
    <dd class="col-sm-8" data-testid="diag-is-rela-
        tional">@ViewBag.IsRelational</dd>
</dl>
```

Step 6: *Test Your Implementation*

InMemory (default):

1. `dotnet run --project src/CloudSoft.Auth.Web --launch-profile http`
2. Visit `/Diagnostics/Store` — it reports `InMemory`.
3. Log in as admin, restart the app, log in again — works each time because the seeder recreates users on every boot.

SQLite:

1. Stop the app.
2. Set the flag and start again:

```
IdentityStore__Provider=SQLite dotnet run --project src/
        CloudSoft.Auth.Web --launch-profile http
```

3. `/Diagnostics/Store` now reports `SQLite`, with EF Core provider `Microsoft.EntityFrameworkCore.Sqlite`.

4. A new `cloudsoft-auth.db` file appears in `src/CloudSoft.Auth.Web/`.
5. Log in as admin. Stop the app. Start it again **without wiping the DB file**. Admin still exists — the seeder's `FindByNameAsync` returned the existing row and took no action.

> √ ***Success indicators:***
>
> - *The diagnostics page reflects the flag's value*
> - *InMemory resets every boot; SQLite persists*
> - *Tests pass under both harnesses:*
> - `./run-playwright-tests.sh headless` — *default InMemory*
> - `./run-playwright-tests-sqlite.sh headless` — *SQLite (wipes the DB first)*

19.4.4 Common Issues

> ***If you encounter problems:***
> **`InvalidOperationException: The current store provider is not registered.`***: You referenced a provider (e.g.* `UseSqlServer`*) without installing its NuGet package. Add the matching EF Core provider package.*
> ***SQLite DB file not appearing:*** *Check the working directory when you ran the app. Relative connection strings (*`Data Source=cloudsoft-auth.db`*) are relative to the process's CWD — typically the project directory when launched via* `dotnet run`*.*
> **`EnsureCreated` *throws when migrations exist:*** `EnsureCreated` *and migrations don't mix. If you later adopt migrations (*`dotnet ef migrations add ...`*), replace* `EnsureCreated()` *with* `Migrate()`*.*
> ***Test passes under InMemory but fails under SQLite:*** *InMemory is lenient about foreign keys and transactions. Real databases are strict. Failures here usually indicate a bug your tests caught — good news.*

19.4.5 Summary

You've given the app two user stores and a flag to pick between them:

- √ `AddDbContext` is the single point of substitution
- √ `IdentityStore:Provider` in configuration chooses at startup
- √ `EnsureCreated` bootstraps relational schemas without migrations

- ✓ A diagnostics page makes the choice visible

> **KEY TAKEAWAY:**
> Persistence is an implementation detail behind the `DbContext`. The same is true for many infrastructure concerns (logging, messaging, caching). Design toward interfaces, register implementations behind flags, and you keep deployment-time flexibility.

19.4.6 Done!

InMemory and SQLite both work. **Exercise 5.3** replaces the hardcoded `TestUserSeeder` with a config-driven admin seeder — the real-world pattern for bootstrapping the first privileged account in a fresh database.

19.5 3. Seeding the First Admin

19.6 Seeding the First Admin

19.6.1 Goal

A fresh database has no users. In Exercise 5.1 we cheated by hardcoding test credentials in `TestUserSeeder`. This exercise replaces that with the **real-world pattern**: the admin user's details live in configuration (typically `user-secrets` or environment variables), the seeder reads them on every boot, and it's idempotent — creating the admin on a fresh DB and doing nothing on subsequent restarts.

The same code path handles both stores: - **InMemory**: the admin user disappears every restart; the seeder recreates from config. - **SQLite**: the admin user persists; the seeder sees the existing row and takes no action.

Same idempotent check, different persistence behavior. That's the point.

WHAT YOU'LL LEARN:

- Why public "bootstrap" endpoints are dangerous and why config-driven seeding is the right pattern
- How `UserManager.FindByNameAsync` makes seeding idempotent
- Why `user-secrets` is the right place for development admin credentials
- How `IHostEnvironment.IsDevelopment()` lets you mix production patterns with dev conveniences

19.6.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed Exercises 5.1 and 5.2 — Identity with both stores works
- ✓ The diagnostics page correctly reports the active provider

19.6.3 Exercise Steps

Overview

1. **Write** an `IdentitySeeder` that replaces `TestUserSeeder`
2. **Move** admin credentials out of code and into configuration
3. **Keep** the dev-only candidate user for lab convenience
4. **Switch** `Program.cs` to call the new seeder
5. **Test Your Implementation** — including the real `user-secrets` pattern

Step 1: *Write IdentitySeeder*

The seeder does three things: 1. Ensures the `Admin` and `Candidate` roles exist 2. Creates the admin user from `AdminSeed:*` config — or logs a warning if not configured 3. In Development only, creates a convenience candidate user

1. **Create** `src/CloudSoft.Auth.Web/Data/IdentitySeeder.cs`:

> `src/CloudSoft.Auth.Web/Data/IdentitySeeder.cs`

```
using System.Security.Claims;
using Microsoft.AspNetCore.Identity;

namespace CloudSoft.Auth.Web.Data;

public static class IdentitySeeder
{
    private static readonly string[] RequiredRoles =
        ["Admin", "Candidate"];

    public static async Task SeedAsync(IServiceProvider
        services, IHostEnvironment env)
    {
        using var scope = services.CreateScope();
        var userManager = scope.ServiceProvider.GetRe-
        quiredService<UserManager<ApplicationUser>>();
        var roleManager = scope.ServiceProvider.GetRe-
        quiredService<RoleManager<IdentityRole>>();
        var config = scope.ServiceProvider.GetRequired-
        Service<IConfiguration>();
        var logger = scope.ServiceProvider.GetRequired-
        Service<ILogger<ApplicationDbContext>>();

        foreach (var role in RequiredRoles)
        {
            if (!await roleMan-
        ager.RoleExistsAsync(role))
                await roleManager.CreateAsync(new Iden-
        tityRole(role));
        }

        await SeedAdminFromConfigAsync(userManager,
        config, logger);

        if (env.IsDevelopment())
            await SeedDevCandidateAsync(userManager);
    }

    private static async Task SeedAdminFromConfigAsync(
        UserManager<ApplicationUser> userManager,
        IConfiguration config,
        ILogger logger)
    {
        var username = config["AdminSeed:Username"];
```

```
        var email    = config["AdminSeed:Email"];
        var password = config["AdminSeed:Password"];
        var department =
        config["AdminSeed:Department"] ??
        "Engineering";

        if (string.IsNullOrWhiteSpace(username) ||
            string.IsNullOrWhiteSpace(email) ||
            string.IsNullOrWhiteSpace(password))
        {
            logger.LogWarning(
                "AdminSeed is not fully configured. Set
        AdminSeed:Username, " +

        "AdminSeed:Email, and AdminSeed:Password via
        user-secrets or " +
                "environment variables to enable admin
        bootstrap.");
            return;
        }

        if (await userManager.FindByNameAsync(username)
        is not null)
            return; // idempotent

        var admin = new ApplicationUser
        {
            UserName = username,
            Email = email,
            EmailConfirmed = true,
        };

        var create = await userMan-
        ager.CreateAsync(admin, password);
        if (!create.Succeeded)
            throw new InvalidOperationException(
                $"Failed to seed admin '{username}': " +
                string.Join("; ",
        create.Errors.Select(e => e.Description)));

        await userManager.AddToRoleAsync(admin,
        "Admin");
        await userManager.AddClaimAsync(admin, new
        Claim("Department", department));
```

```
        logger.LogInformation("Seeded admin user '{User-
        name}' from AdminSeed config.", username);
    }

    private static async Task
        SeedDevCandidateAsync(UserManager<Applicatio-
        nUser> userManager)
    {
        const string username = "candidate";
        if (await userManager.FindByNameAsync(username)
        is not null) return;

        var candidate = new ApplicationUser
        {
            UserName = username,
            Email = "candidate@example.com",
            EmailConfirmed = true,
        };

        var create = await userManager.CreateAsync(can-
        didate, "candidate");
        if (!create.Succeeded) return;

        await userManager.AddToRoleAsync(candidate,
        "Candidate");
        await userManager.AddClaimAsync(candidate, new
        Claim("Department", "Sales"));
    }
}
```

Step 2: *Point Program.cs at the new seeder*

1. **Open** `src/CloudSoft.Auth.Web/Program.cs`
2. **Replace** the `TestUserSeeder.SeedAsync(...)` call with:

> `src/CloudSoft.Auth.Web/Program.cs`

```
await IdentitySeeder.SeedAsync(app.Services, app.Envir-
        onment);
```

3. **Delete** the old `Data/TestUserSeeder.cs` — it's dead code.

Step 3: *Provide dev defaults, prepare for prod*

Production credentials never live in `appsettings.json`. Development convenience values can.

1. **Edit** `src/CloudSoft.Auth.Web/appsettings.Development.json`:

   > `src/CloudSoft.Auth.Web/appsettings.Development.json`

   ```
   {
     "Logging": {
       "LogLevel": {
         "Default": "Information",
         "Microsoft.AspNetCore": "Warning"
       }
     },
     "AdminSeed": {
       "Username": "admin",
       "Email": "admin@example.com",
       "Password": "admin",
       "Department": "Engineering"
     }
   }
   ```

2. `appsettings.json` stays clean — no `AdminSeed` section. Production configuration comes from a different source.

Step 4: *Configure user-secrets (the production-quality pattern)*

Even in dev, `user-secrets` is how most real teams store secrets. The file lives **outside** the repo under `~/.microsoft/usersecrets/…/secrets.json` and is never checked in.

```
cd src/CloudSoft.Auth.Web
dotnet user-secrets
         init
         # once per project
dotnet user-secrets set "AdminSeed:Username" "admin"
dotnet user-secrets set "AdminSeed:Email"
         "admin@example.com"
dotnet user-secrets set "AdminSeed:Password" "ChangeMe-in-
         prod"
```

With `user-secrets` set, you can remove `AdminSeed` from `appsettings.Development.json` and the seeder still finds the values. The configuration system merges secrets on top of JSON configuration during development.

In production (Azure, Kubernetes, anywhere), the same keys come from environment variables (`AdminSeed__Username=...`) or a secret store (Azure Key Vault).

Step 5: *Test Your Implementation*

InMemory (default): 1. `dotnet run` — the admin is seeded on every boot. 2. Log in as `admin / admin`. WhoAmI shows role + Department claim. 3. Stop, restart. Seeder runs again against an empty store; admin is recreated.

SQLite: 1. Wipe any existing DB: `rm src/CloudSoft.Auth.Web/cloudsoft-auth.db*` 2. `IdentityStore__Provider=SQLite dotnet run` 3. First boot: seeder creates admin in the new DB. 4. Stop, start again. The seeder's `FindByNameAsync` returns the existing admin; no action taken. Login still works.

Missing config: 1. Remove or rename the `AdminSeed` section. 2. Start the app — a `WARN` log entry appears explaining what's missing. 3. Admin doesn't exist; `/Account/Login` with admin credentials fails.

> √ ***Success indicators:***
>
> - *Admin login works under InMemory and SQLite*
> - *SQLite persists the admin across restarts*
> - *Missing* `AdminSeed` *produces a warning instead of a crash*
> - `dotnet user-secrets list` *shows the keys your seeder reads*

19.6.4 Common Issues

> ***If you encounter problems:***
> ***"Failed to seed admin: PasswordTooShort":*** *The password doesn't meet the configured policy. Either relax* `PasswordOptions` *(we already did, back in Exercise 5.1) or use a stronger seed password.*
> `user-secrets` ***values not picked up:*** *Confirm* `dotnet user-secrets list` *shows the keys. The* `UserSecretsId` *in the* `.csproj` *must exist —* `dotnet user-secrets init` *generates and writes it.*
> ***Admin created twice with different passwords:*** *A stale* `FindByNameAsync` *result or race across boots. In a single-node dev app this shouldn't happen; if it does, the idempotency check is wrong.*

19.6.5 Summary

The admin bootstrap is now production-shaped:

- √ Admin credentials live in configuration, not source

- √ `user-secrets` (dev) / env vars (prod) handle secret material
- √ The seeder is idempotent — works on fresh and existing databases
- √ The dev candidate user stays a dev convenience, isolated behind `IHostEnvironment.IsDevelopment()`

> **KEY TAKEAWAY:**
> "How does the first privileged user get into the system?" is a real operations question. The answer in almost every real app is config-driven idempotent startup seeding — exactly what you've built.

19.6.6 Done!

Admin exists on every boot without any bypass endpoint. The final exercise — **Registration and Role Promotion** — completes the picture: ordinary users self-register as Candidates, and admins promote them when appropriate.

19.7 4. Registration and Role Promotion

19.8 Registration and Role Promotion

19.8.1 Goal

Complete the user management picture. Anyone can now **register** — landing in the Candidate role by default. Admins get a **User admin** page that lists all users and promotes Candidates to Admin (or demotes them back). The current admin can never modify their own roles, which prevents the last-admin-locked-out scenario.

This pulls together every primitive from the previous exercises:

- `UserManager.CreateAsync` for registration
- `SignInManager.SignInAsync` to log the new user in without a second round-trip
- `[Authorize(Roles = "Admin")]` to gate the administration page
- `AddToRoleAsync` / `RemoveFromRoleAsync` for promotion and demotion
- Explicit self-check to refuse modifications to the current principal

WHAT YOU'LL LEARN:

- How to build a registration flow without Identity's default Razor UI
- How to enforce "admin privilege is conferred, not self-claimed"
- Why self-role-modification is always explicitly disallowed
- How to keep your test suite green while changing the user creation story

19.8.2 Prerequisites

BEFORE STARTING, ENSURE YOU HAVE:

- ✓ Completed Exercises 5.1–5.3 — Identity, feature-flagged stores, admin seeding
- ✓ An admin account (from `AdminSeed` config) that can log in

19.8.3 Exercise Steps

Overview

1. **Add** `UserManager` to `AccountController`
2. **Add** the `Register` GET/POST actions
3. **Create** the Register view
4. **Link** to Register from Login
5. **Build** `UserAdminController` for promotion/demotion
6. **Create** the UserAdmin view
7. **Add** a WhoAmI link to UserAdmin for admins
8. **Test Your Implementation**

Step 1: *Inject UserManager into AccountController*

The Register action needs to create a user, which is `UserManager.CreateAsync` — a separate service from `SignInManager`. Take both through the constructor.

1. **Open** `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`
2. **Replace** the constructor with:

> `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`

```
private readonly SignInManager<ApplicationUser> _signIn-
        Manager;
private readonly UserManager<ApplicationUser> _userMan-
        ager;

public AccountController(
    SignInManager<ApplicationUser> signInManager,
    UserManager<ApplicationUser> userManager)
{
    _signInManager = signInManager;
    _userManager = userManager;
}
```

Step 2: *Add the Register actions*

The POST creates an `ApplicationUser`, calls `UserManager.CreateAsync(user, password)`, assigns the Candidate role, and signs the new user in. Any Identity errors (short password, duplicate username) are surfaced via `ModelState`.

1. **Inside** `AccountController`, below `AccessDenied`, **add**:

> `src/CloudSoft.Auth.Web/Controllers/AccountController.cs`

```
[HttpGet]
public IActionResult Register() => View();

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(string user-
        name, string email, string password)
{
    if (string.IsNullOrWhiteSpace(username) ||
        string.IsNullOrWhiteSpace(password))
    {
        ModelState.AddModelError(string.Empty,
        "Username and password are required.");
        return View();
    }

    var user = new ApplicationUser
    {
        UserName = username,
        Email = email,
        EmailConfirmed = true,
    };

    var create = await _userManager.CreateAsync(user,
        password);
    if (!create.Succeeded)
    {
        foreach (var error in create.Errors)
            ModelState.AddModelError(string.Empty,
        error.Description);
        return View();
    }

    // New users always start as Candidate. Promotion to
        Admin is an
    // explicit administrator action (UserAdminControl-
        ler).
    await _userManager.AddToRoleAsync(user,
        "Candidate");

    await _signInManager.SignInAsync(user, isPersist-
        ent: false);
    return RedirectToAction("Index", "WhoAmI");
}
```

Step 3: *Create the Register view*

A trivial form with testid hooks for Playwright. The form posts to itself and displays model errors at the top.

1. **Create** `src/CloudSoft.Auth.Web/Views/Account/Register.cshtml`:

> `src/CloudSoft.Auth.Web/Views/Account/Register.cshtml`

```
@{
    ViewData["Title"] = "Register";
}

<h1>Register</h1>

@if (!ViewData.ModelState.IsValid)
{
    <div class="alert alert-danger" data-
        testid="register-error">
        @Html.ValidationSummary()
    </div>
}

<form method="post" data-testid="register-form"
        style="max-width: 420px;">
    @Html.AntiForgeryToken()

    <div class="mb-3">
        <label for="username" class="form-label">User-
        name</label>
        <input type="text" id="username"
        name="username" class="form-control"
               data-testid="register-username" autocom-
        plete="username" />
    </div>

    <div class="mb-3">
        <label for="email" class="form-label">Email</
        label>
        <input type="email" id="email" name="email"
        class="form-control"
               data-testid="register-email" autocom-
        plete="email" />
    </div>

    <div class="mb-3">
        <label for="password" class="form-label">Pass-
        word</label>
        <input type="password" id="password" name="pass-
        word" class="form-control"
               data-testid="register-password" autocom-
        plete="new-password" />
    </div>
```

```
    <button type="submit" class="btn btn-primary" data-
        testid="register-submit">Register</button>
</form>
```

Step 4: *Link to Register from Login*

Below the existing login form, add a single link so new users find the register page.

1. **In** `src/CloudSoft.Auth.Web/Views/Account/Login.cshtml`, **after** the login `<button>`, **add**:

> `src/CloudSoft.Auth.Web/Views/Account/Login.cshtml`

```
<p class="mt-3">
    No account yet?
    <a asp-controller="Account" asp-action="Register"
        data-testid="login-register-link">Register
        here</a>.
</p>
```

Step 5: *Build UserAdminController*

The controller is gated by `[Authorize(Roles = "Admin")]`. The Promote and Demote actions both refuse to operate on the current user — this is the key safety invariant.

1. **Create** `src/CloudSoft.Auth.Web/Controllers/UserAdminController.cs`:

> `src/CloudSoft.Auth.Web/Controllers/UserAdminController.cs`

```
using CloudSoft.Auth.Web.Data;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace CloudSoft.Auth.Web.Controllers;

[Authorize(Roles = "Admin")]
public class UserAdminController : Controller
{
    private readonly UserManager<ApplicationUser> _user-
        Manager;

    public UserAdminController(UserManager<Applicatio-
        nUser> userManager)
    {
        _userManager = userManager;
    }

    [HttpGet]
    public async Task<IActionResult> Index()
    {
        var users = await _userMan-
        ager.Users.ToListAsync();
        var rows = new List<UserRow>();
        foreach (var user in users)
        {
            var roles = await _userMan-
        ager.GetRolesAsync(user);
            rows.Add(new UserRow(user.Id,
        user.UserName ?? "(unknown)", user.Email,
        roles.ToArray()));
        }
        return View(rows);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Promote(string
        userId)
    {
        var user = await _userMan-
        ager.FindByIdAsync(userId);
```

```
        if (user is null) return NotFound();

        var currentUser = await _userManager.GetUserA-
        sync(User);
        if (currentUser is not null && currentUser.Id
        == user.Id)
        {
            TempData["UserAdminMessage"] = "You cannot
        modify your own roles.";
            return RedirectToAction(nameof(Index));
        }

        if (!await _userManager.IsInRoleAsync(user,
        "Admin"))
        {
            await _userManager.AddToRoleAsync(user,
        "Admin");
            TempData["UserAdminMessage"] = $"{user.User-
        Name} is now Admin.";
        }

        return RedirectToAction(nameof(Index));
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Demote(string
        userId)
    {
        var user = await _userMan-
        ager.FindByIdAsync(userId);
        if (user is null) return NotFound();

        var currentUser = await _userManager.GetUserA-
        sync(User);
        if (currentUser is not null && currentUser.Id
        == user.Id)
        {
            TempData["UserAdminMessage"] = "You cannot
        demote yourself.";
            return RedirectToAction(nameof(Index));
        }

        if (await _userManager.IsInRoleAsync(user,
        "Admin"))
        {
```

```
            await _userMan-
        ager.RemoveFromRoleAsync(user, "Admin");
            TempData["UserAdminMessage"] = $"{user.User-
        Name} is no longer Admin.";
        }

        return RedirectToAction(nameof(Index));
    }

    public record UserRow(string Id, string Username,
        string? Email, string[] Roles);
}
```

COMMON MISTAKES

- Omitting the self-check. Without it, an admin could demote themselves; if they were the only admin, the app has no way back.
- Using `string.Equals(username, currentUsername)` instead of comparing IDs. Usernames can be renamed; user IDs are immutable.

Step 6: *Build the UserAdmin view*

The view is a table of users. For each row we show promote or demote buttons depending on the current roles, and the caller's own row gets a static "(cannot modify self)" label.

1. **Create** `src/CloudSoft.Auth.Web/Views/UserAdmin/Index.cshtml`:

> `src/CloudSoft.Auth.Web/Views/UserAdmin/Index.cshtml`

```
@model IEnumerable<Cloud-
        Soft.Auth.Web.Control-
        lers.UserAdminController.UserRow>
@{
    ViewData["Title"] = "User administration";
    var currentUserName = User.Identity?.Name;
}

<h1 data-testid="useradmin-heading">User administra-
        tion</h1>

@if (TempData["UserAdminMessage"] is string msg)
{
    <div class="alert alert-info" data-testid="userad-
        min-message">@msg</div>
}

<table class="table table-striped" data-testid="userad-
        min-table">
    <thead>
        <tr>
            <th>Username</th><th>Email</th><th>Roles</
        th><th>Actions</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var row in Model)
        {
            var isCurrent = string.Equals(row.Username,
        currentUserName, StringComparison.Ordinal);
            <tr data-testid="useradmin-row-@row.User-
        name">
                <td>@row.Username @(isCurrent ? "
        (you)" : "")</td>
                <td>@row.Email</td>
                <td data-testid="useradmin-roles-
        @row.Username">
                    @(row.Roles.Length == 0 ? "(none)" :
        string.Join(", ", row.Roles))
                </td>
                <td>
                    @if (isCurrent)
                    {
                        <span class="text-
        muted">(cannot modify self)</span>
```

```
                    }
                    else if (!row.Roles.Con-
        tains("Admin"))
                    {
                        <form asp-action="Promote"
        method="post" style="display:inline">
                            @Html.AntiForgeryToken()
                            <input type="hidden"
        name="userId" value="@row.Id" />
                            <button type="submit"
        class="btn btn-sm btn-outline-primary"
                                    data-testid="pro-
        mote-@row.Username">
                                Promote to Admin
                            </button>
                        </form>
                    }
                    else
                    {
                        <form asp-action="Demote"
        method="post" style="display:inline">
                            @Html.AntiForgeryToken()
                            <input type="hidden"
        name="userId" value="@row.Id" />
                            <button type="submit"
        class="btn btn-sm btn-outline-warning"
                                    data-testid="demote-
        @row.Username">
                                Demote
                            </button>
                        </form>
                    }
                </td>
            </tr>
        }
    </tbody>
</table>
```

Step 7: *Add the User admin link to Who Am I*

1. **In** `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`, inside the `User.IsInRole("Admin")` block, **add** a second link beside the AdminOnly one:

> `src/CloudSoft.Auth.Web/Views/WhoAmI/Index.cshtml`

```
<a asp-controller="UserAdmin" asp-action="Index"
        class="btn btn-outline-primary me-2" data-
        testid="whoami-useradminlink">
    User admin
</a>
```

Step 8: *Test Your Implementation*

1. **Run** the app. Log in as `admin`.
2. On Who Am I, click **User admin**. You see a table of all users. Your row shows `(you)` and `(cannot modify self)`.
3. Log out. Click **Register here**. Create a user `alice` / `alice@example.com` / `alice`.
4. WhoAmI immediately shows you signed in as `alice` with role `Candidate`.
5. Log out. Log in as admin. On User admin, click **Promote to Admin** next to `alice`.
6. Alice's row now lists both `Candidate` and `Admin` (or just `Admin` depending on how you interpret "promote"). A green banner confirms the change.
7. Click **Demote** to revert. The row returns to `Candidate`.

> √ ***Success indicators:***
>
> - *A fresh registration lands on Who Am I as a signed-in Candidate*
> - *Admin can promote and demote other users*
> - *Admin cannot modify their own row*
> - *Candidates get HTTP 403 / access-denied when hitting* `/UserAdmin`

19.8.4 Common Issues

> ***If you encounter problems:***
> ***Register returns 400:*** *Missing antiforgery token — check that* `@Html.AntiForgeryToken()` *is inside the form.*
> ***"Password too short":*** *The* `PasswordOptions` *you relaxed in Exercise 5.1 apply here too. If you tightened them for production, the dev seed passwords will fail. Either use stronger test passwords or keep the relaxed options under Development.*
> ***Promote button produces "You cannot modify your own roles.":*** *You clicked it on your own row. That's working as designed; use a different user's row.*

19.8.5 Summary

The full user-management story now works end-to-end:

- √ Anyone can register; new users default to `Candidate`
- √ Admins have a dedicated page to promote or demote others
- √ Admins cannot modify their own roles
- √ Access to the admin page is gated by the same role-based attribute you've been using since Chapter 4

> **KEY TAKEAWAY:**
> Administrative privilege is **conferred, not self-claimed**. Registration creates normal users; promotion is an action only an existing admin can perform. The `AdminSeed` mechanism from Exercise 5.3 supplies the bootstrap admin — the rest grow from it.

19.8.6 Done!

You've completed Chapter 5 and the full eight-exercise arc. The app has:

- Cookie authentication built from primitives (Chapter 4)
- Roles, claims, policies, CSRF protection (Chapter 4)
- External login via Google OIDC (Chapter 4)
- ASP.NET Core Identity with a feature-flagged user store (Chapter 5)
- Config-driven idempotent admin seeding (Chapter 5)
- Self-service registration and admin-only role promotion (Chapter 5)

This covers every authentication and authorization mechanism you're likely to use in typical ASP.NET Core applications.

PART VIII

Code Collaboration

Exercises for version control with Git and collaboration workflows with GitHub.

PART IX

Docker

Learn container fundamentals through progressive exercises, from running your first container to orchestrating a multi-service development environment with Docker Compose.

19.9 Learning Path

1. ***Run Your First Container*** *— Pull, run, and manage containers with nginx*
2. ***Build and Push Your First Image*** *— Create a custom image and publish to Docker Hub*
3. ***Containerize CloudSoft Recruitment*** *— Single-stage and multi-stage Dockerfiles for a .NET app*

4. ***Docker Compose — Local Development Stack*** *— Orchestrate app + MongoDB with volumes and networking*
5. ***Multi-Platform Builds*** *— Build for both AMD64 and ARM64 with buildx*